

大模型推理系统与核心原理

从零基础数学推导到现代工业工程落地

深入自注意力、KV Cache、Roofline 模型、FlashAttention、量化与分布式并行

涵盖 14 篇系统理论全景推导 + 14 组完整可运行实验源码精解

Transformer · RoPE · SwiGLU · GQA/MLA · PagedAttention · Speculative Decoding ·
Tensor Parallelism

sharp-volta 开源系统学习工程组编著

2026 年 9 月 · 第一版

前言：大模型推理的底层物理规律

0.1 [起步] 大模型推理（LLM Inference）系统学习指南

欢迎开启大模型推理学习之旅！

在现代 AI 系统中，模型训练是「一次性成本」，而模型推理是「长期且持续膨胀的在线成本」。大模型推理的瓶颈通常不是算力（Compute Bound），而是显存容量与内存带宽（Memory Bandwidth Bound）。理解并掌握大模型推理的核心机制，是从事 AI Infra、算法部署和高性能计算（HPC）的关键技能。

0.2 [指南] 从这里开始

→ 先读 **LEARNING_PATH.md**：完整学习路径。它规定了先学什么、后学什么、每个阶段需要先补哪些外部资源（具体到哪一集、哪一章），以及每个阶段的”出关检验”和”卡住了回去补哪里”。下面的列表只是索引。

环境准备（RTX 50 系显卡必须使用 CUDA 12.8 版本的 PyTorch）：

```
1 python3 -m venv .venv && source .venv/bin/activate
2 pip install -r requirements.txt
3 pip install torch --index-url https://download.pytorch.org/whl/cu128
```

0.3 [重点] 【零基础首选】小白到精通·原理与数学公式完整推导教程

核心数学定理推导

适合人群：没有任何深度学习基础、对公式推导和历史背景充满好奇的零基础学习者。包含从 1950 年代规则系统到 Transformer 的完整演进史，以及所有核心公式的逐行代数证明：

1. [概览] 第 0 篇（导读）：AI 深度学习全景族谱与零基础入门指南

- 从感知机到 CNN、ResNet、RNN、Transformer 的科技树全景族谱，各种“Net”到底都是干什么的，不踩坑入门资源与学习路线。

1. [阅读] 第 1 篇：计算机如何读懂语言？从人工规则到大模型演进史

- 计算机的本质、人工规则为何崩溃、N-gram 语言模型、神经网络与反向传播底座、RNN/LSTM 的长程遗忘与并行死穴、Transformer 的颠覆。

1. [几何] 第 2 篇：零基础必备数学直觉：向量、空间变换、点积与 Softmax 深度推导
 - 标量/向量/张量物理本质、点积与余弦夹角几何意义（为什么代表相似度）、矩阵乘法的空间投影本质、Softmax 的指数来源与防溢出平移不变性证明。
1. [反向] 第 3 篇：神经网络如何学习：导数、链式法则、反向传播与交叉熵
 - 梯度为什么指向下山方向（证明）、反向传播为什么比其它求导方法快、矩阵乘反向公式 $X^T G / G W^T$ 、交叉熵 = 最大似然、困惑度、Softmax 雅可比与 $p - y$ 梯度的完整推导、梯度消失与”加法通路”、训练 6N vs 推理 2N、训练与推理的显存账本。
1. [推导] 第 4 篇：Transformer 核心公式彻底推导：每一个符号从何而来
 - Q/K/V 检索设计、【重点推导】为什么必须除以 $\sqrt{d_k}$ （独立变量方差爆炸与 Softmax 梯度消失严格证明）、因果掩码 $-\infty$ 、残差连接常数导数高速公路、RoPE 相对位置内积恒等性证明。
1. [架构] 第 5 篇：从原版 Transformer 到 Llama / DeepSeek：每一处改动的历史动机
 - 注意力的真正起源（Bengio 2003 $\rightarrow$ Seq2Seq 瓶颈 $\rightarrow$ Bahdanau）、BPE 分词与词表大小的推理代价、为什么仅解码器胜出、规模定律与”推理成本成为主问题”、Pre-LN / RMSNorm / SwiGLU / RoPE / MQA $\rightarrow$ GQA $\rightarrow$ MLA / MoE 每一处改动的动机。
1. [加速] 第 6 篇：什么是推理？自回归生成全生命周期与 KV Cache 代数证明
 - 训练与推理的本质鸿沟、Prefill（算力受限）vs Decode（访存受限）、KV Cache 代数展开冗余消除证明、显存爆炸计算公式与 PagedAttention 诞生背景。
1. [决策] 第 7 篇：大模型的决策大脑：Logits、温度系数极限证明与采样算法
 - LM Head 词表投影、【定理推导】温度参数 $T \rightarrow 0$ （唯一最大值时收敛到贪婪）与 $T \rightarrow \infty$ （均匀分布）严格数学极限证明、Top-K 截断、Top-P 动态核采样与重复惩罚。
1. [硬件] 第 8 篇：硬件视角：GPU 存储层级、FLOPs、算术强度与浮点格式
 - Roofline 模型、推导”Decode 线性层算术强度 $\approx$ batch size”与”Decode 注意力算术强度 $\approx$ GQA 分组大小”、Decode 耗时模型、算子融合、FP32 / BF16 / FP16 / FP8 / FP4 的位布局与精度陷阱。
1. [加速] 第 9 篇：FlashAttention：Online Softmax 的严格推导与 IO 复杂度
 - Online Softmax 的归纳法证明、分块注意力、IO 复杂度、FlashDecoding 的 split-K 合并公式及其结合律。
1. [采样] 第 10 篇：投机解码：无偏性证明与期望加速比

- 投机采样无偏性定理的完整证明、接受率与总变差距离、期望加速比公式与最优 K、为什么大 batch 时收益消失。

1. [量化] 第 11 篇：量化：从每位 6 dB 到 SmoothQuant、AWQ 与 GPTQ

- 每位 6 dB 规律、激活离群值、量化权重还是激活（由 Roofline 决定）、SmoothQuant / AWQ / GPTQ 的数学原理（含两变量的 GPTQ 直观例子）。

1. [系统] 第 12 篇：推理服务系统：PagedAttention、前缀缓存与调度

- TTFT / TPOT / Goodput 指标、Little 定律、显存碎片与分页、写时复制、前缀缓存、Chunked Prefill 与 P/D 分离。

1. [并行] 第 13 篇：多卡并行推理：张量并行、流水线并行与专家并行

- 通信原语与 Ring All-Reduce、Megatron ”先列后行”切法的推导、PP 为什么不降延迟、MoE 的专家并行。

0.4 学习路线图 (Learning Roadmap)

完整的阶段划分、前置条件与外部资源见 `LEARNING_PATH.md`。简版：

0.5 [目录] 本学习仓库目录结构

实验目录 `labNN_*` 的 NN 就是它配套的那一篇（一篇配多个实验时用 a/b 区分）。每个目录都有说明文档和可直接运行的代码：

对应篇	目录	核心内容	重点代码
第 3 篇	lab03_autograd_from_scratch/	约 300 行纯 Python 自动求导, 数值验证 Softmax 雅可比、交叉熵梯度、矩阵乘反向公式、梯度消失、XOR	<code>micrograd_from_scratch.py</code>
第 4 篇	lab04_transformer_microscope/	【极致基础】从分词到下一个字的微观全貌、张量维度流动、RoPE 旋转、置换等变性与长程衰减的正确表述	<code>01_tensor_trace_step_by_step.py</code> <code>02_rope_and_attention_microscope.py</code>
第 5 篇	lab05_train_tiny_llama/	用 HuggingFace 仓库教程做语料训练一个迷你 Llama (RM-SNorm / RoPE / GQA / SwiGLU), 再做 KV Cache 对拍与采样	<code>model.py</code> <code>train.py</code> <code>generate.py</code>
第 6 篇	lab06_kv_cache/	Prefill 与 Decode 阶段区别、自回归生成的重复计算痛点、KV Cache 机制与代码对比	<code>kv_cache_benchmark.py</code>
第 7 篇	lab07a_sampling/	Greedy、温度、Top-K、Top-P、重复惩罚的概率分布前后对比	<code>sampling_strategies.py</code>
第 7 篇	lab07b_qwen_from_scratch/	【进阶】只读原始权重、手写 Qwen2.5 推理, 与 HF 官方逐位对拍 logits 与生成结果	<code>qwen_from_scratch.py</code>
第 8 篇	lab08_memory_and_roofline/	模型权重、KV Cache、激活值显存计算, 计算密度 (Arithmetic Intensity) 与 Roofline 模型分析	<code>memory_calculator.py</code>
第 9 篇	lab09_flash_attention/	Online Softmax、分块 FlashAttention、FlashDecoding 合并, GPU 上朴素 vs SDPA; 真机 CUDA 附录: 同一套 online softmax 写成 CUDA C++ kernel, 实测 DRAM 带宽与占用率	<code>flash_attention_numpy.py</code> <code>cuda_softmax/softmax_kernels.cu</code>
第 10 篇	lab10_speculative_decoding/	投机采样 (Speculative Decoding) 核心思想、草稿模型生成与主模型并行验证算法	<code>toy_speculative_decoding.py</code>
第 11 篇	lab11_quantization/	对称与非对称量化、Per-Tensor / Per-Channel / Per-Group 量化、AWQ / GPTQ 原理	<code>quant_basics.py</code>
第 12 篇	lab12a_continuous_batching/	Continuous Batching 的「气泡 (Padding/Bubble)」问题、迭代级调度 (Continuous Batching) 仿真	<code>continuous_batching_sim.py</code>
第 12 篇	lab12b_paged_attention/	注意力分配器、热表、分页注意力	<code>paged_kv_cache_sim.py</code>

Lab 08、10、11 还各新增了进阶脚本: `measure_gpu_roofline.py` (在你的显卡上实测 Roofline)、`verify_unbiasedness.py` (蒙特卡洛验证投机采样定理)、`advanced_quant_numpy.py` (从零实现 SmoothQuant / AWQ / GPTQ)。

全仓库唯一一份真 CUDA C++ 在 `lab09_flash_attention/cuda_softmax/`: 把第 9 篇的 Online Softmax 写成 kernel (`__global__ / __shared__ / __syncthreads()`), 量出 DRAM 带宽、L2 命中与占用率。用 PyTorch 自带的 NVRTC 编译, 不需要装 CUDA toolkit。

0.6 [决策] 核心概念速查手册

0.6.1 Prefill (首字) vs Decode (增量生成) 阶段

- **Prefill 阶段 (首字延迟 TTFT: Time to First Token):**
 - 输入所有 prompt tokens, 能够并行计算。
 - 特征: **Compute-bound (算力受限)**, GPU 利用率高, 矩阵乘法规模大 ($M \times K \times N$, $M > 1$)。
- **Decode 阶段 (每个后续 token 的延迟 TPOT: Time Per Output Token):**
 - 串行逐字生成, 每次输入仅 1 个 token。
 - 特征: **Memory-bound (访存带宽受限)**。每次生成 1 个 token 都需要把数十亿参数和全部历史 KV Cache 从显存搬运进 GPU 计算核心。

0.6.2 KV Cache 显存占用计算公式

对于标准 Transformer 结构模型:

- 参数量: P
- 层数: L
- 隐藏层大小: H 或注意力头数 $n_{\text{heads}} \times d_{\text{head}}$
- 上下文总长度 (Prompt + Generated): S
- 批大小: B
- 精度字节数: b (FP16/BF16 为 2 字节, FP8/INT8 为 1 字节, INT4 为 0.5 字节)

每个 Token 产生的 Key 和 Value 向量显存 (bytes):

$$\text{KV per token} = 2 \times L \times H \times b$$

(注：如果采用 MQA/GQA ，隐藏层维度 H 需按 KV 组数比例缩减，例如 $GQA-8$ 缩减为原来的 $1/8$)

批次总 KV Cache 显存 (bytes)：

$$\text{Total KV Cache} = B \times S \times 2 \times L \times H \times b$$

核心导读与背景

示例 (Llama-3-8B, 32 层, 隐藏层 4096, GQA 8 个 KV 头 vs 32 个 Q 头, FP16)：
每个 token 仅需要 $2 \times 32 \times (4096/4) \times 2 = 131,072 \text{ bytes} \approx 128 \text{ KB}$ 。长度 8192 时，单并发约 1 GB，若并发达 32，则需 32 GB 显存 (远超模型权重自身的 16 GB)！

0.7 [工具] 主流推理引擎横向对比

框架	适用场景	核心亮点	推荐初学者体验
vLLM	工业界高吞吐部署标准	PagedAttention 显存零浪费、Continuous Batching、广泛支持各类模型	★★★★★
SGLang	复杂 Prompt、Agent/结构化生成、超高吞吐	RadixAttention (自动 KV 复用)、高性能 Runtime	★★★★★
TensorRT-LLM	NVIDIA 极致性能压榨	深度融合 Kernel、In-flight Batching、FP8/FP4 极致调优	★★★★
llama.cpp	边缘设备、CPU、Mac、本地轻量推理	零依赖纯 C++、极致 GGUF 量化支持	★★★★★
TGI (Text Generation Inference)	Hugging Face 生态生产部署	简单可靠、支持 FlashAttention、生产级运维特性	★★★★

0.8 [目标] 推荐第一步

1. 读 **LEARNING_PATH.md**，对照”阶段 0”的表格检查自己缺哪块数学基础。
2. 读第 0、1 篇建立全局地图，然后按路径进入阶段 2 (第 2 篇 → 第 3 篇 → Lab 03)。
3. 想先看到”真东西”提提兴趣的话，可以先跑一下：

```
1 .venv/bin/python lab05_train_tiny_llama/train.py && .venv/bin/python lab05_train_tiny_llama/generate.py
```

花一分钟训练出一个会”背诵本教程”的迷你 Llama——然后再回到路径上，把它的每一行都弄懂。

目录

前言：大模型推理的底层物理规律	i
0.1 [起步] 大模型推理 (LLM Inference) 系统学习指南	i
0.2 [指南] 从这里开始	i
0.3 [重点] 【零基础首选】小白到精通·原理与数学公式完整推导教程	i
0.4 学习路线图 (Learning Roadmap)	iii
0.5 [目录] 本学习仓库目录结构	iii
0.6 [决策] 核心概念速查手册	v
0.6.1 Prefill (首字) vs Decode (增量生成) 阶段	v
0.6.2 KV Cache 显存占用计算公式	v
0.7 [工具] 主流推理引擎横向对比	vi
0.8 [目标] 推荐第一步	vi
 第一部分 核心原理与数学推导篇 (Theory & Derivations)	 1
第 1 章 AI 深度学习全景族谱与零基础入门指南	2
1.1 [概览] 一、深度学习终极演化科技树 (全景族谱)	2
1.2 [指南] 二、各种 “Net” 究竟都是干什么的?	2
1.3 [目标] 三、新手零基础想真正入门, 该看什么? (绝不踩坑三部曲)	4
1.3.1 [第一阶段] 阶段 1: 建立直观的数学与几何世界观 (耗时约 2~3 天)	5
1.3.2 [第二阶段] 阶段 2: 动手写代码与经典教科书 (耗时约 1~2 周)	5
1.3.3 [第三阶段] 阶段 3: 大模型推理与系统实战 (面向生产与高性能)	5
 第 2 章 计算机如何读懂语言? 从人工规则到大模型演进史	 7
2.1 [代码] 一、计算机的天然缺陷: 它只懂数字, 不懂人类语言	7
2.2 [文献] 二、第一代: 基于人工规则与专家系统 (1950s - 1980s)	7
2.2.1 核心做法:	7
2.2.2 为什么彻底失败?	8
2.3 [图表] 三、第二代: 统计语言模型与 N-gram (1990s - 2000s)	8
2.3.1 什么是 “语言模型 (Language Model)”?	8
2.3.2 N-gram 的妥协 (马尔可夫假设)	8
2.3.3 致命死穴:	9
2.4 [决策] 四、第三代: 分布式词向量与循环神经网络 (2010s)	9
2.4.1 核心飞跃: 词嵌入向量 (Word Embedding)	9

2.4.2	补齐底座：什么是神经网络？它是怎么“学会”知识的？	9
2.4.3	为什么传统神经网络处理文本会崩溃？循环神经网络（RNN）的诞生	12
2.4.4	为什么强大的 RNN 和 LSTM 最终被历史无情淘汰？	12
2.5	[加速] 五、第四代：Transformer 的横空出世（2017 至今）	13
2.5.1	Transformer 为什么能一统天下？	13
2.6	[目标] 总结与下一章预告	14
第 3 章	零基础必备数学直觉：向量、空间变换、点积与 Softmax 深度推导	15
3.1	[重点] 一、从标量到张量：数据的物理载体	15
3.2	[几何] 二、点积（Dot Product）：大模型如何计算两个词的关联度？	16
3.2.1	代数定义（怎么算）	16
3.2.2	几何意义与余弦定理（为什么代表相似度？）	16
3.3	三、矩阵乘法：空间的旋转、拉伸与投影	17
3.3.1	物理直觉：矩阵是一个“空间投影仪”	17
3.4	[推导] 四、Softmax 函数：从任意实数打分到合法的概率分布	18
3.4.1	为什么不用最简单的“除以总和”？	18
3.4.2	指数函数 $\exp(x) = e^x$ 的登场与推导	19
3.5	[防护] 五、工程避坑细节：Softmax 的平移不变性与防止数值溢出	20
3.5.1	为什么会溢出？	20
3.5.2	数学拯救：平移不变性证明	20
3.5.3	工业级标准实现（Safe Softmax）：	20
3.6	[目标] 总结与下一章预告	20
第 4 章	神经网络如何学习：导数、链式法则、反向传播与交叉熵	22
4.1	[重点] 一、导数：函数在一点附近的“最佳线性近似”	22
4.1.1	为什么梯度的反方向就是“下山最快”的方向？（严格证明）	22
4.2	[链接] 二、链式法则：误差如何“顺藤摸瓜”找到每个旋钮	23
4.2.1	单变量链式法则	23
4.2.2	多变量链式法则：多条路径要相加	23
4.2.3	手算一个完整例子	23
4.3	[加速] 三、反向传播：为什么它能让训练百亿参数成为可能？	24
4.3.1	方案 A：数值差分（最笨）	24
4.3.2	方案 B：前向模式（Forward-mode AD）	24
4.3.3	方案 C：反向模式（Backpropagation）[OK]	24
4.3.4	一个对推理工程师很重要的推论：训练 $\approx 6N$ ，推理 $\approx 2N$	24
4.4	[推导] 四、矩阵形式的反向传播：只需记住一个公式	25
4.5	[目标] 五、损失函数为什么是交叉熵？——最大似然估计	25
4.5.1	语言模型的训练目标	25
4.5.2	为什么取对数、再取负号？	25

4.5.3	困惑度 (Perplexity): 一个有物理含义的指标	26
4.6	[循环] 六、Softmax 与交叉熵的梯度: 本系列最重要的一组推导	26
4.6.1	Softmax 的雅可比矩阵	26
4.6.2	Softmax + 交叉熵: 一个美得不可思议的结果	27
4.6.3	工程细节: 永远不要先算 Softmax 再取 log	27
4.7	[下降] 七、梯度消失与爆炸: 从 Sigmoid、RNN 到残差与 LSTM 的一条暗线	27
4.7.1	激活函数的导数决定了梯度能走多远	27
4.7.2	RNN 为什么会忘记 (第 1 篇”传话游戏”的数学版)	28
4.7.3	LSTM 与残差连接: 同一个思想——”加法通路”	28
4.8	八、初始化与归一化: 与 $\sqrt{d_k}$ 同一个方差推导	28
4.9	[演练] 九、优化器与”为什么训练比推理贵这么多显存”	29
4.10	[OK] 本篇自测 (答不出来就回去重读对应小节)	29
4.11	[资料] 外部资源 (按推荐顺序)	30
第 5 章 Transformer 核心公式彻底推导: 每一个符号从何而来		31
5.1	[架构] 一、终极核心公式全貌	31
5.2	[观察] 二、为什么需要拆分成 Q、K、V? (搜索引擎直觉)	31
5.2.1	类比: YouTube / 百度搜索引擎的底层逻辑	31
5.2.2	为什么不能直接用 $X \cdot X^T$?	32
5.3	[爆炸] 三、【重磅数学推导】: 为什么必须除以 $\sqrt{d_k}$?	32
5.3.1	假设输入是标准正态分布	33
5.3.2	计算点积 $Z = \vec{q} \cdot \vec{k}$ 的均值与方差	33
5.3.3	方差爆炸的灾难后果: Softmax 梯度消失	34
5.3.4	救世主: 除以 $\sqrt{d_k}$	34
5.4	[禁用] 四、因果掩码 (Causal Mask): 为什么必须填充 $-\infty$?	34
5.4.1	数学逻辑:	35
5.5	[路径] 五、残差连接 (Residual Connection): 信息的高速公路	35
5.5.1	为什么不能直接 $X_{\text{output}} = \text{SubLayer}(X_{\text{input}})$?	35
5.5.2	加入残差后的神奇变化:	35
5.6	[循环] 六、RoPE (旋转位置编码): 复平面上的几何芭蕾	36
5.6.1	核心目标	36
5.6.2	二维复平面的几何推导	36
5.6.3	为什么内积只取决于相对距离?	36
5.7	[目标] 总结与下一章预告	37
第 6 章 从原版 Transformer 到 Llama / DeepSeek: 每一处改动的历史动机		39
6.1	[指南] 一、注意力的真正起源 (2003 $\rightarrow$ 2017)	39
6.1.1	2003: 第一个神经语言模型 (Bengio NPLM)	39
6.1.2	2014: Seq2Seq 与”信息瓶颈”	40

6.1.3	2014: Bahdanau 注意力——”别压缩了，每一步回头看”	40
6.1.4	2017: Transformer 的真正创新是”只要注意力”	40
6.2	二、分词 (Tokenization): 为什么”词表大小”会影响推理速度	41
6.2.1	为什么不按”词”或”字”切?	41
6.2.2	BPE: 从数据压缩借来的算法	41
6.2.3	词表大小是一个推理工程上的权衡	41
6.3	[架构] 三、三条路线: 为什么最后是”只有解码器”的 GPT 赢了?	42
6.4	[增长] 四、规模定律: 为什么”推理成本”成了整个行业的主问题	42
6.5	[工具] 五、从原版 Transformer 到现代 LLM: 逐个改动的”为什么”	43
6.5.1	Post-LN → Pre-LN: 把归一化挪出”高速公路”	43
6.5.2	LayerNorm → RMSNorm: 去掉”减均值”	43
6.5.3	ReLU → GELU → SwiGLU: 门控前馈网络	43
6.5.4	绝对位置编码 → RoPE: 为了”外推”与”相对位置”	44
6.5.5	MHA → MQA → GQA → MLA: 一部”为 KV Cache 减肥”的历史	44
6.5.6	稠密 → MoE (混合专家): 参数多 ≠ 算得多	46
6.5.7	其它细节改动	46
6.6	六、一张总表: 同一个”Transformer”, 七年里变了什么	47
6.7	[OK] 本篇自测	47
6.8	[资料] 外部资源	48
第 7 章	什么是推理? 自回归生成全生命周期与 KV Cache 代数证明	50
7.1	[对比] 一、训练 (Training) vs 推理 (Inference) 的物理鸿沟	50
7.1.1	训练时的”上帝视角” (Teacher Forcing)	50
7.1.2	推理时的”摸黑前行” (自回归 Autoregressive Loop)	51
7.2	[加速] 二、推理的两大完全不同阶段: Prefill 与 Decode	51
7.3	[观察] 三、KV Cache 的代数展开与冗余消除证明	51
7.3.1	朴素自回归计算中的重复劳动 (无 KV Cache)	51
7.3.2	KV Cache 的工程解法: 以显存换算力	52
7.3.3	复杂度代数对比:	53
7.4	[隐患] 四、KV Cache 的代价: 显存爆炸与 PagedAttention 的诞生	53
7.4.1	KV Cache 显存占用精确公式推导	53
7.4.2	现代模型的架构救赎: GQA 与 PagedAttention	54
7.5	[目标] 总结与下一章预告	55
第 8 章	大模型的决策大脑: Logits、温度系数极限证明与采样算法	56
8.1	[目标] 一、最后的终点线: LM Head (语言模型头)	56
8.1.1	词表矩阵乘法	56
8.2	[剖析] 二、决策方法 1: 贪婪搜索 (Greedy Search)	57
8.2.1	优缺点分析:	57

8.3	[温度] 三、【重磅数学推导】：温度系数 (Temperature) 的极限证明	57
8.3.1	定理 1: 若最大 logit 唯一, 当温度 $T \rightarrow 0^+$ 时, 概率分布严格退化为贪婪搜索 (确定性决策)	58
8.3.2	定理 2: 当温度 $T \rightarrow +\infty$ 时, 概率分布严格退化为均匀分布 (纯随机乱码)	60
8.4	[目标] 四、工业级截断双子星: Top-K 与 Top-P	60
8.4.1	Top-K 采样	60
8.4.2	Top-P 采样 (核采样 Nucleus Sampling, 更智能的动态截断)	61
8.5	五、防复读机制: 重复惩罚 (Repetition Penalty)	61
8.6	[检验] 前七篇小结	62
第 9 章 硬件视角: GPU 存储层级、FLOPs、算术强度与浮点格式		63
9.1	[硬件] 一、GPU 的基本构造: 计算很便宜, 搬数据很贵	63
9.1.1	计算单元	63
9.1.2	存储层级 (越往下越大、越慢)	64
9.2	[数字] 二、数清楚两件事: 算了多少 (FLOPs)、搬了多少 (Bytes)	64
9.2.1	矩阵乘的 FLOPs	64
9.2.2	矩阵乘的访存量	64
9.2.3	算术强度 (Arithmetic Intensity)	64
9.3	[几何] 三、Roofline 模型: 一张图判断你被谁卡住	65
9.4	[目标] 四、核心推导: Decode 的算术强度 $\approx$ Batch Size	65
9.4.1	线性层 (占 Decode 权重读取的绝大部分)	65
9.4.2	Prefill	66
9.4.3	Decode 中的注意力: batch 救不了它	66
9.4.4	把两部分合起来: 一步 Decode 的耗时模型	67
9.5	[链接] 五、小算子与”算子融合”	67
9.6	[实验] 六、浮点格式: 量化与混合精度的前提	67
9.6.1	浮点数的结构	67
9.6.2	为什么 BF16 打败了 FP16?	68
9.6.3	低精度的陷阱: 累加必须用高精度	68
9.6.4	整数格式与浮点格式的区别	68
9.7	[OK] 本篇自测	68
9.8	[资料] 外部资源	69
第 10 章 FlashAttention: Online Softmax 的严格推导与 IO 复杂度		70
10.1	[低速] 一、朴素注意力慢在哪里?	70
10.2	[推导] 二、Online Softmax: 一遍扫描算出 Softmax	71
10.2.1	传统的”安全 Softmax”要扫三遍	71
10.2.2	把前两遍合成一遍	71
10.3	[加速] 三、把输出也”在线”算出来: FlashAttention 的核心	72

10.3.1	FlashAttention 算法（前向，单个头）	72
10.3.2	它省下了什么？	72
10.4	[组件] 四、FlashDecoding：把同一个技巧用在 Decode 上	73
10.4.1	问题	73
10.4.2	解法：沿 K/V 的序列维度切分（split-K）	73
10.4.3	与 PagedAttention 的关系	74
10.5	五、版本演进一览	74
10.6	[OK] 本篇自测	74
10.7	[资料] 外部资源	75
第 11 章	投机解码：无偏性证明与期望加速比	76
11.1	[加速] 一、为什么”猜 K 个再一起验证”会更快？	76
11.2	[几何] 二、单个 token 的无偏性定理	76
11.3	[链接] 三、推广到 K 个 token	77
11.4	[增长] 四、期望加速比公式	78
11.5	[启蒙] 五、演进	78
11.6	[OK] 本篇自测	79
11.7	[资料] 外部资源	79
第 12 章	量化：从每位 6 dB 到 SmoothQuant、AWQ 与 GPTQ	80
12.1	一、量化误差的基本规律：每多 1 位，信噪比 +6 dB	80
12.2	[瓶颈] 二、大模型为什么难量化：激活离群值	81
12.2.1	先想清楚：你要量化什么？	81
12.3	[基础] 三、SmoothQuant：把激活的难度”转移”给权重	81
12.4	[目标] 四、AWQ：保护 1% 的关键权重	82
12.5	[工具] 五、GPTQ：用二阶信息补偿舍入误差	82
12.5.1	先看一个只有两个权重的例子	82
12.5.2	目标	82
12.5.3	核心思想（来自 1993 年的 Optimal Brain Surgeon）	82
12.5.4	让它能用在 175B 模型上的工程技巧	83
12.6	[OK] 六、小结：选哪个？	83
12.7	[OK] 本篇自测	83
12.8	[资料] 外部资源	83
第 13 章	推理服务系统：PagedAttention、前缀缓存与调度	85
13.1	一、先定义”好”：推理服务的指标体系	85
13.1.1	Little 定律：并发数从哪来	86
13.2	[反向] 二、迭代级调度回顾：为什么连续批处理成立	86
13.3	[模块] 三、KV Cache 的显存碎片：PagedAttention 要解决的问题	86
13.3.1	传统做法：按最大长度预留连续显存	86

13.3.2	PagedAttention: 把操作系统的虚拟内存搬过来	87
13.3.3	代价: 注意力 kernel 要”查表读 KV”	87
13.3.4	显存不够时怎么办: 抢占	87
13.4	四、共享: 引用计数与写时复制	87
13.5	五、前缀缓存: 跨请求复用 KV	88
13.6	六、Prefill 与 Decode 的冲突: Chunked Prefill 与 P/D 分离	88
13.6.1	问题: 一个长 Prefill 会让所有人卡住	88
13.6.2	Chunked Prefill (Sarathi-Serve, 2024)	89
13.6.3	P/D 分离 (DistServe、Mooncake, 2024)	89
13.7	七、一个现代推理引擎里还有什么	89
13.8	[OK] 本篇自测	90
13.9	[资料] 外部资源	90
第 14 章	多卡并行推理: 张量并行、流水线并行与专家并行	91
14.1	一、先认识”通信原语”和它们的代价	91
14.2	二、张量并行 (Tensor Parallelism, TP): 把每一层的矩阵切开	92
14.2.1	第一个矩阵按列切	92
14.2.2	第二个矩阵按行切	92
14.2.3	为什么不能反过来?	92
14.2.4	注意力: 按”头”切	93
14.2.5	代价与收益	93
14.3	[硬件] 三、流水线并行 (Pipeline Parallelism, PP): 按层切	93
14.4	[人员][实验] 四、专家并行 (Expert Parallelism, EP): MoE 的切法	93
14.5	[并行] 五、其它并行方式	94
14.6	[指南] 六、怎么组合? 一些经验法则	94
14.7	[OK] 本篇自测	94
14.8	[资料] 外部资源	95
第二部分	工业实战与实验源码精解篇 (Practical Labs & Implementations)	96
第 15 章	从零手写自动微分与反向传播引擎 (Micrograd)	97
15.1	Lab 03: 从零手写自动求导 (Autograd)	97
15.2	为什么要自己写一遍?	97
15.3	运行	97
15.4	动手练习 (做完才算过关)	98
15.5	学完之后	98
15.6	实验核心源码精选与解析	98
15.6.1	源码实现: micrograd_from_scratch.py	98

第 16 章 Transformer 推理显微镜：逐层张量变化与 RoPE 实测	108
16.1 Lab 04: 大模型推理的微观全貌（零起点极细拆解）	108
16.2 [指南] 1. 宏观全貌：从一句话到下一个字的 8 个步骤	108
16.3 [观察] 2. 张量形状流（Tensor Shape Flow Tracker）	109
16.4 [实验] 3. 逐个核心算子的底层细节与数学公式	111
16.4.1 (1) 为什么注意力机制必须有「因果掩码（Causal Mask）」？	111
16.4.2 (2) RMSNorm（均方根归一化）到底算什么？	111
16.4.3 (3) SwiGLU（门控前馈网络）算什么？	111
16.4.4 (4) 采样：从 Logits 到文字	112
16.5 [执行] 运行显微镜级追踪脚本	112
16.6 [链接] 说明与下一步	112
16.7 实验核心源码精选与解析	112
16.7.1 源码实现：01_tensor_trace_step_by_step.py	112
16.7.2 源码实现：02_rope_and_attention_microscope.py	120
第 17 章 从零训练与推理最小完整 Llama 模型	125
17.1 Lab 05: 亲手训练一个迷你 Llama，再在它身上做推理	125
17.2 为什么需要这个模块？	125
17.3 文件	125
17.4 运行	125
17.5 你会看到什么	125
17.6 动手练习	126
17.7 下一步	127
17.8 实验核心源码精选与解析	127
17.8.1 源码实现：model.py	127
17.8.2 源码实现：train.py	131
17.8.3 源码实现：generate.py	135
第 18 章 KV Cache 性能基准测试与吞吐加速比实测	141
18.1 Lab 06: KV Cache 原理与自回归解码基准测试	141
18.2 [思考] 为什么需要 KV Cache？	141
18.2.1 朴素实现的问题（无 KV Cache）：	141
18.2.2 KV Cache 的优化思路：	141
18.3 [执行] 运行测试	142
18.4 [链接] 继续深入	142
18.5 实验核心源码精选与解析	142
18.5.1 源码实现：kv_cache_benchmark.py	142

第 19 章 工业级文本采样策略实操 (Top-K / Top-P / Min-P)	149
19.1 Lab 07a: 采样策略显微镜	149
19.2 运行	149
19.3 速查	149
19.4 下一步	150
19.5 实验核心源码精选与解析	150
19.5.1 源码实现: <code>sampling_strategies.py</code>	150
第 20 章 纯 Python 零依赖 Qwen 模型真实权重推理	155
20.1 Lab 07b: 亲手实现 Qwen2.5 的推理, 并与官方逐位对拍	155
20.2 规则	155
20.3 运行	155
20.4 本机 (RTX 5060 Ti) 一次运行的关键结果	155
20.4.1 为什么 Decode 只达到理论上限的一成左右?	156
20.5 动手练习 (按难度递增)	156
20.6 学完之后	157
20.7 实验核心源码精选与解析	157
20.7.1 源码实现: <code>qwen_from_scratch.py</code>	157
第 21 章 显存占用严谨核算与 GPU 算力 Roofline 实测	166
21.1 Lab 08: 显存开销与 Roofline 性能模型	166
21.2 [推导] 1. 显存开销组成详解	166
21.2.1 (1) 模型权重 (Model Weights)	166
21.2.2 (2) KV Cache 显存	166
21.2.3 (3) 激活值 (Activations)	167
21.2.4 (4) CUDA 上下文与运行时开销	167
21.3 [高速] 2. Roofline 模型与为什么 Decode 是访存受限?	167
21.3.1 计算密度 (Arithmetic Intensity)	167
21.3.2 单 Token 解码耗时下界理论估算	168
21.4 [工具] 显存与性能计算工具使用	168
21.5 [实验] 在你自己的显卡上实测 Roofline (需要 PyTorch)	169
21.6 实验核心源码精选与解析	169
21.6.1 源码实现: <code>memory_calculator.py</code>	170
21.6.2 源码实现: <code>measure_gpu_roofline.py</code>	174
第 22 章 FlashAttention 算法仿真与 CUDA Softmax 算子	180
22.1 Lab 09: FlashAttention 与 FlashDecoding	180
22.2 运行	180
22.3 本机 (RTX 5060 Ti) 实验 5 结果	180
22.4 [附件] 附录: 真机 CUDA 版 (强烈建议做)	181

22.5 动手练习	181
22.6 实验核心源码精选与解析	181
22.6.1 源码实现: <code>flash_attention_numpy.py</code>	182
第 23 章 投机解码仿真实现与无偏性实证校验	189
23.1 Lab 10: 投机采样 (Speculative Decoding) 原理与算法仿真	189
23.2 为什么需要投机采样?	189
23.3 核心算法流程	189
23.3.1 严格的数学无偏性保证 (Unbiased Sampling)	189
23.4 [执行] 运行仿真实验	190
23.5 实验核心源码精选与解析	191
23.5.1 源码实现: <code>toy_speculative_decoding.py</code>	191
23.5.2 源码实现: <code>verify_unbiasedness.py</code>	195
第 24 章 工业量化机制 (RTN / SmoothQuant / AWQ / GPTQ) 实战	199
24.1 Lab 11: 模型量化技术 (INT8 / INT4 / FP8, AWQ, GPTQ)	199
24.2 [目标] 什么是量化? 为什么对推理如此重要?	199
24.3 [几何] 核心量化数学公式	199
24.3.1 对称量化 (Symmetric Quantization)	199
24.3.2 非对称量化 (Asymmetric / Affine Quantization)	200
24.4 [模块] 量化粒度 (Granularity)	200
24.5 [思考] 工业级算法速览	200
24.6 [执行] 运行量化实验	201
24.7 [实验] 进阶: 从零实现 SmoothQuant / AWQ / GPTQ	201
24.8 实验核心源码精选与解析	202
24.8.1 源码实现: <code>quant_basics.py</code>	202
24.8.2 源码实现: <code>advanced_quant_numpy.py</code>	206
第 25 章 Continuous Batching (持续批处理) 动态调度仿真	212
25.1 Lab 12a: 显存调度与连续批处理 (Continuous Batching)	212
25.2 [施工] 为什么静态批处理 (Static Batching) 在 LLM 中失效?	212
25.2.1 静态 Batching 的两大致命缺陷:	212
25.3 [加速] 连续批处理 (Continuous Batching / In-flight Batching)	212
25.4 [文档] 内存革命: PagedAttention 原理	213
25.5 [执行] 运行仿真实验	214
25.6 [链接] 继续深入	214
25.7 实验核心源码精选与解析	214
25.7.1 源码实现: <code>continuous_batching_sim.py</code>	214

第 26 章 PagedAttention (分页显存分配机制) 仿真	220
26.1 Lab 12b: PagedAttention、写时复制、前缀缓存与 Chunked Prefill	220
26.2 运行	220
26.3 代码结构	220
26.4 动手练习	221
26.5 实验核心源码精选与解析	221
26.5.1 源码实现: <code>paged_kv_cache_sim.py</code>	221
第 27 章 多卡张量并行 (Tensor Parallelism) 切分与 All-Reduce 仿真	229
27.1 Lab 13: 多卡并行推理仿真	229
27.2 运行	229
27.3 动手练习	229
27.4 实验核心源码精选与解析	230
27.4.1 源码实现: <code>tensor_parallel_sim.py</code>	230
第 28 章 生产级推理引擎 (vLLM / SGLang / TRT-LLM) 压测实践	236
28.1 毕业设计: 工业级推理引擎实践 (vLLM / llama.cpp)	236
28.2 原则: 先预测, 再测量, 再解释	236
28.3 [测试] 任务 1: 用 vLLM 部署, 并测出吞吐-延迟曲线	236
28.4 [测试] 任务 2: 前缀缓存	237
28.5 [测试] 任务 3: 长 Prefill 对 Decode 的干扰 (Chunked Prefill)	238
28.6 [测试] 任务 4: llama.cpp + GGUF 量化	238
28.7 [测试] 任务 5: 投机解码	238
28.8 [阅读] 任务 6: 源码阅读路线	238
28.9 [OK] 毕业标准	239
28.10 实验核心源码精选与解析	239
28.10.1 源码实现: <code>bench_openai_server.py</code>	240

第一部分

核心原理与数学推导篇 (Theory & Derivations)

第 1 章 AI 深度学习全景族谱与零基础入门指南

核心导读与背景

前言：深度学习领域充斥着各种各样的“Net”（如 CNN、RNN、ResNet、Transformer、Diffusion），初学者极易迷失在浩瀚的术语海洋中。本篇为你梳理整张「AI 科技树」，讲清楚各种网络各司其职的关系，并给出最少弯路、最落地的零基础入门路线。

1.1 [概览] 一、深度学习终极演化科技树（全景族谱）

整个现代 AI 的发展，本质上只由两条主线驱动：**视觉线（看图像）**和 **语言时序线（读文本）**。它们在经历各自数十年的探索后，最终被 **Transformer** 完成大一统！

1.2 [指南] 二、各种“Net”究竟都是干什么的？

当你看到不同论文或文章里提到不同的 Net 时，可以对照下表快速定位：

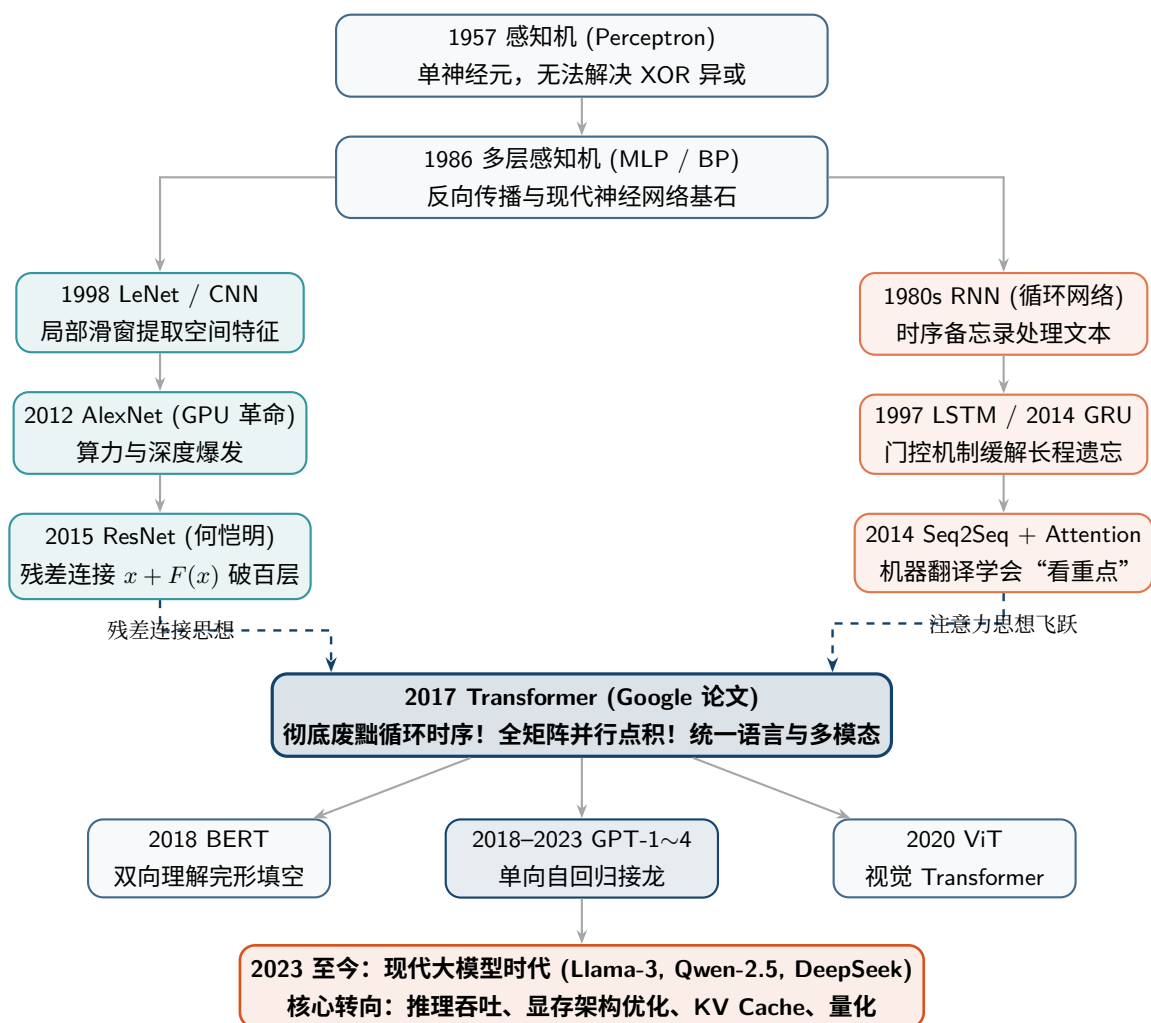


图 1.1: AI 深度学习演化科技树全景族谱

模型代号	全称	诞生初衷与解决的核心痛点	状态与历史地位
MLP	Multi-Layer Perceptron (多层感知机 / 全连接网络)	最基础的万能函数拟合器，全连接矩阵乘法。但输入长度固定，无法直接处理大图像和长文章。	所有现代网络的基本积木（如 Transformer 里的 FFN）
CNN	Convolutional Neural Network (卷积神经网络)	专为图像而生 。一张 1000x1000 的图片直接用全连接会导致参数爆炸，CNN 用一个小滑窗（卷积核）扫过图片提取边缘和纹理。	统治计算机视觉领域(2012-2020)，经典代表：LeNet, AlexNet, VGG
ResNet	Residual Network (残差网络)	网络堆得太深（超过 20 层）会导致梯度消失退化。何恺明发明 残差短路连接 ($x + F(x)$), 让网络轻松堆到 152 层甚至 1000 层！	现代所有大模型的标配机制 （Transformer 每层都有残差连接）
RNN	Recurrent Neural Network (循环神经网络)	专为文本时序而生 。像穿糖葫芦一样逐字传递隐藏状态备忘录，解决文本长度不一的问题。	因串行无法利用 GPU 并行，且长程记忆衰减，已被淘汰
LSTM	Long Short-Term Memory (长短期记忆网络)	在 RNN 基础上引入遗忘门、输入门、输出门，缓解了 RNN 读到后面忘了前面的死穴。	在 2017 年前是机器翻译、语音识别的绝对霸主
Transformer	基于 Self-Attention 的全新网络架构	彻底废弃循环连接 ，任意两词距离 $O(1)$ ，纯矩阵运算极致利用 GPU。不仅统领自然语言，还通过 ViT 杀入计算机视觉，通过 Sora/Diffusion 杀入视频生成。	当今全人类所有大模型的唯一通用底座

1.3 [目标] 三、新手零基础想真正入门，该看什么？（绝不踩坑三部曲）

网上的 AI 教程铺天盖地，90% 都是死记硬背公式或调包调库，初学者往往陷入“懂了但又没完全懂”的痛苦中。

想要扎实、高效、不浪费时间地入门，**推荐且仅推荐以下三部封神级的黄金资源：**

1.3.1 [第一阶段] 阶段 1：建立直观的数学与几何世界观（耗时约 2~3 天）

- **【绝对首选】** 3Blue1Brown 经典动画教程《深度学习的本质》(Bilibili / YouTube 免费看)
- 搜索关键词：3Blue1Brown 深度学习
- 为什么神：全网最好的几何可视化，用生动的动态图形讲清楚：
- 什么是神经网络与激活函数？
- 什么是梯度下降（走下山坡）？
- 反向传播微积分链式法则的几何动画。
- 注意力机制（Attention）到底在几何空间中拉伸了什么。
- **建议：**把这 4 集视频当作看电影，看上两遍，你脑海里的数学直觉就彻底打通了！

1.3.2 [第二阶段] 阶段 2：动手写代码与经典教科书（耗时约 1~2 周）

- **【教材推荐】** 李沐（前亚马逊首席科学家）团队《动手学深度学习》(Dive into Deep Learning / d2l.ai)
- 中文官网完全免费在线阅读，配备完整可运行代码。
- 从基础张量、线性回归、MLP 讲到现代注意力机制，文字极度平实。
- **【硬核进阶】** Andrej Karpathy（OpenAI 联合创始人、前特斯拉 AI 总监）的视频系列《Neural Networks: Zero to Hero》
- 他在油管上从一个只有几行的微型框架 micrograd 开始，用纯 Python 逐行写出一个微型 GPT（nanoGPT）。
- 跟着他敲一遍，大模型的黑盒彻底透明。

1.3.3 [第三阶段] 阶段 3：大模型推理与系统实战（面向生产与高性能）

- **【避坑指南：不要一上来就学“从零预训练大模型”】**
- 训练一个 70B 模型需要几千张英伟达 H100 显卡和几千万美元预算，对绝大多数工程师而言，学如何从头训练一个大模型完全脱离现实。
- **【正确的工程路径：深耕大模型推理（LLM Inference）】**
- 理解模型的前向传播逻辑与微观张量流(参见本项目的 lab04_transformer_microscope);
- 理解为什么推理的核心瓶颈在于显存容量与内存带宽（Memory Bandwidth）（参见 lab08_memory_and_roofline);

- 深入业界工业标准框架 (vLLM、SGLang、llama.cpp)，学习它们的核心技术：Page-dAttention、连续批处理 (Continuous Batching)、投机采样、量化；
- 这也是当前 AI Infra、大模型私有化部署、高性能边缘计算最紧缺、含金量最高的技术方向！

核心导读与背景

[指南] **本仓库的完整路线**：上面三部曲是外部资源的推荐；本仓库自己的 14 篇理论（第 0~13 篇）+ 配套实验，以及它们与外部资源如何穿插学习，见 **LEARNING_PATH.md**。族谱里没有展开的细节——注意力在 2014 年的真正起源、从原版 Transformer 到 Llama/DeepSeek 的每一处改动——在 **第 5 篇**。

下一篇：第 1 篇：计算机如何读懂语言？从人工规则到大模型演进史 | 总路线：LEARNING_PATH.md

第 2 章 计算机如何读懂语言？从人工规则到大模型演进史

核心导读与背景

阅读前须知：本篇不需要任何高等数学或编程背景。我们将从人类对计算机的最初探索讲起，带你一步步理解大模型（LLM）究竟是在什么背景下被发明出来的，以及它究竟解决了前人什么无法逾越的难题。

2.1 [代码] 一、计算机的天然缺陷：它只懂数字，不懂人类语言

首先请思考一个最本质的事实：计算机的硬件核心（CPU 和 GPU），本质上只是无数由晶体管组成的微型电路，它唯一擅长的事情，就是极速计算浮点数的「加法」和「乘法」。

计算机根本不知道什么是“猫”，什么是“开心”，什么是“我爱你”。对于计算机而言，屏幕上的汉字或英文字符，只是一串无意义的二进制编码（比如 UTF-8 编码中，“大”这个字的十六进制代码是 0xE5 0xA4 0xA7）。

人类要想让计算机能对话、能写诗、能编程，就必须完成一个根本性的跨越：如何把人类丰富、带有歧义、充满上下文语境的自然语言，转化为数字？并让计算机通过纯粹的数学计算，“理解”语言并生成语言？

为了解决这个问题，人类科学家经历了整整四代探索。

2.2 [文献] 二、第一代：基于人工规则与专家系统（1950s - 1980s）

在计算机科学诞生初期，科学家们认为人类语言是有严格语法规则的。

2.2.1 核心做法：

语言学家人工编写成千上万条逻辑规则：

- 句子 = 主语 + 谓语 + 宾语
- 如果主语是“我”，谓语可以是“吃”，宾语可以是“苹果”。
- 遇到英语单词 "bank"，如果前文有 "river"，就翻译为“河岸”；如果有 "money"，就翻译为“银行”。

2.2.2 为什么彻底失败？

1. **语言规则无穷无尽**：自然语言充斥着成语、倒装句、隐喻、俚语和错别字，规则写得越多，规则之间就越互相冲突。
2. **缺乏泛化能力**：人类只要说一句规则库里没有的表达（例如：“今天我被老板炒了鱿鱼”），系统就会瞬间崩溃。

事实证明：靠人类肉眼穷举语言规则是死路一条。

2.3 [图表] 三、第二代：统计语言模型与 N-gram (1990s - 2000s)

既然人工写规则走不通，统计学家们提出了全新思路：不要人工写规则，让计算机去统计真实人类书写的大量文章！

2.3.1 什么是“语言模型 (Language Model)”？

数学家对语言模型的定义极其质朴：

核心导读与背景

语言模型，就是一个计算“一句话在现实世界中出现的概率”的数学函数。

例如，对于两句话：

- S_1 = “今天天气真好”
- S_2 = “好真天气今天”

一个好的语言模型应该计算出： $P(S_1) = 0.05$ （很常见），而 $P(S_2) = 0.0000001$ （人类几乎不这么说）。

根据概率论的**链式法则 (Chain Rule)**，一句话出现的联合概率，等于逐个词出现的条件概率相乘：

$$P(w_1, w_2, w_3, \dots, w_n) = P(w_1) \times P(w_2 | w_1) \times P(w_3 | w_1, w_2) \cdots \times P(w_n | w_1, \dots, w_{n-1})$$

这个公式看似完美，但计算机在实际计算时崩溃了：当句子长达 20 个字时，条件概率 $P(w_{20} | w_1, \dots, w_{19})$ 要求计算机去统计：在历史上所有文章中，前面 19 个字完全一模一样的情况下，第 20 个字是 w_{20} 的频率。现实中，哪怕整个互联网的所有文本加起来，前面 19 个字完全一样的概率也几乎是 0！这就是著名的「数据稀疏与维度灾难」。

2.3.2 N-gram 的妥协 (马尔可夫假设)

为了能算出来，统计学家做了一个妥协：假定第 t 个词出现的概率，只与它前面的 $N - 1$ 个词有关，跟更早以前的词完全无关！

- **2-gram (Bi-gram)**: 每个词只看前 1 个词: $P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-1})$
- **3-gram (Tri-gram)**: 每个词只看前 2 个词: $P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-2}, w_{t-1})$

2.3.3 致命死穴:

1. **短视, 无法理解长文本**: 如果你写一篇文章, 开头说“小明是个法国人.....”, 在经过 500 字后问“他最擅长的母语是 _____”, N-gram 只看前两个词, 根本不可能答对。
2. **没有任何概念理解**: 在统计模型中, “苹果”和“梨”是两个完全不相关的独立符号。即使模型见过“我吃了一个苹果”, 它也无法推断出“我吃了一个梨”是否合理。

2.4 [决策] 四、第三代: 分布式词向量与循环神经网络 (2010s)

2013 年, Google 提出了里程碑式的 Word2Vec, 彻底颠覆了符号表示法。

2.4.1 核心飞跃: 词嵌入向量 (Word Embedding)

人类想通了: 不要用离散的孤立代号表示词, 而是把每个词表示为高维空间里的一个坐标点 (一个向量, 比如 512 个数字)!

- 著名的语言学假设: “一个词的含义, 由它经常与哪些词同时出现所决定 (*Distributional Hypothesis*)。”
- 如果把词映射到多维空间中, 在大量文本训练后, 你会惊奇地发现数学几何与人类常识奇迹般地重合了:

$$\vec{v}_{\text{国王}} - \vec{v}_{\text{男人}} + \vec{v}_{\text{女人}} \approx \vec{v}_{\text{王后}}$$

“苹果”和“梨”在空间中的距离会非常近, 而“苹果”和“量子力学”距离极远。

2.4.2 补齐底座: 什么是神经网络? 它是怎么“学会”知识的?

核心直觉与思考

[思考] 很多教材直接从词向量跳到 RNN, 导致零基础同学一头雾水。到底什么是“神经元”? 什么是“隐藏层”? 它到底是怎么从数据中学习的? 我们用最接地气的生活例子彻底讲明白。

(1) 单个人工神经元: 一个带旋钮的打分器

生物学中, 人脑神经元通过树突接收多个电信号, 在细胞核汇总, 超过一定阈值后通过轴突激发传递。数学家把这个过程抽象成了一个极其简单的数学公式:

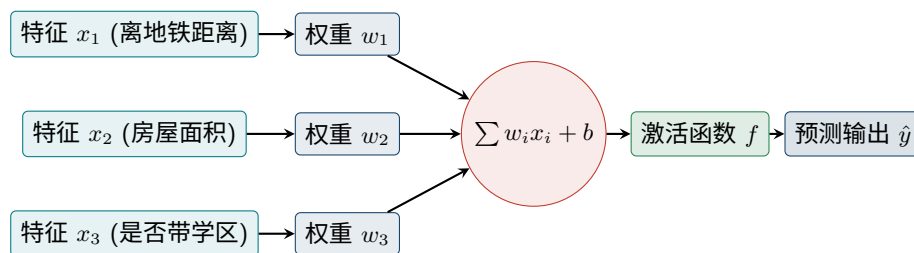


图 2.1: 单神经元（感知机）加权求和与非线性激活计算图

1. **输入** (x_1, x_2, x_3): 客观事实数据。
2. **权重** (Weights, w_1, w_2, w_3): 每个因素在决策中占多大分量（重要程度）。
 - 如果你极度看重学区，学区权重 w_3 就会调得非常大；
 - 如果你不在乎价格，价格权重 w_2 就会接近 0。
1. **偏置** (Bias, b): 基础门槛/自身倾向性（比如你天生买房意愿就很强，即使条件一般也想买， b 就是正数）。
2. **加权求和**:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b = \sum_{i=1}^n w_ix_i + b$$

(2) 为什么必须有「激活函数 (Activation Function)」?

如果神经元只做 $z = \sum w_ix_i + b$ 这种加减乘除，它在数学上叫**线性变换**。线性变换的致命缺陷是：无论你把多少个神经元串联在一起，100 层的线性变换在代数上依然完全等价于仅仅只有 1 层！

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = W_{\text{new}}x + b_{\text{new}}$$

这意味着，没有非线性激活，多层神经网络就丧失了深度的意义，根本无法拟合现实世界中千变万化的曲折规律！

激活函数（如 ReLU、Sigmoid、tanh）就像是一个“非线性开关”：

- 它将输入进来的线性加权，通过曲线进行扭曲与折叠；
- 有了激活函数，多层神经网络就能像折纸一样，在多维空间中折叠出极其复杂的曲面，**理论上可以逼近宇宙中任意复杂的数学函数**（通用近似定理 Universal Approximation Theorem）！

(3) 什么是多层神经网络与“隐藏层 (Hidden Layer)”?

把成百上千个神经元分层排列，前一层的输出作为后一层的输入，就组成了**多层感知机 (MLP / 神经网络)**：

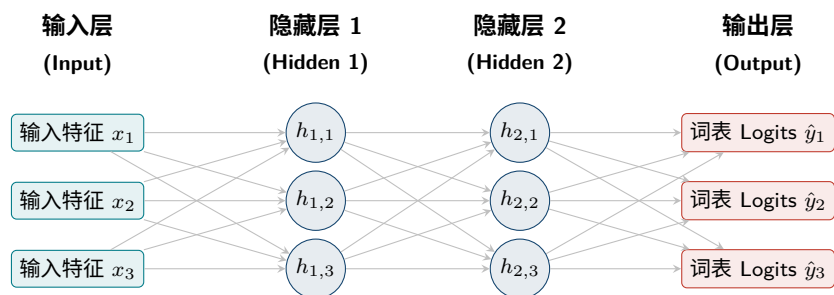


图 2.2: 多层感知机 (MLP) 特征变换与全连接信息流动图

• 为什么叫“隐藏层”？

因为输入是人类给的文字，输出是最终的预测结果，这两头都是看得见摸得着的。而中间这些层，是计算机内部自主形成的高维抽象概念表征 (Representation)，人类很难直接用一两个词去定义中间神经元到底在想什么，因此被称为“隐藏状态 (Hidden State)”。

(4) 神经网络是怎么“学习”的？(损失函数与反向传播)

网络搭好了，但一开始里面的权重旋钮 (W 和 b) 全是随机初始化的乱码。它怎么学会人类知识的？这个学习过程完全模拟了人类的“犯错-反省-改正”机制：

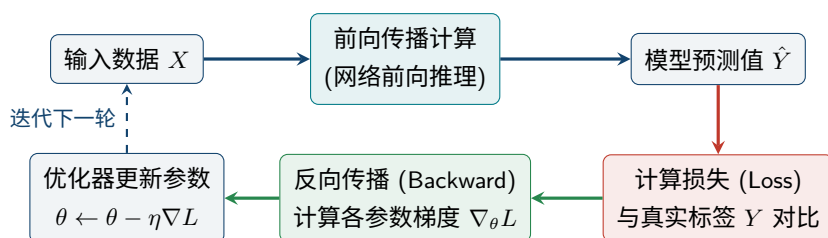


图 2.3: 深度学习模型训练闭环：前向计算、损失衡量与反向梯度传播

1. **第一步 (前向瞎猜)**：给模型输入“太阳从东边”，模型一顿乱算，吐出下一个字是“西瓜”。
2. **第二步 (算差距 Loss)**：损失函数发现标准答案是“升”，预测概率和事实差了十万八千里，给出一个很大的惩罚分数 (Loss)。
3. **第三步 (反向算责任)**：利用高等数学的链式求导法则 (Chain Rule)，从后向前计算输出误差对每一个权重参数的偏导数 (称为**梯度 Gradient**)。梯度告诉我们：“如果把第 354 号旋钮稍微拧大一点，总误差就会减小！”
4. **第四步 (更新旋钮)**：按照梯度的相反方向，微调所有的权重参数 (这个过程叫**梯度下降**)。
5. **重复数十亿次**：当把互联网上几万亿个词喂给模型，历经成百上千卡 GPU 几个月的日夜调整，所有的旋钮都达到了极致精密的和谐状态——大模型就此诞生！

2.4.3 为什么传统神经网络处理文本会崩溃？循环神经网络（RNN）的诞生

搞懂了基础神经网络，我们立刻面临一个致命挑战：**传统的神经网络，输入大小必须是固定的！**

- 比如设计一个能输入 3 个词的网络，它就只能吃 3 个词；
- 但人类说话有长有短，有的问句只有 2 个字（“在吗？”），有的文章长达 1 万字！
- 怎么让同一个神经网络，既能读短句，又能读长文呢？

科学家为此设计了**循环神经网络（Recurrent Neural Network, RNN）**。RNN 的核心构想是：**网络结构固定不变，但配一个随时更新的“备忘录小本子（隐藏状态 h_t ）”！**

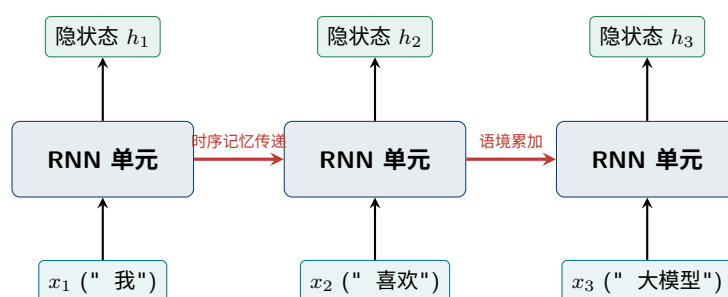


图 2.4: 循环神经网络（RNN）沿时序展开的信息流动机制

每个时间步 t ，同一个 RNN 网络只接收两样东西：

1. 上一步传过来的备忘录 h_{t-1}
2. 当前进来的这 1 个新词 x_t

通过矩阵乘法与激活函数更新备忘录：

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

其中：

- W_x 是针对新词的权重矩阵；
- W_h 是针对历史备忘录的权重矩阵；
- $\tanh$ 是激活函数，把所有数字压缩在 $[-1, 1]$ 之间，防止数值无休止放大。

2.4.4 为什么强大的 RNN 和 LSTM 最终被历史无情淘汰？

尽管 LSTM 和 GRU 引入了复杂的门控机制，但在大模型时代，RNN 撞上了一堵无法逾越的物理高墙：

1. 信息串行的「传话游戏」缺陷（长程遗忘与梯度消失）：

- 文本必须一个词接一个词地顺序传递。如果一句话长达 1000 个词，第 1 个词的信息必须经历 1000 次矩阵乘法才能到达末尾。
- 在数学上，多次连乘会导致浮点数的梯度消失（数值缩小到 0）或梯度爆炸（数值溢出为 NaN）。哪怕最先进的 LSTM，在超过 100~200 个词后，对前文的记忆也基本荡然无存。

1. 算力杀手：完全无法在 GPU 上并行训练！

- 现代 GPU 内部有上万个计算核心，GPU 最强大的地方在于同一毫秒同时做数千个矩阵相乘。
- 但是 RNN 要求：必须先算完第 1 个词，拿到 h_1 之后，才能算第 2 个词；算完第 2 个词才能算第 3 个词！
- 意味着 99% 的 GPU 核心只能闲置排队等待！训练速度慢如蜗牛，根本无法利用海量互联网数据进行缩放（Scale）。

2.5 [加速] 五、第四代：Transformer 的横空出世（2017 至今）

2017 年，Google 发表了震动整个 AI 领域的著名论文：《Attention Is All You Need》（注意力就是你所需要的一切）。

Transformer 做出了一个在当时看似离经叛道、甚至疯狂的决定：

核心导读与背景

彻底抛弃所有的循环（RNN）和时间步串行连接！一个都不留！

2.5.1 Transformer 为什么能一统天下？

1. 任意两词之间距离永远是 $O(1)$ ：

- 不管一篇文章有 10 个词还是 10 万个词，Transformer 的自注意力机制（Self-Attention）让文章中任意一个词与另一个词之间，直接通过矩阵点积相连。
- 第 1 个词和第 10000 个词可以直接打分产生关联，不再有任何“传话游戏”的信息衰减！

1. 天然契合 GPU 的暴力并行计算：

- 一整篇文章输入进来，所有词的 Q、K、V 投影矩阵在一瞬间同时相乘完成！
- GPU 的成千上万个核心全负荷拉满运转，训练吞吐量相比 RNN 提升了几个数量级。

1. 奠定了如今大模型（LLM）的基石：

- 正是因为能在海量 GPU 集群上并行计算，才使得人类能够把整个维基百科、GitHub 源码、电子书全部喂给模型，催生了 GPT-3、GPT-4、Llama、Qwen 等如今震撼世界的超级大模型。

2.6 [目标] 总结与下一章预告

回顾人类探寻机器理解语言的 70 年历程：

1. **人工规则**：因语言的无限歧义性与复杂性而崩溃；
2. **统计 N-gram**：因缺乏语义理解与上下文数据稀疏而停滞；
3. **循环神经网络 RNN**：引入连续词向量，但因信息串行传递无法长久记忆，且无法利用现代 GPU 并行算力而遭淘汰；
4. **Transformer**：以完全并行的自注意力机制一统天下，成为当今所有大模型的唯一通用底座。

在下一篇中，我们将从**最基础的初等数学起步**，推导并理解：

- 什么是向量与空间变换？
- 两个词向量为什么做「点积」就能代表语义相似度？
- 深度学习中最常用的归一化函数 Softmax 是怎么来的？（它的导数为什么那么优雅，在第 3 篇推导）

避坑指南与核心警示

[重点] 本篇为了控制篇幅，跳过了两段重要的历史：**2003 年第一个神经语言模型**，以及**2014 年 Seq2Seq 的信息瓶颈如何催生了注意力机制**。另外本篇 §4.2 讲神经网络如何“学习”只是比喻，严格的数学在第 3 篇。它们分别在：- **第 3 篇：导数、链式法则、反向传播与交叉熵** - **第 5 篇：从原版 Transformer 到 Llama / DeepSeek §1**

上一篇：第 0 篇（导读）：AI 深度学习全景族谱与零基础入门指南 | 下一篇：第 2 篇：零基础必备数学直觉：向量、空间变换、点积与 Softmax 深度推导 | 总路线：LEARNING_PATH.md

第 3 章 零基础必备数学直觉：向量、空间变换、点积与 Softmax 深度推导

核心直觉与思考

阅读前须知：许多同学被 AI 算法劝退，往往不是因为逻辑理解不了，而是被一堆看似高深莫测的数学符号吓退。本篇将把大模型涉及的所有核心数学工具（向量、矩阵乘法、点积、Softmax）用最通俗的几何直觉和一步步代数推导彻底讲透。

3.1 [重点] 一、从标量到张量：数据的物理载体

在代码和论文中，你经常会看到这四个名词：



图 3.1: 数据物理维度的层级跃迁（标量 → 向量 → 矩阵 → 张量）

1. 标量 (Scalar)：单个数值。

- 例如：今天的气温 25.5°C 。

1. 向量 (Vector)：一列按顺序排列的数字，通常表示高维空间里的一个坐标点或一个带方向的箭头。

- 例如：描述一个人： $[180\text{cm}, 75\text{kg}, 25\text{岁}]$ ，这是一个 3 维向量。
- 在大模型中，一个词（如“大”）会被表示为一个包含 4096 个数字的向量，这 4096 个数字就是该词在人类语义空间中的精细坐标。

1. 矩阵 (Matrix)：由行和列构成的二维表格。

- 形状写作 $[M, N]$ ，表示有 M 行、 N 列。
- 矩阵在大模型里通常有两种角色：
- **批量数据**：比如一句话包含 10 个词，每个词是 4096 维，整句话就是 $[10, 4096]$ 的矩阵；
- **空间变换器（权重参数）**：用来把向量从一个空间变换到另一个空间。

1. 张量 (Tensor): 超过二维的多维数组。

- 例如在推理系统中, 常见形状 [Batch, Seq_Len, Hidden_Dim]:
- Batch = 4 (同时处理 4 个用户的提问)
- Seq_Len = 512 (每句话有 512 个词)
- Hidden_Dim = 4096 (每个词的维度)
- 这是一个三维张量, 形状为 [4, 512, 4096]。

3.2 [几何] 二、点积 (Dot Product): 大模型如何计算两个词的关联度?

在自注意力机制中, 最重要的核心计算就是点积。为什么计算两个向量的点积, 就能代表两个词的语义相关性?

3.2.1 代数定义 (怎么算)

假设有两个 n 维向量 $\vec{a} = [a_1, a_2, \dots, a_n]$ 和 $\vec{b} = [b_1, b_2, \dots, b_n]$, 它们的点积等于对应位置元素相乘再全部相加:

$$\vec{a} \cdot \vec{b} = a_1 b_1 + a_2 b_2 + \dots + a_n b_n = \sum_{i=1}^n a_i b_i$$

核心导读与背景

例题: $\vec{a} = [1, 2]$, $\vec{b} = [3, 4]$ 点积 $\vec{a} \cdot \vec{b} = 1 \times 3 + 2 \times 4 = 3 + 8 = 11$ 。

3.2.2 几何意义与余弦定理 (为什么代表相似度?)

在欧氏空间几何学中, 点积还有一个极其重要的等价公式:

$$\vec{a} \cdot \vec{b} = |\vec{a}| |\vec{b}| \cos(\theta)$$

其中:

- $|\vec{a}|$ 是向量 $\vec{a}$ 的长度 (模长);
- $|\vec{b}|$ 是向量 $\vec{b}$ 的长度;
- θ 是两个向量之间的夹角 (从 0° 到 180°);
- $\cos(\theta)$ 是夹角的余弦值。

让我们来看看 $\cos(\theta)$ 随夹角的变化规律:

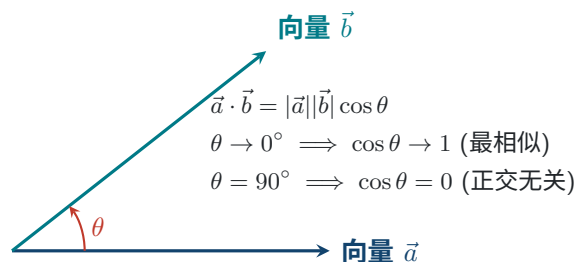


图 3.2: 高维向量内积与夹角余弦相似度直观几何解释

1. 方向完全相同 ($\theta = 0^\circ$):

$\cos(0^\circ) = 1$, 点积取得最大正值! (表示两个词的含义高度一致、强烈共鸣)

1. 方向垂直/正交 ($\theta = 90^\circ$):

$\cos(90^\circ) = 0$, 点积精确等于 0! (表示两个词风马牛不相及, 毫不相关)

1. 方向完全相反 ($\theta = 180^\circ$):

$\cos(180^\circ) = -1$, 点积为负数! (表示语义冲突或对立)

核心直觉与思考

[思考] **核心结论**: 在大模型中, Query (查询) 向量与 Key (键) 向量做点积:

$$\text{Score} = Q \cdot K^T$$

避坑指南与核心警示

由 $\vec{q} \cdot \vec{k} = |\vec{q}||\vec{k}| \cos \theta$, 它同时由**方向 (夹角)**和**长度 (模长)**决定: 模长相近时, 夹角越小点积越大、分配的注意力越高; 但模长不同时, 一个夹角更大却更”长”的 Key 也可能拿到更高的分数。所以点积是”**未归一化的相似度**”, 并不等于余弦相似度。这一点很重要: 第 4 篇会证明, 正是因为模长会随维度 d_k 增长, 点积的方差才会变大, 所以必须除以 $\sqrt{d_k}$ 。

3.3 三、矩阵乘法：空间的旋转、拉伸与投影

很多人高中或大学学过矩阵乘法, 只记得”左边行乘以右边列”, 却不知道它在物理世界中意味着什么。

3.3.1 物理直觉：矩阵是一个”空间投影仪”

当一个向量 $\vec{x}$ (比如代表一个词的 4096 维特征) 乘以一个权重矩阵 W 时:

$$\vec{y} = \vec{x}W$$

这绝不仅是一堆数字的加减乘除，而是在把向量 $\vec{x}$ 投射到一个全新的语义子空间中！
在 Transformer 中：

- 相同的输入词向量 X ，乘以矩阵 W_q 就被投射到了“提问空间 (Query Space)”；
- 乘以矩阵 W_k 就被投射到了“特征被检索空间 (Key Space)”；
- 乘以矩阵 W_v 就被投射到了“实际语义内容空间 (Value Space)”。

这三个矩阵参数，是大模型在预训练过程中学习到的最重要的核心知识载体。

3.4 [推导] 四、Softmax 函数：从任意实数打分到合法的概率分布

当注意力机制计算完点积后，得到了一组得分（称为 Logits 或 Attention Scores），例如：

$$z = [2.5, -1.0, 5.0]$$

这些数字有正有负，求和不等于 1，计算机无法把它们当作概率或者注意力加权重使用。
我们需要一个函数，将任意实数数组 z 转换为合法的概率分布 p ：

- 条件 1：所有元素必须严格大于 0 ($p_i > 0$)；
- 条件 2：所有元素的和必须严格等于 1 ($\sum_i p_i = 1$)。

3.4.1 为什么不用最简单的“除以总和”？

你可能会想，为什么不直接取绝对值再除以总和呢？例如：

$$p_i = \frac{|z_i|}{\sum |z_j|}$$

为什么不行？

1. **失去符号判别**：-5.0 和 +5.0 会得到相同的权重，但负数代表不相关甚至反对，绝对不能和正数等同。
2. **缺乏数学可导性**：绝对值函数在 $x = 0$ 处不可导，神经网络无法进行梯度反向传播。
3. **无法拉开差距**：简单的线性归一化不能突出高分项的绝对优势。

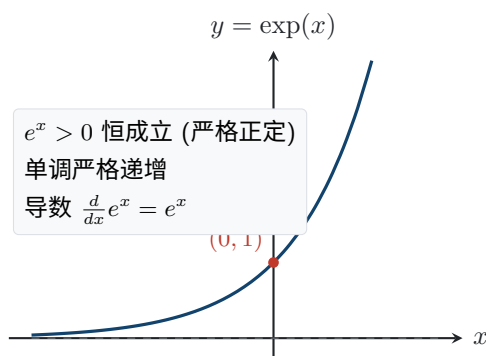


图 3.3: $\exp(x) = e^x$ 的严格正定性与单调性

3.4.2 指数函数 $\exp(x) = e^x$ 的登场与推导

数学家引入了自然常数的指数函数 e^x ($e \approx 2.71828$)，它拥有极其完美的数学特质：

利用这一性质，对每一项先取指数确保严格为正，再除以所有指数项的总和完成归一化，这就诞生了著名的 **Softmax** 公式：

$$p_i = \text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

实例计算：

设 $z = [1.0, 2.0, 3.0]$ ：

1. 分别取指数：

- $e^{1.0} \approx 2.718$
- $e^{2.0} \approx 7.389$
- $e^{3.0} \approx 20.086$

1. 求和： $\sum = 2.718 + 7.389 + 20.086 = 30.193$

2. 归一化概率：

- $p_1 = 2.718/30.193 \approx 9.0\%$
- $p_2 = 7.389/30.193 \approx 24.5\%$
- $p_3 = 20.086/30.193 \approx 66.5\%$
- 总和： $9.0\% + 24.5\% + 66.5\% = 100\%$ ！

核心直觉与思考

[思考] **Softmax 的直观名字含义**：传统的 $\max(1, 2, 3)$ 是“硬最大值 (Hard-max)”，只会输出 3（概率为 $[0, 0, 1]$ ）。而 **Softmax** 是“平滑软化版的最大值”，最大项拿到绝大部分概率（66.5%），但同时也给次高项保留了一定机会，且整体函数处处光滑可导！

3.5 [防护] 五、工程避坑细节：Softmax 的平移不变性与防止数值溢出

在工业级大模型推理引擎中，如果直接按照数学公式 $\text{math.exp}(z)$ 编写代码，你的服务会在运行几秒钟后彻底崩溃（报错 `OverflowError: math range error` 或输出 `NaN`）。

3.5.1 为什么会溢出？

浮点数（如 `float32`）能表示的最大数字约为 3.4×10^{38} 。当大模型某一步算出的未缩放点积较大时（例如 $z_i = 100$ ），计算 $e^{100} \approx 2.68 \times 10^{43}$ ，直接突破硬件浮点数表示上限，导致指数爆炸溢出（Overflow）！

3.5.2 数学拯救：平移不变性证明

科学家们发现了一个美妙的代数恒等式：给输入向量的所有元素同时减去一个常数 C ，**Softmax** 的计算结果完全不发生任何改变！

【数学严格证明】：设新的输入为 $z'_i = z_i - C$ ，代入 Softmax 公式：

$$\text{Softmax}(z - C)_i = \frac{e^{z_i - C}}{\sum_{j=1}^n e^{z_j - C}} = \frac{e^{z_i} \cdot e^{-C}}{\sum_{j=1}^n (e^{z_j} \cdot e^{-C})}$$

提取公因式 e^{-C} ：

$$= \frac{e^{z_i} \cdot e^{-C}}{e^{-C} \cdot \sum_{j=1}^n e^{z_j}} = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} = \text{Softmax}(z)_i \quad \blacksquare$$

3.5.3 工业级标准实现 (Safe Softmax)：

令常数 $C = \max(z)$ （整个向量中的最大值）。此时每一个被计算的数值 $z_i - C \leq 0$ ，其指数 $e^{z_i - C} \in (0, 1]$ ，最大项为 $e^0 = 1$ 。无论输入的数字多么大，指数永远被锁死在 0 到 1 之间，彻底根除浮点数溢出 Bug！

3.6 [目标] 总结与下一章预告

在本篇中，我们建立了大模型所需的全部底层数学直觉：

1. 向量：语义空间的几何坐标点；

2. **点积**：由**夹角与模长共同**决定的未归一化相似度 ($\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos \theta$ ，**不等于余弦相似度**)；
3. **矩阵乘法**：将特征投射到不同的语义子空间；
4. **Softmax**：利用指数函数的单调性与正值性，将打分安全映射为加权概率。

掌握了这些基石，下一篇（第 3 篇）我们先回答”神经网络到底怎么学习”：导数、反向传播、交叉熵，以及 **Softmax 的导数为什么那么优雅**。再下一篇（第 4 篇）直面 Transformer 最核心的王牌公式——它”为什么必须除以 $\sqrt{d_k}$ ”的论证，正要用到第 3 篇推导的 Softmax 导数：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

我们将推导：**为什么必须除以 $\sqrt{d_k}$ ？如果没有这根小小的平方根号，大模型为什么会彻底崩溃？**

上一篇：第 1 篇：计算机如何读懂语言？从人工规则到大模型演进史 | 下一篇：第 3 篇：神经网络如何学习：导数、链式法则、反向传播与交叉熵 | 总路线：LEARNING_PATH.md

第 4 章 神经网络如何学习：导数、链式法则、反向传播与交叉熵

核心数学定理推导

阅读前须知：第 1 篇用”拧旋钮”的比喻讲了神经网络怎么学习，但没有真正推导；下一篇（第 4 篇）讲 Transformer 时，还要用到 Softmax 的导数 $p_i(\delta_{ij} - p_j)$ 。本篇把”学习”这件事从数学上彻底讲透：导数到底是什么、为什么梯度指向下山方向、反向传播为什么比别的求导方法快几十亿倍、损失函数为什么偏偏是交叉熵、以及 Sigmoid / RNN 的梯度为什么会消失。

前置知识：会求 x^2 、 e^x 、 $\ln x$ 的导数即可（高中水平）。如果对”导数是什么”本身没有直觉，先看 [3Blue1Brown《微积分的本质》](#) 第 1~4 集，再回来。

配套实验：`lab03_autograd_from_scratch/micrograd_from_scratch.py`——用约 350 行纯 Python 实现一个自动求导引擎，并用数值差分验证本篇的每一个公式。

4.1 [重点] 一、导数：函数在一点附近的”最佳线性近似”

很多人记得导数是”切线斜率”，但在深度学习里更有用的理解是：

$$f(x+h) \approx f(x) + f'(x) \cdot h \quad (h \text{ 很小})$$

导数告诉你：输入挪动一点点 h ，输出大约挪动 $f'(x) \cdot h$ 。它是一个”灵敏度”。

多元函数 $f(x_1, \dots, x_n)$ 同理，每个输入各有一个灵敏度，称为**偏导数** $\frac{\partial f}{\partial x_i}$ ，把它们排成向量就是**梯度**：

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right], \quad f(\vec{x} + \vec{h}) \approx f(\vec{x}) + \nabla f \cdot \vec{h}$$

4.1.1 为什么梯度的反方向就是”下山最快”的方向？（严格证明）

设我们沿单位方向 $\vec{u}$ ($|\vec{u}| = 1$) 走一小步 ϵ ，函数变化量约为：

$$f(\vec{x} + \epsilon \vec{u}) - f(\vec{x}) \approx \epsilon (\nabla f \cdot \vec{u})$$

由第 2 篇的点积几何公式 $\nabla f \cdot \vec{u} = |\nabla f| |\vec{u}| \cos \theta = |\nabla f| \cos \theta$ ：

- $\theta = 0$ （沿梯度方向）时增长最快，增量 $= \epsilon |\nabla f|$ ；

- $\theta = 180^\circ$ (沿**负梯度**方向) 时下降最快, 减量 $= \epsilon |\nabla f|$ 。■

于是得到**梯度下降** (Gradient Descent) 更新规则, η 叫学习率:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

核心直觉与思考

[思考] 为什么学习率不能太大? 因为上面整个推导依赖” $f(x+h) \approx f(x) + f'(x)h$ ” 这个一阶近似, 只在 h 很小时成立。步子一大, 近似失效, 损失反而可能上升甚至发散。

4.2 [链接] 二、链式法则：误差如何”顺藤摸瓜”找到每个旋钮

4.2.1 单变量链式法则

若 $y = g(x)$, $L = f(y)$, 则:

$$\frac{dL}{dx} = \frac{dL}{dy} \cdot \frac{dy}{dx}$$

用”灵敏度”理解: x 动一点 $\rightarrow y$ 动 $\frac{dy}{dx}$ 倍 $\rightarrow L$ 再动 $\frac{dL}{dy}$ 倍, 灵敏度沿路径相乘。

4.2.2 多变量链式法则：多条路径要相加

若 x 同时影响了 y_1 和 y_2 , 而 L 依赖 y_1, y_2 :

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y_1} \frac{\partial y_1}{\partial x} + \frac{\partial L}{\partial y_2} \frac{\partial y_2}{\partial x}$$

规则只有两条：同一条路径上相乘，不同路径之间相加。这就是反向传播的全部数学。

4.2.3 手算一个完整例子

模型 $\hat{y} = wx + b$, 损失 $L = (\hat{y} - y)^2$ 。取 $x = 2$, $w = 3$, $b = 1$, $y = 10$:

步骤	前向 (从左到右算值)	反向 (从右到左算梯度)
$u = wx$	$u = 6$	$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial u} \cdot x = -6 \times 2 = -12$
$\hat{y} = u + b$	$\hat{y} = 7$	$\frac{\partial L}{\partial u} = \frac{\partial L}{\partial \hat{y}} \cdot 1 = -6, \quad \frac{\partial L}{\partial b} = -6 \times 1 = -6$
$e = \hat{y} - y$	$e = -3$	$\frac{\partial L}{\partial \hat{y}} = \frac{\partial L}{\partial e} \cdot 1 = -6$
$L = e^2$	$L = 9$	$\frac{\partial L}{\partial e} = 2e = -6$

梯度 $\frac{\partial L}{\partial w} = -12 < 0$: 说明把 w 调大, 损失会下降。取 $\eta = 0.01$, $w \leftarrow 3 - 0.01 \times (-12) = 3.12$ 。预测值从 7 向 10 靠近了。你可以在配套实验里用代码逐行复现这张表。

4.3 [加速] 三、反向传播：为什么它能让训练百亿参数成为可能？

”对每个参数求偏导”有很多种做法，为什么全世界都用反向传播（Backpropagation，本质是反向模式自动微分 Reverse-mode AD）？

4.3.1 方案 A：数值差分（最笨）

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta + h e_i) - L(\theta - h e_i)}{2h}$$

每个参数要跑 2 次前向。一个 7B 模型就要跑 140 亿次前向才能得到一次梯度——完全不可行。（但它是检验梯度实现是否写对的金标准，配套实验里会用到。）

4.3.2 方案 B：前向模式（Forward-mode AD）

从输入往输出推”每个输入对当前节点的灵敏度”。代价与输入个数成正比——对参数有几十亿、输出只有 1 个标量损失的神经网络，依然要几十亿次。

4.3.3 方案 C：反向模式（Backpropagation）[OK]

从唯一的输出 L 出发，反向计算 $\frac{\partial L}{\partial(\text{每个中间节点})}$ 。每个节点的梯度只需要它”下游”节点的梯度（已算好，直接复用）乘以局部导数，这就是动态规划。一次反向传播就得到全部参数的梯度，代价只是前向的约 2 倍。

核心导读与背景

[文献] 历史：反向模式自动微分在 1970 年由 Linnainmaa 提出；1986 年 Rumelhart、Hinton、Williams 在《Nature》上把它用于多层神经网络并展示了隐藏层能学到有意义的表示，这才让 MLP 真正可训练（第 0 篇族谱里的”1986 BP 算法”）。

4.3.4 一个对推理工程师很重要的推论：训练 $\approx 6N$ ，推理 $\approx 2N$

- 一个参数量为 N 的模型，处理 1 个 token 的前向，每个参数参与一次乘法 + 一次加法 $\rightarrow$ 约 $2N$ FLOPs。
- 反向传播要对”激活”和”权重”各求一次梯度（见下一节的 GW^T 与 X^TG ），约 $4N$ FLOPs。
- 所以训练每个 token 约 $6N$ FLOPs，推理每个 token 约 $2N$ FLOPs（不计注意力的 $O(\text{上下文长度})$ 项）。第 8 篇会用这个数字来估算 Prefill 的耗时。

4.4 [推导] 四、矩阵形式的反向传播：只需记住一个公式

神经网络里 99% 的计算是矩阵乘法 $Y = XW$ ($X : [B, K]$, $W : [K, N]$, $Y : [B, N]$)。设上游已经传来了 $G = \frac{\partial L}{\partial Y}$ (形状同 Y , $[B, N]$)，求 $\frac{\partial L}{\partial W}$ 与 $\frac{\partial L}{\partial X}$ 。

逐元素推导： $Y_{ij} = \sum_k X_{ik} W_{kj}$ ，所以 W_{kj} 影响了第 j 列所有的 Y_{ij} ($i = 1..B$ ，多条路径相加)：

$$\frac{\partial L}{\partial W_{kj}} = \sum_i \frac{\partial L}{\partial Y_{ij}} \frac{\partial Y_{ij}}{\partial W_{kj}} = \sum_i G_{ij} X_{ik} = (X^T G)_{kj}$$

同理 X_{ik} 影响了第 i 行所有的 Y_{ij} ：

$$\frac{\partial L}{\partial X_{ik}} = \sum_j G_{ij} W_{kj} = (GW^T)_{ik}$$

$$\boxed{\frac{\partial L}{\partial W} = X^T G, \quad \frac{\partial L}{\partial X} = GW^T}$$

核心导读与背景

[工具] **形状对齐记忆法：** $\frac{\partial L}{\partial W}$ 必须和 W 同形状 $[K, N]$ ，手上只有 $X : [B, K]$ 和 $G : [B, N]$ ，唯一能拼出 $[K, N]$ 的写法就是 $X^T G$ 。

[思考] 一个会在推理里反复出现的事实： $\frac{\partial L}{\partial W} = X^T G$ 需要前向时的输入 X ——这就是训练时必须把每层激活值存下来的原因（训练显存大头之一）。推理不做反向，所以激活用完即扔，这也是第 6 篇对比表中”训练显存杀手是激活值，推理显存杀手是 KV Cache”的根源。

4.5 [目标] 五、损失函数为什么是交叉熵？——最大似然估计

4.5.1 语言模型的训练目标

回到第 1 篇的概率链式法则：一段文本的概率 $P(x_1, \dots, x_T) = \prod_t P(x_t | x_{<t})$ 。模型 p_θ 在每个位置输出一个词表上的分布。我们希望模型给真实出现过的文本尽可能高的概率，这就是最大似然估计 (MLE)：

$$\max_{\theta} \prod_{t=1}^T p_{\theta}(x_t | x_{<t})$$

4.5.2 为什么取对数、再取负号？

1. **连乘 → 连加：** 几千个小于 1 的概率相乘会下溢成 0 (float32 最小正数约 10^{-45})，取对数后变成求和，数值稳定；

2. $\log$ 单调递增，最大化 $\prod p$ 等价于最大化 $\sum \log p$;
3. 优化器习惯”最小化”，于是加负号：

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t})$$

这就是**负对数似然(NLL)**,也等于”真实分布(one-hot)与模型分布之间的**交叉熵**”: $H(y, p) = -\sum_i y_i \log p_i = -\log p_{\text{正确词}}$ 。

4.5.3 困惑度 (Perplexity): 一个有物理含义的指标

$$\text{PPL} = \exp(\mathcal{L})$$

- 如果模型对词表 V 个词**完全均匀地瞎猜**, $p = 1/V$, $\mathcal{L} = \ln V$, $\text{PPL} = V$ 。
- 所以 PPL 可以理解为”**模型平均在多少个词之间犹豫**”。 $\text{PPL} = 10$ 意味着模型的不确定性相当于每步从 10 个等可能的词里挑。

核心导读与背景

[工具] **调试黄金法则** (Lab 05 的 `train.py` 会真的断言它): 一个刚随机初始化的模型, 初始损失应该约等于 $\ln V$ 。字符级小模型 $V \approx 1500$ (**会随语料增删而变**, 因为语料就是本仓库的文档), 初始损失应 $\approx \ln 1500 \approx 7.3$ 。如果远大于这个值, 说明初始化让模型”自信地瞎猜”, 训练前就有 bug。

[链接] **与推理的联系**: 量化(第 11 篇)后要评估精度损失, 工业界最常用的指标就是量化前后 PPL 的变化。

4.6 [循环] 六、Softmax 与交叉熵的梯度: 本系列最重要的一组推导

4.6.1 Softmax 的雅可比矩阵

$p_i = \frac{e^{z_i}}{S}$, 其中 $S = \sum_k e^{z_k}$ 。注意 $\frac{\partial S}{\partial z_j} = e^{z_j}$ 。用商的求导法则 $(\frac{u}{v})' = \frac{u'v - uv'}{v^2}$:

情况 $i = j$:

$$\frac{\partial p_i}{\partial z_i} = \frac{e^{z_i} S - e^{z_i} e^{z_i}}{S^2} = \frac{e^{z_i}}{S} - \left(\frac{e^{z_i}}{S}\right)^2 = p_i - p_i^2 = p_i(1 - p_i)$$

情况 $i \neq j$ (分子 e^{z_i} 不含 z_j , 导数为 0):

$$\frac{\partial p_i}{\partial z_j} = \frac{0 \cdot S - e^{z_i} e^{z_j}}{S^2} = -p_i p_j$$

合并成一个式子 (δ_{ij} 在 $i = j$ 时为 1, 否则为 0):

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j)$$

第 4 篇解释”为什么必须除以 $\sqrt{d_k}$ ”时, 用的就是这个公式。当某个 $p_i \rightarrow 1$ 、其余 $p_j \rightarrow 0$ 时, 每一项 $p_i(\delta_{ij} - p_j)$ 都 $\rightarrow 0$: $i = j$ 时是 $1 \times (1 - 1)$, $i \neq j$ 时含因子 $p_j \rightarrow 0$ 或 $p_i \rightarrow 0$ 。这就严格说明了”Softmax 饱和 $\rightarrow$ 梯度消失”, 第 4 篇会据此推出”必须除以 $\sqrt{d_k}$ ”。

4.6.2 Softmax + 交叉熵: 一个美得不可思议的结果

设正确类别为 y , $\mathcal{L} = -\log p_y$ 。对 logit z_j 求导 (链式法则, 经过所有 p_i 的路径相加——但 $\mathcal{L}$ 只依赖 p_y , 所以只有一条路径):

$$\frac{\partial \mathcal{L}}{\partial z_j} = -\frac{1}{p_y} \cdot \frac{\partial p_y}{\partial z_j} = -\frac{1}{p_y} \cdot p_y(\delta_{yj} - p_j) = p_j - \delta_{yj}$$

$$\nabla_z \mathcal{L} = \vec{p} - \vec{y}_{\text{one-hot}}$$

梯度就是”模型预测的分布减去真实分布”。预测对了多少, 梯度就小多少; 而且它永远不会因为 Softmax 饱和而消失——只要预测错了 (p_y 小), 梯度就大。这正是分类与语言模型都用”Softmax + 交叉熵”组合的深层原因。

4.6.3 工程细节: 永远不要先算 Softmax 再取 log

$\log(\text{softmax}(z))_y = z_y - \log \sum_k e^{z_k}$ 。后一项叫 **LogSumExp**, 用第 2 篇的平移技巧稳定计算:

$$\text{LSE}(z) = m + \log \sum_k e^{z_k - m}, \quad m = \max_k z_k$$

先算 softmax 再取 log, 当某个 p 下溢成 0 时会得到 $\log 0 = -\infty$ 。PyTorch 的 `F.cross_entropy` 直接吃 logits、内部用 LSE, 就是这个原因。

4.7 [下降] 七、梯度消失与爆炸: 从 Sigmoid、RNN 到残差与 LSTM 的一条暗线

4.7.1 激活函数的导数决定了梯度能走多远

激活函数	公式	导数	导数上界
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma(x)(1 - \sigma(x))$	1/4 (在 $x = 0$)
Tanh	$\tanh(x)$	$1 - \tanh^2(x)$	1
ReLU	$\max(0, x)$	0 或 1	1
SiLU / Swish	$x\sigma(x)$	$\sigma(x)(1 + x(1 - \sigma(x)))$	约 1.1

用 Sigmoid 堆 10 层，即使权重完美，梯度也至少被乘上 $(1/4)^{10} \approx 10^{-6}$ 。这就是 2012 年以前深层网络训练不动、AlexNet 改用 ReLU 后一举成功的重要原因之一。现代大模型的 SwiGLU 里用的是 SiLU（第 5 篇讲它的来历）。

4.7.2 RNN 为什么会忘记（第 1 篇“传话游戏”的数学版）

$h_t = \tanh(W_h h_{t-1} + W_x x_t)$ ，对前一步求导：

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(\tanh'(\cdot)) W_h$$

从第 T 步传回第 1 步要连乘 $T - 1$ 个这样的矩阵：

$$\frac{\partial h_T}{\partial h_1} = \prod_{t=2}^T \text{diag}(\tanh'_t) W_h$$

粗略地说，若 W_h 的最大奇异值 < 1 ，连乘指数级趋于 0（梯度消失，远处的词学不到）；若 > 1 则可能指数级爆炸（Pascanu et al., 2013）。

4.7.3 LSTM 与残差连接：同一个思想——“加法通路”

- **LSTM (1997)**：细胞状态更新为 $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ ，于是 $\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t)$ 。遗忘门 f_t 可以学到接近 1，梯度就能几乎无衰减地沿 c 流过很多步。
- **ResNet / Transformer 残差 (2015/2017)**： $x_l = x_{l-1} + F(x_{l-1})$ ， $\frac{\partial x_l}{\partial x_{l-1}} = I + \frac{\partial F}{\partial x_{l-1}}$ （第 4 篇 §5 会在 Transformer 里再推导一次）。

两者都是把“乘法连乘的通路”换成“加法直通的通路”。理解了这一点，你就理解了为什么深度学习史上几次关键突破长得如此相似。

4.8 八、初始化与归一化：与 $\sqrt{d_k}$ 同一个方差推导

独立随机变量之和的方差等于各自方差之和，所以 d 个独立的单位方差项相加，方差变为 d （第 4 篇推导“除以 $\sqrt{d_k}$ ”时用的也是这一条）。把它用在一层线性变换 $y_j = \sum_{i=1}^n w_{ij} x_i$ 上（ w, x 独立、零均值）：

$$\text{Var}(y_j) = n \cdot \text{Var}(w) \cdot \text{Var}(x)$$

想让信号逐层既不放大也不衰减，就需要 $\text{Var}(w) = 1/n$ 。这就是 **Xavier 初始化 (2010)** 的核心；配合 ReLU（砍掉一半信号）时要 $\text{Var}(w) = 2/n$ ，即 **He 初始化 (2015)**。即使初始化得当，训练过程中分布也会漂移，于是有了 **BatchNorm (2015)** $\rightarrow$ **LayerNorm (2016)** $\rightarrow$ **RMSNorm (2019)**，把每层输入的尺度重新“拉回来”（演进原因见第 5 篇）。

核心数学定理推导

[反向] 同一个推导出现了三次： $\sqrt{d_k}$ 缩放、Xavier/He 初始化、归一化层，本质都是“控制方差，让数值待在激活函数和 Softmax 的敏感区间里”。

4.9 [演练] 九、优化器与“为什么训练比推理贵这么多显存”

- **SGD**: $\theta \leftarrow \theta - \eta g_o$
- **Momentum**: $m \leftarrow \beta m + g$, $\theta \leftarrow \theta - \eta m$ (在峡谷形的损失面上抑制来回震荡)。
- **Adam (2014)**: 同时维护一阶矩 m 和二阶矩 v :

$$m \leftarrow \beta_1 m + (1 - \beta_1)g, \quad v \leftarrow \beta_2 v + (1 - \beta_2)g^2, \quad \theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

($\hat{m}, \hat{v}$ 是偏差修正后的值。) 相当于给每个参数一个自适应的学习率。

显存账本 (混合精度 + Adam, 每个参数):

项目	训练	推理
BF16 权重	2 B	2 B
BF16 梯度	2 B	—
FP32 主权重副本	4 B	—
Adam m (FP32)	4 B	—
Adam v (FP32)	4 B	—
合计	16 B / 参数 (还没算激活)	2 B / 参数 (+ KV Cache)

所以 7B 模型训练至少 112 GB，推理只要 14 GB。这就是为什么你在一张 16GB 的 5060 Ti 上能做推理研究、不能从头训练 7B——也是本仓库路线把重心放在推理上的原因。

4.10 [OK] 本篇自测 (答不出来就回去重读对应小节)

1. 为什么学习率过大时损失会上升? (提示: 一阶近似)
2. 一个 $[B, K] \times [K, N]$ 的矩阵乘, 反向传播需要做哪两个矩阵乘? 各自的形状是什么? 总 FLOPs 是前向的几倍?
3. 用 $p_i(\delta_{ij} - p_j)$ 解释: 为什么 Softmax 各输入之间的差距越大, 梯度越小? (为什么不能说“输入数值越大”? 提示: 给所有输入同时加 100, 雅可比变不变?)
4. 字符级模型词表 $V = 65$, 随机初始化时损失大约应是多少? PPL 呢?

5. LSTM 的细胞状态和 Transformer 的残差连接，在”梯度流”意义上共享了哪个设计？
 6. 为什么 7B 模型推理只要约 14GB，训练却要 100GB 以上？
-

4.11 [资料] 外部资源（按推荐顺序）

1. [3Blue1Brown · 神经网络系列](#) 第 2~4 集（梯度下降、反向传播直觉、反向传播微积分）——先建立图像。
2. [Andrej Karpathy · The spelled-out intro to neural networks and backpropagation: building micrograd](#) (Zero to Hero 第 1 集，约 2.5 小时) ——**强烈建议跟着敲**，配套实验就是它的中文注释精简版。
3. [CS231n 反向传播讲义](#) ——计算图与”门”的局部梯度视角，含矩阵求导的形状技巧。
4. [The Matrix Calculus You Need For Deep Learning](#) ——想把矩阵求导彻底看懂时看。
5. 《动手学深度学习》4.7 前向传播、反向传播和计算图、4.8 数值稳定性和模型初始化。
6. 论文（选读）：Rumelhart et al. 1986 *Learning representations by back-propagating errors*；Pascanu et al. 2013 [On the difficulty of training RNNs](#)；Kingma & Ba 2014 [Adam](#)。

上一篇：第 2 篇：零基础必备数学直觉：向量、空间变换、点积与 Softmax 深度推导 | 下一篇：第 4 篇：Transformer 核心公式彻底推导：每一个符号从何而来 | 总路线：LEARNING_PATH.md

第 5 章 Transformer 核心公式彻底推导：每一个符号从何而来

核心数学定理推导

阅读前须知：在这篇教程中，我们将直面大模型皇冠上的明珠——Transformer 的数学核心。我们不会像很多教材那样直接甩给你公式，而是像侦探破案一样，带你一步一步把每一个符号、每一个分母、每一个负号的数学来源推导得清清楚楚。

5.1 [架构] 一、终极核心公式全貌

在所有大模型（GPT-4、Llama-3、Qwen-2.5）的心脏位置，跳动着的都是这行经典的注意力公式：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

请盯住这个公式 3 秒钟。它包含 5 个核心要素：

1. Q （Query，查询）
2. K （Key，键）
3. V （Value，值）
4. QK^T （点积打分矩阵）
5. 分母 $\sqrt{d_k}$ （缩放因子，最精妙的数学设计）

下面我们逐一拆解。

5.2 [观察] 二、为什么需要拆分成 Q 、 K 、 V ？（搜索引擎直觉）

很多人初学时都有一个巨大的困惑：明明输入的只有一句话的词嵌入 X ，为什么偏偏要用三个不同的矩阵把 X 变成 Q 、 K 、 V ？直接拿 X 算不行吗？

5.2.1 类比：YouTube / 百度搜索引擎的底层逻辑

想象你在 YouTube 搜索框里搜视频：

1. **你的搜索词**：" 如何学习大模型推理" ——这就是 **Query (查询)**。它代表当前词“迫切想寻找什么信息”。
2. **库里每个视频的标题/标签**：" 大模型实战教程", " 美食烹饪视频" ——这就是 **Key (索引键)**。它代表每个词用来被别人检索的“特征标签”。
3. **视频真正的画面和声音**：几十分钟的真实视频内容——这就是 **Value (真实值)**。它代表每个词真正承载的信息。

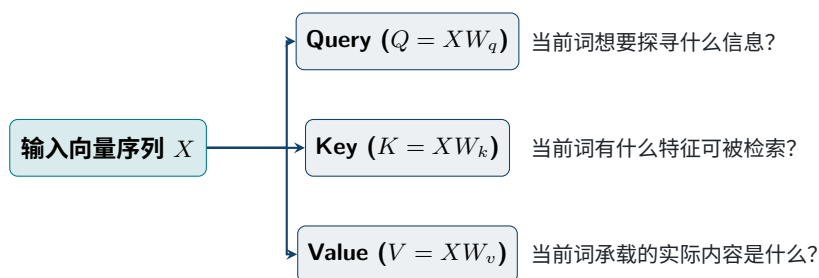


图 5.1: 自注意力机制三元组 Q, K, V 投影与语义物理意义

5.2.2 为什么不能直接用 $X \cdot X^T$?

如果直接拿原始特征自己和自己做点积：

- 相似度矩阵就变成了对称矩阵：词 A 对词 B 的注意力，必须强制等于词 B 对词 A 的注意力。
- 但在人类语言中，关联往往是极其不对称的！
- 例如句子：“小猫跳上了桌子，因为它饿了”。
- 当模型读到代词“它”时，“它”需要极其强烈地关注前文的“小猫”（消歧代词）；
- 但前面的“小猫”在被输入时，根本不需要关注后面很远的“它”。
- 引入独立的 W_q 和 W_k 投影，彻底打破了对称性束缚，让信息流动的指向性更加灵活、自由！

5.3 [爆炸] 三、【重磅数学推导】：为什么必须除以 $\sqrt{d_k}$?

这是面试中最高频、最考验底层功底的一道经典推导题：论文作者为什么不除以 d_k ，也不除以 10，偏偏要除以根号 $\sqrt{d_k}$ ？

我们来进行完整的方差演化数学推导。

5.3.1 假设输入是标准正态分布

设 Query 向量的一个分量为 q_i , Key 向量的一个分量为 k_i 。在经过层归一化 (LayerNorm) 后, 模型内部特征通常满足均值为 0、方差为 1 的独立标准正态分布:

- 期望 (均值): $E[q_i] = 0, E[k_i] = 0$
- 方差: $\text{Var}(q_i) = 1, \text{Var}(k_i) = 1$
- 单个维度的长度为 d_k (例如 $d_k = 64$ 或 128)。

5.3.2 计算点积 $Z = \vec{q} \cdot \vec{k}$ 的均值与方差

点积公式展开为:

$$Z = \sum_{i=1}^{d_k} q_i k_i = q_1 k_1 + q_2 k_2 + \cdots + q_{d_k} k_{d_k}$$

(1) 计算点积的数学期望 $E[Z]$:

根据独立变量期望的可加性与可乘性:

$$E[Z] = \sum_{i=1}^{d_k} E[q_i k_i] = \sum_{i=1}^{d_k} E[q_i] \cdot E[k_i] = \sum_{i=1}^{d_k} (0 \times 0) = 0$$

均值仍然是 0, 没有偏置。

(2) 计算单项乘积 $q_i k_i$ 的方差 $\text{Var}(q_i k_i)$:

根据方差定义公式 $\text{Var}(X) = E[X^2] - (E[X])^2$:

$$\text{Var}(q_i k_i) = E[(q_i k_i)^2] - (E[q_i k_i])^2 = E[q_i^2] E[k_i^2] - 0$$

因为 $E[q_i^2] = \text{Var}(q_i) + (E[q_i])^2 = 1 + 0 = 1$, 同理 $E[k_i^2] = 1$:

$$\text{Var}(q_i k_i) = 1 \times 1 = 1$$

(3) 计算 d_k 项累加后的总方差 $\text{Var}(Z)$:

由于这 d_k 个维度相互独立, 总方差等于各项方差相加:

$$\text{Var}(Z) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = \underbrace{1 + 1 + \cdots + 1}_{d_k \uparrow} = d_k$$

根据标准差是方差的平方根:

$$\sigma(Z) = \sqrt{\text{Var}(Z)} = \sqrt{d_k}$$

5.3.3 方差爆炸的灾难后果：Softmax 梯度消失

请看当隐藏维度变大时发生的事情：

- 现代大模型每个头的维度 d_k 通常是 128。
- 点积未缩放前的标准差就是 $\sqrt{128} \approx 11.31!$
- 按照正态分布 3σ 原则，大部分点积数值会在 $[-34, +34]$ 之间剧烈震荡。

如果把 $[+30, -10, 0]$ 送入 Softmax 会发生什么？

- $e^{30} \approx 1.06 \times 10^{13}$
- $e^0 = 1$
- 最大的一项占据了 99.9999999999% 的概率，其他所有项几乎全被压为 0！

更致命的是梯度反向传播公式：Softmax 的导数公式为：

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j)$$

当某个 $p_i \approx 1$ 而其他 $p_j \approx 0$ 时，这个导数项直接逼近于 0！梯度彻底消失，网络所有参数锁死，无法学习任何深层知识！

5.3.4 救世主：除以 $\sqrt{d_k}$

根据随机变量的方差性质：常数 c 提出来要平方，即 $\text{Var}(cX) = c^2 \text{Var}(X)$ 。如果我们将点积 Z 除以 $\sqrt{d_k}$ ：

$$\text{Var}\left(\frac{Z}{\sqrt{d_k}}\right) = \left(\frac{1}{\sqrt{d_k}}\right)^2 \cdot \text{Var}(Z) = \frac{1}{d_k} \cdot d_k = 1$$

完美的代数闭环！ 无论模型的维度 d_k 做得多大（64、128、256），除以 $\sqrt{d_k}$ 之后，点积结果的方差被死死压制在 1.0！使得输入进 Softmax 的数值始终落在 $[-3, +3]$ 的黄金活跃区间，Softmax 既能分出高下，又保持了充沛的梯度流动！

5.4 [禁用] 四、因果掩码 (Causal Mask)：为什么必须填充 $-\infty$ ？

在生成文本时，第 1 个词绝对不能看到第 2 个词的内容。我们通过一个掩码矩阵 M 与原始点积得分相加：

$$S_{\text{masked}} = \frac{QK^T}{\sqrt{d_k}} + M$$

其中上三角区域（代表未来时刻）全部被填入 $-\infty$ （代码中通常用 -10000.0 或 $-\mathbf{1e9}$ ）：

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

5.4.1 数学逻辑：

根据极限：

$$\lim_{x \rightarrow -\infty} e^x = 0$$

在计算第 1 行的 Softmax 时：

$$p_{1,2} = \frac{e^{S_{1,2} + (-\infty)}}{\sum e^{\dots}} = \frac{0}{\sum e^{\dots}} = 0$$

加法叠加负无穷，经指数变换后精确转化为乘法意义上的 0 权重，彻底斩断对未来信息的窥探！

5.5 [路径] 五、残差连接 (Residual Connection)：信息的高速公路

在 Transformer 的每一层后面，都有这样一行代码：

$$X_{\text{output}} = X_{\text{input}} + \text{SubLayer}(X_{\text{input}})$$

5.5.1 为什么不能直接 $X_{\text{output}} = \text{SubLayer}(X_{\text{input}})$ ？

假设一个大模型有 80 层（如 Llama-3-70B）：在反向传播计算梯度时，根据微积分链式法则：

$$\frac{\partial \mathcal{L}}{\partial X_0} = \frac{\partial \mathcal{L}}{\partial X_{80}} \times \prod_{l=1}^{80} \frac{\partial X_l}{\partial X_{l-1}}$$

80 个矩阵导数连乘，只要每一层的雅可比矩阵范数稍微小于 1（例如 0.9）， $0.9^{80} \approx 0.0002$ ，底层的参数梯度瞬间衰减归零，大模型根本训练不起来！

5.5.2 加入残差后的神奇变化：

因为 $X_l = X_{l-1} + F(X_{l-1})$ ，对 X_{l-1} 求导：

$$\frac{\partial X_l}{\partial X_{l-1}} = \mathbf{I} + \frac{\partial F(X_{l-1})}{\partial X_{l-1}}$$

展开到前向损失：

$$\frac{\partial \mathcal{L}}{\partial X_{l-1}} = \frac{\partial \mathcal{L}}{\partial X_l} \left(\mathbf{I} + \frac{\partial F}{\partial X_{l-1}} \right) = \frac{\partial \mathcal{L}}{\partial X_l} + \frac{\partial \mathcal{L}}{\partial X_l} \frac{\partial F}{\partial X_{l-1}}$$

看到了吗？式子中永远出现了一个干净纯粹的单位矩阵 **I**！它宛如一条畅通无阻的高速公路，无论模型叠到 100 层还是 1000 层，梯度都能沿着这条高速路无损地一路冲回第一层！

5.6 [循环] 六、RoPE（旋转位置编码）：复平面上的几何芭蕾

最后我们来看现代开源模型（LLaMA/Qwen/DeepSeek）一致采用的 **RoPE**（Rotary Position Embedding）。

5.6.1 核心目标

我们希望找到一个变换函数 $R(x, m)$ ，给向量 x 注入位置序号 m 。并且满足一个梦幻般的特性：两个向量注入位置 m 和 n 后的点积，只和它们的相对距离 $(m - n)$ 有关！

$$\langle R(q, m), R(k, n) \rangle = g(q, k, m - n)$$

5.6.2 二维复平面的几何推导

在复数几何中，将一个二维向量 (x_0, x_1) 视为复数 $z = x_0 + ix_1$ 。根据欧拉公式，旋转角度 θ 相当于乘以 $e^{i\theta}$ ：

$$z_{\text{rot}} = z \cdot e^{i\theta} = (x_0 + ix_1)(\cos \theta + i \sin \theta) = (x_0 \cos \theta - x_1 \sin \theta) + i(x_0 \sin \theta + x_1 \cos \theta)$$

转换为实数矩阵乘法形式：

$$R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

在位置 m 处，旋转角度为 $m\theta$ ：

$$q_m = R_{m\theta} q$$

在位置 n 处，旋转角度为 $n\theta$ ：

$$k_n = R_{n\theta} k$$

5.6.3 为什么内积只取决于相对距离？

让我们计算旋转后的内积：

$$\langle q_m, k_n \rangle = (R_{m\theta} q)^T (R_{n\theta} k) = q^T (R_{m\theta}^T R_{n\theta}) k$$

注意旋转矩阵的正交性质：逆时针旋转 $m\theta$ 的转置矩阵，就等于顺时针旋转，即 $R_{m\theta}^T = R_{-m\theta}$ ！根据旋转矩阵的相乘性质（先转 $-m\theta$ 再转 $n\theta$ ）：

$$R_{m\theta}^T R_{n\theta} = R_{-m\theta} R_{n\theta} = R_{(n-m)\theta}$$

代入内积式：

$$\langle q_m, k_n \rangle = q^T R_{(n-m)\theta} k = g(q, k, n - m) \quad \blacksquare$$

数学证明完毕！ 通过在二维平面上纯粹旋转一个角度 $m\theta$ ：

1. 向量本身的长度（模长）没有被拉伸或破坏；
2. 两个词点积时，两者的绝对位置 m 和 n 自动相消，**唯独留下了相对距离 $n - m$ ！**
3. 当高维空间拆成许多个 2D 坐标对，每个对使用不同的旋转频率 $\theta_i = 10000^{-2(i-1)/d}$ 时，会出现一种**有条件的”长程衰减”**——这一点经常被说错，要精确表述：
 - 把第 i 对分量的贡献写成 $A_i \cos(\Delta\theta_i) + B_i \sin(\Delta\theta_i)$ (Δ 是相对距离)。如果 q 与 k **是相关的**（语义上”匹配”， A_i 平均为正），那么点积的**期望**是一堆不同频率余弦的正权重叠加， Δ 变大后各频率相位错开，**期望值随距离衰减**：同样相似的两个词，离得越远得到的注意力加成越小。
 - 但如果 q 与 k **不相关**，点积的分布与距离**完全无关**（旋转不改变各向同性随机向量的分布），没有任何衰减。对某一对具体的向量，点积随距离剧烈震荡，远处的值完全可能比近处更大。
 - RoFormer 论文 §3.4.3 证明的是点积的一个**上界**随距离衰减，并不是点积本身一定变小。
 - 所以 RoPE 的衰减是”对匹配信号的软性偏置”，不是硬性的距离惩罚。长上下文扩展（PI / NTK / YaRN，第 5 篇 §5.4）要解决的是另一个问题：超过训练长度后，低频维度会转到训练时从未见过的角度（分布外）。

5.7 [目标] 总结与下一章预告

在本篇中，我们逐一完成了 4 大核心公式的严格数学推导：

1. **Q, K, V 拆分**：打破对称性，建立检索信息流；
2. **除以 $\sqrt{d_k}$** ：证明了独立变量点积方差随维度线性增长，通过平方根归一化彻底拯救了 Soft-max 的梯度消失；
3. **因果掩码 $-\infty$** ：利用 $e^{-\infty} = 0$ 斩断未来信息泄露；
4. **残差连接**：通过常数导数高速公路根治深度退化；
5. **RoPE 旋转矩阵**：通过二维复平面旋转证明了相对位置内积恒等性。

至此，Transformer 的核心公式你已经推导了一遍！

核心直觉与思考

[重点] 但注意：本篇讲的是 2017 年原版论文的公式。现代大模型（Llama、Qwen、DeepSeek）在此基础上做了许多改动——Pre-LN、RMSNorm、SwiGLU、GQA、MLA、MoE——每一处都有明确的历史动机，见 **第 5 篇**。想亲手验证本篇的 RoPE 性质（包括“衰减只发生在 q 、 k 相关时的期望值上，不相关时完全不衰减”这一容易被误解的点），运行 `lab04_transformer_microscope/02_rope_and_attention_microscope.py`。

在下一篇（第 5 篇）中，我们先看这些公式在 2017 年之后被改了哪些地方、为什么改；再下一篇（第 6 篇）跨越到实际运行：**到底什么是大模型推理（Inference）？为什么它在首字（Prefill）和后续生成（Decode）表现出完全不同的运行规律？KV Cache 在代数上是如何推演出来的？**

上一篇：第 3 篇：神经网络如何学习：导数、链式法则、反向传播与交叉熵 | 下一篇：第 5 篇：从原版 Transformer 到 Llama / DeepSeek：每一处改动的历史动机 | 总路线：LEARNING_PATH.md

第 6 章 从原版 Transformer 到 Llama / DeepSeek：每一处改动的历史动机

核心数学定理推导

阅读前须知：第 1 篇讲到“2017 年 Transformer 横空出世”就结束了，第 4 篇讲的是 Transformer 的公式，Lab 04 里的代码却已经是 **RMSNorm + RoPE + SwiGLU**——这些东西 2017 年的原版 Transformer 里一个都没有。它们是怎么来的？本篇补上两段缺失的历史：1. **Transformer 之前：**注意力机制到底是为了解决什么问题被发明的？（它并不是 2017 年凭空出现的）2. **Transformer 之后：**从原版到 Llama-3、Qwen2.5、DeepSeek-V3，每一处改动被什么问题逼出来，以及它对推理意味着什么。

前置：第 1、3、4 篇。特别是第 3 篇的“加法通路”和“方差控制”，本篇会反复用到。

6.1 [指南] 一、注意力的真正起源（2003 → 2017）

6.1.1 2003：第一个神经语言模型（Bengio NPLM）

第 1 篇说 N-gram 的死穴是“苹果”和“梨”毫无关系。2003 年 Bengio 等人提出**神经概率语言模型**：

$$P(w_t \mid w_{t-n+1}, \dots, w_{t-1}) = \text{softmax}(b + Wx + U \tanh(d + Hx)), \quad x = [e_{t-n+1}; \dots; e_{t-1}]$$

（ x 是前 $n-1$ 个词向量的拼接； $U \tanh(d + Hx)$ 是一个单隐藏层 MLP； Wx 是论文特意加入的**从输入直连输出的线性项**，可以设为 0。）

- 每个词先查表得到一个**可学习的向量** e （这就是**词嵌入的真正起点**，比 Word2Vec 早 10 年）；
- 相似的词学到相似的向量，于是“我吃了苹果”的统计经验能**泛化到**“我吃了梨”。
- 但它仍然只看**固定窗口**的 $n-1$ 个词，并且 softmax 覆盖整个词表，在当时的硬件上训练极慢。

之后 2010 年 Mikolov 用 RNN 做语言模型（RNNLM）去掉了固定窗口；2013 年 Word2Vec 把“只学词向量”这件事做到极致地便宜。

6.1.2 2014: Seq2Seq 与“信息瓶颈”

Sutskever 等人用两个 LSTM 做翻译：**编码器**读完整个源句子，把它压成一个固定长度的向量 c ；**解码器**从 c 出发逐词生成译文。问题马上暴露：无论源句子 5 个词还是 50 个词，都要塞进同一个几百维的向量里。Cho 等人（2014）的实验显示**句子越长，翻译质量下降越明显**。这个固定长度向量就是**信息瓶颈**。

6.1.3 2014: Bahdanau 注意力——“别压缩了，每一步回头看”

Bahdanau、Cho、Bengio 的想法是：解码器生成第 t 个词时，**不要只看那个压缩向量**，而是回头看编码器每个位置的隐藏状态 h_j ，**按需加权**：

$$e_{tj} = a(s_{t-1}, h_j), \quad \alpha_{tj} = \frac{\exp(e_{tj})}{\sum_k \exp(e_{tk})}, \quad c_t = \sum_j \alpha_{tj} h_j$$

请对照第 4 篇的注意力公式：

Bahdanau 2014	Transformer 2017
解码器当前状态 s_{t-1} (“我现在要翻译什么”)	Query
编码器各位置 h_j (用来算匹配分数)	Key
编码器各位置 h_j (被加权求和的内容)	Value
打分函数 $a(\cdot)$ 是一个小 MLP (加性注意力)	打分函数是点积 $q \cdot k / \sqrt{d_k}$

2015 年 Luong 等人把打分函数换成**点积**（乘性注意力）——更便宜，而且整批计算恰好是一次矩阵乘，GPU 最擅长。**注意**：2014 年 Bahdanau 已经有了「打分 + softmax 加权求和」这个两段结构，但那时 Key 和 Value 是**同一个** h_j （没有 W_k/W_v 两套投影），Query 就是解码器状态而非可学习的投影。**Q/K/V 三套独立投影的分工是 2017 年 Transformer 才定型的**——上面表格里的 Q/K/V 是后人回填的框架。

6.1.4 2017: Transformer 的真正创新是“只注意力”

既然注意力这么有用，而 RNN 又串行、又会遗忘（第 1、3 篇），Vaswani 等人问：**能不能把 RNN 整个扔掉，只用注意力？**

- **自注意力**：Query、Key、Value 都来自同一个序列自身（而不是解码器看编码器）；
- **位置编码**：扔掉 RNN 后，模型丧失了顺序感(lab04_transformer_microscope/02_rope_and_attn实验 1 证明了这一点)，必须显式注入位置；
- **多头**：多组独立的 W_q, W_k, W_v ，让不同的头关注不同类型的关系；
- **缩放点积**：除以 $\sqrt{d_k}$ （第 4 篇的推导）。

原版 Transformer 是**编码器-解码器**结构,用来做翻译,6 层编码器 + 6 层解码器, $d_{\text{model}} = 512$, FFN 用 ReLU, **Post-LN**, 正弦位置编码。

6.2 二、分词 (Tokenization): 为什么” 词表大小” 会影响推理速度

6.2.1 为什么不按” 词” 或” 字” 切?

- **按词切**: 词表爆炸 (英文有各种变形, 中文没有天然空格), 总会遇到没见过的词 (OOV, Out-Of-Vocabulary) 只能标记为 `<unk>`;
- **按字符 / 字节切**: 永远不会 OOV, 但序列变得很长——注意力是 $O(n^2)$, 而且 **Decode 步数 = token 数**, 每多一个 token 就要多读一遍全部权重 (第 6 篇)。

6.2.2 BPE: 从数据压缩借来的算法

2016 年 Sennrich 等人把 1994 年的数据压缩算法 **BPE (Byte Pair Encoding)** 用到翻译上:

1. 从单个字符 (或字节) 出发作为初始词表;
2. 统计语料里**最常相邻出现的一对符号**, 合并成一个新符号加入词表;
3. 重复第 2 步, 直到词表达到目标大小 (例如 32000、128000、151936)。

结果: 常见词是一个 token, 罕见词被拆成几个子词, **永远不会 OOV**。GPT-2 进一步在**字节**上做 BPE (Byte-level BPE), 任何 UTF-8 文本都能编码。

6.2.3 词表大小是一个推理工程上的权衡

词表更大	词表更小
[OK] 同样的文本 token 更少 → Decode 步数更少、KV Cache 更小	[OK] Embedding 与 LM Head 更小
[X] Embedding 表和 LM Head 更大: LM Head 每个 token 都要算 $d \times V$ 的矩阵乘	[X] 同样文本 token 更多

例子: Qwen2.5-0.5B 的 $V = 151936$ 、 $d = 896$, 光是词嵌入矩阵就有 **1.36 亿参数**, 占全模型约 **28%** (Lab 07b 会打印出来)。Llama-2 词表只有 32000, 对中文很不友好 (一个汉字常被切成多个 token); Llama-3 扩到 128256、Qwen2.5 小尺寸 (0.5B/1.5B/3B) 用 151936 (且词嵌入与 LM Head 共享权重), 大尺寸 (7B 及以上) 用 152064 (不共享), 中文同样的内容 token 数显著减少——**这直接等于中文推理变快、变便宜**。

核心导读与背景

[链接] 深入: Karpathy 的视频 *Let's build the GPT Tokenizer* 与代码 [minbpe](#)。

6.3 [架构] 三、三条路线：为什么最后是”只有解码器”的 GPT 赢了？

路线	代表	训练目标	能力特点
仅编码器	BERT (2018)	完形填空（遮住 15% 的词，双向看上下文去猜）	理解任务极强，但不能自回归生成
编码器-解码器	原版 Transformer、T5 (2019)	输入 → 输出	适合翻译、摘要等”有明确输入输出”的任务
仅解码器	GPT 系列、Llama、Qwen、DeepSeek	预测下一个词	什么文本都能当训练数据；生成与理解统一成”续写”

主流的解释（不是定理，而是历史经验）：

1. **目标最简单、数据最多**：任何一段文本都是”预测下一个词”的训练数据，无需标注；
2. **一切任务都能写成绩写**：GPT-3 (2020) 展示了上下文学习 (In-Context Learning) —— 在提示里给几个例子，不改参数就能做新任务；
3. **规模化最顺**：同样算力下，仅解码器结构简单、易于扩展；
4. **对推理友好**：只有一种因果注意力，KV Cache 结构统一、实现简单。

6.4 [增长] 四、规模定律：为什么”推理成本”成了整个行业的主问题

- **2019 GPT-2 (1.5B)**：发现模型够大后，不微调也能做一些任务 (zero-shot)。
- ****2020 Kaplan 等 *Scaling Laws*：损失随参数量 N 、数据量 D 、算力 C 呈幂律 ** 下降，例如 $L(N) \propto N^{-0.076}$ 。只要按规律加大规模，效果就可预测地变好。**
- **2020 GPT-3 (175B)**：上下文学习能力涌现。
- **2022 Chinchilla (Hoffmann 等)**：在固定训练算力下，最优做法是参数量与训练 token 数同比例增长，约 **每个参数 20 个 token**。此前的大模型普遍”参数太多、数据太少”。
- **2022 InstructGPT → ChatGPT**：用指令微调 + RLHF 让模型”听话”，大模型从研究品变成亿级用户产品。

关键转折：Chinchilla 只优化了**训练成本**。但一个模型训练一次，却要被调用成千上万亿次。2023 年的 Llama 明确把目标改成”**给定推理预算下效果最好**”——宁可用远超 Chinchilla 比例的

数据去过度训练一个小模型 (Llama-3-8B 用了 15T token ≈ 1870 token/参数, 是 Chinchilla 比例 20 的约 93 倍; 早期的 Llama-1-7B 只用了 1T ≈ 143 token/参数, 约 7 倍)。从这一刻起, 推理效率成为模型设计本身的一等公民: 下面第五节的 MQA/GQA/MLA、MoE、词表扩大, 全部带着”推理更便宜”的动机。

6.5 [工具] 五、从原版 Transformer 到现代 LLM: 逐个改动的”为什么”

6.5.1 Post-LN $\rightarrow$ Pre-LN: 把归一化挪出”高速公路”

- **原版 Post-LN:** $x_l = \text{LN}(x_{l-1} + F(x_{l-1}))$ 。LN 挡在残差路径上, 第 3 篇说的”I 直通路径”被每一层的 LN 打断, 梯度必须穿过 L 个 LN 的雅可比。深层时训练不稳定, 必须用学习率预热 (warmup) 小心伺候。
- **Pre-LN:** $x_l = x_{l-1} + F(\text{LN}(x_{l-1}))$ 。残差路径上什么都没有, 梯度可以原封不动流回第一层。Xiong 等 (2020) 从理论上分析了这一点; 实际上 GPT-2 (2019) 就已经改成了 Pre-LN。
- **代价:** Pre-LN 的残差流数值会随层数累积变大, 所以最后要再加一个 **Final Norm** (你在 lab04_transformer_microscope 里看到的 Step 8)。

6.5.2 LayerNorm $\rightarrow$ RMSNorm: 去掉”减均值”

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta \quad \longrightarrow \quad \text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_i x_i^2 + \epsilon}} \odot \gamma$$

Zhang & Sennrich (2019) 发现 LayerNorm 起作用的主要是”缩放” (re-scaling), ”减均值” (re-centering) 贡献很小。RMSNorm 少一次求均值的归约、没有 β , 效果相当。**对推理的意义:** 归一化是典型的访存受限小算子 (读一遍、写一遍, 算得很少), 少一次归约就少一次同步, 也更容易和前后算子融合 (fuse) 成一个 kernel (第 8 篇)。

6.5.3 ReLU $\rightarrow$ GELU $\rightarrow$ SwiGLU: 门控前馈网络

- 原版 FFN: $\text{ReLU}(xW_1)W_2$, 隐藏维度 $4d$ 。
- BERT / GPT-2 换成更平滑的 **GELU**。
- Shazeer (2020) 系统比较了多种门控线性单元 (GLU) 变体, SwiGLU 在同等计算量下效果最好:

$$\text{SwiGLU}(x) = (\text{SiLU}(xW_{\text{gate}}) \odot xW_{\text{up}})W_{\text{down}}$$

门控的直觉：一路计算”内容”，另一路计算”让多少内容通过”，两者相乘，比单纯的逐元素非线性表达力更强。有趣的是，论文作者自己写道：“我们不解释这些架构为何有效，把它们的成功和其他一切一样，归功于上天的仁慈。”——深度学习很多改动的”为什么”首先是**实验结果**，理论解释往往滞后。

- 一个你在配置文件里会看到的数字：SwiGLU 有 3 个矩阵而不是 2 个。为了保持参数量不变，隐藏维度从 $4d$ 缩成 $\frac{2}{3} \times 4d = \frac{8}{3}d$ 。Llama-7B 的 $d = 4096$ ， $\frac{8}{3} \times 4096 \approx 10923$ ，向上取整到 256 的倍数就是 **11008**——这就是 Llama 配置里那个”奇怪数字”的来历。

6.5.4 绝对位置编码 → RoPE：为了”外推”与”相对位置”

- 原版用正弦函数的绝对位置编码，加在输入上；GPT-2 改成**可学习的绝对位置向量**——但训练时最长 1024，就**根本没有**第 1025 个位置的向量，无法外推。
- T5（相对位置偏置）、ALiBi（按距离线性惩罚注意力分数）走向了”相对位置”。
- **RoPE**（苏剑林等，2021）：对 Q、K 按位置旋转，内积只依赖相对距离（第 4 篇已证明），**没有额外参数**。

RoPE 对推理的一个关键影响：K 在写入 **KV Cache** 之前就已经按它自己的位置旋转好了。之后无论新 token 走到哪个位置，缓存里的 K 都不用再动——只需要旋转新来的 q 和 k。Lab 07b 里你会亲手实现这一点。

长上下文扩展：RoPE 的频率 $\theta_i = b^{-2i/d}$ 中的底数 b 决定了”最慢的那一对分量转多快”。训练长度为 L 时，低频维度只转过了很小的角度；推理时位置一旦超过 L ，这些维度会转到**训练中从未见过的角度**（分布外），模型行为随之失控——这才是需要扩展方法的原因（RoPE 对”匹配信号”的期望衰减是另一回事，见第 4 篇 §6）。要把训练时的 4K 上下文扩到 32K、128K，常见做法是**调整频率**：位置插值（PI, 2023）、NTK-aware 缩放、YaRN（2023）。这也是为什么 Llama-3 把 b 从 10000 调到 500000，Qwen2.5 用 1000000。

6.5.5 MHA → MQA → GQA → MLA：一部”为 KV Cache 减肥”的历史

这一条演进完全由**推理驱动**，是本篇和第 6 篇之间最直接的桥梁。

方案	年份	KV 头数	每 token 每层 KV 存储	动机与代价
MHA 多头注意力	2017	$= n_h$	$2n_h d_h$	原版
MQA 多查询注意力	2019	1	$2d_h$	Shazeer 的论文标题就叫 <i>One Write-Head is All You Need</i> , 明确为了解决增量解码的显存带宽瓶颈。代价: 质量下降、训练不稳
GQA 分组查询注意力	2023	g (如 8)	$2gd_h$	Ainslie 等人的折中: 每 n_h/g 个 Q 头共享一组 KV。质量接近 MHA, KV 缩小 n_h/g 倍。Llama-2-70B、Llama-3、Qwen2.5 均采用
MLA 多头潜在注意力	2024	—	$d_c + d_h^R$ (如 512 + 64)	DeepSeek-V2: 不存 K、V 本身, 而是存一个低秩的压缩潜向量 c^{KV} , 用时再“解压”; 解压矩阵还能代数上吸收进 W_q 与 W_o 。RoPE 与低秩压缩不兼容, 所以额外存一个 64 维的带位置小 key

用数字感受一下 (BF16, 每 token, 全部层):

- Llama-3-8B (32 层, GQA-8, $d_h = 128$): $2 \times 32 \times 8 \times 128 \times 2 = 131,072$ 字节 = **128 KB**
- DeepSeek-V3 (671B 总参数, 61 层, MLA): $(512 + 64) \times 61 \times 2 \approx 70,272$ 字节 $\approx$ **69 KB**
- 如果 DeepSeek-V3 用 MHA (128 头 $\times$ 128 维): $2 \times 128 \times 128 \times 61 \times 2 \approx$ **3.8 MB**

一个 671B 的模型, 每 token 的 KV Cache 比 8B 的 Llama-3 还小。这就是”模型架构为推理服务”的最佳例证。

6.5.6 稠密 $\rightarrow$ MoE (混合专家): 参数多 $\neq$ 算得多

把每层的 FFN 换成 E 个”专家” FFN, 再加一个**路由器**: 每个 token 只挑得分最高的 k 个专家计算 (例如 $E = 256$ 选 $k = 8$)。

- Mixtral 8x7B: 总参数约 47B, 每 token 激活约 13B;
- DeepSeek-V3: 总参数 671B, 每 token 激活 37B。

对推理的意义 (很反直觉):

- **算力**只跟”激活参数”有关 $\rightarrow$ Prefill 很便宜;
- **显存**要装下**全部**参数 $\rightarrow$ 必须多卡;
- **Decode 带宽**: batch 很小时每步只读被选中专家的权重; 但 batch 一大, 几乎所有专家都会被某个 token 选中, 每一步又要读全部权重——于是出现了**专家并行 (EP)**等专门的部署方案 (第 13 篇)。

6.5.7 其它细节改动

- **去掉偏置项**: Llama 系列去掉了线性层的 bias (Qwen2 保留了 Q/K/V 投影的 bias, Lab 07b 实现时要注意)。
- **词嵌入与 LM Head 共享权重 (tied embeddings)**: 小模型里词表矩阵占比极大, 共享可省下几亿参数。Qwen2.5-0.5B 就是共享的。
- **滑动窗口注意力** (Mistral-7B, 2023): 每个 token 只看最近 W 个 token, KV Cache 大小封顶。

6.6 六、一张总表：同一个“Transformer”，七年里变了什么

组件	原版 Transformer (2017)	GPT-2/3 (2019-20)	Llama-1 (2023)	Llama-3 Qwen2.5 (2024)	DeepSeek-V3 (2024)
结构	编码器-解码器	仅解码器	仅解码器	仅解码器	仅解码器
归一化位置	Post-LN	Pre-LN	Pre-LN	Pre-LN	Pre-LN
归一化类型	LayerNorm	LayerNorm	RMSNorm	RMSNorm	RMSNorm
FFN 激活	ReLU	GELU	SwiGLU	SwiGLU	SwiGLU (MoE)
位置编码	正弦绝对	可学习绝对	RoPE	RoPE (大底数)	RoPE (解耦 + YaRN)
注意力	MHA	MHA	MHA	GQA	MLA
词表	~37K (BPE)	50257 (字节 BPE)	32000	128256 / 151936 (Qwen2.5 小尺寸) 或 152064 (7B+)	129280

核心导读与背景

[目标] 读完本篇你应该能做到：打开任意一个模型的 config.json（例如 Hugging Face 上 Qwen2.5 的配置），逐个字段说出它是什么、为什么有它、对推理的显存和速度有什么影响。Lab 07b 的第一个任务就是这个。

6.7 [OK] 本篇自测

1. Bahdanau 注意力要解决 Seq2Seq 的什么问题？它和 Transformer 的注意力，Q/K/V 分别对应什么？
2. 为什么 Transformer 必须有位置编码，而 RNN 不需要？
3. 为什么 Llama-7B 的 FFN 中间维度是 11008？
4. Pre-LN 比 Post-LN 好训练，用第 3 篇的”加法通路”解释。
5. GQA-8 的 70B 模型比 MHA 版本省多少 KV Cache？为什么说 MQA 的动机来自推理而不是训练？
6. MoE 模型”参数 671B、激活 37B”，在 batch=1 与 batch=256 时，Decode 每一步需要读的权重量分别大约是多少？
7. Llama-3 为什么要用远超 Chinchilla 最优比例的数据训练 8B 模型？

6.8 [资料] 外部资源

入门可视化

- Jay Alammar: [The Illustrated Transformer](#)、[Visualizing A Neural Machine Translation Model \(Seq2Seq + Attention\)](#)
- Lilian Weng: [Attention? Attention!](#) (从 Bahdanau 到 Transformer 的注意力家谱)
- 3Blue1Brown: [神经网络系列](#) 第 5~7 集 (GPT、注意力、MLP 如何存储知识)

跟着写代码

- Karpathy: *Let's build GPT: from scratch, in code, spelled out* 与 *Let's build the GPT Tokenizer* ([Zero to Hero](#) 第 7、8 集)
- Harvard NLP: [The Annotated Transformer](#) (原版论文逐段对照代码)
- Sebastian Raschka: [LLMs-from-scratch](#) (含 GPT → Llama → Qwen 的逐步改造笔记本)

中文深度

- 苏剑林 (RoPE 作者) 博客: [Transformer 升级之路: 2、博采众长的旋转式位置编码](#)

论文清单 (按本篇出现顺序)

- Bengio et al. 2003 [A Neural Probabilistic Language Model](#)
- Mikolov et al. 2013 [Word2Vec](#)
- Sutskever et al. 2014 [Seq2Seq](#); Cho et al. 2014 [On the Properties of NMT](#)
- Bahdanau et al. 2014 [Neural Machine Translation by Jointly Learning to Align and Translate](#); Luong et al. 2015 [Effective Approaches to Attention-based NMT](#)
- Vaswani et al. 2017 [Attention Is All You Need](#)
- Sennrich et al. 2016 [BPE for NMT](#)
- Devlin et al. 2018 [BERT](#); Raffel et al. 2019 [T5](#)
- Radford et al. 2018 [GPT-1](#); 2019 [GPT-2](#); Brown et al. 2020 [GPT-3](#)
- Kaplan et al. 2020 [Scaling Laws](#); Hoffmann et al. 2022 [Chinchilla](#); Ouyang et al. 2022 [InstructGPT](#)
- Xiong et al. 2020 [On Layer Normalization in the Transformer Architecture](#); Zhang & Sennrich 2019 [RMSNorm](#)

- Hendrycks & Gimpel 2016 [GELU](#); Shazeer 2020 [GLU Variants Improve Transformer](#)
- Su et al. 2021 [RoFormer](#); Press et al. 2021 [ALiBi](#); Chen et al. 2023 [Position Interpolation](#); Peng et al. 2023 [YaRN](#)
- Shazeer 2019 [MQA](#); Ainslie et al. 2023 [GQA](#); DeepSeek-AI 2024 [DeepSeek-V2 \(MLA\)](#)、[DeepSeek-V3](#)
- Fedus et al. 2021 [Switch Transformer](#); Jiang et al. 2024 [Mixtral](#)
- Touvron et al. 2023 [Llama](#)、[Llama 2](#); Llama Team 2024 [The Llama 3 Herd of Models](#)

上一篇：第 4 篇：Transformer 核心公式彻底推导：每一个符号从何而来 | 下一篇：第 6 篇：什么是推理？自回归生成全生命周期与 KV Cache 代数证明 | 总路线：LEARNING_PATH.md

第 7 章 什么是推理？自回归生成全生命周期与 KV Cache 代数证明

核心数学定理推导

阅读前须知：许多人以为大模型“训练”和“推理”是一回事，只是输入不同而已。本篇我们将彻底厘清大模型推理（Inference）的物理本质，推导为什么模型只能逐字生成，以及为什么一个小小的 KV Cache 会成为大模型显存爆炸的万恶之源与工程关键。

7.1 [对比] 一、训练（Training）vs 推理（Inference）的物理鸿沟

在讨论推理之前，我们必须先对比两者在底层运行模式上的根本差异：

对比维度	训练阶段 (Training)	推理阶段 (Inference)
已知信息	全部已知（整篇文章的所有字都摆在眼前）	完全未知（只知道用户的 Prompt，答复一无所知）
参数权重	频繁变动（通过反向传播不断更新梯度）	完全冻结（只读参数，绝不更改）
计算模式	全序列完全并行（所有 Token 一瞬间同时算完）	自回归逐字串行（像挤牙膏一样一个词一个词蹦）
显存杀手	激活值与优化器状态（Adam 动量、权重梯度）	模型权重与 KV Cache（历史上下文缓存）

7.1.1 训练时的“上帝视角”（Teacher Forcing）

训练时，我们给模型一段完整的文字：“人工智能彻底改变了世界”。因为整句话都在显卡里，模型可以在一次前向计算中，同时预测所有位置的下一个词：

- 位置 1 的特征去预测“智能”；
- 位置 2 的特征去预测“彻底”；
- 位置 3 的特征去预测“改变”

成千上万个词的预测误差在这一瞬间同时算出，然后梯度反向传播一次性更新全部参数。这就是为什么训练高度并行。

7.1.2 推理时的“摸黑前行”（自回归 Autoregressive Loop）

推理时，用户发来一个问题：“北京的特产是什么？”模型此时根本不知道后续几百个字是什么，它只能像盲人摸象一样，玩一个接龙游戏：

解码步数	模型前向输入上下文（Context Tokens）	预测输出下一个 Token
第 1 步	[" 北京", " 的", " 特产", " 是", " 什么", "? "]	” 烤”
第 2 步	[" 北京", " 的", " 特产", " 是", " 什么", "? ", " 烤"]	” 鸭”
第 3 步	[" 北京", " 的", " 特产", " 是", " 什么", "? ", " 烤", " 鸭"]	”。”（触发 EOS 停止）

上一步自己吐出来的词，拼入原句作为下一步的输入，周而复始，直到吐出结束标记 <eos> 为止。这就是大模型自回归推理的全部物理本质！

7.2 [加速] 二、推理的两大完全不同阶段：Prefill 与 Decode

在大模型推理系统（如 vLLM、TensorRT-LLM）中，你会每天听到两个术语：Prefill（预填充）与 Decode（解码）。它们两者的硬件瓶颈截然相反！

特征维度	阶段 1: Prefill 阶段 (首字计算)	阶段 2: Decode 阶段 (增量生成)
输入形态	一次性输入全部 Prompt (如 1000 字)	每次仅输入最新生成的 1 个 Token
计算模式	大规模矩阵乘法 (GEMM)	极小规模矩阵-向量乘法 (GEMV)
硬件瓶颈	算力受限 (Compute-Bound), 核心满载	访存受限 (Memory-Bound), 带宽跑满
核心考核指标	TTFT (首字时间, 用户等待延迟)	TPOT (每字时间, 流式打字流畅度)

7.3 [观察] 三、KV Cache 的代数展开与冗余消除证明

现在我们进入推理系统中最核心的概念：KV Cache。

7.3.1 朴素自回归计算中的重复劳动（无 KV Cache）

假设当前已经生成到了第 t 个词，序列为 $[x_1, x_2, \dots, x_{t-1}, x_t]$ 。我们要预测第 $t + 1$ 个词。根据 Transformer 注意力机制：

- 1. 计算整个序列的 Query、Key、Value：

$$Q_{1:t} = [x_1 W_q; x_2 W_q; \dots; x_t W_q]$$

$$K_{1:t} = [x_1 W_k; x_2 W_k; \dots; x_t W_k]$$

$$V_{1:t} = [x_1 W_v; x_2 W_v; \dots; x_t W_v]$$

1. 预测下一个词时，我们只关心最后一个位置的 Query: $q_t = x_t W_q$ 。
2. 最后一个位置的注意力加权计算为：

$$\text{Output}_t = \text{softmax} \left(\frac{q_t K_{1:t}^T}{\sqrt{d_k}} \right) V_{1:t}$$

仔细看这里的 $K_{1:t}$ 和 $V_{1:t}$ ：

$$K_{1:t} = [\underbrace{x_1 W_k, x_2 W_k, \dots, x_{t-1} W_k}_{\text{历史前 } t-1 \text{ 个词的 Key}}, x_t W_k]$$

$$V_{1:t} = [\underbrace{x_1 W_v, x_2 W_v, \dots, x_{t-1} W_v}_{\text{历史前 } t-1 \text{ 个词的 Value}}, x_t W_v]$$

发现了什么惊人的事实？

- 权重矩阵 W_k 和 W_v 是固定冻结的；
- 历史上的词 $x_1, x_2, \dots, x_{t-1}$ 是已经说出来的话，绝不会再改变！
- 这意味着：前面 $t-1$ 个词的 Key 和 Value 向量，在第 $t-1$ 步时就已经原模原样算过一次了！
- 如果不保存它们，每生成一个新词，都要把前面所有历史词重新投影一遍。当文本长达 4000 字时，第 4001 步要把前 4000 个词的投影全量重算一遍！总计算量呈二次方 $O(T^2)$ 暴增，生成速度越来越慢！

7.3.2 KV Cache 的工程解法：以显存换算力

既然历史的 Key 和 Value 永远不会变，那我们在第一次算出它们时，就直接保存在显存的缓存池中（KV Cache）！

第 t 步推理：KV Cache 增量运算过程

1. 单向量投影：仅对最新生成的单个 Token 向量 $x_t \in \mathbb{R}^{1 \times d_{\text{model}}}$ 计算投影：

$$q_t = x_t W_q, \quad k_t = x_t W_k, \quad v_t = x_t W_v$$

2. 更新增量缓存：将新产生的单步 k_t, v_t 追加到已有显存缓存中：

$$K_{\text{all}} = \begin{bmatrix} K_{\text{cache}} \\ k_t \end{bmatrix}, \quad V_{\text{all}} = \begin{bmatrix} V_{\text{cache}} \\ v_t \end{bmatrix}$$

3. 注意力打分：单向量 q_t 与整个长序列 K_{all} 做矩阵乘法，得到 $[1, t]$ 注意力分布，再加权 V_{all} 得出输出向量！

7.3.3 复杂度代数对比：

- 无 KV Cache：每次前向需要做整个矩阵的投影 $O(t \cdot d_{\text{model}}^2)$ ，总共生成 T 步需要累计 $O(T^2 \cdot d_{\text{model}}^2)$ 次计算。
- 有 KV Cache：每次前向只需做单个向量的投影 $O(1 \cdot d_{\text{model}}^2)$ ，总计算量降为 $O(T \cdot d_{\text{model}}^2)$ ！

投影与 FFN 这类”逐 token”矩阵乘法的总计算量少了一个因子 T （而不是常数倍）。

避坑指南与核心警示

[注意] 严谨补充（容易被忽略）：KV Cache 消除的是”对历史 token 重复做投影 / FFN”的冗余，并没有消除注意力本身的二次方：第 t 步里，新的 q_t 仍然要和缓存中全部 t 个 k 做点积、再对 t 个 v 加权，代价是 $O(t \cdot d)$ 。累加 T 步后注意力部分仍是 $O(T^2 d)$ 。所以”有 Cache 后每步耗时恒定”只在 $t \cdot d \ll d_{\text{model}}^2$ （上下文不长）时近似成立；上下文变长后，每一步都要从显存读一遍整个 KV Cache，这正是长上下文 Decode 变慢、以及 FlashDecoding / GQA / MLA / KV 量化出现的原因（见第 8、9、12 篇）。

7.4 [隐患] 四、KV Cache 的代价：显存爆炸与 PagedAttention 的诞生

天下没有免费的午餐。KV Cache 用空间换取了时间，但带来了一个极度致命的副作用：显存空间被疯狂吞噬！

7.4.1 KV Cache 显存占用精确公式推导

对于一个大模型：

- 层数 L （每一层 Transformer 都有自己独立的 K 和 V ）
- 隐藏层维度 H
- 上下文总长度 S （输入的 Prompt 词数 + 已生成的词数）

- 并发请求数 B (Batch Size, 同时服务的用户数)
- 浮点精度字节数 b (FP16/BF16 每个数字占 2 字节)

每个词需要存储一个 Key 向量 (H 个数字) 和一个 Value 向量 (H 个数字), 跨越全部 L 层:

$$\text{每个 Token 的 KV 显存} = 2 \times L \times H \times b \quad (\text{字节})$$

如果同时服务 B 个用户, 平均每个请求长 S 个词:

$$\text{总 KV Cache 显存} = B \times S \times 2 \times L \times H \times b$$

实例计算 (一个 80 层、隐藏维度 8192、采用原始 MHA 的 70B 级模型):

核心导读与背景

注: 真实的 Llama-2-70B 已经使用了 GQA (64 个 Q 头、8 个 KV 头), 下面的数字是”假如它仍用原始 MHA” 的反事实计算——正是这个数字吓人, 才逼出了 GQA。LLaMA-1-65B (80 层、8192 维、MHA) 的量级与之相同。

- $L = 80, H = 8192, b = 2$ (FP16)
- 单个 Token 的 KV Cache 显存:

$$2 \times 80 \times 8192 \times 2 = 2,621,440 \text{ 字节} \approx 2.5 \text{ MB / token}$$

- 如果有 16 个用户并发 ($B = 16$), 每个人对话长度为 4096 字 ($S = 4096$):

$$16 \times 4096 \times 2.5 \text{ MB} \approx 163,840 \text{ MB} = \mathbf{160 \text{ GB 显存!}}$$

看到了吗? 光是保存这一批对话的 KV Cache, 就需要整整 **160 GB** 显存, 这比 70B 模型本身的权重 (140 GB) 还要大! 而换成 GQA-8 (Llama-2/3-70B 的真实配置) 后, 每 token 只需 $2 \times 80 \times (8 \times 128) \times 2 = 327,680$ 字节 $\approx 320 \text{ KB}$, 同样场景只要 **20 GB**——这就是 GQA 的价值。

7.4.2 现代模型的架构救赎: GQA 与 PagedAttention

为了解决这个显存无底洞, 学术界与工业界爆发了两场革命:

1. 模型结构革命: GQA (分组查询注意力)

- 既然存储全部 64 个头太费显存, 那就让 8 个 Query 头共享 1 组 Key/Value 头 (KV 头数压缩为 1/8)。

- 现代 Llama-3、Qwen2.5 广泛采用 GQA，将 KV Cache 显存直接缩减为原来的 1/4 到 1/8!

1. 显存管理革命：PagedAttention (vLLM 破圈之作)

- 传统框架必须为每个请求提前申请连续的显存块（例如预留 4096 个词的空间），但大部分请求只聊了 100 句就退出了。vLLM 论文测到 60%~80% 的 KV 显存被浪费——其中既有按 max_len 预留的闲置，也有内部碎片与外部碎片（见本节的下一段），三者要分开看。
- vLLM 借鉴了操作系统的虚拟分页技术，将 KV Cache 切成 16 个 token 一块的物理页，按需分配，零碎片浪费，让服务器并发能力暴增数倍！

7.5 [目标] 总结与下一章预告

在本篇中，我们彻底理清了大模型推理的微观全貌：

1. **推理与训练的区别**：推理是权重冻结的自回归单向接龙；
2. **Prefill vs Decode**：Prefill 是首字并行算力受限（Compute-Bound），Decode 是逐字生成访存受限（Memory-Bound）；
3. **KV Cache 代数展开**：证明了历史 Key 和 Value 的不变性，通过缓存把逐 token 重复做投影/FFN 的部分从 $O(T^2)$ 降到 $O(T)$ （注意力本身仍是 $O(T^2d)$ ，长上下文下它反过来成为主导，见本节 §3 的补充）；
4. **显存挑战**：KV Cache 随并发与长度线性膨胀，催生了 GQA 与 PagedAttention。

在下一篇中，我们将探讨：模型输出 Logits 之后，最终的文字是怎么被选择出来的？温度系数（Temperature）在数学极限下到底证明了什么？Top-K 和 Top-P 是怎么把胡言乱语切除的？

核心数学定理推导

[重点] 本篇说“Decode 访存受限”，但没有给出定量推导。为什么 batch 开大几乎不增加每一步的耗时？GQA 除了省显存还能提速吗？这些在 **第 8 篇：硬件视角** 中推导，并在你的显卡上实测。显存管理（PagedAttention）的完整机制与实现见 **第 12 篇** 与 Lab 12b。

上一篇：第 5 篇：从原版 Transformer 到 Llama / DeepSeek：每一处改动的历史动机 | 下一篇：第 7 篇：大模型的决策大脑：Logits、温度系数极限证明与采样算法 | 总路线：LEARNING_PATH.md

第 8 章 大模型的决策大脑：Logits、温度系数极限证明与采样算法

核心数学定理推导

阅读前须知：经过了前面四篇的学习，大模型的 Transformer 网络已经完成了前向传播，把前文所有的上下文浓缩进了一个最终的隐藏层向量中。在这最后一公里，这个向量是如何变成真实世界的文字的？为什么温度调高了模型会胡思乱想？调低了就会一本正经？本篇将为你带来严格的数学极限证明与算法解析。

8.1 [目标] 一、最后的终点线：LM Head（语言模型头）

在模型的最后一层，针对序列的最后一个位置，输出一个经过归一化的隐藏特征向量：

$$h_{\text{last}} \in \mathbb{R}^{d_{\text{model}}} \quad (\text{例如 } 4096 \text{ 维})$$

这个向量就像是一团高度压缩的“思考精华”，里面包含了模型对“下一个字应该是什么”的所有语义判断。但它还不是人类能看懂的字，我们需要把它映射回整个词表（Vocabulary）。

8.1.1 词表矩阵乘法

假设模型的词表里包含 V 个词（现代大模型如 Llama-3 的词表大小 $V = 128,256$ ；Qwen2.5 小尺寸 151,936、7B 及以上 152,064）。我们使用一个投影矩阵 $W_{\text{head}} \in \mathbb{R}^{d_{\text{model}} \times V}$ 与之相乘：

$$z = h_{\text{last}} W_{\text{head}} \in \mathbb{R}^V$$

这个输出向量 z 的长度恰好等于词表大小 V 。向量里的每一个数值，被称为一个 **Logit**（未归一化对数打分）：

$$z = [z_1, z_2, \dots, z_V]$$

- z_{1024} 对应词表里的“的”，数值可能是 14.5（极高）；
- z_{305} 对应词表里的“烤鸭”，数值可能是 8.2；
- z_{9999} 对应词表里的“宇宙飞船”，数值可能是 -3.1（极低）。

现在，模型拿到了对全人类十几万个词汇的打分单。接下来的关键问题是：**怎么做最终的选词决策？**

8.2 [剖析] 二、决策方法 1：贪婪搜索（Greedy Search）

最直观的方法，就是直接挑选打分最高的那一个词：

$$\hat{y} = \arg \max_i (z_i)$$

候选 Token	模型输出 Logit 打分	选择策略判定
" 的"	14.5	最高分! Greedy 策略直接选中
" 是"	12.1	次高分
" 在"	9.3	低分
" 飞船"	-2.0	极低概率

8.2.1 优缺点分析：

- **优点：**计算最快，结果 100% 确定，非常适合**数学推理、编写代码、事实性问答**等有唯一确定答案的场景。
- **缺点：**极其死板，缺乏创造力。更致命的是，贪婪算法只看眼前单步最优，容易陷入局部最优陷阱，导致模型在长文本生成时陷入可怕的**无脑复读死循环**（“我认为.....我认为.....我认为.....”）。

为了打破这种死板，人类引入了**随机概率采样（Stochastic Sampling）**。

8.3 [温度] 三、【重磅数学推导】：温度系数（Temperature）的极限证明

为了在采样中可控地调节模型的“严肃度”与“创造力”，科学家在 Softmax 中引入了一个正实数参数 T （Temperature，温度系数）：

$$p_i(T) = \frac{\exp\left(\frac{z_i}{T}\right)}{\sum_{j=1}^V \exp\left(\frac{z_j}{T}\right)}$$

你一定在各类大模型界面中调节过这个滑动条（通常在 0.0 ~ 2.0 之间）。但你是否知道，在严谨的数学分析中，温度参数在极限状态下到底意味着什么？

下面我们给出两个经典的**数学定理与极限推导**。

8.3.1 定理 1：若最大 logit 唯一，当温度 $T \rightarrow 0^+$ 时，概率分布严格退化为贪婪搜索（确定性决策）

核心数学定理推导

【证明】：设 Logits 向量中存在唯一的最大值 $z_{\max} = \max_j(z_j)$ 。对任意其他非最大项 $i \neq \max$ ，均有：

$$z_i - z_{\max} < 0$$

核心导读与背景

我们将 Softmax 公式分子与分母同时除以 $\exp\left(\frac{z_{\max}}{T}\right)$ （利用平移不变性）：

$$p_i(T) = \frac{\exp\left(\frac{z_i - z_{\max}}{T}\right)}{\sum_{j=1}^V \exp\left(\frac{z_j - z_{\max}}{T}\right)}$$

核心导读与背景

考察当 $T \rightarrow 0^+$ 时的极限行为：

情况 A：对于最大值项自身 ($i = \max$)：

$$z_{\max} - z_{\max} = 0 \implies \exp\left(\frac{0}{T}\right) = e^0 = 1$$

核心导读与背景

情况 B：对于任意非最大值项 ($i \neq \max$)：因为 $z_i - z_{\max} < 0$ （为一个严格负常数 $-c$ ），当 $T \rightarrow 0^+$ 时：

$$\lim_{T \rightarrow 0^+} \frac{z_i - z_{\max}}{T} = -\infty$$

核心导读与背景

根据自然指数极限：

$$\lim_{T \rightarrow 0^+} \exp\left(\frac{z_i - z_{\max}}{T}\right) = \lim_{x \rightarrow -\infty} e^x = 0$$

核心导读与背景

现在把分子和分母的极限合并：- 分母的总和：

$$\lim_{T \rightarrow 0^+} \sum_{j=1}^V \exp\left(\frac{z_j - z_{\max}}{T}\right) = 1 + \underbrace{0 + 0 + \cdots + 0}_{V-1 \text{ 个}} = 1$$

核心导读与背景

- 最大值项的概率：

$$\lim_{T \rightarrow 0^+} p_{\max}(T) = \frac{1}{1} = \mathbf{1.0} \quad (100\%)$$

核心导读与背景

- 所有非最大值项的概率：

$$\lim_{T \rightarrow 0^+} p_{i \neq \max}(T) = \frac{0}{1} = \mathbf{0.0} \quad (0\%)$$

核心导读与背景

Q.E.D. 证毕！

物理直觉：

避坑指南与核心警示

[注意] ”最大值唯一”这个前提不能省：如果有 k 个词的 logit 并列最大，同样的推导给出极限是这 k 个词各占 $1/k$ 的均匀分布，而不是确定性地选其中一个。真实模型里 logit 恰好相等很少见，但 BF16 精度下前几名”数值上相等”并不罕见，这也是低温采样有时仍有随机性的原因之一。

温度越低 ($T \rightarrow 0$)，最大值项的优势被无限放大，概率瞬间形成尖锐的独峰（全部概率集中在一个词上的”单点分布”，即 one-hot），模型 100% 必然挑选中打分最高的词！这就是为什么调低温度能让回答严谨、稳定、绝不胡说八道！

8.3.2 定理 2：当温度 $T \rightarrow +\infty$ 时，概率分布严格退化为均匀分布（纯随机乱码）

核心数学定理推导

【证明】：考察指数内部项在 $T \rightarrow +\infty$ 时的极限：对于任意有限实数 z_i ：

$$\lim_{T \rightarrow +\infty} \frac{z_i}{T} = 0$$

核心导读与背景

因此：

$$\lim_{T \rightarrow +\infty} \exp\left(\frac{z_i}{T}\right) = e^0 = 1$$

核心数学定理推导

代入 Softmax 公式：

$$\lim_{T \rightarrow +\infty} p_i(T) = \frac{1}{\sum_{j=1}^V 1} = \frac{1}{V}$$

核心导读与背景

Q.E.D. 证毕！

物理直觉：

温度无穷大 ($T \rightarrow \infty$) 时，词表中每一个词（不论是常见字还是生僻乱码）被选中的概率全部变成了毫无差别的 $1/V$ ！模型丧失了一切前向推理学习到的知识，完全退化为一个均匀扔骰子的随机数发生器！

8.4 [目标] 四、工业级截断双子星：Top-K 与 Top-P

单纯靠 Temperature 调节依然有风险：即使把温度设为正常的 0.7，长尾里那些概率只有 0.0001% 的荒谬生僻字，依然有极其微小的概率被“不幸抽中”，导致模型偶尔冒出一句胡话。

为了彻底扫清这些低质噪声，工业界标配了两种截断算法：

8.4.1 Top-K 采样

- 逻辑：不管词表有多大，永远只保留概率最高的前 K 个候选词（例如 $K = 40$ 或 $K = 50$ ）。
- 将第 $K + 1$ 到第 V 个词的 Logit 强行赋值为 $-\infty$ （概率归零）。

- 在保留的这 K 个词中重新归一化 Softmax。

高概率候选词（保留采样池）	截断阈值	长尾低概率词（强制置 $-\infty$ ）
前 K 个最高概率词: [词 1, 词 2, ..., 词 K]	$\longleftrightarrow$	剩余词: [词 $K + 1$, ..., 词 V] 全部过滤

8.4.2 Top-P 采样（核采样 Nucleus Sampling，更智能的动态截断）

Top-K 有一个天然的死穴：固定 K 值无法适应不同语境的灵活性！

- **场景 A：** 用户输入“人工智能的英文缩写是 _____”。
- 这种问题几乎只有唯一正确答案“AI”。此时理想的候选池大小应该是 $K = 1$ 。如果固定 $K = 50$ ，模型反而可能把后 49 个错误答案放进抽奖箱。
- **场景 B：** 用户输入“请描述一个美丽的黄昏景色：_____”。
- 此时有成百上千个优美的辞藻（“霞光”、“落日”、“晚风”、“云彩”）都是合理的，此时 $K = 50$ 又显得太少，扼杀了多样性。

Top-P 的破局之道（Holtzman et al., ICLR 2020）：

核心导读与背景

不要固定候选词的个数，而是固定候选词的「累积置信度阈值 P 」（例如 $P = 0.9$ 或 90%）！

语境场景	候选 Token 概率累积	动态候选池判定
极确定语境（如代码/事实）	"AI": 92% $\geq$ 90%	立即截断！候选池仅保留 1 个词
发散创意语境（如诗歌/创作）	"落日": 25%, "晚霞": 20%, ...	累积至 90.5% 截断！动态保留 15 个候选词

- **语境确定时：** 头部词概率极高，累积迅速破 90%，候选池自动收缩为 1~2 个词，极其精准；
- **语境发散时：** 头部词概率分散，需要累积几十个词才破 90%，候选池自动扩大，充分保障多样性！

8.5 五、防复读机制：重复惩罚（Repetition Penalty）

你一定见过模型陷入“复读机”状态。怎么在数学上给它一记当头棒喝？

在生成第 t 步时，算法会检查前文中已经出现过的所有词的索引集合 $\mathcal{S}_{\text{history}}$ 。对于集合中的词，对其打分 z_i 施加一个惩罚系数 $\theta > 1.0$ （通常取 $1.1 \sim 1.3$ ）：

$$z'_i = \begin{cases} z_i/\theta & \text{若 } z_i > 0 \quad (\text{主动压低正向打分}) \\ z_i \times \theta & \text{若 } z_i \leq 0 \quad (\text{加大负向惩罚力度}) \end{cases}$$

通过这一轻巧的代数缩放，曾经说过的词在下一步重新争夺第一名的难度被大幅提高，模型被迫转向寻找新的表达方式，显著缓解复读机问题。

避坑指南与核心警示

[注意] 但它是一把钝器：它惩罚的是“出现过的所有 token”，不区分该不该重复。人名、专有名词、标点、代码里的变量名本来就需要反复出现，惩罚过重会让模型改用同义词、写错变量名，甚至破坏格式。而且它连提示词里的 token 也一起惩罚 (Lab 07b 实验 5 里 HF 的实现就是这样)。所以实际服务中惩罚系数通常只取 1.05~1.2，另有 frequency / presence penalty 等更细的变体。

8.6 [检验] 前七篇小结

到这里，你已经**从零起点、不依赖任何黑盒**，贯通了“推理在算什么”的整条链路：

1. 第 1 篇：计算机如何从规则、统计到被 Transformer 彻底征服
2. 第 2 篇：向量空间几何、点积的几何意义与 Safe Softmax 溢出防护
3. 第 3 篇：导数、反向传播、交叉熵，Softmax 雅可比与 $p - y$ 梯度
4. 第 4 篇：严格证明 Attention 为什么必须除以 $\sqrt{d_k}$ ，推导 RoPE 相对位置内积恒等性
5. 第 5 篇：从原版 Transformer 到 Llama / DeepSeek，每一处改动的历史动机
6. 第 6 篇：推理与训练的本质区别，代数展开 KV Cache 消除冗余并剖析显存挑战
7. 第 7 篇：温度参数在 $T \rightarrow 0$ 和 $T \rightarrow \infty$ 下的数学极限，Top-K、Top-P 与重复惩罚

但要真正理解推理**为什么快、为什么慢、怎么优化**，还需要继续：**第 8 篇：硬件视角** → **第 9 篇：FlashAttention** → **第 10 篇：投机解码** → **第 11 篇：量化** → **第 12 篇：推理服务系统** → **第 13 篇：多卡并行**

完整顺序（含实验与外部资源）见 `LEARNING_PATH.md`。

上一篇：第 6 篇：什么是推理？自回归生成全生命周期与 KV Cache 代数证明 | 下一篇：第 8 篇：硬件视角：GPU 存储层级、FLOPs、算术强度与浮点格式 | 总路线：LEARNING_PATH.md

第 9 章 硬件视角：GPU 存储层级、FLOPs、算术强度与浮点格式

核心数学定理推导

阅读前须知：第 6 篇和 Lab 08 告诉你一个结论——”Prefill 算力受限，Decode 访存受限”。但为什么？受限到什么程度？batch 开多大才能把 GPU 喂饱？GQA 除了省显存还有没有别的好处？量化为什么能加速？这些问题都需要**把硬件放进公式里**才能回答。本篇是从”懂模型”走向”懂推理系统”的分水岭。

前置：第 6 篇（Prefill/Decode、KV Cache）、第 3 篇（ $2N$ FLOPs/token）、第 5 篇（GQA/MLA）。

配套实验：`lab08_memory_and_roofline/measure_gpu_roofline.py`——在你自己的 RTX 5060 Ti 上实测带宽、峰值算力、Decode 形状矩阵乘随 batch 的变化曲线，以及 BF16/FP16 的精度陷阱。

9.1 [硬件] 一、GPU 的基本构造：计算很便宜，搬数据很贵

9.1.1 计算单元

- GPU 由几十到上百个 SM（Streaming Multiprocessor，流式多处理器）组成。RTX 5060 Ti 有 36 个 SM，H100 有 132 个。
- 每个 SM 里有两类计算单元：
 - **CUDA Core：**做普通的标量浮点运算（FP32 加乘）；
 - **Tensor Core：**专门做小矩阵乘累加（例如一次完成 16×16 的块乘），低精度（BF16/FP16/FP8/FP4）吞吐是 CUDA Core 的十几倍以上。大模型的矩阵乘几乎全部跑在 Tensor Core 上。
- 线程以 32 个为一组（warp）同步执行同一条指令。

9.1.2 存储层级（越往下越大、越慢）

层级	容量量级	带宽量级	谁能访问
寄存器	每 SM 256 KB	最快	单个线程
共享内存 / L1 (SRAM)	每 SM 约 100~228 KB	全卡合计数十 TB/s	同一个 SM 内的线程块
L2 缓存	全卡数十 MB	数 TB/s	全卡
显存 (HBM / GDDR)	16 ~ 192 GB	0.4 ~ 8 TB/s	全卡

RTX 5060 Ti 的显存是 16GB GDDR7、128 位总线，标称带宽 448 GB/s。作为对比，H100 SXM 的 HBM3 带宽是 3.35 TB/s。

核心直觉与思考

[思考] 一个工厂比喻（来自 Horace He 的 *Making Deep Learning Go Brrrr*）：Tensor Core 是一座效率极高的工厂，显存是远处的仓库，显存带宽是连接两者的公路。工厂再快，原料运不过来也只能停工。大模型 Decode 的处境就是：工厂 90% 以上的时间在等货车。

9.2 [数字] 二、数清楚两件事：算了多少（FLOPs）、搬了多少（Bytes）

9.2.1 矩阵乘的 FLOPs

$[M, K] \times [K, N]$ ：输出有 MN 个元素，每个是 K 次乘法 + K 次加法：

$$\text{FLOPs} = 2MKN$$

一个参数量为 N_{param} 的模型，处理一个 token 约 $2N_{\text{param}}$ FLOPs（第 3 篇）。

9.2.2 矩阵乘的访存量

每个元素 b 字节（BF16 为 2），至少要读两个输入、写一个输出：

$$\text{Bytes} = b(MK + KN + MN)$$

9.2.3 算术强度（Arithmetic Intensity）

$$\text{AI} = \frac{\text{FLOPs}}{\text{Bytes}} \quad (\text{单位: FLOP/Byte, 每搬 1 字节做多少次运算})$$

9.3 [几何] 三、Roofline 模型：一张图判断你被谁卡住

硬件有两个上限：峰值算力 π (FLOP/s) 和显存带宽 β (Byte/s)。一个算术强度为 AI 的算子，能达到的性能是：

可达性能 = min(π , AI $\times$ β)

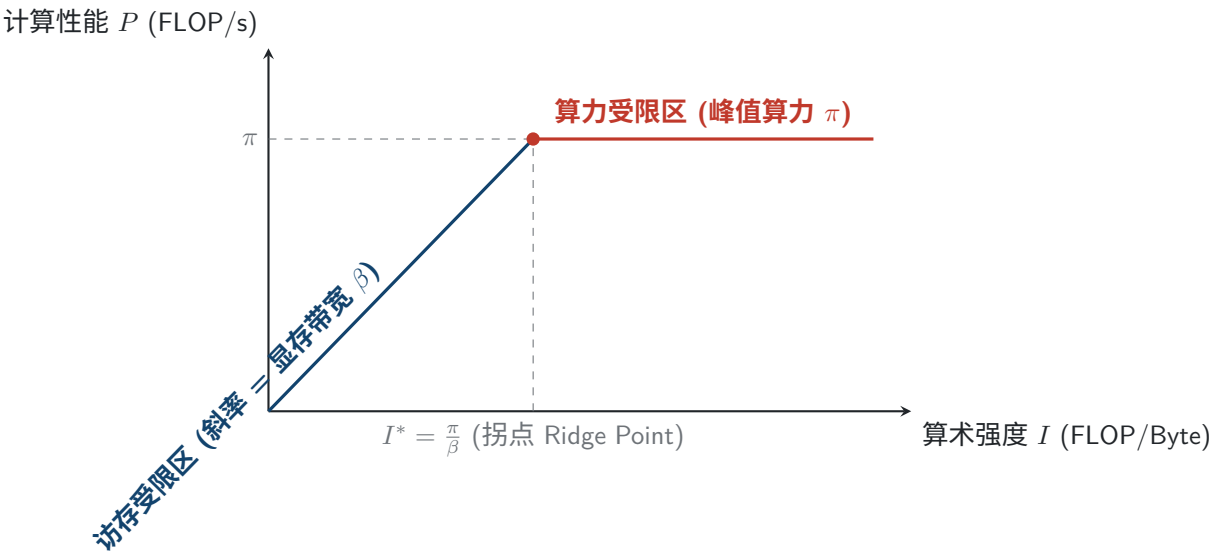


图 9.1: GPU Roofline 模型与瓶颈判定边界

拐点 $AI^* = \pi/\beta$ ：算子的 AI 低于它，就是访存受限；高于它，就是算力受限。

硬件	BF16 稠密峰值 π	带宽 β	拐点 AI^*
A100 SXM	312 TFLOP/s	2.04 TB/s	≈ 153
H100 SXM	989 TFLOP/s	3.35 TB/s	≈ 295
RTX 5060 Ti	用配套实验实测	448 GB/s (标称)	实测后自己算

避坑指南与核心警示

[注意] 厂商宣传的 Tensor 算力常常是”稀疏”或”FP16 累加”口径，GeForce 卡在 FP32 累加（训练/推理实际用的模式）下常常只有宣传值的一半。永远以实测为准——这也是配套实验存在的意义。

9.4 [目标] 四、核心推导：Decode 的算术强度 $\approx$ Batch Size

9.4.1 线性层（占 Decode 权重读取的绝大部分）

Decode 时 batch 里有 B 个请求，每个请求只有 1 个新 token，所以线性层是 $[B, K] \times [K, N]$ ，而权重 KN 远大于激活 BK 、 BN ($B \ll K, N$):

$$AI = \frac{2BKN}{b(BK + KN + BN)} \approx \frac{2BKN}{b \cdot KN} = \frac{2B}{b} \stackrel{\text{BF16}}{=} \boxed{B}$$

Decode 时线性层的算术强度约等于 batch size。 这一个结论解释了一大串现象：

- batch = 1 时 $AI \approx 1$ ，而拐点在一两百 $\rightarrow$ GPU 算力利用率只有 **1% 量级**；
- 把 batch 从 1 提到 64，每一步读的权重**几乎不变**，时间几乎不变，吞吐却提升接近 64 倍——这就是**批处理的全部意义**，也是 Lab 12a 连续批处理仿真里”每一步耗时视为常数”这个假设的来源；
- 只有当 B 接近拐点（A100 上约 150）后，再加 batch 才会让每一步变慢。

9.4.2 Prefill

Prefill 时 $M = \text{prompt 长度}$ 。例如 $M = 2000$ 、 $K = N = 4096$ ，代入完整公式：

$$AI = \frac{2MKN}{b(MK + KN + MN)} = \frac{2 \cdot 2000 \cdot 4096^2}{2(2000 \cdot 4096 + 4096^2 + 2000 \cdot 4096)} \approx 1012$$

（注意这里 M 已经不再远小于 K, N ，上面 Decode 用的近似 $2M/b$ 只是 $M \rightarrow 0$ 时的上界，会高估一倍。）即便如此，1012 也远超拐点 $\rightarrow$ **算力受限**。所以 Prefill 的耗时估算是：

$$T_{\text{prefill}} \approx \frac{2N_{\text{param}} \cdot S_{\text{prompt}}}{\pi \cdot \text{MFU}}$$

MFU（Model FLOPs Utilization）是实际达到峰值的比例，优秀实现通常 40%~70%。

9.4.3 Decode 中的注意力：batch 救不了它

线性层的权重被 batch 里所有请求**共享**，所以 batch 能摊薄权重读取。但**每个请求的 KV Cache 是它自己的**，只有它自己的 query 会用。单个请求、单层、上下文长度 S ：Q 头数 n_h 、KV 头数 n_{kv} 、头维度 d_h ：

$$\begin{aligned} \text{FLOPs} &= \underbrace{2n_h S d_h}_{QK^T} + \underbrace{2n_h S d_h}_{PV} = 4n_h S d_h, & \text{Bytes} &= \underbrace{2S n_{kv} d_h}_{K \text{ 和 } V} \cdot b \\ AI_{\text{attn}} &= \frac{4n_h S d_h}{2S n_{kv} d_h b} = \frac{2}{b} \cdot \frac{n_h}{n_{kv}} \stackrel{\text{BF16}}{=} \boxed{\frac{n_h}{n_{kv}}} \end{aligned}$$

Decode 注意力的算术强度 = GQA 的分组大小，与 batch 无关、与上下文长度无关。

- MHA： $AI = 1$ ，永远被带宽卡死；
- Llama-3-8B（32 Q 头 / 8 KV 头）： $AI = 4$ ；
- MLA 让大量 Q 头共享同一个压缩潜向量， AI 更高。

避坑指南与核心警示

[目标] 所以 GQA/MLA 的价值有两层：第 5 篇讲的是省显存（能放更多请求、更长上下文），这里推出的是提速（同样的 KV 字节被更多次计算复用）。长上下文时注意力的 KV 读取会超过权重读取，成为 Decode 的主要耗时——这时 GQA 分组大小直接决定速度。

9.4.4 把两部分合起来：一步 Decode 的耗时模型

$$T_{\text{step}}(B) \approx \max\left(\frac{W_{\text{bytes}} + B \cdot S \cdot \text{KV}_{\text{bytes/token}}}{\beta}, \frac{2N_{\text{param}}B}{\pi}\right)$$

系统吞吐 = $B/T_{\text{step}}(B)$ ，单用户速度 = $1/T_{\text{step}}(B)$ 。加大 batch：总吞吐上升，单用户速度下降——这就是推理服务永恒的“吞吐-延迟权衡”，Lab 08 的 `memory_calculator.py` 已经按这个公式输出了结果。

9.5 [链接] 五、小算子与“算子融合”

RMSNorm、残差相加、SiLU、逐元素乘.....这些算子每个元素只做几次运算，AI 远小于 1。如果每个都单独启动一个 kernel：

执行模式	显存（HBM）与片上缓存（SRAM）数据流动过程
朴素未融合算子	读 x (HBM) $\rightarrow$ RMSNorm $\rightarrow$ 写回 HBM $\rightarrow$ 读回 $\rightarrow$ 乘 γ $\rightarrow$ 写回 $\rightarrow$ 读回 $\rightarrow$ 残差加法 $\rightarrow$ 写回 HBM（访存往返 6 次!）
Kernel Fusion	读 x (HBM) $\rightarrow$ 片上 SRAM / 寄存器一次性完成计算 $\rightarrow$ 写回 HBM（访存仅 1 次，打满带宽!）

大部分时间花在显存来回搬运上。算子融合（Kernel Fusion）把多个逐元素操作合成一个 kernel：数据只从显存读一次，在寄存器里做完所有运算，再写回一次。

- `torch.compile`、Triton、TensorRT 做的核心优化之一就是自动融合；
- FlashAttention 本质上就是把“ $\text{QK}^T \rightarrow$ 掩码 $\rightarrow$ Softmax $\rightarrow$ 乘 V ”整个融合成一个 kernel，而且巧妙地让 $S \times S$ 的注意力矩阵根本不落到显存上——这需要一个数学技巧（Online Softmax），见第 9 篇。

9.6 [实验] 六、浮点格式：量化与混合精度的前提

9.6.1 浮点数的结构

一个浮点数由三部分组成：符号位 s 、指数位 e （决定范围）、尾数位 m （决定精度）：

$$x = (-1)^s \times 2^{e-\text{bias}} \times (1.m)_2$$

格式	符号/指数/尾数	最大值	相对精度 (机器 ϵ)	主要用途
FP32	1 / 8 / 23	3.4×10^{38}	$2^{-23} \approx 1.2 \times 10^{-7}$	累加器、优化器状态
FP16	1 / 5 / 10	65504	$2^{-10} \approx 9.8 \times 10^{-4}$	早期混合精度
BF16	1 / 8 / 7	3.4×10^{38}	$2^{-7} \approx 7.8 \times 10^{-3}$	现代训练与推理的默认格式
FP8 E4M3	1 / 4 / 3	448	2^{-3}	前向的权重与激活
FP8 E5M2	1 / 5 / 2	57344	2^{-2}	梯度 (需要更大范围)
FP4 E2M1	1 / 2 / 1	6	—	Blackwell (含你的 5060 Ti) 原生支持, 需配合分块缩放

9.6.2 为什么 BF16 打败了 FP16?

BF16 就是把 FP32 的尾数直接砍掉 16 位: 指数位和 FP32 一样多, 范围与 FP32 相同, 只是精度低。FP16 精度更高, 但最大只能表示 65504, 训练时激活或梯度很容易溢出成 `inf`, 需要”损失缩放 (loss scaling)”等额外技巧。深度学习对范围比对精度敏感得多, 所以 BF16 胜出。

9.6.3 低精度的陷阱: 累加必须用高精度

BF16 只有 8 位有效精度 (7 位尾数 + 隐含的 1)。在 256 附近, 相邻两个可表示数之间的间隔是 2:

$$256 + 1 \stackrel{\text{BF16}}{=} 256$$

如果你把 4096 个数在 BF16 里逐个累加, 后面加的小数会被全部吞掉。所以:

- Tensor Core 做 BF16 矩阵乘时, 内部用 FP32 累加;
- Softmax、RMSNorm 的求和通常在 FP32 下完成 (Lab 07b 的实现里你会看到 `.float()`)。

配套实验会让你亲眼看到这个现象。

9.6.4 整数格式与浮点格式的区别

INT8/INT4 是均匀网格 (相邻值间隔固定为缩放因子 S), 浮点是对数网格 (越靠近 0 越密)。神经网络的权重大多集中在 0 附近, 所以同样 8 位, FP8 对小值更友好、INT8 对均匀分布更友好。这是第 11 篇量化的出发点。

9.7 [OK] 本篇自测

1. $[1, 4096] \times [4096, 4096]$ 的 BF16 矩阵乘, FLOPs 和访存字节各是多少? AI 是多少?

2. 为什么说 Decode 时”batch 从 1 提到 32，每一步耗时几乎不变”？什么时候这个结论不再成立？
 3. 推导 Decode 注意力的 AI，并说明为什么增大 batch 对它没有帮助。
 4. 在一张带宽 448 GB/s 的卡上，BF16 的 Qwen2.5-7B（约 15.2 GB 权重）batch=1 Decode 的理论上限是多少 token/s？换成 INT4 权重呢？
 5. BF16 和 FP16 都是 16 位，为什么训练更偏爱 BF16？
 6. 为什么 Softmax 的求和要在 FP32 里做？
-

9.8 [资料] 外部资源

- Horace He: [Making Deep Learning Go Brrrr From First Principles](#) ——算力/带宽/开销三分法，**必读**。
- NVIDIA: [GPU Performance Background User's Guide](#) ——官方的算术强度与 Roofline 讲解。
- kippily: [Transformer Inference Arithmetic](#) ——用本篇的方法把推理的每一项算清楚，**必读**。
- Google DeepMind: [How to Scale Your Model](#) ——第 1 章 Rooflines、第 4 章 Transformer Math、第 7 章 Inference，是本篇最好的进阶读物。
- [GPU MODE 讲座](#) ——从 CUDA 入门到 FlashAttention 的社区课程；配合教材 *Programming Massively Parallel Processors* (PMPP, 第 4 版)。
- [NVIDIA CUDA C++ Programming Guide](#) 第 1~5 章（编程模型、存储层级）。
- Goldberg: [What Every Computer Scientist Should Know About Floating-Point Arithmetic](#) (经典，选读)。
- Williams, Waterman, Patterson 2009: *Roofline: An Insightful Visual Performance Model for Multicore Architectures* (CACM, Roofline 的原始论文)。

上一篇：第 7 篇：大模型的决策大脑：Logits、温度系数极限证明与采样算法 | 下一篇：第 9 篇：FlashAttention：Online Softmax 的严格推导与 IO 复杂度 | 总路线：LEARNING_PATH.md

第 10 章 FlashAttention: Online Softmax 的严格推导与 IO 复杂度

避坑指南与核心警示

阅读前须知：FlashAttention (2022) 可能是近几年影响最大的一个推理/训练优化——它不改变任何数学结果，却让注意力快了数倍、显存从 $O(N^2)$ 降到 $O(N)$ ，从而让长上下文成为可能。它的核心是一个非常漂亮的代数技巧：**Online Softmax**。本篇把它从头推导一遍。

前置：第 2 篇 (Softmax 平移不变性)、第 4 篇 (注意力公式)、第 8 篇 (存储层级、算术强度、算子融合)。

配套实验：`lab09_flash_attention/flash_attention_numpy.py`——用 NumPy 实现 Online Softmax、分块 FlashAttention 与 FlashDecoding 的 split-K 合并，逐一验证与朴素注意力完全相等；并在 GPU 上实测朴素注意力与 PyTorch SDPA 的显存差距。**真机版：**`lab09_flash_attention/cuda_softmax/`——把本节和下一节的 Online Softmax 写成真正的 CUDA C++ kernel，在你自己显卡上量出 DRAM 带宽、L2 命中与占用率。不需要装 CUDA toolkit。

10.1 [低速] 一、朴素注意力慢在哪里？

对序列长度 N 、头维度 d ，朴素实现分三步，每一步都是一个独立的 kernel：

标准 Attention 步骤	数学算子公式	朴素显存读写量 (HBM IO)
1. 计算打分矩阵	$S = QK^T/\sqrt{d}$	读 Q, K ($2Nd$) $\rightarrow$ 写 S 到显存 (N^2)
2. 概率归一化	$P = \text{softmax}(S)$	读 S (N^2) $\rightarrow$ 写 P 到显存 (N^2)
3. 加权求和输出	$O = PV$	读 P (N^2), V (Nd) $\rightarrow$ 写 O 到显存 (Nd)
总访存量	高达 $4Nd + 2N^2$ 字节 (受 $O(N^2)$ 显存访问严重瓶颈束缚)	

- **显存：**必须存下 $N \times N$ 的 S 和 P 。 $N = 32K$ 时单个头就是 10^9 个元素，BF16 下 2GB——而一层有几十个头。
- **访存：**总共读写 $\Theta(Nd + N^2)$ 个元素。而 d 通常只有 64~128， N^2 项完全主导。Softmax 这一步每个元素只做几次运算，算术强度极低 (第 8 篇)，GPU 大部分时间在搬运 N^2 的中间矩阵。

自然的想法：像第 8 篇说的那样做算子融合——把 Q、K、V 分块读进片上 SRAM，算完直接输出 O，S 和 P 永远不落到显存。**障碍：**Softmax 需要整行的最大值和总和，才能归一化任何一个元素。只看到一个块时，你不知道这一行后面还有没有更大的值。

10.2 [推导] 二、Online Softmax：一遍扫描算出 Softmax

10.2.1 传统的“安全 Softmax”要扫三遍

对一行 $x_1, \dots, x_N$ (第 2 篇)：

1. 第一遍求 $m = \max_i x_i$;
2. 第二遍求 $\ell = \sum_i e^{x_i - m}$;
3. 第三遍输出 $p_i = e^{x_i - m} / \ell$ 。

10.2.2 把前两遍合成一遍

维护两个“运行中”的量：到目前为止看过的最大值 m_j 和对应的和 ℓ_j ：

$$m_j = \max(m_{j-1}, x_j), \quad \ell_j = \ell_{j-1} \cdot e^{m_{j-1} - m_j} + e^{x_j - m_j}$$

初始 $m_0 = -\infty$, $\ell_0 = 0$ 。

定理：对所有 j , $\ell_j = \sum_{i=1}^j e^{x_i - m_j}$ 。

证明 (归纳法)：

- $j = 1$: $m_1 = x_1$, $\ell_1 = 0 + e^{x_1 - x_1} = 1 = \sum_{i=1}^1 e^{x_i - m_1}$ [OK]
- 假设 $\ell_{j-1} = \sum_{i=1}^{j-1} e^{x_i - m_{j-1}}$, 则

$$\ell_j = \left(\sum_{i=1}^{j-1} e^{x_i - m_{j-1}} \right) e^{m_{j-1} - m_j} + e^{x_j - m_j} = \sum_{i=1}^{j-1} e^{x_i - m_j} + e^{x_j - m_j} = \sum_{i=1}^j e^{x_i - m_j} \quad \blacksquare$$

关键就是那个**修正因子** $e^{m_{j-1} - m_j}$ ：当最大值变大时，之前累加的和是以旧的最大值为基准的，乘上这个因子就把它“换算”到新基准下。因为 $m_j \geq m_{j-1}$ ，这个因子 ≤ 1 ，**永远不会溢出**。

上面是逐个元素更新；换成逐块更新（一次处理一个块 B ）形式完全一样：

$$m_{\text{new}} = \max \left(m_{\text{old}}, \max_{i \in B} x_i \right), \quad \ell_{\text{new}} = \ell_{\text{old}} e^{m_{\text{old}} - m_{\text{new}}} + \sum_{i \in B} e^{x_i - m_{\text{new}}}$$

10.3 [加速] 三、把输出也”在线”算出来：FlashAttention 的核心

注意力的一行输出是 $o = \sum_i p_i v_i = \frac{1}{\ell} \sum_i e^{x_i - m} v_i$ (x_i 是这一行第 i 个分数 $q \cdot k_i / \sqrt{d}$)。我们连最后的归一化都不急着做，只维护一个未归一化的累加器：

$$\text{acc}_j = \text{acc}_{j-1} \cdot e^{m_{j-1} - m_j} + \sum_{i \in B_j} e^{x_i - m_j} v_i$$

定理：处理完第 j 块后， $\text{acc}_j = \sum_{i \in B_1 \cup \dots \cup B_j} e^{x_i - m_j} v_i$ 。证明与上一节完全相同（把标量 1 换成向量 v_i ）。于是全部块处理完后：

$$o = \frac{\text{acc}_{\text{final}}}{\ell_{\text{final}}}$$

这和一次性算完整的 Softmax 再乘 V 在数学上严格相等，不是近似。

10.3.1 FlashAttention 算法（前向，单个头）

```

1 把 Q 按行切成若干块 Q_1..Q_Tr (每块 Br 行)，K、V 切成 K_1..K_Tc (每块 Bc 行)
2 for 每个 Q 块 Q_i (不同的 Q 块互相独立 → 分给不同的 SM 并行)：
3     把 Q_i 读进 SRAM；初始化 m = -∞, l = 0, acc = 0 (都在 SRAM / 寄存器里)
4     for 每个 K/V 块 j：
5         把 K_j、V_j 读进 SRAM
6         S_ij = Q_i K_jT / √d                                     # Br × Bc 的小矩阵，只存在片上
7         (因果掩码：若 K_j 整块都在 Q_i 之后，直接跳过；对角块内部再逐元素掩码)
8         m_new = max(m, rowmax(S_ij))
9         P_ij = exp(S_ij - m_new)
10        l = l · exp(m - m_new) + rowsum(P_ij)
11        acc = acc · exp(m - m_new) + P_ij V_j
12        m = m_new
13    O_i = acc / l, 写回显存；另外存下 logsumexp = m + log l (训练的反向传播要用)
```

(这是 FlashAttention-2 的循环顺序：外层循环 Q，内层循环 K/V。FlashAttention-1 的外层是 K/V。)

10.3.2 它省下了什么？

- **显存：** S 、 P 从不写入显存，额外空间只有每行的 m 、 ℓ ： $O(N)$ 。
- **访存：** Q 、 O 各读写一次； K 、 V 对每个 Q 块读一次，共 N/B_r 次。设片上 SRAM 大小为 M （按元素计），块大小 $B_r \sim M/d$ ，论文证明 FlashAttention 的 HBM 访问量为 $\Theta\left(\frac{N^2 d^2}{M}\right)$ ，而朴素实现为 $\Theta(Nd + N^2)$ 。

两者之比的量级是 $\frac{M}{d^2}$ ——这是一个与 N 无关的常数。按配套实验 3 的简化模型（约 100 KB SRAM 要同时放下 Q 、 K 、 V 、 O 四个块）， $d = 64$ 约省 6 倍， $d = 128$ 约省 1.5 倍。

核心导读与背景

注：实验 3 用的是**元素个数**口径，它数出的是 $2B_r/d = M/(2d^2)$ ，和论文的 M/d^2 差一个 2。这不是矛盾——实验 3 的朴素项数了 4 趟 N^2 （写 S 、读 S 、写 P 、读 P ），而论文的 $\Theta(Nd + N^2)$ 只按元素数算一趟。**量级和”与 N 无关”这个结论都不受影响**，但引用具体倍数时要说清用的是哪套账。另外，小 N 时这个常数还到不了（尾块占比大）： $d = 64$ 时要到 $N \gtrsim 16K$ 才收敛到 6 倍， $N = 1024$ 时只有 4.9 倍。真实 kernel 的块形状与 L2 缓存命中也会让具体数字不同，但”常数倍、与 N 无关”这一点不变。

- **计算量**：FLOPs 与朴素完全相同（甚至略多一点点 exp 修正）。
- **随 N 增长的真实收益是显存**：朴素实现需要 $O(N^2)$ 的额外显存，FlashAttention 只要 $O(N)$ 。再加上多个 kernel 融合成一个、不再产生 FP32 的 N^2 中间张量，实测加速往往比访存模型预测的还大（配套实验 5 在 5060 Ti 上 $N = 4096$ 时约 16 倍）。

核心直觉与思考

[思考] 一个反直觉的教训：FlashAttention 没有减少任何计算，只是**减少了数据搬运**，就换来了数倍加速。这正是第 8 篇 Roofline 的核心观点——在访存受限的区域，优化的对象是字节而不是 FLOPs。它的反向传播更进一步：**宁可重新计算 S 和 P ，也不去读存下来的 N^2 矩阵**，因为重算比读显存更便宜。

想亲手验证这句话？`lab09_flash_attention/cuda_softmax/` 把本节的 Online Softmax 写成真正的 CUDA C++ kernel，在你自己显卡上量出：三个实现都贴着 **380 GB/s 的 Roofline**（也就是都”优化到位了”），差别只在搬运了 268 MB 还是 134 MB——约 1.9x。里面还有一个反直觉的结果：把整行缓存进 shared memory 的版本在 $N = 8192$ 时**反而掉到 74%**，因为 shared 占用把 SM 的常驻 block 数从 6 压到 2。这正是上面说的”分块”为什么必须存在。

10.4 [组件] 四、FlashDecoding：把同一个技巧用在 Decode 上

10.4.1 问题

Decode 时每个请求的 Q 只有 **1 行**。FlashAttention 靠”不同的 Q 块分给不同的 SM”来并行，现在只剩 $\text{batch} \times \text{头数}$ 这么多个独立任务。 $\text{batch} = 1$ 、32 个头、上下文 64K 时，只有 32 个任务，而 H100 有 132 个 SM——大部分 SM 在闲着，每个任务却要顺序扫完 64K 个 key。

10.4.2 解法：沿 K/V 的序列维度切分 (split-K)

把长度为 S 的 KV Cache 切成 s 段，**每段独立算出自己的 $(m^{(r)}, \ell^{(r)}, \text{acc}^{(r)})$** ，分给不同的 SM 并行。最后用 Online Softmax 的规则合并：

$$m = \max_r m^{(r)}, \quad \ell = \sum_r \ell^{(r)} e^{m^{(r)} - m}, \quad \text{acc} = \sum_r \text{acc}^{(r)} e^{m^{(r)} - m}, \quad o = \text{acc} / \ell$$

为什么能这样合并？ 因为第三节的定理说明 $\text{acc}^{(r)} = \sum_{i \in \text{段}r} e^{x_i - m^{(r)}} v_i$ ，乘以 $e^{m^{(r)} - m}$ 后正好变成 $\sum_{i \in \text{段}r} e^{x_i - m} v_i$ ，所有段相加就是完整的和。这个合并操作**满足结合律**，所以切成几段、按什么顺序合并都不影响结果——这也是为什么它能用在分布式场景（例如把超长上下文的 KV 分到多张卡上，Ring Attention）。

10.4.3 与 PagedAttention 的关系

PagedAttention（第 12 篇）把 KV Cache 分成固定大小的物理块，块在显存里**不连续**。它的 kernel 本质上就是”一块一块读 KV、用 Online Softmax 累加”——**Online Softmax 让注意力天然可以分块计算，这是分页 KV Cache 能高效工作的数学前提。**

10.5 五、版本演进一览

版本	年份	关键改进
Online max (Milakov & Gimershein)	2018	一遍扫描求 Softmax 的归一化因子
FlashAttention	2022	分块 + Online Softmax + 反向重计算, IO 感知
FlashAttention-2	2023	交换内外循环、减少非矩阵乘运算、更好的并行划分, 约 2 倍提速
FlashDecoding	2023	Decode 时沿 KV 序列切分并行
FlashAttention-3	2024	针对 Hopper: 异步数据搬运 (TMA)、warp 专门化、FP8

在 PyTorch 里, `F.scaled_dot_product_attention` 会自动选择 FlashAttention 等融合 kernel; vLLM、SGLang 则使用 FlashAttention / FlashInfer 等专门的推理注意力库。

10.6 [OK] 本篇自测

1. 朴素注意力的显存和 HBM 访存量各是多少量级？为什么 d 很小时问题特别严重？
2. 写出 Online Softmax 的更新公式，并用归纳法证明 $\ell_j = \sum_{i \leq j} e^{x_i - m_j}$ 。
3. 修正因子 $e^{m_{\text{old}} - m_{\text{new}}}$ 为什么永远不会溢出？
4. FlashAttention 的 FLOPs 比朴素实现少吗？那它为什么快？
5. FlashDecoding 解决的是什么场景下的什么问题？写出两段部分结果的合并公式。

6. 为什么说 Online Softmax 是 PagedAttention 的数学前提?

10.7 [资料] 外部资源

- Zihao Ye: [From Online Softmax to FlashAttention](#) (华盛顿大学 CSE 599M 讲义, 推导最清晰的短文, 必读)
- Milakov & Gimelshein 2018: [Online normalizer calculation for softmax](#)
- Dao et al. 2022: [FlashAttention](#); Dao 2023: [FlashAttention-2](#); Shah et al. 2024: [FlashAttention-3](#)
- Stanford CRFM 博客: [Flash-Decoding for long-context inference](#)
- Triton 官方教程: [Fused Attention](#) (用 Python 写出真正在 GPU 上跑的 FlashAttention; 先做完前面的矩阵乘教程)
- 代码: [Dao-AILab/flash-attention](#); [GPU MODE 讲座](#) 中关于 FlashAttention 的几讲

上一篇: 第 8 篇: 硬件视角: GPU 存储层级、FLOPs、算术强度与浮点格式 | 下一篇: 第 10 篇: 投机解码: 无偏性证明与期望加速比 | 总路线: [LEARNING_PATH.md](#)

第 11 章 投机解码：无偏性证明与期望加速比

核心导读与背景

阅读前须知：Lab 10 给出了投机采样的算法与仿真，但最关键的”为什么”没有讲：它凭什么说”输出分布与大模型完全一致”？加速比能不能事先算出来？什么时候它反而不划算？

前置：第 7 篇（采样）、第 8 篇（访存受限、拐点）。

配套实验：`lab10_speculative_decoding/verify_unbiasedness.py`——蒙特卡洛验证无偏性定理与期望加速比公式，并演示两种常见错误写法如何产生偏差。

11.1 [加速] 一、为什么”猜 K 个再一起验证”会更快？

第 8 篇推出：Decode 时线性层的算术强度约等于**这一步处理的 token 数**。batch=1 时一次只处理 1 个 token，GPU 算力利用率只有百分之一左右。如果让大模型**一次前向验证 K 个 token**，读的权重还是那一份，算术强度变成 K，只要 K 远小于拐点（5060 Ti 上约 130），耗时几乎不变。

于是问题变成：**从哪里弄来 K 个”大概率正确”的候选 token？**——让一个小得多的草稿模型先猜。关键是：猜错了不能影响结果的正确性。

11.2 [几何] 二、单个 token 的无偏性定理

设在同一个前缀下，目标（大）模型给出分布 $p(x)$ ，草稿（小）模型给出分布 $q(x)$ 。算法：

1. 从 q 采样一个 token x ；
2. 以概率 $\min\left(1, \frac{p(x)}{q(x)}\right)$ **接受** x ；
3. 若拒绝，则从**残差分布** $r(x) = \frac{\max(0, p(x)-q(x))}{Z}$ 重新采样，其中 $Z = \sum_{x'} \max(0, p(x') - q(x'))$ 。

（约定：若 $q(x) = 0$ ，草稿根本不会提出 x ，第 2 步的比值不会被计算，所以不会出现除以 0。定理对 q 唯一的要求是它是一个合法的概率分布。）

定理：最终输出的 token 服从分布 p ，与草稿模型 q 的好坏无关。

证明：输出 x 有两种途径——草稿恰好是 x 且被接受；或者草稿被拒绝、然后从残差分布抽中了 x 。

(1) 草稿是 x 且被接受的概率：

$$q(x) \cdot \min\left(1, \frac{p(x)}{q(x)}\right) = \min(q(x), p(x))$$

(2) 发生拒绝的总概率（对所有可能的草稿 token 求和）：

$$P(\text{拒绝}) = 1 - \sum_{x'} \min(q(x'), p(x')) = \sum_{x'} \left(p(x') - \min(q(x'), p(x'))\right) = \sum_{x'} \max(0, p(x') - q(x')) = Z$$

（第二个等号用了 $\sum_{x'} p(x') = 1$ ；第三个等号是因为 $p - \min(p, q) = \max(0, p - q)$ 。）

(3) 拒绝后从残差分布抽中 x 的概率：

$$P(\text{拒绝}) \cdot r(x) = Z \cdot \frac{\max(0, p(x) - q(x))}{Z} = \max(0, p(x) - q(x))$$

两条途径相加：

$$P(\text{输出} = x) = \min(q(x), p(x)) + \max(0, p(x) - q(x)) = p(x) \quad \blacksquare$$

（最后一步分两种情况验证：若 $p(x) \leq q(x)$ ，是 $p(x) + 0$ ；若 $p(x) > q(x)$ ，是 $q(x) + p(x) - q(x)$ 。若 $Z = 0$ ，说明 $p = q$ ，永远不会拒绝，结论同样成立。）

直觉：草稿”猜多了”的地方（ $q > p$ ）按比例拒掉，”猜少了”的地方（ $p > q$ ）由残差分布精确补上。

11.3 [链接] 三、推广到 K 个 token

草稿模型自回归地猜 $x_1, \dots, x_K$ ；大模型一次前向同时得到 $p(\cdot | \text{前缀})$ 、 $p(\cdot | \text{前缀}, x_1)$ 、.....、 $p(\cdot | \text{前缀}, x_1..x_K)$ 共 $K + 1$ 个分布。然后从左到右逐个用第二节的规则检验：

- 第 i 个被接受 $\rightarrow$ 继续检验第 $i + 1$ 个；
- 第 i 个被拒绝 $\rightarrow$ 从残差分布采样一个 token 替代它，**丢弃后面所有草稿**，本轮结束；
- K 个全部被接受 $\rightarrow$ 用已经算好的第 $K + 1$ 个分布**免费再采样一个** token。

由于每个位置都在”正确的前缀”下应用了单 token 定理，按位置归纳可得：**整段输出的联合分布与大模型逐个 token 自回归采样完全一致。**

核心导读与背景

贪婪解码（温度 0）是特例： p 是 one-hot，接受规则退化为”草稿 token 等于大模型的 argmax 才接受”。

11.4 [增长] 四、期望加速比公式

设每个草稿 token 的接受率为 α （假设各位置独立同分布）。由第二节，

$$\alpha = \sum_x \min(p(x), q(x)) = 1 - \text{TV}(p, q)$$

其中 TV 是两个分布的总变差距离——草稿越像目标，接受率越高。

一轮能产出多少 token？”至少前 i 个草稿都被接受”的概率是 α^i ，每轮还一定会多出 1 个 token（要么是拒绝后的替代 token，要么是全接受后的免费 token）：

$$\mathbb{E}[\text{每轮产出}] = 1 + \sum_{i=1}^K \alpha^i = \frac{1 - \alpha^{K+1}}{1 - \alpha}$$

设草稿模型单步耗时与大模型单步耗时之比为 c ($c < 1$ ，例如 $c = 0.1$ 即十分之一)，一轮耗时 $= KcT + T$ (K 次草稿 + 1 次验证)。相对于大模型逐个生成（每 token 耗时 T ）的加速比：

$$\text{加速比} = \frac{1 - \alpha^{K+1}}{(1 - \alpha)(Kc + 1)}$$

例： $\alpha = 0.8$ 、 $K = 4$ 、 $c = 0.1$ ：期望每轮产出 $\frac{1 - 0.8^5}{0.2} \approx 3.36$ ，加速比 $\frac{3.36}{1.4} \approx 2.4$ 倍。

两个重要推论：

1. K 不是越大越好： α^i 衰减很快，而草稿成本随 K 线性增加，存在最优 K （配套实验会扫描出来）。
2. **batch 越大，投机解码越不划算**：它的前提是”验证 K 个 token 和验证 1 个一样快”，而 batch 很大时 Decode 本身已接近拐点（第 8 篇），多验证的 token 不再免费。所以投机解码主要用于**低延迟、小 batch** 的场景。

11.5 [启蒙] 五、演进

方法	草稿从哪来
Blockwise Parallel Decoding (2018)	模型自己加多个预测头（最早的思想）
Speculative Decoding / Sampling (2022/2023)	独立的小模型，并给出无偏性证明
Medusa (2024)	在大模型上加几个轻量解码头，一次猜多个位置，用树形注意力并行验证多条候选
EAGLE (2024)	在 特征层 （倒数第二层隐藏状态）上做自回归草稿，接受率更高
Prompt Lookup / N-gram	直接从提示词里找重复片段当草稿（代码编辑、摘要中效果很好），零成本

11.6 [OK] 本篇自测

1. 写出投机采样单 token 无偏性的完整证明。关键的两步等式各用到了什么？
 2. 接受率 α 与总变差距离是什么关系？
 3. 推导每轮期望产出 token 数 $\frac{1-\alpha^{K+1}}{1-\alpha}$ 。为什么 batch 大时投机解码收益变小？
 4. ”拒绝后直接从 p 重新采样” 为什么是错的？（提示：Lab 10 实验 3）
-

11.7 [资料] 外部资源

投机采样

- Leviathan et al. 2022: [Fast Inference from Transformers via Speculative Decoding](#); Chen et al. 2023: [Accelerating LLM Decoding with Speculative Sampling](#)（两篇独立提出，证明方式与本篇一致）
 - Stern et al. 2018: [Blockwise Parallel Decoding](#); Cai et al. 2024: [Medusa](#); Li et al. 2024: [EAGLE](#)
-

上一篇: 第 9 篇: FlashAttention: Online Softmax 的严格推导与 IO 复杂度 | 下一篇: 第 11 篇: 量化: 从每位 6 dB 到 SmoothQuant、AWQ 与 GPTQ | 总路线: LEARNING_PATH.md

第 12 章 量化：从每位 6 dB 到 SmoothQuant、AWQ 与 GPTQ

核心数学定理推导

阅读前须知：Lab 11 给出了量化公式与分组量化的仿真，但没有讲：大模型为什么**特别难**量化？什么时候该量化权重、什么时候该量化激活？GPTQ、AWQ、SmoothQuant 分别用什么数学手段解决问题？

前置：第 3 篇（矩阵乘的梯度与形状）、第 8 篇（访存受限、拐点、浮点格式）、Lab 11 的 `quant_basics.py`。

配套实验：`lab11_quantization/advanced_quant_numpy.py`——量化误差的 6 dB/bit 规律、激活离群值、SmoothQuant、AWQ 缩放、GPTQ 误差补偿，全部用 NumPy 从零实现。

12.1 一、量化误差的基本规律：每多 1 位，信噪比 +6 dB

均匀量化（Lab 11 的公式）把一个区间切成 2^b 格，格宽为 S 。舍入误差近似在 $[-S/2, S/2]$ 上均匀分布，方差为：

$$\sigma_{\text{err}}^2 = \frac{S^2}{12}$$

（均匀分布 $U(-a, a)$ 的方差是 $a^2/3$ ，代入 $a = S/2$ 。）位数每加 1，格宽 S 减半，误差方差变为 $1/4$ ，信噪比提高 $10 \log_{10} 4 \approx 6.02$ dB。INT8 到 INT4 理论上损失约 24 dB——这就是为什么 INT4 必须配合分组量化和下面的算法，而 INT8 往往直接可用。

避坑指南与核心警示

[注意] 这条“6 dB 规律”依赖三个前提：(1) 量化区间固定（这里由最大绝对值决定，不随位数变）；(2) **高分辨率近似**——格子足够细，每个格子里数据的分布近似平坦，误差才近似均匀分布；(3) 信号功率不变。位数很低时前提 (2) 不再成立：高斯分布的数据集中在 0 附近，几个粗格子根本覆盖不了分布形状，误差不再是均匀的。所以 Lab 11 实测（高斯数据、最大绝对值定标）里，高位宽时每位约 +6.1 dB，INT3→INT4 却有 +7.35 dB，INT8→INT4 共 25.2 dB 而非 24 dB。

误差方差正比于 S^2 ，而 S 由该组数据的**最大绝对值**决定：一个离群值会拉大整组的 S ，让组里其它正常值的精度全部变差。这是理解后面所有算法的钥匙。

12.2 [瓶颈] 二、大模型为什么难量化：激活离群值

Dettmers 等 (LLM.int8(), 2022) 发现：模型规模超过约 6.7B 后，**激活值**里会出现“涌现的离群特征”——少数几个**固定的通道（隐藏维度）**，数值比其它通道大几十倍，而且几乎在每个 token 上都出现。

- 如果对激活做 per-tensor 量化，整个张量的 S 被这几个通道决定，其它通道几乎全被舍入成 0；
- 而**权重**的分布通常比较规整，容易量化。

LLM.int8() 的解法是**混合精度分解**：把离群通道拿出来用 FP16 算，其余用 INT8 算。效果好但实现复杂、速度不理想，于是有了下面的方法。

12.2.1 先想清楚：你要量化什么？

第 8 篇的 Roofline 决定了答案：

方案	量化对象	加速了什么	代表
W4A16 / W8A16 (仅权重)	权重存成低位，计算前反量化成 BF16	Decode (访存受限, 少读字节 = 加速)	GPTQ、AWQ
W8A8 / FP8 (权重 + 激活)	两者都低精度，直接用 INT8 / FP8 Tensor Core	Prefill 与大 batch (算力受限，低精度算力翻倍)	SmoothQuant、FP8
KV Cache 量化	K、V 存成 FP8 / INT8 / INT4	长上下文 Decode(读 KV 的字节减少)，能放更多请求	FP8 KV、KIVI

12.3 [基础] 三、SmoothQuant：把激活的难度”转移”给权重

对线性层 $Y = XW$ ($X: [T, C_{in}]$, $W: [C_{in}, C_{out}]$)，对每个输入通道 j 选一个缩放因子 $s_j > 0$ ：

$$Y = XW = (X \operatorname{diag}(s)^{-1})(\operatorname{diag}(s)W) = \hat{X}\hat{W}$$

这是严格的数学恒等式。选择

$$s_j = \frac{\max |X_{:,j}|^\alpha}{\max |W_{j,:}|^{1-\alpha}}, \quad \alpha \approx 0.5$$

离群通道的激活被除以一个大数、变得温和；对应的权重行被乘以同一个数、变大一些。两边的”量化难度”被拉平，两者都能用 INT8 表示。

为什么没有额外开销？ X 来自前面的 RMSNorm, $\operatorname{diag}(s)^{-1}$ 可以直接合并进 RMSNorm 的权重 γ ; $\operatorname{diag}(s)$ 合并进 W 。推理时的计算图完全没有变化。

12.4 [目标] 四、AWQ：保护 1% 的关键权重

AWQ 的观察：权重里只有很少一部分（约 1%）是”关键”的——它们对应的输入激活很大，舍入误差会被放大。把这 1% 保留为 FP16 效果很好，但混合精度对硬件不友好。**替代方案：把关键通道的权重放大 s 倍再量化，同时把对应激活缩小 s 倍**（与 SmoothQuant 同样的恒等变换）：

$$Q(w \cdot s) \cdot \frac{x}{s}$$

量化的绝对误差上界由格宽 S 决定，大致不变（只要这个通道不是组内最大值， S 就不变），但它乘的激活缩小到了 $1/s$ ，所以**这个通道贡献的输出误差约缩小为 $1/s$** 。AWQ 用少量校准数据，在 $s = (\text{激活平均幅度})^\beta$ 这一族里网格搜索 $\beta \in [0, 1]$ ，选输出误差最小的那个。它不需要反向传播、不依赖校准数据的具体内容，泛化性好，是目前 W4A16 最常用的方法之一。

12.5 [工具] 五、GPTQ：用二阶信息补偿舍入误差

12.5.1 先看一个只有两个权重的例子

设某个输出 $y = w_1x_1 + w_2x_2$ ，权重 $w_1 = 0.34$ ， $w_2 = 0.10$ ，量化格宽 $S = 0.25$ （只能取 0、 ± 0.25 、 ± 0.5）。再假设两个输入总是相等： $x_1 = x_2 = x$ （极端的”相关”）。真实输出 $y = 0.44x$ 。

- RTN（各自四舍五入）： $w_1 \rightarrow 0.25$ ， $w_2 \rightarrow 0$ ，输出 $0.25x$ ，误差 $0.19x$ 。
- GPTQ 的做法：先量化 $w_1 \rightarrow 0.25$ ，误差 0.09 ；因为 $x_1 = x_2$ ，把这 0.09 加到还没量化的 w_2 上， $w_2 = 0.19$ ；再量化 $w_2 \rightarrow 0.25$ ，输出 $0.5x$ ，误差 $0.06x$ 。
- 单看权重，GPTQ 把 w_2 从 0.10 改成了 0.25 ，改得更多；但输出误差从 $0.19x$ 降到 $0.06x$ 。下面的公式就是把”该往哪列补、补多少”在一般相关结构下算到最优。

12.5.2 目标

逐层求解：让量化后的层在校准数据 X 上输出尽可能不变（下面把权重写成 $[C_{out}, C_{in}]$ ， X 为 $[C_{in}, n]$ ）：

$$\min_{\hat{W}} \|WX - \hat{W}X\|_F^2$$

这是一个关于 $\hat{W}$ 的二次函数，它的 Hessian 是 $H = 2XX^T$ （ $[C_{in}, C_{in}]$ ，每一行输出共享同一个 H ）。

12.5.3 核心思想（来自 1993 年的 Optimal Brain Surgeon）

一个一个地量化输入通道（列）。量化第 q 列时产生误差 $w_q - \text{quant}(w_q)$ ；因为输入通道之间是相关的，可以调整还没量化的其它列来补偿这个误差。最优补偿量为：

$$\delta = -\frac{w_q - \text{quant}(w_q)}{[H^{-1}]_{qq}} \cdot [H^{-1}]_{q,:}$$

然后把第 q 列从问题中移除（对 H^{-1} 做一步高斯消元），继续下一列。

直觉：如果第 3 个输入通道和第 7 个输入通道高度相关，第 3 列舍入多了一点，就让第 7 列少一点，输出几乎不变。**RTN**（直接四舍五入）每一列各自为战；**GPTQ** 让后面的列替前面的列“还债”。

12.5.4 让它能用在 175B 模型上的工程技巧

GPTQ 论文的贡献在于把 OBS 做得足够快：所有行共享同一个 H^{-1} 、按固定顺序量化、批量延迟更新、用 Cholesky 分解保证数值稳定。175B 模型在单卡上约 4 小时量化完。

12.6 [OK] 六、小结：选哪个？

场景	推荐
单卡本地部署、batch 小（例如你的 5060 Ti）	W4A16: AWQ / GPTQ / llama.cpp 的 GGUF Q4_K 等
数据中心、大 batch、H100 / Blackwell	FP8 (W8A8)，可加 FP8 KV Cache
超长上下文	在权重量化基础上加 KV Cache 量化

评估量化好坏：先看 PPL 变化（第 3 篇），再看下游任务准确率——PPL 对小损失不够敏感。

12.7 [OK] 本篇自测

1. 为什么每多 1 位信噪比提高约 6 dB？
2. Decode 为什么适合“只量化权重”，而 Prefill 需要“权重 + 激活都量化”才能加速？
3. 为什么说 SmoothQuant 的缩放“没有推理开销”？
4. AWQ 放大关键通道的权重，为什么反而减小了误差？
5. GPTQ 相比 RTN，利用了输入通道之间的什么性质？为什么它的权重误差更大、输出误差却更小？

12.8 [资料] 外部资源

- Maarten Grootendorst: [A Visual Guide to Quantization](#)（图解入门，先看这个）

- Dettmers et al. 2022: [LLM.int8\(\)](#) (离群值的发现)
- Xiao et al. 2022: [SmoothQuant](#)
- Lin et al. 2023: [AWQ](#)
- Frantar et al. 2022: [GPTQ](#); 以及它的前身 [Optimal Brain Compression](#)
- Liu et al. 2024: [KIVI \(KV Cache 2-bit 量化\)](#)

上一篇: 第 10 篇: 投机解码: 无偏性证明与期望加速比 | 下一篇: 第 12 篇: 推理服务系统: PagedAttention、前缀缓存与调度 | 总路线: [LEARNING_PATH.md](#)

第 13 章 推理服务系统：PagedAttention、前缀缓存与调度

核心导读与背景

阅读前须知：前面九篇都在讲”一个请求怎么算得又对又快”。真实的推理服务同时面对成百上千个请求：它们长短不一、随时到达、随时结束，还常常共享同样的系统提示词。本篇讲推理引擎（vLLM、SGLang、TensorRT-LLM）在**系统层面**解决的问题：怎么衡量好坏、显存怎么管、请求怎么排。

前置：第 6 篇（KV Cache 显存公式）、Lab 12a（连续批处理仿真）、第 8 篇（Decode 耗时模型 $T_{\text{step}}(B)$ ）、第 9 篇（Online Softmax 可分块计算）。

配套实验：`lab12b_paged_attention/paged_kv_cache_sim.py`——实现块分配器、块表、分页注意力、写时复制、前缀缓存与 Chunked Prefill 的仿真。

13.1 一、先定义”好”：推理服务的指标体系

指标	含义	用户感受	主要受谁影响
TTFT (Time To First Token)	从请求到达，到第一个 token 返回	” 它开始回答了吗”	排队时间 + Prefill 时间
TPOT / ITL (Time Per Output Token / Inter-Token Latency)	相邻两个输出 token 的间隔	” 打字速度够不够流畅”	每一步 Decode 的耗时
端到端延迟	$TTFT + TPOT \times (n_{\text{out}} - 1)$	总等待时间	两者之和
吞吐 (Throughput)	每秒生成的总 token 数 / 完成的请求数	老板关心的成本	batch 大小、显存利用率
Goodput	满足延迟目标(SLO, 如 $TTFT < 1s$ 且 $TPOT < 50ms$) 的请求吞吐	真正有意义的吞吐	以上全部

工业界通常看 **P50 / P99 分位数**，而不是平均值——一次偶发的 2 秒卡顿会被用户记住。

13.1.1 Little 定律：并发数从哪来

排队论的 Little 定律：**系统中平均请求数 = 到达率 × 平均停留时间**， $L = \lambda W$ 。例：每秒到达 10 个请求，每个请求平均要生成 10 秒 → 系统里同时约有 **100 个请求** 在跑。每个请求上下文 2K token、每 token KV 128 KB (Llama-3-8B)，光 KV Cache 就要 $100 \times 2048 \times 128\text{KB} \approx 25\text{GB}$ 。**并发数由业务决定，显存必须装得下它**——这就是为什么 KV Cache 的显存管理是推理系统的核心问题。

13.2 [反向] 二、迭代级调度回顾：为什么连续批处理成立

Lab 12a 已经仿真过：静态批处理要等整批最长的请求结束，连续批处理在**每一步**都可以让结束的请求离开、新请求加入 (Orca, 2022)。它能成立的物理基础是第 8 篇的推导：**Decode 线性层的算术强度 $\approx$ batch size**，在 batch 远小于拐点时，一步的耗时几乎与 batch 无关。所以”多塞一个请求进这一步”几乎是免费的。

但连续批处理立刻带来两个新问题：

1. 请求随时进出，KV Cache 的显存怎么分配和回收？（第三 ~ 五节）
 2. 新请求的 Prefill 很重，插进正在 Decode 的批次里会怎样？（第六节）
-

13.3 [模块] 三、KV Cache 的显存碎片：PagedAttention 要解决的问题

13.3.1 传统做法：按最大长度预留连续显存

请求到达时，并不知道它最终会生成多少 token，于是只能按 `max_len`（例如 2048）预留一整块连续显存：

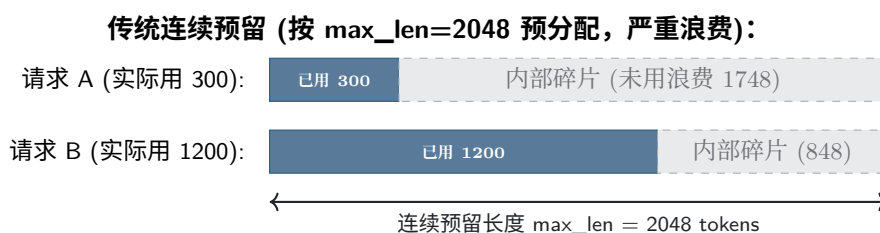


图 13.1: 传统连续显存分配的内部碎片问题

- **内部碎片**：预留了却没用上的部分；
- **外部碎片**：不同大小的连续块反复分配释放后，空闲显存被切成零碎的小段，总量够但凑不出一整块。

vLLM 论文测量发现，当时的系统里 KV Cache 显存只有 **20%~38% 真正存了 token**。

13.3.2 PagedAttention：把操作系统的虚拟内存搬过来

操作系统早在几十年前就解决过同样的问题——分页：

操作系统	PagedAttention
进程	请求（序列）
虚拟页	逻辑块（第 0 块 = 第 0~15 个 token）
物理页框	显存里的物理块（每块存 16 个 token 的 K、V）
页表	块表（Block Table）：逻辑块号 → 物理块号
缺页时分配	写满一块时才分配下一块

请求 ID	逻辑块到物理块映射表（Block Table）
请求 A	逻辑块 0 → 物理块 7, 逻辑块 1 → 物理块 2, 逻辑块 2 → 物理块 9
请求 B	逻辑块 0 → 物理块 4, 逻辑块 1 → 物理块 0, 逻辑块 2 → 物理块 5

核心优势	按需分配固定大小的物理块（如 16 tokens），物理显存无需连续，内部碎片近乎为零！
------	--

- 每个请求最多只浪费**最后一个块里没写满的部分**（< 16 个 token）。注意这只是上界： $\frac{15}{S} < 4\%$ 要求序列长度 $S \gtrsim 400$ token；短对话（ $S = 100$ 时最坏 15%）远达不到这个数；
- 没有外部碎片：所有块一样大，任何空闲块都能用；
- 结果：同样的显存能容纳 **2~4 倍** 的并发请求，吞吐相应提升。

13.3.3 代价：注意力 kernel 要“查表读 KV”

KV 不再连续，注意力计算时要按块表逐块读取——而第 9 篇已经证明，Online Softmax 让注意力可以**逐块累加**，所以分页几乎不影响计算效率。这是 PagedAttention 在数学上可行的原因。

13.3.4 显存不够时怎么办：抢占

请求在运行中不断变长，块可能被分完。此时调度器**抢占**一部分请求：

- **换出（Swap）**：把它的 KV 块拷到 CPU 内存，之后再换回；
- **重算（Recompute）**：直接丢掉它的 KV，之后重新 Prefill（Prefill 是算力受限的，对短序列往往比经 PCIe 拷贝更快）。

13.4 四、共享：引用计数与写时复制

同一个提示词需要生成多个回答时（并行采样 $n > 1$ 、束搜索），它们的提示词 KV **完全相同**。有了块表，共享变得非常自然：

- 多个序列的块表指向**同一组物理块**，每个物理块维护一个**引用计数**；

- 当某个序列要往一个被共享且未写满的块里追加 token 时，先复制一份给自己再写（写时复制，Copy-on-Write）——这同样是操作系统 fork() 的经典技术。

并行采样 4 个回答时，提示词部分的 KV 只存 1 份而不是 4 份。

13.5 五、前缀缓存：跨请求复用 KV

很多请求的开头是一样的：

- 所有请求共享同一段几千 token 的系统提示词；
- 多轮对话中，第 k 轮的输入 = 前 $k - 1$ 轮的全部内容 + 新问题；
- Few-shot 示例、RAG 中被反复检索到的同一篇文档、Agent 的工具描述。

这些前缀的 KV 完全相同（同样的 token、同样的位置 → 同样的 K、V，RoPE 也一样）。前缀缓存让请求结束后 KV 块不立即释放，而是留着等下一个相同前缀的请求复用：

- vLLM 的自动前缀缓存：对每个写满的块，用“它自己的 token + 它之前所有 token”算一个哈希值作为键；新请求逐块查哈希，命中就直接复用物理块；
- SGLang 的 RadixAttention：用基数树（Radix Tree）组织所有缓存的前缀，按 LRU 淘汰。

收益：命中的部分完全跳过 Prefill。系统提示词 4000 token、用户问题 50 token 时，TTFT 可能降低一个数量级。

避坑指南与核心警示

[注意] 前缀必须逐 token 完全相同且从第一个 token 开始。把会变化的内容（例如当前时间）放在系统提示词开头，会让整个缓存失效——这是写提示词时要注意的工程细节。

13.6 六、Prefill 与 Decode 的冲突：Chunked Prefill 与 P/D 分离

13.6.1 问题：一个长 Prefill 会让所有人卡住

连续批处理把新请求的 Prefill 和其他请求的 Decode 放在同一步里。第 8 篇告诉我们：

- Decode 一步耗时 $\approx$ 读一遍权重的时间（例如 10ms）；
- 一个 8000 token 的 Prefill 是算力受限的，可能要几百毫秒。

它们在同一步里，这一步的耗时就由 Prefill 决定，正在 Decode 的几十个用户会同时看到一次几百毫秒的“卡顿”（TPOT 尖刺）。

13.6.2 Chunked Prefill (Sarathi-Serve, 2024)

把长 Prefill 切成若干块（例如每块 512 token），每一步只做一块，和其他请求的 Decode 拼在一起。每一步设一个 token 预算：

- 每一步的耗时有了上界，Decode 用户不再卡顿；
- 而且 Decode 本来算力闲置（访存受限），顺便塞进一段 Prefill，正好把闲置的算力用上——两种瓶颈互补。

13.6.3 P/D 分离 (DistServe、Mooncake, 2024)

更彻底的做法：Prefill 和 Decode 放到不同的 GPU 上，Prefill 算完后把 KV Cache 通过高速网络传给 Decode 节点。

- 理由正是第 8 篇：两者的瓶颈完全相反（算力 vs 带宽），硬件配置和并行策略可以分别优化；
- 代价：KV Cache 需要跨机传输，对网络带宽要求高。

13.7 七、一个现代推理引擎里还有什么

技术	解决的问题	本仓库对应
CUDA Graph	Decode 一步要启动几百个小 kernel，每个都有微秒级开销 (Lab 07b 实测只跑到带宽下界的一成左右，约 7%~14%)	Lab 07b 练习 5
融合的注意力 kernel (FlashAttention / FlashInfer)	第 9 篇	Lab 09
量化（权重 / KV Cache）	显存与带宽	Lab 11、第 11 篇
投机解码	Decode 串行、算力闲置	Lab 10、第 10 篇
张量并行 / 专家并行	单卡放不下	第 13 篇、Lab 13
结构化输出 (JSON Schema 约束解码)	让输出满足格式	第 7 篇的延伸：对 logits 做掩码
多 LoRA 服务	一个基座模型服务多个微调版本	选读 S-LoRA

13.8 [OK] 本篇自测

1. TTFT 和 TPOT 分别由什么决定？为什么服务要看 P99 而不只看平均？
 2. 用 Little 定律估算：每秒 5 个请求、每个请求平均耗时 20 秒、平均上下文 4K、每 token KV 128 KB，需要多少 KV Cache 显存？
 3. 预留连续显存为什么会浪费？PagedAttention 每个请求最多浪费多少？
 4. 为什么说 Online Softmax（第 9 篇）是 PagedAttention 的数学前提？
 5. 写时复制在什么时候触发？
 6. 为什么把”当前时间”写在系统提示词开头会破坏前缀缓存？
 7. 一个长 Prefill 为什么会让所有正在 Decode 的请求卡顿？Chunked Prefill 如何解决？为什么说它让”两种瓶颈互补”？
-

13.9 [资料] 外部资源

- Kwon et al. 2023: [Efficient Memory Management for LLM Serving with PagedAttention](#) (vLLM 论文, 必读), 以及 [vLLM 博客](#)
 - Yu et al. 2022: [Orca: A Distributed Serving System for Transformer-Based Generative Models](#) (OSDI'22, 迭代级调度)
 - Anyscale 博客: [How continuous batching enables 23x throughput in LLM inference](#)
 - Zheng et al. 2023: [SGLang \(RadixAttention\)](#)
 - Agrawal et al. 2023/2024: [Sarathi](#)、[Sarathi-Serve \(Chunked Prefill\)](#)
 - Zhong et al. 2024: [DistServe \(P/D 分离\)](#); Qin et al. 2024: [Mooncake](#)
 - vLLM 官方文档: [docs.vllm.ai](#) 中的 Design / Architecture 部分
 - Lilian Weng: [Large Transformer Model Inference Optimization](#) (推理优化综述)
-

上一篇: 第 11 篇: 量化: 从每位 6 dB 到 SmoothQuant、AWQ 与 GPTQ | 下一篇: 第 13 篇: 多卡并行推理: 张量并行、流水线并行与专家并行 | 总路线: [LEARNING_PATH.md](#)

第 14 章 多卡并行推理：张量并行、流水线并行与专家并行

核心导读与背景

阅读前须知：一张 16GB 的卡能跑 7B（量化后），但 70B 的 BF16 权重就要 140GB，DeepSeek-V3 更是 671B。模型放不下一张卡时，必须把它切开放到多张卡上。怎么切、切完之后卡与卡之间要传什么、传多少——这些决定了多卡推理的速度。

前置：第 3 篇（矩阵乘的形状）、第 5 篇（MoE）、第 8 篇（带宽与耗时模型）、第 9 篇（Online Softmax 合并）。

配套实验：`lab13_parallelism/tensor_parallel_sim.py`——用 NumPy 模拟多卡：张量并行的 MLP 与注意力、错误切法的反例、Ring All-Reduce、多卡 Decode 延迟模型。

14.1 一、先认识“通信原语”和它们的代价

多卡之间交换数据的几种标准操作（NCCL 库实现）：

原语	做什么	典型用途
All-Reduce	每张卡有一个同形状张量，结果是 所有卡的和 ，每张卡都拿到	张量并行合并部分和
All-Gather	每张卡有一块，结果是 拼接后 的完整张量	拼回被切开的激活
Reduce-Scatter	先求和，再把结果切块，每张卡拿一块	All-Reduce 的前半段
All-to-All	每张卡把自己的数据切成 n 份分别发给每张卡	MoE 把 token 发给对应专家
Send / Recv	点对点	流水线并行传激活

Ring All-Reduce 的代价： n 张卡连成环，先做 Reduce-Scatter 再做 All-Gather，每张卡发送的数据量为

$$2 \cdot \frac{n-1}{n} \cdot \text{数据大小}$$

几乎与卡数无关——这是它被广泛采用的原因。但每一次通信还有一个**固定延迟**（几微秒到几十微秒），数据很小时，延迟占主导。

互联带宽差距巨大：

互联	单向带宽量级
NVLink (H100, 同机 8 卡)	约 450 GB/s
PCIe 4.0 x16 / 5.0 x16	约 25 / 50 GB/s
InfiniBand 400 Gb/s (跨机)	约 50 GB/s

所以”切法”必须和”连接方式”匹配：通信频繁的切法只能用在 NVLink 连接的同一台机器内。

14.2 二、张量并行（Tensor Parallelism, TP）：把每一层的矩阵切开

Megatron-LM (2019) 给出了 Transformer 最经典的切法。以 FFN 为例： $Y = f(XA)B$, f 是逐元素的非线性（GeLU / SiLU）。

14.2.1 第一个矩阵按列切

把 A 按列切成 $[A_1, A_2]$, 卡 i 持有 A_i ：

$$XA = [XA_1, XA_2] \Rightarrow f(XA) = [f(XA_1), f(XA_2)]$$

因为 f 是逐元素的，每张卡可以独立地对自己那半算非线性，不需要任何通信。

14.2.2 第二个矩阵按行切

把 B 按行切成 $\begin{bmatrix} B_1 \\ B_2 \end{bmatrix}$, 卡 i 持有 B_i ：

$$Y = [f(XA_1), f(XA_2)] \begin{bmatrix} B_1 \\ B_2 \end{bmatrix} = f(XA_1)B_1 + f(XA_2)B_2$$

每张卡算出一个部分和，最后做一次 All-Reduce 求和。

14.2.3 为什么不能反过来？

如果 A 按行切（即切分输入维度），每张卡得到的是 $X_i A_i$ 这样的部分和，而

$$f(X_1 A_1 + X_2 A_2) \neq f(X_1 A_1) + f(X_2 A_2)$$

非线性函数对加法不分配，必须先 All-Reduce 求和才能做 f ——多了一次通信。”先列后行”让两次矩阵乘之间的非线性完全在本地完成，每个子层只通信一次。

SwiGLU 同理： W_{gate} 、 W_{up} 按列切， W_{down} 按行切（ $\text{SiLU}(\cdot) \odot (\cdot)$ 是逐元素的）。

14.2.4 注意力：按“头”切

多个注意力头本来就互相独立，这恰好就是“按列切”： W_q, W_k, W_v 按头分给不同的卡（列切），每张卡独立算自己那几个头的注意力， W_o 按行切，最后一次 All-Reduce。附带好处：每张卡只存自己那些头的 KV Cache，KV 显存也被均分。（GQA 的 KV 头数少于 TP 卡数时，KV 头需要复制到多张卡上。）

14.2.5 代价与收益

- **每层 2 次 All-Reduce**（注意力后一次、FFN 后一次），数据量是 $[\text{token 数}, d_{\text{model}}]$ 。
- **Decode 收益**：每张卡只读 $1/n$ 的权重，第 8 篇的访存时间直接除以 n ——TP 能降低单请求延迟，这是它在推理中最重要价值。
- **Decode 代价**：每步 $2L$ 次 All-Reduce，每次数据量很小（ $\text{batch} \times 8192 \times 2$ 字节），延迟主导。在 PCIe 上，通信时间可能超过省下的计算时间，TP 加卡反而变慢（配套实验会算给你看）。

14.3 [硬件] 三、流水线并行（Pipeline Parallelism, PP）：按层切

把 80 层分成 4 段，每张卡放 20 层，激活依次在卡之间传递。

- **通信少**：每段边界只传一次激活（点对点），适合跨机器；
- **不降低单请求延迟**：一个 token 仍要依次穿过所有层，第 1 张卡算完第 2 张卡才能开始；
- **气泡**：如果只有一个请求，任一时刻只有一张卡在工作。推理服务里有大量请求时，可以把不同请求（micro-batch）错开塞进流水线，让每张卡都忙起来——PP 提升吞吐，不提升延迟。

llama.cpp 在多张消费级显卡之间“按层分配”，本质上就是流水线式切分。

14.4 [人员][实验] 四、专家并行（Expert Parallelism, EP）：MoE 的切法

MoE 层有几十上百个专家（第 5 篇），把不同的专家放在不同的卡上：

1. 路由器算出每个 token 要去哪 k 个专家；
2. **All-to-All**：把 token 发到对应专家所在的卡；
3. 各卡计算自己的专家；
4. 再一次 **All-to-All** 把结果发回原卡，按路由权重加权求和。

难点：

- **负载均衡**: 热门专家所在的卡会成为瓶颈, 训练时要加负载均衡损失或偏置调整 (DeepSeek-V3 使用无辅助损失的偏置调整);
- **All-to-All 通信量大**, 需要高带宽网络; DeepSeek 为此专门开发了通信库 DeepEP。

14.5 [并行] 五、其它并行方式

- **数据并行 (DP)**: 每张卡放一份完整模型, 各自服务不同请求。最简单, 吞吐线性扩展, 但每张卡都要放得下整个模型。
- **序列 / 上下文并行**: 超长上下文 (百万 token) 时, 把一个序列的 KV 切到多张卡上。**Ring Attention** 让 KV 块在卡之间环形传递, 每张卡用 **Online Softmax** 的合并公式 (第 9 篇 §4) 累加部分结果——又一次用到了那个满足结合律的合并操作。

14.6 [指南] 六、怎么组合？一些经验法则

场景	常见方案
单机多卡 (NVLink), 模型放不下一张卡	机内 TP
跨多台机器的超大模型	机内 TP + 机间 PP
MoE 大模型 (DeepSeek-V3 等)	专家用 EP, 注意力用 DP 或 TP
消费级多卡 (PCIe, 无 NVLink)	优先 PP / 按层切分 (通信少), 或者量化后单卡
吞吐优先、单卡放得下	多副本 DP

14.7 [OK] 本篇自测

1. Ring All-Reduce 每张卡发送多少数据? 为什么说它”几乎与卡数无关”?
2. 为什么 FFN 的第一个矩阵按列切、第二个按行切? 反过来会怎样?
3. 张量并行时每层需要几次 All-Reduce? 数据量多大?
4. 为什么 TP 能降低 Decode 延迟而 PP 不能?
5. 在两张通过 PCIe 连接的 5060 Ti 上跑 TP=2, Decode 一定比单卡快吗? 用第 8 篇的耗时模型加上通信项分析。

6. Ring Attention 和第 9 篇的哪个公式有关?

14.8 [资料] 外部资源

- Shoeybi et al. 2019: [Megatron-LM](#) (张量并行的原始论文, 第 3 节图 3 就是本篇第二节)
- Pope et al. 2022: [Efficiently Scaling Transformer Inference](#) (专门讲推理的多卡切分与延迟/成本权衡, 强烈推荐)
- Google DeepMind: [How to Scale Your Model](#) 第 3 章 (Sharding) 与第 7~8 章 (推理)
- Hugging Face: [The Ultra-Scale Playbook](#) (训练视角, 但通信原语与并行方式讲得非常直观)
- Huang et al. 2018: [GPipe](#) (流水线并行); Liu et al. 2023: [Ring Attention](#)
- DeepSeek-AI 2024: [DeepSeek-V3 技术报告](#) (EP、负载均衡、部署方案)
- [NCCL 文档](#): 各通信原语的定义

上一篇: 第 12 篇: 推理服务系统: PagedAttention、前缀缓存与调度 | 总路线: LEARNING_PATH.md

第二部分

工业实战与实验源码精解篇 (Practical Labs & Implementations)

第 15 章 从零手写自动微分与反向传播引擎 (Micrograd)

15.1 Lab 03: 从零手写自动求导 (Autograd)

核心导读与背景

对应理论：第 3 篇：神经网络如何学习：导数、链式法则、反向传播与交叉熵前置：读完第 3 篇第一 ~ 六节。依赖：纯 Python 标准库。

15.2 为什么要自己写一遍？

PyTorch 的 `loss.backward()` 是整个深度学习最大的”黑盒”。只要你亲手写过一次，就会明白：

- 反向传播 = 链式法则 + 拓扑排序 + 梯度累加，没有任何魔法；
- 为什么每一步训练前必须 `zero_grad()`（梯度是 += 累加的）；
- 为什么训练要保存激活值（`_backward` 闭包里用到了前向时的 `self.data`）；
- 为什么推理时可以用 `torch.no_grad()` / `torch.inference_mode()` 省掉大量显存（不建图、不存激活）。

15.3 运行

```
1 python3 lab03_autograd_from_scratch/micrograd_from_scratch.py
```

实验	验证的结论	对应章节
1	手算表格 $dL/dw = -12$, $dL/db = -6$	第 3 篇 §2.3
2	自动求导 = 数值差分 (梯度检验)	第 3 篇 §3
3	Softmax 雅可比 $p_i(\delta_{ij} - p_j)$; 交叉熵梯度 $p - y$; logits 放大后梯度坍塌	第 3 篇 §6、第 4 篇 §3
4	$\partial L / \partial W = X^T G$, $\partial L / \partial X = G W^T$	第 3 篇 §4
5	Sigmoid 连乘梯度消失 vs 残差畅通	第 3 篇 §7、第 4 篇 §5
6	单层学不会 XOR, 两层可以	第 1 篇 §4.2、第 0 篇族谱

15.4 动手练习（做完才算过关）

1. 给 Value 加一个 `silu()` 算子 ($x\sigma(x)$), 自己推导导数, 并用实验 2 的方法做梯度检验。
2. 把实验 6 的激活函数从 `tanh` 换成 `sigmoid`, 学习率不变, 观察收敛速度变化, 并用第 3 篇 §7 的导数上界解释。
3. 把实验 5 的深度改成 30, 看纯 Sigmoid 的梯度变成多少。
4. (进阶) 跟着 [Karpathy 的 micrograd 视频](#) 实现 `Neuron` / `Layer` / `MLP` 类, 并训练一个二分类的”月牙”数据集。

15.5 学完之后

你已经可以放心使用 `torch.autograd` 了——它做的事和这 350 行完全一样, 只是算子换成了张量、实现换成了 C++/CUDA。下一步: 第 5 篇 → Lab 05: 亲手训练一个迷你 GPT。

15.6 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码, 重点实现细节如下:

15.6.1 源码实现: `micrograd_from_scratch.py`

Listing 15.1: `micrograd_from_scratch.py`

```

1 """
2 Lab 03: 从零手写自动求导引擎 (Autograd), 并用数值差分验证第 3 篇的每一个公式 (零外部依赖)
3
4 对应教程: zero_to_hero_tutorial/03_神经网络如何学习: 导数、链式法则、反向传播与交叉熵.md

```

```

5 思路来自 Karpathy 的 micrograd (https://github.com/karpathy/micrograd), 这里精简并加上了中文注释与验证实验。
6
7 实验清单:
8   1. 复现第 3 篇"手算表格":  $w=3, x=2, b=1, y=10$  时的梯度  $dL/dw = -12, dL/db = -6$ 
9   2. 梯度检验: 自动求导 vs 中心差分数值导数 (验证你写的反向传播是对的)
10  3. 验证 Softmax 雅可比  $p_i(\delta_{ij} - p_j)$  与 "Softmax+交叉熵梯度 =  $p - y$ "
11  4. 验证矩阵乘反向传播公式  $dL/dW = X^T G, dL/dX = G W^T$ 
12  5. 梯度消失: 多层 Sigmoid 连乘 vs 加上残差连接
13  6. 历史重演: 单层感知机学不会 XOR, 加一层隐藏层 + 非线性后学会了
14  """
15
16 import math
17 import random
18
19
20 # =====
21 # 核心: 一个标量节点 Value, 记录"值 + 梯度 + 它是怎么算出来的"
22 # =====
23 class Value:
24     def __init__(self, data, _children=(), _op=""):
25         self.data = float(data)
26         self.grad = 0.0 # dL/d(本节点), 反向传播时累加
27         self._backward = lambda: None # 把本节点的梯度分发给输入节点的函数
28         self._prev = set(_children)
29         self._op = _op
30
31     # ----- 基本运算: 每个运算都要写"局部导数 × 上游梯度" -----
32     def __add__(self, other):
33         other = other if isinstance(other, Value) else Value(other)
34         out = Value(self.data + other.data, (self, other), "+")
35
36         def _backward():
37             #  $d(a+b)/da = 1, d(a+b)/db = 1$ 。用 += 是因为一个节点可能被多条路径使用 (多路
            径相加)
38             self.grad += 1.0 * out.grad
39             other.grad += 1.0 * out.grad
40             out._backward = _backward
41         return out
42
43     def __mul__(self, other):
44         other = other if isinstance(other, Value) else Value(other)
45         out = Value(self.data * other.data, (self, other), "*")
46
47         def _backward():
48             #  $d(ab)/da = b, d(ab)/db = a$ 
49             self.grad += other.data * out.grad
50             other.grad += self.data * out.grad

```

```

51     out._backward = _backward
52     return out
53
54     def __pow__(self, k):
55         assert isinstance(k, (int, float))
56         out = Value(self.data ** k, (self,), f"**{k}")
57
58         def _backward():
59             self.grad += k * (self.data ** (k - 1)) * out.grad
60         out._backward = _backward
61         return out
62
63     def exp(self):
64         out = Value(math.exp(self.data), (self,), "exp")
65
66         def _backward():
67             self.grad += out.data * out.grad # (e^x)' = e^x
68         out._backward = _backward
69         return out
70
71     def log(self):
72         out = Value(math.log(self.data), (self,), "log")
73
74         def _backward():
75             self.grad += (1.0 / self.data) * out.grad
76         out._backward = _backward
77         return out
78
79     def tanh(self):
80         t = math.tanh(self.data)
81         out = Value(t, (self,), "tanh")
82
83         def _backward():
84             self.grad += (1 - t * t) * out.grad
85         out._backward = _backward
86         return out
87
88     def sigmoid(self):
89         s = 1.0 / (1.0 + math.exp(-self.data))
90         out = Value(s, (self,), "sigmoid")
91
92         def _backward():
93             self.grad += s * (1 - s) * out.grad # 上界 1/4
94         out._backward = _backward
95         return out
96
97     def relu(self):
98         out = Value(max(0.0, self.data), (self,), "relu")

```

```

99
100     def _backward():
101         self.grad += (1.0 if self.data > 0 else 0.0) * out.grad
102     out._backward = _backward
103     return out
104
105     # ----- 反向传播: 拓扑排序后, 从输出往输入逐个调用 _backward -----
106     def backward(self):
107         topo, visited = [], set()
108
109         def build(v):
110             if v not in visited:
111                 visited.add(v)
112                 for child in v._prev:
113                     build(child)
114                 topo.append(v)
115         build(self)
116
117         self.grad = 1.0 # dL/dL = 1
118         for v in reversed(topo):
119             v._backward()
120
121     # ----- 语法糖 -----
122     def __neg__(self): return self * -1
123     def __sub__(self, other): return self + (-other)
124     def __rsub__(self, other): return other + (-self)
125     def __radd__(self, other): return self + other
126     def __rmul__(self, other): return self * other
127     def __truediv__(self, other): return self * (other ** -1 if isinstance(other,
Value) else 1.0 / other)
128     def __rtruediv__(self, other): return other * self ** -1
129     def __repr__(self): return f"Value(data={self.data:.6f}, grad={self.grad:.6f})"
130
131
132     def title(s):
133         print("\n" + "=" * 74)
134         print(f"[实验] {s}")
135         print("=" * 74)
136
137
138     def numeric_grad(f, xs, i, h=1e-5):
139         """中心差分: (f(x+h) - f(x-h)) / 2h, 只用来"检验", 不用来训练"""
140         xp = list(xs); xp[i] += h
141         xm = list(xs); xm[i] -= h
142         return (f(xp) - f(xm)) / (2 * h)
143
144

```

```

145 # =====
146 # 实验 1: 复现第 3 篇手算表格
147 # =====
148 def exp1_hand_table():
149     title("实验 1: 复现第 3 篇的手算反向传播表格")
150     x, w, b, y = Value(2.0), Value(3.0), Value(1.0), Value(10.0)
151     u = w * x
152     y_hat = u + b
153     e = y_hat - y
154     L = e ** 2
155     L.backward()
156     print(f"    前向: u = {u.data}, y_hat = {y_hat.data}, e = {e.data}, L = {L.data}")
157     print(f"    反向: dL/de = {e.grad}, dL/dy_hat = {y_hat.grad}, dL/du = {u.grad}")
158     print(f"          dL/dw = {w.grad}    (表格中手算: -12)")
159     print(f"          dL/db = {b.grad}    (表格中手算: -6)")
160     assert (w.grad, b.grad) == (-12.0, -6.0)
161     lr = 0.01
162     print(f"    一步梯度下降 ( $\eta=\{lr\}$ ): w: 3  $\rightarrow$  {3 - lr * w.grad:.2f}, b: 1  $\rightarrow$  {1 - lr * b.grad:.2f}, "
163           f"    新预测 = {(3 - lr * w.grad) * 2 + (1 - lr * b.grad):.2f} (向真实值 10 靠近) ")
164
165
166 # =====
167 # 实验 2: 梯度检验
168 # =====
169 def exp2_gradcheck():
170     title("实验 2: 梯度检验 —— 自动求导 vs 数值差分")
171
172     def f_value(v):
173         a, b, c = v
174         return ((a * b).tanh() + (c ** 2) * a.exp() - (b / c).log() * 0.5).sigmoid()
175
176     def f_float(xs):
177         a, b, c = xs
178         return 1 / (1 + math.exp(-(math.tanh(a * b) + c ** 2 * math.exp(a) - 0.5 * math.log(b / c))))
179
180     xs = [0.3, 1.7, 0.9]
181     vs = [Value(x) for x in xs]
182     out = f_value(vs)
183     out.backward()
184     print(f"    f(a,b,c) = sigmoid( tanh(a·b) + c²·e^a - 0.5·ln(b/c) )")
185     for i, name in enumerate("abc"):
186         ng = numeric_grad(f_float, xs, i)

```

```

187         rel = abs(vs[i].grad - ng) / max(1e-12, abs(ng))
188         print(f"     $\partial f / \partial \{name\}$ : 自动求导 = {vs[i].grad:+.8f} | 数值差分 = {ng:+.8f} | 相对误差 = {rel:.1e}")
189         assert rel < 1e-6, f" $\partial f / \partial \{name\}$  梯度检验失败"
190     print("    --> 相对误差远小于 1e-6: 说明每个算子的 _backward 都写对了。以后你写任何新算子, 都应该这样检验。")
191
192
193 # =====
194 # 实验 3: Softmax 雅可比 & Softmax + 交叉熵
195 # =====
196 def softmax_values(zs):
197     m = max(z.data for z in zs)                # 平移不变性, 防溢出 (常数不参与求导)
198     exps = [(z - m).exp() for z in zs]
199     s = sum(exps, Value(0.0))
200     return [e / s for e in exps]
201
202
203 def exp3_softmax_ce():
204     title("实验 3: Softmax 雅可比  $p_i(\delta_{ij} - p_j)$  与交叉熵梯度  $p - y$ ")
205     z_raw = [2.0, 1.0, 0.1, -1.5]
206
207     # (a) 对  $p_0$  反向传播, 得到雅可比矩阵的第 0 行  $\partial p_0 / \partial z_j$ 
208     zs = [Value(z) for z in z_raw]
209     ps = softmax_values(zs)
210     ps[0].backward()
211     p = [q.data for q in ps]
212     print(f"    z = {z_raw}")
213     print(f"    p = softmax(z) = {[round(q, 4) for q in p]}")
214     print("    雅可比第 0 行  $\partial p_0 / \partial z_j$ : ")
215     for j in range(len(z_raw)):
216         formula = p[0] * ((1.0 if j == 0 else 0.0) - p[j])
217         print(f"        j={j}: 自动求导 = {zs[j].grad:+.6f} | 公式  $p_0(\delta_{0j} - p_j) = \{formula:+.6f\}$ ")
218         assert abs(zs[j].grad - formula) < 1e-12
219
220     # (b) 交叉熵: 真实类别  $y = 2$ 
221     y = 2
222     zs = [Value(z) for z in z_raw]
223     ps = softmax_values(zs)
224     loss = -ps[y].log()
225     loss.backward()
226     print(f"\n    真实类别  $y = \{y\}$ , 交叉熵  $L = -\log p_y = \{loss.data:.4f\}$ ")
227     for j in range(len(z_raw)):
228         formula = p[j] - (1.0 if j == y else 0.0)
229         print(f"         $\partial L / \partial z_{\{j\}}$ : 自动求导 = {zs[j].grad:+.6f} | 公式  $p_j - 1[j=y] = \{formula:+.6f\}$ ")
230         assert abs(zs[j].grad - formula) < 1e-12

```

```

231     print("    --> 梯度 = 预测分布 - 真实分布。正确类别的 logit 被推高，其余被压低，力度正比
232         于它们各自的概率。")
233
234     # (c) 饱和现象: logits 放大 10 倍后, Softmax 雅可比数值全体变小
235     zs = [Value(z * 10) for z in z_raw]
236     ps = softmax_values(zs)
237     ps[0].backward()
238     print(f"\n    把 logits 放大 10 倍 (模拟没有除以  $\sqrt{d_k}$ ) 后,  $\partial p_0 / \partial z_j = \{[f'\{z.\text{grad} : +.2e\}' \text{ for } z \text{ in } zs]\}$ ")
239     print("    -->  $p_0 \approx 1$ , 雅可比整体坍缩到接近 0 —— 这就是第 4 篇所说的 Softmax 饱和导致梯度消失。")
240
241 # =====
242 # 实验 4: 矩阵乘的反向传播公式
243 # =====
244 def exp4_matmul_backward():
245     title("实验 4: 验证矩阵乘反向传播公式  $dL/dW = X^T G$ ,  $dL/dX = G W^T$ ")
246     rng = random.Random(0)
247     B, K, N = 2, 3, 2
248     X = [[Value(rng.uniform(-1, 1)) for _ in range(K)] for _ in range(B)]
249     W = [[Value(rng.uniform(-1, 1)) for _ in range(N)] for _ in range(K)]
250     Y = [[sum((X[i][k] * W[k][j] for k in range(K)), Value(0.0)) for j in range(N))
251          ] for i in range(B)]
252     # 随便定义一个标量损失:  $L = \sum c_{ij} * Y_{ij}$ , 于是  $G = \partial L / \partial Y = c$ 
253     C = [[rng.uniform(-1, 1) for _ in range(N)] for _ in range(B)]
254     L = sum((Y[i][j] * C[i][j] for i in range(B) for j in range(N)), Value(0.0))
255     L.backward()
256
257     G = C
258     dW_formula = [[sum(X[i][k].data * G[i][j] for i in range(B)) for j in range(N))
259                   ] for k in range(K)]
260     dX_formula = [[sum(G[i][j] * W[k][j].data for j in range(N)) for k in range(K))
261                   ] for i in range(B)]
262     err_w = max(abs(W[k][j].grad - dW_formula[k][j]) for k in range(K) for j in range(N))
263     err_x = max(abs(X[i][k].grad - dX_formula[i][k]) for i in range(B) for k in range(K))
264     print(f"    X: [{B},{K}], W: [{K},{N}], Y = XW: [{B},{N}]")
265     print(f"    dL/dW 自动求导 vs  $X^T G$  最大误差: {err_w:.1e}")
266     print(f"    dL/dX 自动求导 vs  $G W^T$  最大误差: {err_x:.1e}")
267     assert err_w < 1e-12 and err_x < 1e-12
268     print("    --> 反向传播需要两个矩阵乘 (各与前向同样大小), 所以反向  $\approx 2$  倍前向 FLOPs, 训练  $\approx 6N$ 、推理  $\approx 2N$ 。")
269
270 # =====
271 # 实验 5: 梯度消失 & 残差

```

```

270 # =====
271 def exp5_vanishing():
272     depth = 10
273     title(f"实验 5: 梯度消失 —— {depth} 层 Sigmoid 连乘 vs 加上残差连接")
274
275     x = Value(0.5)
276     h = x
277     for _ in range(depth):
278         h = (h * 1.0).sigmoid()
279     h.backward()
280     print(f"    纯 Sigmoid 堆 {depth} 层: dOut/dx = {x.grad:.3e}    (每层导数  $\leq 1/4$ ,  $(1/4)^{\{depth\}} \approx \{0.25 ** depth:.1e\})")
281     assert x.grad <= 0.25 ** depth, "Sigmoid 连乘的梯度不应超过  $(1/4)^{depth}$ "
282
283     x = Value(0.5)
284     h = x
285     for _ in range(depth):
286         h = h + (h * 1.0).sigmoid() # 残差:  $x + F(x)$ 
287     h.backward()
288     print(f"    加残差后 {depth} 层:          dOut/dx = {x.grad:.3e}    (每层导数  $= 1 + F'(x) \geq 1$ , 梯度一路畅通)")
289     assert x.grad >= 1.0, "有残差时梯度不应小于 1"
290
291 # =====
292 # 实验 6: 历史重演 —— XOR
293 # =====
294 def train_xor(hidden):
295     rng = random.Random(42)
296     data = [( [0.0, 0.0], 0.0), ([0.0, 1.0], 1.0), ([1.0, 0.0], 1.0), ([1.0, 1.0], 0.0) ]
297
298     if hidden == 0:
299         # 单层:  $y = \text{sigmoid}(w \cdot x + b)$ , 只能画一条直线
300         W1 = [Value(rng.uniform(-1, 1)) for _ in range(2)]
301         b1 = Value(0.0)
302         params = W1 + [b1]
303
304         def forward(x):
305             return (W1[0] * x[0] + W1[1] * x[1] + b1).sigmoid()
306     else:
307         W1 = [[Value(rng.uniform(-1, 1)) for _ in range(hidden)] for _ in range(2)]
308
309         b1 = [Value(0.0) for _ in range(hidden)]
310         W2 = [Value(rng.uniform(-1, 1)) for _ in range(hidden)]
311         b2 = Value(0.0)
312         params = [w for row in W1 for w in row] + b1 + W2 + [b2]
313$ 
```

```

314     def forward(x):
315         hs = [(W1[0][j] * x[0] + W1[1][j] * x[1] + b1[j]).tanh() for j in
range(hidden)]
316         return (sum((hs[j] * W2[j] for j in range(hidden)), Value(0.0)) + b2).
sigmoid()
317
318     lr = 0.5
319     for step in range(2000):
320         # 二分类交叉熵:  $-[y \log p + (1-y) \log(1-p)]$ 
321         loss = Value(0.0)
322         for x, y in data:
323             p = forward(x)
324             loss = loss - (p.log() * y + (1 - p).log() * (1 - y))
325         loss = loss / len(data)
326         for q in params:
327             q.grad = 0.0 # [注意] 梯度是累加的, 每步必须清零 (PyTorch 里的
optimizer.zero_grad())
328         loss.backward()
329         for q in params:
330             q.data -= lr * q.grad
331     preds = [forward(x).data for x, _ in data]
332     return loss.data, preds
333
334
335 def exp6_xor():
336     title("实验 6: 历史重演 —— 1969 年感知机学不会 XOR, 1986 年多层网络 + 反向传播学会了"
)
337     for hidden in [0, 4]:
338         loss, preds = train_xor(hidden)
339         if hidden == 0:
340             assert abs(loss - 0.6931) < 0.01, "单层模型在 XOR 上应停在 ln2"
341         else:
342             assert loss < 0.05 and [round(p) for p in preds] == [0, 1, 1, 0], "两
层网络应学会 XOR"
343         name = "单层 (无隐藏层)" if hidden == 0 else f"两层 (隐藏层 {hidden} 个 tanh
神经元)"
344         print(f"    {name:<28}: 训练后损失 = {loss:.4f} | 对 (00,01,10,11) 的预测 = {[
round(p, 3) for p in preds]} (目标 0,1,1,0)")
345         print("    --> 单层模型损失卡在 ln2 ≈ 0.693 (等于瞎猜): 一条直线分不开 XOR。")
346         print("    加一层非线性隐藏层后损失趋近 0 —— 这就是第 1 篇说的'激活函数让网络能折出曲
面'。")
347
348
349 if __name__ == "__main__":
350     exp1_hand_table()
351     exp2_gradcheck()
352     exp3_softmax_ce()
353     exp4_matmul_backward()

```

```
354     exp5_vanishing()  
355     exp6_xor()
```

第 16 章 Transformer 推理显微镜：逐层张量变化与 RoPE 实测

16.1 Lab 04：大模型推理的微观全貌（零起点极细拆解）

如果你对「大模型推理到底在算什么」、「张量每一步怎么变」感到模糊，本模块就是为你量身打造的显微镜。

无论模型是 0.5B、7B 还是 70B，无论叫 Llama-3、Qwen2.5 还是 DeepSeek，其自回归推理的微观数学本质完全一致。

16.2 [指南] 1. 宏观全貌：从一句话到下一个字的 8 个步骤

假设你对模型说：" 人工智能"，模型要推理出下一个字 " 的"。整个过程分为严格的 8 步：

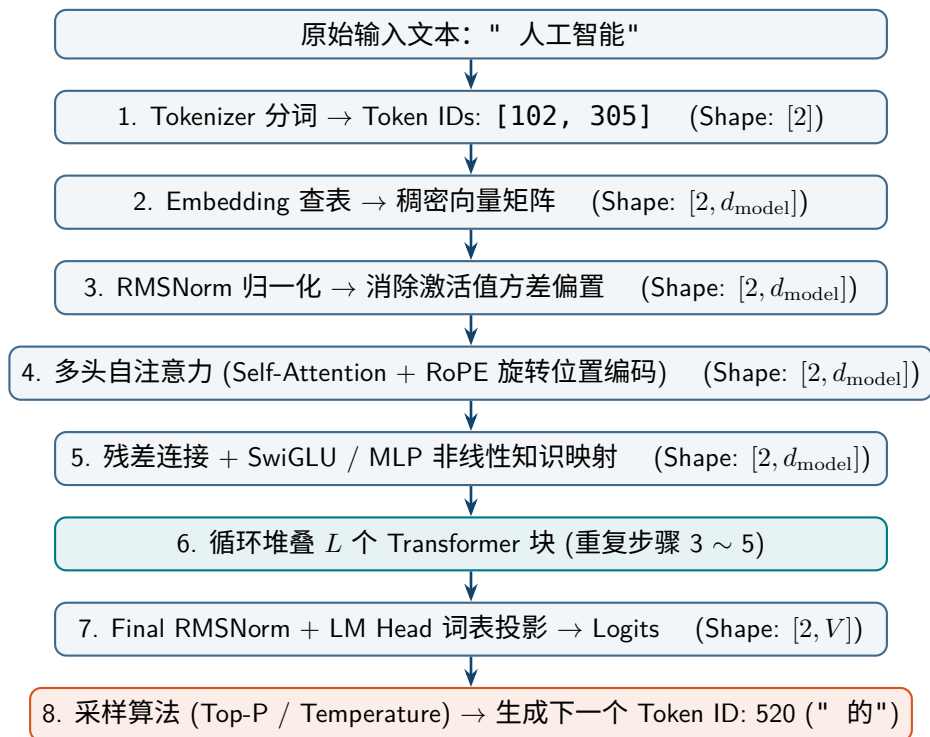


图 16.1: Transformer 单步推理张量生命周期微观全貌

16.3 [观察] 2. 张量形状流 (Tensor Shape Flow Tracker)

理解大模型推理的关键，是死死盯住**每一步张量的形状 (Shape)**。下表用一个**假设的**微型配置来说明形状如何流动（注意：脚本 `01_tensor_trace_step_by_step.py` 为了能把每个数字打印出来，用的是更小的配置： $d_{\text{model}} = 16$ 、2 个头、词表 11 个字、只有 1 层；下面的流程图描述的是真实模型的 L 层结构）。

假设微型模型配置：

- 序列长度 $T = 3$ （例如输入 3 个词）
- 隐藏层维度 $d_{\text{model}} = 64$
- 注意力头数 $n_{\text{heads}} = 4$ ，每个头的维度 $d_{\text{head}} = 64/4 = 16$
- 词表大小 $\text{vocab_size} = 1000$

处理阶段	算子名称	输入形状 (Input Shape)	输出形状 (Output Shape)	物理含义解释
分词	Tokenizer	字符串文本	[3]	文本转整数 ID
词嵌入	Embedding Lookup	[3]	[3, 64]	每个词映射成 64 维空间的一个点
归一化	RMSNorm	[3, 64]	[3, 64]	均方根缩放, 保持方差稳定
QKV 投影	Linear (W_q, W_k, W_v)	[3, 64]	[3, 64] (各 1 个)	生成查询、键、值向量
多头拆分	Reshape & Transpose	[3, 64]	[4, 3, 16]	4 个注意力头并行关注不同维度的特征
位置编码	RoPE (旋转编码)	[4, 3, 16]	[4, 3, 16]	对 Q 和 K 按位置角频率旋转, 注入顺序感
注意力打分	$Q \cdot K^T / \sqrt{d_k}$	Q:[4, 3, 16], K:[4, 3, 16]	[4, 3, 3]	第 i 个词与第 j 个词的关联度矩阵
因果掩码	Causal Mask	[4, 3, 3]	[4, 3, 3]	将右上角置为 $-\infty$ (禁止看到未来的词)
注意力归一	Softmax	[4, 3, 3]	[4, 3, 3]	转换成概率百分比分布 (每一行和为 1)
值加权	Score $\cdot V$	Scores:[4, 3, 3], V:[4, 3, 16]	[4, 3, 16]	提取汇聚上下文信息
多头合并	Transpose & Reshape	[4, 3, 16]	[3, 64]	拼接 4 个头的特征
输出投影	Linear (W_o)	[3, 64]	[3, 64]	特征融合
残差连接	$X + \text{Attn}(X)$	[3, 64] + [3, 64]	[3, 64]	保留原始输入, 防止梯度/特征退化
前馈网络	SwiGLU / MLP	[3, 64]	[3, 64]	升维到例如 172 维激活后再降维回 64 维
残差连接	$X + \text{FFN}(X)$	[3, 64] + [3, 64]	[3, 64]	第二层残差
词表投影	LM Head (W_{head})	[3, 64]	[3, 1000]	将每个位置映射为词表中每个词的得分
预测下个词	Slice Last Position	[3, 1000]	[1000]	我们只取最后一个位置 (第 3 个词) 的输出
采样策略	Greedy / Top-P	[1000]	整数 (标量)	采样出第 4 个词的 Token ID!

16.4 [实验] 3. 逐个核心算子的底层细节与数学公式

16.4.1 (1) 为什么注意力机制必须有「因果掩码 (Causal Mask)」?

在文本生成中，我们不能让前面的词“偷看”后面的词。对于序列长度为 3 的输入：

$$\text{Mask} = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

当注意力分数矩阵与该 Mask 相加后，在执行 Softmax 时：

$$e^{-\infty} = 0$$

被掩码的位置权重精确变成 0！这样第 1 个词只能看第 1 个词，第 2 个词只能看第 1 和第 2 个词，保证严格的单向自回归因果性。

16.4.2 (2) RMSNorm（均方根归一化）到底算什么？

现代大模型 (LLaMA/Qwen/Mistral) 舍弃了传统的 LayerNorm，改用更高效的 RMSNorm：

- 不需要计算均值 μ ，不需要减均值；
- 仅计算向量元素的均方根 (Root Mean Square)：

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

- 归一化公式 (γ 是可学习的缩放向量)：

$$\bar{x}_i = \frac{x_i}{\text{RMS}(x)} \odot \gamma_i$$

计算量少了一半，硬件执行速度更快，数值更平稳。

16.4.3 (3) SwiGLU（门控前馈网络）算什么？

现代模型基本不再使用经典的 $\text{ReLU}(W_1 x) W_2$ ，而是采用 SwiGLU 结构：它包含三个权重矩阵： $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$ ：

$$\text{SwiGLU}(x) = (\text{SiLU}(xW_{\text{gate}}) \otimes (xW_{\text{up}})) W_{\text{down}}$$

- xW_{gate} 通过 SiLU 激活函数充当“开关门 (Gate)”，控制哪些特征信息可以通过。

- 与 xW_{up} 做逐元素乘法 (Hadamard product $\otimes$)。
- 最终经过 W_{down} 降维回隐藏层维度。

16.4.4 (4) 采样：从 Logits 到文字

最后一步”从 Logits 选出下一个 token”单独放在 **Lab 07a**，配合第 7 篇学习。

16.5 [执行] 运行显微镜级追踪脚本

本目录下提供了可以直接运行的 Python 代码：

1. 01_tensor_trace_step_by_step.py:

单步前向传播显微镜。完整实现了一个微型 Transformer，单步逐行打印所有输入/输出张量 Shape、注意力矩阵数值、Logits 排行榜以及生成循环。

1. 02_rope_and_attention_microscope.py:

位置编码与因果掩码显微镜。无位置编码时的置换等变性、RoPE 的相对位置性质、长程衰减的正确表述（只在 q、k 相关时的期望上成立）、因果掩码。

对应理论：第 4 篇。依赖：纯 Python 标准库。

16.6 [链接] 说明与下一步

- 脚本 01 的”Decode”步骤为了看清张量，故意把整段序列重算一遍（没有 KV Cache）；权重是随机的，所以输出没有语义。
- 脚本 02 已修正：旧版把”点积交换律”当成了”没有语序感”的证明，并声称单一频率的 RoPE 有长程衰减（实际上单一频率下点积是 $\cos(\Delta\theta)$ ，是周期函数）。新版演示了真正的置换等变性；长程衰减实验对比了 q、k 完全相关 / 部分相关 / 独立随机三种情况——只有相关时点积的期望才随距离衰减，独立随机时完全不衰减。
- 想看一个真正训练过的模型，请继续做 **Lab 05** 与 **Lab 07b**。完整顺序见 **LEARNING_PATH.md**。

16.7 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码，重点实现细节如下：

16.7.1 源码实现：01_tensor_trace_step_by_step.py

Listing 16.1: 01_tensor_trace_step_by_step.py

```

1  """
2  Lab 04 - 脚本 1: 大模型推理前向传播「显微镜级」单步跟踪器 (Zero-Dependency Pure Python)
3
4  本脚本为你彻底揭开大模型推理的黑盒:
5  输入一段极小文本: ["大", "模", "型"] (Token IDs: [5, 6, 7])
6  逐行打印每个算子的张量形状 (Shape)、物理含义、因果注意力权重图与 Logits 排行榜。
7  包含 Prefill 首字阶段与 Decode 增量阶段的真实计算过程。
8  """
9
10 import math
11 import random
12
13
14 # =====
15 # 1. 微型分词器与词表 (Micro Tokenizer & Vocab)
16 # =====
17 VOCAB = {
18     0: "<pad>",
19     1: "我",
20     2: "爱",
21     3: "学",
22     4: "习",
23     5: "大",
24     6: "模",
25     7: "型",
26     8: "推",
27     9: "理",
28     10: "<eos>"
29 }
30 WORD2ID = {v: k for k, v in VOCAB.items()}
31 VOCAB_SIZE = len(VOCAB)
32
33 # 模型超参数 (超微型配置, 方便逐个数字打印查看)
34 D_MODEL = 16          # 隐藏层维度
35 N_HEADS = 2           # 注意力头数
36 D_HEAD = D_MODEL // N_HEADS # 每个头的维度: 16 // 2 = 8
37 HIDDEN_DIM = 32       # FFN 中间层维度
38
39
40 # =====
41 # 2. 基础张量算子实现 (手写纯 Python 矩阵运算, 无任何黑盒)
42 # =====
43 def create_random_matrix(rows, cols, seed):
44     rng = random.Random(seed)
45     return [[rng.gauss(0, 0.2) for _ in range(cols)] for _ in range(rows)]
46

```

```

47 def matmul_2d(A, B):
48     """[N, K] x [K, M] -> [N, M]"""
49     n, k = len(A), len(A[0])
50     m = len(B[0])
51     res = [[0.0] * m for _ in range(n)]
52     for i in range(n):
53         for p in range(k):
54             val = A[i][p]
55             for j in range(m):
56                 res[i][j] += val * B[p][j]
57     return res
58
59 def rms_norm(X, weight, eps=1e-5):
60     """
61     RMSNorm: 均方根归一化
62     X: [seq_len, d_model]
63     weight: [d_model]
64     """
65     out = []
66     for row in X:
67         mean_square = sum(x * x for x in row) / len(row)
68         scale = 1.0 / math.sqrt(mean_square + eps)
69         normed_row = [x * scale * w for x, w in zip(row, weight)]
70         out.append(normed_row)
71     return out
72
73 def silu(x):
74     """SiLU (Swish) 激活函数:  $x * \text{sigmoid}(x)$ """
75     return x / (1.0 + math.exp(-max(-20.0, min(20.0, x))))
76
77 def apply_rope(vec, pos, base=10000.0):
78     """
79     RoPE: 把 vec 的第 i 对分量 (vec[2i], vec[2i+1]) 旋转角度  $\text{pos} * \text{base}^{(-2i/d)}$ 
80     推导见 zero_to_hero_tutorial 第 4 篇第六节; 数值性质见 02
81     _rope_and_attention_microscope.py
82     """
83     d = len(vec)
84     out = [0.0] * d
85     for i in range(d // 2):
86         theta = pos * base ** (-2.0 * i / d)
87         c, s = math.cos(theta), math.sin(theta)
88         out[2 * i] = vec[2 * i] * c - vec[2 * i + 1] * s
89         out[2 * i + 1] = vec[2 * i] * s + vec[2 * i + 1] * c
90     return out
91
92 def softmax(row):
93     """防溢出 Softmax"""
94     max_val = max(row)

```

```

94     exp_vals = [math.exp(v - max_val) for v in row]
95     sum_exp = sum(exp_vals)
96     return [v / sum_exp for v in exp_vals]
97
98
99 # =====
100 # 3. 显微镜跟踪类
101 # =====
102 class MicroLlamaTracer:
103     def __init__(self):
104         # 权重初始化
105         self.embed_table = create_random_matrix(VOCAB_SIZE, D_MODEL, seed=1) #
[11, 16]
106         self.norm1_w = [1.0] * D_MODEL
107         self.w_q = create_random_matrix(D_MODEL, D_MODEL, seed=2) # [16, 16]
108         self.w_k = create_random_matrix(D_MODEL, D_MODEL, seed=3)
109         self.w_v = create_random_matrix(D_MODEL, D_MODEL, seed=4)
110         self.w_o = create_random_matrix(D_MODEL, D_MODEL, seed=5)
111
112         self.norm2_w = [1.0] * D_MODEL
113         # SwiGLU 权重
114         self.w_gate = create_random_matrix(D_MODEL, HIDDEN_DIM, seed=6)
115         self.w_up = create_random_matrix(D_MODEL, HIDDEN_DIM, seed=7)
116         self.w_down = create_random_matrix(HIDDEN_DIM, D_MODEL, seed=8)
117
118         self.final_norm_w = [1.0] * D_MODEL
119         self.lm_head = create_random_matrix(D_MODEL, VOCAB_SIZE, seed=9) # [16,
120 11]
121
122     def forward_trace(self, token_ids, phase_name="Prefill"):
123         T = len(token_ids)
124         tokens_str = [VOCAB[tid] for tid in token_ids]
125
126         print("\n" + "=" * 76)
127         print(f"[实验] [{phase_name} 阶段] 显微镜单步追踪")
128         print(f" 输入文本: {' + '.join(tokens_str)} (Token IDs: {token_ids})")
129         print(f"[几何] 序列长度 T = {T} | 隐藏层维度 D = {D_MODEL} | 头数 Heads = {
N_HEADS} (单头维度 {D_HEAD})")
130         print("=" * 76)
131
132         # -----
133         # Step 1: Embedding 查表
134         # -----
135         x = [self.embed_table[tid][:] for tid in token_ids] # [T, D_MODEL]
136         print(f"\n[Step 1] Embedding 词嵌入查表:")
137         print(f"      输入形状: [{T}] --> 输出形状: [{T}, {D_MODEL}]" )
138         print(f"      说明: 每一个整数 Token ID 被映射为 {D_MODEL} 维稠密特征向量。")
139         print(f"      示例 (Token '{tokens_str[0]}' 前 4 维特征): {[round(v, 3)

```

```

139     for v in x[0][:4]]...)
140
141     # -----
142     # Step 2: 第 1 层 RMSNorm
143     # -----
144     x_norm1 = rms_norm(x, self.norm1_w)
145     print(f"\n[Step 2] Attention 前 RMSNorm 归一化:")
146     print(f"        输入形状: [{T}], {D_MODEL}] --> 输出形状: [{T}], {D_MODEL}]")
147
148     print(f"        说明: 消除特征幅值异常波动, 稳定注意力输入方差。")
149
150     # -----
151     # Step 3: Q, K, V 投影矩阵乘法
152     # -----
153     Q = matmul_2d(x_norm1, self.w_q) # [T, D_MODEL]
154     K = matmul_2d(x_norm1, self.w_k) # [T, D_MODEL]
155     V = matmul_2d(x_norm1, self.w_v) # [T, D_MODEL]
156     print(f"\n[Step 3] Q, K, V 投影 (Linear Projections):")
157     print(f"        Q 形状: [{T}], {D_MODEL}] | K 形状: [{T}], {D_MODEL}] | V 形")
158     print(f"状: [{T}], {D_MODEL}]")
159     print(f"        物理含义:")
160     print(f"        - Query (Q): 当前位置想要'询问'什么? ")
161     print(f"        - Key   (K): 当前位置拥有什么'检索特征'? ")
162     print(f"        - Value (V): 当前位置真正携带的'内容信息'。")
163
164     # -----
165     # Step 4: 多头注意力拆分与因果注意力打分
166     # -----
167     print(f"\n[Step 4] 多头注意力计算: RoPE 旋转 Q/K + 因果掩码 (Causal Attention) :")
168     print(f"        每个头取出 [{T}], {D_HEAD}] 的 Q_h/K_h, 按位置 0..{T-1} 旋转后")
169     print(f"再做  $Q_h \cdot K_h^T / \sqrt{D\_HEAD}$ ")
170     # 针对每个 Head 独立计算
171     head_outputs = []
172     for h in range(N_HEADS):
173         col_start = h * D_HEAD
174         col_end = (h + 1) * D_HEAD
175         # 提取属于头 h 的子矩阵 [T, D_HEAD]
176         Q_h = [[row[c] for c in range(col_start, col_end)] for row in Q]
177         K_h = [[row[c] for c in range(col_start, col_end)] for row in K]
178         V_h = [[row[c] for c in range(col_start, col_end)] for row in V]
179
180         # RoPE: 只旋转 Q 和 K (V 不旋转), 第 i 个位置旋转  $i * \theta$ 
181         Q_h = [apply_rope(Q_h[i], i) for i in range(T)]
182         K_h = [apply_rope(K_h[i], i) for i in range(T)]
183
184         # 计算注意力分数:  $Q_h \cdot K_h^T / \sqrt{D\_HEAD}$ 
185         scale = math.sqrt(D_HEAD)

```

```

182     attn_scores = [[0.0] * T for _ in range(T)]
183     for i in range(T):
184         for j in range(T):
185             dot = sum(Q_h[i][d] * K_h[j][d] for d in range(D_HEAD))
186             attn_scores[i][j] = dot / scale
187
188     # 注入因果掩码 (Causal Masking): j > i 的未来位置设为 -inf
189     for i in range(T):
190         for j in range(T):
191             if j > i:
192                 attn_scores[i][j] = -1e9 # 代表 -inf
193
194     # Softmax 归一化
195     attn_weights = [softmax(row) for row in attn_scores]
196     assert all(attn_weights[i][j] == 0.0 for i in range(T) for j in range(
197 i + 1, T)), "因果掩码失效: 看到了未来"
198     assert all(abs(sum(row) - 1) < 1e-12 for row in attn_weights)
199
200     if h == 0:
201         print(f"                【头 0 的注意力热力权重矩阵 (Attention Map)】 (行=当前
202 词, 列=被关注词):")
203         header = f"                {'':<6} | " + " | ".join([f"{tokens_str[c]:^6}"
204 " for c in range(T)])
205         print(header)
206         print("                " + "-" * len(header))
207         for r in range(T):
208             row_str = f"                {tokens_str[r]:<6} | " + " | ".join([f"{
209 attn_weights[r][c]*100:5.1f}%" for c in range(T)])
210             print(row_str)
211             print(f"                --> 观察: 右上角全部为 0.0%, 严格杜绝向后偷看未来! ")
212
213     # 加权求和 context = attn_weights * V_h -> [T, D_HEAD]
214     context_h = matmul_2d(attn_weights, V_h)
215     head_outputs.append(context_h)
216
217     # -----
218     # Step 5: 多头拼接与 Output 投影
219     # -----
220     concat_context = []
221     for i in range(T):
222         row = []
223         for h in range(N_HEADS):
224             row.extend(head_outputs[h][i])
225         concat_context.append(row) # [T, D_MODEL]
226
227     attn_out = matmul_2d(concat_context, self.w_o) # [T, D_MODEL]
228     print(f"\n[Step 5] 多头特征拼接与输出线性映射 (Out Projection):")
229     print(f"                多头拼接形状: [{T}, {D_MODEL}] --> w_o 投影输出: [{T}, {

```

```

D_MODEL}])")
226
227     # -----
228     # Step 6: 第 1 层残差连接
229     # -----
230     x_residual1 = [[x[i][d] + attn_out[i][d] for d in range(D_MODEL)] for i in
range(T)]
231     print(f"\n[Step 6] 残差连接 1 (Residual Connection):")
232     print(f"        公式:  $X_{new} = X + \text{Attention}(X)$  --> 形状: [{T}, {D_MODEL}]")
233     print(f"        说明: 保持深层网络信息畅通传递, 防止表征退化。")
234
235     # -----
236     # Step 7: 第 2 层 RMSNorm + SwiGLU 前馈网络 (MLP)
237     # -----
238     x_norm2 = rms_norm(x_residual1, self.norm2_w)
239     gate = matmul_2d(x_norm2, self.w_gate) # [T, HIDDEN_DIM]
240     up    = matmul_2d(x_norm2, self.w_up)   # [T, HIDDEN_DIM]
241
242     # SwiGLU:  $\text{silu}(\text{gate}) * \text{up}$ 
243     swiglu_act = []
244     for i in range(T):
245         swiglu_row = [silu(gate[i][k]) * up[i][k] for k in range(HIDDEN_DIM)]
246         swiglu_act.append(swiglu_row)
247
248     ffn_out = matmul_2d(swiglu_act, self.w_down) # [T, D_MODEL]
249     x_residual2 = [[x_residual1[i][d] + ffn_out[i][d] for d in range(D_MODEL)]
for i in range(T)]
250     print(f"\n[Step 7] SwiGLU 门控前馈网络 (MLP) 与残差连接 2:")
251     print(f"        升维中间层形状: [{T}, {HIDDEN_DIM}] --> 降维输出形状: [{T}, {D_MODEL}]")
252     print(f"        公式:  $X_{final} = X_{res1} + (\text{SiLU}(X \cdot W_{gate}) \odot (X \cdot W_{up})) \cdot W_{down}$ ")
253
254     # -----
255     # Step 8: Final RMSNorm + LM Head 投影到词表
256     # -----
257     x_final = rms_norm(x_residual2, self.final_norm_w)
258     logits_all = matmul_2d(x_final, self.lm_head) # [T, VOCAB_SIZE]
259     print(f"\n[Step 8] 最终归一化与 LM Head (输出投影):")
260     print(f"        输入形状: [{T}, {D_MODEL}] --> 输出 Logits 形状: [{T}, {VOCAB_SIZE}]")
261     print(f"        说明: 每个位置都产生了一个大小为 {VOCAB_SIZE} 的未归一化打分向量!")
262
263     # -----
264     # Step 9: 提取最后一个位置做预测与采样
265     # -----

```

```

266     last_logits = logits_all[-1] # [VOCAB_SIZE]
267     last_probs = softmax(last_logits)
268
269     # 排序选出前 3 名最高可能性的词
270     ranked_indices = sorted(range(VOCAB_SIZE), key=lambda idx: last_probs[idx], reverse=True)
271
272     print(f"\n[Step 9] 预测下一个词 (针对最后一个 Token '{tokens_str[-1]}'):")
273     print(f"        候选词概率分布排行榜 (Top 3):")
274     for rank, idx in enumerate(ranked_indices[:3]):
275         print(f"        [第一阶段] 第 {rank+1} 名: Token '{VOCAB[idx]}' (ID={idx:<2d}) | 原始 Logit: {last_logits[idx]:6.2f} | 概率: {last_probs[idx]*100:5.2f}%")
276
277     best_token_id = ranked_indices[0]
278     print(f"\n[目标] 贪婪解码决策 (Greedy Choice): 采纳 Token '{VOCAB[best_token_id]}' (ID={best_token_id})")
279     return best_token_id
280
281
282 def run_micro_trace():
283     tracer = MicroLlamaTracer()
284
285     # Prefill 阶段: 输入初始 Prompt "大" "模" "型"
286     prompt_ids = [WORD2ID["大"], WORD2ID["模"], WORD2ID["型"]]
287     next_token_1 = tracer.forward_trace(prompt_ids, phase_name="Prefill (首字全量计算)")
288
289     # Decode 阶段 1: 将新生成的 token 拼接入序列, 预测后续词
290     # 注意: 这里为了看清楚张量, 故意"把整段序列重新算一遍" (没有 KV Cache)。
291     # 真正的推理引擎在 Decode 时只输入 1 个新 token, 并复用缓存的 K/V —— 见 lab06_kv_cache/
292     # 以及 lab07b_qwen_from_scratch/ 中对真实模型的实现。
293     full_context_step1 = prompt_ids + [next_token_1]
294     next_token_2 = tracer.forward_trace(full_context_step1, phase_name="Decode Step 1 (生成第 2 个字, 朴素全量重算版)")
295
296     print("\n" + "=" * 76)
297     print(" 完整生成结果追踪:")
298     all_tokens = prompt_ids + [next_token_1, next_token_2]
299     result_text = "".join([VOCAB[tid] for tid in all_tokens])
300     print(f"    输入提示词 : {''.join([VOCAB[tid] for tid in prompt_ids])}")
301     print(f"    模型输出   : {result_text}")
302     print(f"    [注意] 权重是随机初始化的, 所以输出没有语义。想看一个【真正训练过】的模型如何推理, ")
303     print(f"    请跑 lab05_train_tiny_llama/ (自己训练) 和 lab07b_qwen_from_scratch/ (加载 Qwen2.5 真实权重)。")
304     print("=" * 76)

```

```

305
306
307 if __name__ == "__main__":
308     run_micro_trace()

```

16.7.2 源码实现：02_rope_and_attention_microscope.py

Listing 16.2: 02_rope_and_attention_microscope.py

```

1  """
2  Lab 04 - 脚本 2: 旋转位置编码 (RoPE) 与自注意力因果掩码极细拆解
3
4  深入解答三个核心疑问:
5  1. 【为什么必须有位置编码?】: 没有位置编码的自注意力是"置换等变"的——
6     把输入顺序打乱, 每个词得到的输出向量完全不变, 只是跟着换了个位置。
7     所以 "猫 吃 鱼" 和 "鱼 吃 猫" 里的 "猫", 模型看到的是同一个东西。
8  2. 【RoPE 怎么旋转向量的?】: 把向量拆成 2 维对, 在复平面按位置  $m$  旋转  $m \cdot \theta_i$ ,
9     旋转后点积只取决于相对距离  $(m - n)$ 。
10 3. 【长程衰减从哪来?】: 单个频率下点积 =  $\cos(\Delta\theta)$ , 是周期函数, 根本不衰减;
11    只有把很多个不同频率  $\theta_i$  叠加起来, 远距离处各分量相位错开、互相抵消, 才出现"平均意义上"
12    的衰减。
13
14  """
15  import math
16  import random
17
18  def print_title(title):
19      print("\n" + "=" * 74)
20      print(f"[实验] {title}")
21      print("=" * 74)
22
23
24  def dot(a, b):
25      return sum(x * y for x, y in zip(a, b))
26
27
28  def softmax(row):
29      m = max(row)
30      e = [math.exp(v - m) for v in row]
31      s = sum(e)
32      return [v / s for v in e]
33
34
35  def self_attention_no_pos(X, Wq, Wk, Wv):
36      """最朴素的单头自注意力（无位置编码、无掩码），X: [T, d]"""
37      def proj(x, W):

```

```

38         return [sum(x[i] * W[i][j] for i in range(len(x))) for j in range(len(W
    [0]))]
39     Q = [proj(x, Wq) for x in X]
40     K = [proj(x, Wk) for x in X]
41     V = [proj(x, Wv) for x in X]
42     d = len(Q[0])
43     out = []
44     for q in Q:
45         w = softmax([dot(q, k) / math.sqrt(d) for k in K])
46         out.append([sum(w[j] * V[j][c] for j in range(len(V))) for c in range(d)])
47     return out
48
49
50 # =====
51 # 1. 揭秘: 没有位置编码时, Attention 为什么是"瞎子"?
52 # =====
53 def demo_permutation_equivariance():
54     print_title("1. 为什么自注意力机制本身'没有语序感'? (置换等变性)")
55
56     rng = random.Random(0)
57     d = 4
58     emb = {w: [rng.gauss(0, 1) for _ in range(d)] for w in ["猫", "吃", "鱼"]}
59     Wq = [[rng.gauss(0, 0.5) for _ in range(d)] for _ in range(d)]
60     Wk = [[rng.gauss(0, 0.5) for _ in range(d)] for _ in range(d)]
61     Wv = [[rng.gauss(0, 0.5) for _ in range(d)] for _ in range(d)]
62
63     s1 = ["猫", "吃", "鱼"]
64     s2 = ["鱼", "吃", "猫"]
65     out1 = self_attention_no_pos([emb[w] for w in s1], Wq, Wk, Wv)
66     out2 = self_attention_no_pos([emb[w] for w in s2], Wq, Wk, Wv)
67
68     print("分别把 '猫 吃 鱼' 与 '鱼 吃 猫' 送入同一个【无位置编码】的自注意力层:")
69     for w in ["猫", "鱼"]:
70         v1 = out1[s1.index(w)]
71         v2 = out2[s2.index(w)]
72         diff = max(abs(a - b) for a, b in zip(v1, v2))
73         print(f"    '{w}' 在句子1中的输出: {[round(v, 4) for v in v1]}")
74         print(f"    '{w}' 在句子2中的输出: {[round(v, 4) for v in v2]}    (最大差值 {
diff:.1e})")
75         assert diff == 0.0, '无位置编码时同一个词的输出应与位置无关'
76
77     print("--> 结论: 同一个词无论在句首还是句尾, 得到的输出向量【完全一样】, 只是换了个位置。")
78
79     print("    原因: 注意力对 K/V 做的是加权求和  $\sum_j w_j v_j$ , 求和与 j 的排列顺序无关。")
80     print("    所以必须显式地给每个 Token 注入位置 m, 否则 '猫吃鱼' 与 '鱼吃猫' 在模型眼里是
    同一个词袋。")
81

```

```

82 # =====
83 # 2. RoPE (Rotary Position Embedding) 旋转实操
84 # =====
85 def rope(x, m, base=10000.0):
86     """
87     对  $d$  维向量  $x$  ( $d$  为偶数) 在位置  $m$  处施加 RoPE:
88     第  $i$  对分量 ( $x[2i]$ ,  $x[2i+1]$ ) 旋转角度  $m * \theta_i$ ,  $\theta_i = \text{base}^{-2i/d}$ 
89     (与 Llama 的实现在"如何配对"上略有不同: HF 实现把前半一半与后半一半配对, 数学性质完全相同)
90     """
91     d = len(x)
92     out = [0.0] * d
93     for i in range(d // 2):
94         theta = base ** (-2.0 * i / d)
95         c, s = math.cos(m * theta), math.sin(m * theta)
96         a, b = x[2 * i], x[2 * i + 1]
97         out[2 * i] = a * c - b * s
98         out[2 * i + 1] = a * s + b * c
99     return out
100
101
102 def demo_rope_relative_property():
103     print_title("2. RoPE: 旋转后点积只取决于相对距离 (m - n)")
104
105     rng = random.Random(1)
106     d = 8
107     q = [rng.gauss(0, 1) for _ in range(d)]
108     k = [rng.gauss(0, 1) for _ in range(d)]
109
110     print(f"随机取一对 {d} 维向量 q、k; 模长 |q| = {math.sqrt(dot(q, q)):.4f}")
111     for m in [0, 5, 100]:
112         print(f"    RoPE(q, m={m:<3d}) 的模长 = {math.sqrt(dot(rope(q, m), rope(q, m))):.4f} (旋转是正交变换, 不改变模长)")
113
114     print("\n固定相对距离  $\Delta = m - n = 3$ , 改变绝对位置:")
115     for n in [0, 10, 1000, 50000]:
116         m = n + 3
117         val = dot(rope(q, m), rope(k, n))
118         print(f"    <RoPE(q, {m:>5d}), RoPE(k, {n:>5d})> = {val:+.6f}")
119         assert abs(val - dot(rope(q, 3), k)) < 1e-9, "RoPE 点积应只依赖相对距离"
120     print("--> 绝对位置从 0 变到 50000, 点积一字不差 —— 只剩下相对距离  $\Delta$  在起作用。")
121
122
123 def demo_rope_long_range_decay():
124     print_title("3. 长程衰减: 只在 q、k 相关时的【期望】上成立")
125
126     print("(a) 只有 1 对分量 (单一频率  $\theta=1$ ) 时,  $q=k=[1,0]$ : 点积 =  $\cos(\Delta)$ , 是周期函数:")
127     for delta in [0, 1, 2, 3, 6, 7]:
128         val = dot(rope([1.0, 0.0], delta, base=1.0), [1.0, 0.0])

```

```

129     print(f"    Δ={delta:<3d} 点积 = {val:+.4f}")
130     print("    --> Δ=3 时是 -0.99, Δ=6 又回到 +0.96 —— 单一频率【没有】长程衰减。")
131
132     d, n_pairs = 64, 300
133     rng = random.Random(2)
134     deltas = [0, 4, 16, 64, 256, 1024, 4096]
135     qs = [[rng.gauss(0, 1) for _ in range(d)] for _ in range(n_pairs)]
136     noise = [[rng.gauss(0, 1) for _ in range(d)] for _ in range(n_pairs)]
137     cases = {
138         "k = q (完全相关)": [q[:] for q in qs],
139         "k = 0.5q + 噪声": [[0.5 * a + 0.866 * b for a, b in zip(q, z)] for q, z
140     in zip(qs, noise)],
141         "q、k 独立随机": noise,
142     }
143     print(f"\n(b) d={d} (32 个频率), 每种情况 {n_pairs} 对随机向量, 统计 RoPE 点积随距离
144     Δ 的变化:")
145     print("    " + f"{'':<16}" + "".join(f"{'Δ=' + str(D):>9}" for D in deltas))
146     for name, ks in cases.items():
147         vals = {D: [dot(rope(q, D), k) for q, k in zip(qs, ks)] for D in deltas}
148         mean = [sum(v) / n_pairs for v in vals.values()]
149         std = [math.sqrt(sum((x - m) ** 2 for x in v) / n_pairs) for v, m in zip(
150             vals.values(), mean)]
151         print(f"    {name:<14} 均值" + "".join(f"{m:9.2f}" for m in mean))
152         se = max(std) / math.sqrt(n_pairs) # 均值的标准误
153         if name.startswith("q、k 独立"):
154             assert all(abs(m) < 5 * se for m in mean), "独立随机时各距离的均值应 ≈ 0
155             (无衰减)"
156         else:
157             assert mean[0] - mean[-2] > 10 * se, "相关时均值应随距离明显衰减"
158         print(f"    {'':<14} 标准差" + "".join(f"{s_:9.2f}" for s_ in std))
159     print("    --> q、k 相关时, 点积的【期望】随距离衰减: 同样'匹配'的两个词, 离得越远注意力加
160     成越小。")
161     print("    q、k 独立时, 均值始终 ≈ 0、标准差始终 ≈ √d: 旋转不改变各向同性随机向量的分
162     布, 【没有任何衰减】。")
163     print("    RoFormer 论文 §3.4.3 证明的是一个【上界】的衰减, 不是点积本身一定变小。")
164     print("    所以 RoPE 的衰减是对'匹配信号'的软性偏置, 不是硬性的距离惩罚。")
165
166 # =====
167 # 4. 因果掩码 (Causal Mask) 数值实验
168 # =====
169 def demo_causal_mask_simulation():
170     print_title("4. 因果掩码 (Causal Mask) 到底怎么做到'禁止向后看'?")
171
172     raw_scores = [
173         [2.0, 1.5, 3.0], # 词 0 对词 0, 1, 2 的原始点积
174         [1.0, 2.5, 1.8], # 词 1 对词 0, 1, 2 的原始点积
175         [0.5, 1.2, 2.2], # 词 2 对词 0, 1, 2 的原始点积

```

```

171 ]
172
173 print("原始未经掩码的点积得分矩阵 (未遮挡未来):")
174 for row in raw_scores:
175     print("    " + str([round(x, 2) for x in row]))
176
177 print("\n叠加因果下三角掩码 (-inf 遮挡未来):")
178 masked_scores = []
179 for i in range(3):
180     row = []
181     for j in range(3):
182         if j > i:
183             row.append(-1e9) # -infinity
184         else:
185             row.append(raw_scores[i][j])
186     masked_scores.append(row)
187
188 for row in masked_scores:
189     row_display = [f"{x:6.1f}" if x > -1e8 else " -inf" for x in row]
190     print("    [" + ", ".join(row_display) + "]")
191
192 print("\n经过 Softmax 归一化后的真实注意力权重矩阵 (每一行和为 1.0):")
193 for r in range(3):
194     probs = softmax(masked_scores[r])
195     print(f"    第 {r} 个词的注意力分布 -> " + str([f"{p*100:5.1f}%" for p in
196 probs]))
197
198 if __name__ == "__main__":
199     demo_permutation_equivariance()
200     demo_rope_relative_property()
201     demo_rope_long_range_decay()
202     demo_causal_mask_simulation()

```

第 17 章 从零训练与推理最小完整 Llama 模型

17.1 Lab 05：亲手训练一个迷你 Llama，再在它身上做推理

避坑指南与核心警示

对应理论：第 4 篇（注意力公式）、第 6 篇（KV Cache）、第 7 篇（采样）、第 3 篇（交叉熵 / 困惑度 / Adam）、第 5 篇（RMSNorm、RoPE、SwiGLU、GQA、Pre-LN）前置：完成 Lab 03（你已经亲手写过反向传播，所以这里可以放心用 `loss.backward()`）。依赖：PyTorch（见根目录 `requirements.txt`，RTX 50 系必须装 cu128 版本）。

17.2 为什么需要这个模块？

`lab04_transformer_microscope` 和 `lab06_kv_cache` 用的都是**随机权重**，输出没有任何意义。这里你会得到一个**真正学过东西**的模型（虽然很小），然后在它身上验证推理理论——这样“KV Cache 输出完全一致”“温度越高越胡说”这些结论就不再是抽象描述。

17.3 文件

文件	内容
<code>model.py</code>	约 180 行的 Llama 架构：RMSNorm、RoPE（半分配对，与 HF 一致）、GQA、因果掩码、SwiGLU、Pre-LN、KV Cache。每个组件的注释都标注了对应的教程章节
<code>train.py</code>	用本仓库的教程 Markdown 做语料，字符级训练；包含“初始损失 $\approx \ln V$ ”检查、验证集 PPL、early stopping
<code>generate.py</code>	朴素生成 vs KV Cache 逐 token 对拍；用真实缓存张量验证显存公式；逐步计时；采样策略对比

17.4 运行

```
1 .venv/bin/python lab05_train_tiny_llama/train.py      # GPU 约 0.5~1.5 分钟（取决于
   显卡时钟状态）
2 .venv/bin/python lab05_train_tiny_llama/generate.py
```

17.5 你会看到什么

以下结论都由脚本自己**断言**（不满足会直接报错），具体数值以运行输出为准：

- 初始损失 $\approx \ln(V)$ ——初始化正确。脚本会打印实测值和理论值，并断言两者相差 < 0.5 。

[注意] **V (字表大小)** 和下面所有 **PPL 数字** 都会随仓库文档的增删而变，因为语料就是本仓库的文档。具体数值以你运行时打印的为准，这里只描述**形状**。

- 验证集 PPL: 从约 V (均匀瞎猜 $= \ln V$) 一路降到十几，随后**过拟合**: 训练损失继续下降

(默认 1000 步结束时约 0.4~0.5)，验证损失却回升。3.5M 参数 vs 十几万字符，数据量只有 Chinchilla 比例的千分之一，模型开始背书——这正是第 5 篇讲规模定律时说的”参数和数据要匹配”。

核心导读与背景

记住这条曲线的**形状** (先降后升、训练/验证分叉)，不要记具体数字：评审实测中，同一份代码在语料增加 120 行后，最优 PPL 从 11.1 变成 14.3 ——不是代码变了，是验证集的切分变了。**这也是”评测集必须冻结”这条工程常识的活例子。**

- 朴素生成与 KV Cache 生成 400 个 token **完全一致**，logits 最大差 $\sim 1e-5$
- KV Cache 真实字节数 = 公式 $2 \times L \times n_{kv} \times d_{head} \times b \times T$ ，**逐字节相等**
- 每步耗时 (本机一次运行):

核心导读与背景

[计时] 以下计时类数字是本机某一次运行的**量级示例**，前提是显卡空闲、已预热；GPU 降频或有其它负载时可能慢 40% 以上。**CPU 列更不可靠**：CPU 首步有首次内存分配/缺页开销，且没有预热路径，实测可能与表里差 5 倍。请看**趋势和比例** (GPU 上 Cache 基本持平 vs 朴素线性增长)，以你自己运行的输出为准。

上下文长度	朴素 (GPU)	Cache (GPU)	朴素 (CPU)	Cache (CPU)
11	3.1 ms	3.3 ms	3.4 ms	1.9 ms
210	4.1 ms	3.1 ms	10.6 ms	3.4 ms
409	3.9 ms	4.7 ms	37.2 ms	2.5 ms

CPU 上完美呈现”朴素线性变慢、缓存持平”；GPU 上两者差不多——因为模型太小，**耗时被 kernel 启动等固定开销主导** (第 8 篇的 overhead 区间)。这是真实世界的现象，不是实验失败：它解释了为什么推理引擎要用 CUDA Graph，以及为什么 Lab 07b 要换成真实的大模型再测。

17.6 动手练习

1. 把 `n_kv_head` 改成 8 (即 MHA) 重新训练，比较参数量、KV Cache 大小和验证 PPL。

2. 在 `model.py` 里把 RoPE 去掉 (`apply_rope` 直接返回 `x`) 重新训练, 看验证 PPL 变差多少——对应 `lab04_transformer_microscope/02` 里”没有位置编码就是词袋”的实验。
3. 把 SwiGLU 换成 $\text{ReLU}(x \cdot W_1) \cdot W_2$, 保持参数量相同 (隐藏维度要改成多少? 见第 5 篇 §5.3)。
4. 生成长度超过训练时的 `block=128` 后, 文本质量有没有明显下降? 这 and 第 5 篇 RoPE 外推的讨论有什么关系?
5. (进阶) 把 `Attention.forward` 里的 `torch.cat` 缓存改成预分配一块 $[B, n_{kv}, \text{max_len}, \text{hd}]$ 的张量、按位置写入, 比较速度。这就是从”动态拼接”走向 PagedAttention (Lab 12b) 的第一步。

17.7 下一步

Lab 07b: 不用 transformers 的模型类, 亲手实现 Qwen2.5 的前向传播并与官方逐位对拍

17.8 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码, 重点实现细节如下:

17.8.1 源码实现: `model.py`

Listing 17.1: `model.py`

```

1  """
2  Lab 05: 一个 Llama 架构的迷你语言模型 (每个组件都对应教程里的一个推导)
3
4  RMSNorm      → 第 5 篇 §5.2 (为什么去掉减均值)、第 8 篇 §6 (为什么求和要转 FP32)
5  RoPE         → 第 4 篇 §6 (相对位置恒等式)、第 5 篇 §5.4 (K 旋转后再进缓存)
6  GQA         → 第 5 篇 §5.5、第 8 篇 §4.3 (KV 头少于 Q 头)
7  因果掩码     → 第 4 篇 §4
8  SwiGLU       → 第 5 篇 §5.3
9  Pre-LN 残差 → 第 3 篇 §7、第 5 篇 §5.1
10 KV Cache    → 第 6 篇 §3
11
12 只用 torch 的基本张量运算和 nn.Linear / nn.Embedding, 不用 nn.MultiheadAttention 等高层封装。
13 """
14
15 import math
16 from dataclasses import dataclass
17

```

```

18 import torch
19 import torch.nn as nn
20 import torch.nn.functional as F
21
22
23 @dataclass
24 class Config:
25     vocab_size: int
26     n_layer: int = 4
27     n_head: int = 8          # Q 头数
28     n_kv_head: int = 2      # KV 头数 (GQA: 每 4 个 Q 头共享 1 组 KV)
29     d_model: int = 256
30     ffn_hidden: int = 688    #  $\approx 8/3 * d_{model}$ , 取 16 的倍数 (第 5 篇 §5.3)
31     max_seq_len: int = 1024 # RoPE 表预先算到这么长
32     rope_base: float = 10000.0
33     norm_eps: float = 1e-5
34     dropout: float = 0.0    # 只在训练时生效 (model.eval() 后自动关闭), 推理代码完全不受影响
35
36     @property
37     def head_dim(self):
38         return self.d_model // self.n_head
39
40
41 class RMSNorm(nn.Module):
42     def __init__(self, dim, eps):
43         super().__init__()
44         self.eps = eps
45         self.weight = nn.Parameter(torch.ones(dim))
46
47     def forward(self, x):
48         # 求均方要在 FP32 里做, BF16 累加会丢精度 (第 8 篇 §6.3)
49         xf = x.float()
50         xf = xf * torch.rsqrt(xf.pow(2).mean(-1, keepdim=True) + self.eps)
51         # 注意这里【不要】写成 xf.to(x.dtype) * self.weight:
52         #   weight 是 FP32 参数, 乘上去会把结果再提升回 FP32 —— 于是那次 .to() 只起到
53         #   "把归一化后的值先截断到 BF16 再算"的作用, 纯属有损往返, 输出依然是 FP32。
54         # 所以整个残差流是 FP32, autocast 只作用在子层内部的 Linear 上。这比纯 BF16 更准,
55         # 代价是显存/带宽占用更高。想改成真正的 BF16 残差流, 需要同时处理 nn.Embedding
56         # (autocast 下它同样输出 FP32, 残差加法会再把它提升回去), 只改本行是不够的。
57         return xf * self.weight
58
59
60 def precompute_rope(head_dim, max_len, base):
61     """返回 cos/sin 表, 形状 [max_len, head_dim/2], 第 m 行第 i 列是 cos(m * base^(-2i/d))"""
62     inv_freq = 1.0 / (base ** (torch.arange(0, head_dim, 2).float() / head_dim))
63     angles = torch.outer(torch.arange(max_len).float(), inv_freq)

```

```

64     return angles.cos(), angles.sin()
65
66
67 def apply_rope(x, cos, sin):
68     """
69     x: [B, H, T, D]; cos/sin: [T, D/2]
70     采用 Hugging Face / Llama 的"前后两半配对"写法: 第 i 维与第 i + D/2 维组成一对做 2D
    旋转。
71     (lab04_transformer_microscope 里用的是相邻两维配对, 两者只是维度排列不同, 数学性质完全
    一样)
72     """
73     d2 = x.shape[-1] // 2
74     x1, x2 = x[..., :d2], x[..., d2:]
75     cos, sin = cos.to(x.dtype), sin.to(x.dtype)
76     return torch.cat([x1 * cos - x2 * sin, x1 * sin + x2 * cos], dim=-1)
77
78
79 class Attention(nn.Module):
80     def __init__(self, cfg: Config):
81         super().__init__()
82         self.cfg = cfg
83         hd = cfg.head_dim
84         self.wq = nn.Linear(cfg.d_model, cfg.n_head * hd, bias=False)
85         self.wk = nn.Linear(cfg.d_model, cfg.n_kv_head * hd, bias=False)
86         self.wv = nn.Linear(cfg.d_model, cfg.n_kv_head * hd, bias=False)
87         self.wo = nn.Linear(cfg.n_head * hd, cfg.d_model, bias=False)
88
89     def forward(self, x, cos, sin, cache=None, start_pos=0):
90         """
91         x: [B, T, d_model], T 在 Prefill / 训练时是整段长度, 在 Decode 时是 1
92         cache: None (训练/朴素推理) 或一个 dict {"k": [B, n_kv, S_past, hd], "v":
    ...}, 原地更新
93         start_pos: 本次输入的第一个 token 在整个序列中的位置 (决定 RoPE 转多少度、掩码怎么
    画)
94         """
95         B, T, _ = x.shape
96         cfg, hd = self.cfg, self.cfg.head_dim
97         q = self.wq(x).view(B, T, cfg.n_head, hd).transpose(1, 2)      # [B,
    n_head, T, hd]
98         k = self.wk(x).view(B, T, cfg.n_kv_head, hd).transpose(1, 2)  # [B, n_kv,
    T, hd]
99         v = self.wv(x).view(B, T, cfg.n_kv_head, hd).transpose(1, 2)
100
101         # RoPE: 只转 Q 和 K, 按它们各自的绝对位置转; K 转好之后才写进缓存, 以后再也不用动它
102         pos_cos, pos_sin = cos[start_pos:start_pos + T], sin[start_pos:start_pos +
    T]
103         q = apply_rope(q, pos_cos, pos_sin)
104         k = apply_rope(k, pos_cos, pos_sin)

```

```

105
106     if cache is not None:
107         if "k" in cache:
108             k = torch.cat([cache["k"], k], dim=2)
109             v = torch.cat([cache["v"], v], dim=2)
110             cache["k"], cache["v"] = k, v
111         S = k.shape[2]  # 总共能看到的 key 数 = start_pos + T
112
113         # GQA: 把每个 KV 头复制给它负责的 n_head / n_kv_head 个 Q 头
114         rep = cfg.n_head // cfg.n_kv_head
115         k = k.repeat_interleave(rep, dim=1)
116         v = v.repeat_interleave(rep, dim=1)
117
118         scores = (q @ k.transpose(-1, -2)) / math.sqrt(hd)  # [B,
119         n_head, T, S]
120         # 因果掩码: 第 i 个 query 的绝对位置是 start_pos + i, 只能看绝对位置 <= 它的 key
121         q_pos = torch.arange(start_pos, start_pos + T, device=x.device).unsqueeze
122         (1)
123         k_pos = torch.arange(S, device=x.device).unsqueeze(0)
124         scores = scores.masked_fill(k_pos > q_pos, float("-inf"))
125         probs = F.softmax(scores.float(), dim=-1).to(q.dtype)  # Softmax
126         在 FP32 里做
127         out = probs @ v  # [B,
128         n_head, T, hd]
129         out = out.transpose(1, 2).reshape(B, T, cfg.n_head * hd)
130         return self.wo(out)
131
132
133 class SwiGLU(nn.Module):
134     def __init__(self, cfg: Config):
135         super().__init__()
136         self.w_gate = nn.Linear(cfg.d_model, cfg.ffn_hidden, bias=False)
137         self.w_up = nn.Linear(cfg.d_model, cfg.ffn_hidden, bias=False)
138         self.w_down = nn.Linear(cfg.ffn_hidden, cfg.d_model, bias=False)
139
140     def forward(self, x):
141         return self.w_down(F.silu(self.w_gate(x)) * self.w_up(x))
142
143
144 class Block(nn.Module):
145     def __init__(self, cfg: Config):
146         super().__init__()
147         self.attn_norm = RMSNorm(cfg.d_model, cfg.norm_eps)
148         self.attn = Attention(cfg)
149         self.ffn_norm = RMSNorm(cfg.d_model, cfg.norm_eps)
150         self.ffn = SwiGLU(cfg)
151         self.drop = nn.Dropout(cfg.dropout)
152

```

```

149     def forward(self, x, cos, sin, cache=None, start_pos=0):
150         # Pre-LN: 残差这条"高速公路"上什么都不放
151         x = x + self.drop(self.attn(self.attn_norm(x), cos, sin, cache, start_pos)
152         )
153         x = x + self.drop(self.ffn(self.ffn_norm(x)))
154         return x
155
156     class TinyLlama(nn.Module):
157         def __init__(self, cfg: Config):
158             super().__init__()
159             self.cfg = cfg
160             self.tok_emb = nn.Embedding(cfg.vocab_size, cfg.d_model)
161             self.blocks = nn.ModuleList([Block(cfg) for _ in range(cfg.n_layer)])
162             self.norm = RMSNorm(cfg.d_model, cfg.norm_eps)
163             self.lm_head = nn.Linear(cfg.d_model, cfg.vocab_size, bias=False)
164             cos, sin = precompute_rope(cfg.head_dim, cfg.max_seq_len, cfg.rope_base)
165             self.register_buffer("rope_cos", cos, persistent=False)
166             self.register_buffer("rope_sin", sin, persistent=False)
167             self.apply(self._init_weights)
168
169         @staticmethod
170         def _init_weights(m):
171             # 小标准差初始化: 让初始 logits 接近 0 → 初始分布接近均匀 → 初始损失  $\approx \ln(V)$  (第 3
172             # 篇 §5.3)
173             if isinstance(m, (nn.Linear, nn.Embedding)):
174                 nn.init.normal_(m.weight, mean=0.0, std=0.02)
175
176         def new_cache(self):
177             """每层一个空 dict; Attention.forward 会往里面写入 k/v"""
178             return [dict() for _ in self.blocks]
179
180         def forward(self, idx, cache=None, start_pos=0):
181             """idx: [B, T] 的 token id; 返回 logits [B, T, vocab]"""
182             x = self.tok_emb(idx)
183             for i, blk in enumerate(self.blocks):
184                 x = blk(x, self.rope_cos, self.rope_sin, None if cache is None else
185                 cache[i], start_pos)
186             return self.lm_head(self.norm(x))
187
188         def num_params(self):
189             return sum(p.numel() for p in self.parameters())

```

17.8.2 源码实现: train.py

Listing 17.2: train.py

```

1  """
2  Lab 05 - 训练：用本仓库自己的教程文本，训练一个字符级的迷你 Llama
3
4  对应教程：第 3 篇（交叉熵、困惑度、Adam、初始损失检查）、第 5 篇（架构）
5
6  运行（GPU 约 0.5~1.5 分钟，取决于显卡时钟状态；CPU 可加 --device cpu --steps 300，效果差一
    些但能跑通）：
7      .venv/bin/python lab05_train_tiny_llama/train.py
8  产物：
9      lab05_train_tiny_llama/tiny_llama.pt    (权重 + 配置 + 字表，供 generate.py 使用)
10 """
11
12 import argparse
13 import glob
14 import math
15 import os
16 import time
17
18 import torch
19 import torch.nn.functional as F
20
21 from model import Config, TinyLlama
22
23 HERE = os.path.dirname(os.path.abspath(__file__))
24 ROOT = os.path.dirname(HERE)
25
26
27 def load_corpus():
28     files = sorted(glob.glob(os.path.join(ROOT, "zero_to_hero_tutorial", "*.md")))
29     files += sorted(glob.glob(os.path.join(ROOT, "*", "README.md")))
30     files.append(os.path.join(ROOT, "README.md"))
31     text = "\n\n".join(open(f, encoding="utf-8").read() for f in files if os.path.
        exists(f))
32     return text, len(files)
33
34
35 def main():
36     ap = argparse.ArgumentParser()
37     ap.add_argument("--steps", type=int, default=1000)
38     ap.add_argument("--batch", type=int, default=64)
39     ap.add_argument("--block", type=int, default=128, help="训练时的上下文长度")
40     ap.add_argument("--lr", type=float, default=1e-3)
41     ap.add_argument("--dropout", type=float, default=0.2, help="数据太少，用 dropout
        缓解过拟合")
42     ap.add_argument("--eval_every", type=int, default=100)
43     ap.add_argument("--device", default="cuda" if torch.cuda.is_available() else "
        cpu")
44     ap.add_argument("--seed", type=int, default=1337)

```

```

45     args = ap.parse_args()
46     torch.manual_seed(args.seed)
47     dev = args.device
48
49     # -----
50     # 1. 数据: 字符级分词 (最简单的 tokenizer: 每个字符一个 id)
51     # -----
52     text, n_files = load_corpus()
53     chars = sorted(set(text))
54     stoi = {c: i for i, c in enumerate(chars)}
55     data = torch.tensor([stoi[c] for c in text], dtype=torch.long)
56     # 按 1000 字符一段切开, 每 10 段取 1 段做验证集: 让验证集覆盖所有章节, 而不是只取文末的几个 README
57     chunks = list(torch.split(data, 1000))
58     train_data = torch.cat([c for i, c in enumerate(chunks) if i % 10 != 5])
59     val_data = torch.cat([c for i, c in enumerate(chunks) if i % 10 == 5])
60     V = len(chars)
61     print(f"[资料] 语料: {n_files} 个 Markdown 文件, 共 {len(text):,} 个字符, 字表大小 V = {V}")
62     print(f"    训练集 {len(train_data):,} 字符 / 验证集 {len(val_data):,} 字符")
63
64     def get_batch(split):
65         d = train_data if split == "train" else val_data
66         ix = torch.randint(len(d) - args.block - 1, (args.batch,))
67         x = torch.stack([d[i:i + args.block] for i in ix])
68         y = torch.stack([d[i + 1:i + args.block + 1] for i in ix]) # 目标 = 输入
69         # 右移一位 (预测下一个字符)
70         return x.to(dev), y.to(dev)
71
72     # -----
73     # 2. 模型
74     # -----
75     cfg = Config(vocab_size=V, dropout=args.dropout)
76     model = TinyLlama(cfg).to(dev)
77     print(f"[决策] 模型: {cfg.n_layer} 层, d_model={cfg.d_model}, Q 头={cfg.n_head},
78           KV 头={cfg.n_kv_head} (GQA), "
79           f"FFN={cfg.ffn_hidden}, dropout={cfg.dropout}, 参数量 {model.num_params()}
80           / 1e6:.2f}M")
81
82     use_bf16 = dev.startswith("cuda")
83     autocast = torch.autocast(device_type="cuda", dtype=torch.bfloat16) if
84     use_bf16 else torch.autocast("cpu", enabled=False)
85
86     @torch.no_grad()
87     def estimate_loss(iters=20):
88         model.eval()
89         out = {}
90         for split in ["train", "val"]:

```

```

87         losses = []
88         for _ in range(iters):
89             x, y = get_batch(split)
90             with autocast:
91                 logits = model(x)
92                 losses.append(F.cross_entropy(logits.float().view(-1, V), y.view
(-1)).item())
93             out[split] = sum(losses) / len(losses)
94         model.train()
95         return out
96
97     # -----
98     # 3. 训练前检查: 初始损失应  $\approx \ln(V)$  (第 3 篇 §5.3 的调试黄金法则)
99     # -----
100     init = estimate_loss(5)
101     print(f"\n[观察] 初始损失检查: 实测 {init['val']:.3f} vs 理论  $\ln(V) = \{{\rm math.log}(V)\}$ 
: .3f} ")
102     f"({'[OK] 正常' if abs(init['val'] - math.log(V)) < 0.5 else '[X] 偏差过
大, 检查初始化'})")
103     assert abs(init['val'] - math.log(V)) < 0.5, '初始损失应  $\approx \ln(V)$ , 检查初始化'
104
105     # -----
106     # 4. 训练循环: AdamW + 线性预热 + 余弦退火
107     # -----
108     opt = torch.optim.AdamW(model.parameters(), lr=args.lr, betas=(0.9, 0.95),
weight_decay=0.1)
109     warmup = max(1, args.steps // 30)
110
111     def lr_at(step):
112         if step < warmup:
113             return args.lr * (step + 1) / warmup
114         progress = (step - warmup) / max(1, args.steps - warmup)
115         return args.lr * (0.1 + 0.9 * 0.5 * (1 + math.cos(math.pi * progress)))
116
117     print(f"\n[演练] 开始训练 {args.steps} 步 (batch={args.batch}, block={args.block
}, device={dev})")
118     path = os.path.join(HERE, "tiny_llama.pt")
119     best = {"val": float("inf"), "step": -1, "train": None}
120     t0 = time.time()
121     for step in range(args.steps + 1):
122         if step % args.eval_every == 0:
123             l = estimate_loss()
124             mark = ""
125             if l["val"] < best["val"]:
126                 # 只保存验证集损失最低的那一刻 (early stopping), 之后的"进步"只是在背训练
集
127
128                 best = {"val": l["val"], "step": step, "train": l["train"]}
129                 torch.save({"model": model.state_dict(), "config": cfg.__dict__, "

```

```

129     chars": chars}, path)
130         mark = "  "
131         print(f"    step {step:5d} | train loss {l['train']:.3f} | val loss {l['val']:.3f} | "
132               f"val PPL {math.exp(l['val']):7.2f} | {time.time() - t0:5.1f}s{mark}")
133     if step == args.steps:
134         break
135     for g in opt.param_groups:
136         g["lr"] = lr_at(step)
137     x, y = get_batch("train")
138     with autocast:
139         logits = model(x)
140         loss = F.cross_entropy(logits.float().view(-1, V), y.view(-1))
141         opt.zero_grad(set_to_none=True)      # 梯度是累加的，每步清零 (Lab 03 实验 6)
142         loss.backward()
143         torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
144         opt.step()
145
146     print(f"\n[思考] 解读:")
147     print(f"    1. 验证集 PPL 从约 {V} (均匀瞎猜) 降到了 {math.exp(best['val']):.1f}: 模型确实学到了中文字符与 Markdown 的统计规律。")
148     print(f"    2. 训练损失一路下降，验证损失却在某一步后不降反升 —— 这就是【过拟合】:")
149     print(f"        {model.num_params() / 1e6:.1f}M 参数 vs 只有 {len(train_data) / 1e3:.0f}K 个训练字符，模型容量远大于数据量，开始逐字背诵训练集。")
150     print(f"    3. 所以我们只保存了验证损失最低的第 {best['step']} 步 (early stopping)。")
151     print(f"\n 已保存到 {os.path.relpath(path, ROOT)}, 下一步运行: .venv/bin/python lab05_train_tiny_llama/generate.py")
152
153
154 if __name__ == "__main__":
155     main()

```

17.8.3 源码实现: generate.py

Listing 17.3: generate.py

```

1  """
2  Lab 05 - 推理: 在一个"真正训练过"的模型上，把第 6、7 篇的推理理论全部跑一遍
3
4  实验清单:
5      1. 朴素自回归 (每步重算整段) vs KV Cache (Prefill 一次 + 每步只算 1 个 token)
6          → 逐 token 比对输出完全一致, logits 误差在浮点噪声量级
7      2. 用真实缓存张量验证第 6 篇的 KV Cache 显存公式  $2 \times L \times n_{kv} \times d_{head} \times bytes \times$ 

```

```

tokens
8   3. 每一步 Decode 的耗时: 朴素方式随长度增长, 缓存方式基本持平
9   4. 采样策略: Greedy / Temperature / Top-P 在真实模型上的效果 (第 7 篇)
10
11 运行 (先跑 train.py 生成 tiny_llama.pt) :
12   .venv/bin/python lab05_train_tiny_llama/generate.py
13   """
14
15   import argparse
16   import os
17   import time
18
19   import torch
20   import torch.nn.functional as F
21
22   from model import Config, TinyLlama
23
24   HERE = os.path.dirname(os.path.abspath(__file__))
25
26
27   def load(device):
28       path = os.path.join(HERE, "tiny_llama.pt")
29       if not os.path.exists(path):
30           raise SystemExit("找不到 tiny_llama.pt, 请先运行: .venv/bin/python
31       lab05_train_tiny_llama/train.py")
32       ckpt = torch.load(path, map_location=device)
33       model = TinyLlama(Config(**ckpt["config"])).to(device)
34       model.load_state_dict(ckpt["model"])
35       model.eval()
36       chars = ckpt["chars"]
37       stoi = {c: i for i, c in enumerate(chars)}
38       return model, chars, stoi
39
40   def sync(device):
41       if device.startswith("cuda"):
42           torch.cuda.synchronize()
43
44
45   @torch.inference_mode()
46   def generate_naive(model, ids, n_new, device):
47       """朴素版: 每一步都把【整段序列】重新送进模型, 只取最后一个位置的 logits"""
48       ids = list(ids)
49       step_logits, step_times = [], []
50       for _ in range(n_new):
51           sync(device); t = time.perf_counter()
52           x = torch.tensor([ids], device=device)
53           logits = model(x)[0, -1]

```

```

54     nxt = int(logits.argmax())
55     sync(device); step_times.append(time.perf_counter() - t)
56     step_logits.append(logits.float().cpu())
57     ids.append(nxt)
58     return ids, step_logits, step_times
59
60
61 @torch.inference_mode()
62 def generate_cached(model, ids, n_new, device):
63     """KV Cache 版: Prefill 整段 prompt 一次, 之后每步只送 1 个 token 进去"""
64     ids = list(ids)
65     cache = model.new_cache()
66     step_logits, step_times = [], []
67
68     sync(device); t = time.perf_counter()
69     logits = model(torch.tensor([ids], device=device), cache=cache, start_pos=0)
70     [0, -1] # Prefill
71     for i in range(n_new):
72         nxt = int(logits.argmax())
73         sync(device); step_times.append(time.perf_counter() - t)
74         step_logits.append(logits.float().cpu())
75         ids.append(nxt)
76         if i == n_new - 1:
77             break
78         sync(device); t = time.perf_counter()
79         # Decode: 只输入刚生成的 1 个 token, 它的位置是 len(ids) - 1
80         logits = model(torch.tensor([[nxt]], device=device), cache=cache,
81 start_pos=len(ids) - 1)[0, -1]
82     return ids, step_logits, step_times, cache
83
84 @torch.inference_mode()
85 def sample(model, ids, n_new, device, temperature=1.0, top_p=1.0, seed=0):
86     """带温度与 Top-P 的采样 (第 7 篇), 使用 KV Cache"""
87     g = torch.Generator(device="cpu").manual_seed(seed)
88     ids = list(ids)
89     cache = model.new_cache()
90     logits = model(torch.tensor([ids], device=device), cache=cache, start_pos=0)
91     [0, -1]
92     for _ in range(n_new):
93         logits = logits.float().cpu()
94         if temperature == 0:
95             nxt = int(logits.argmax())
96         else:
97             probs = F.softmax(logits / temperature, dim=-1)
98             if top_p < 1.0:
99                 sp, si = probs.sort(descending=True)
100                 keep = sp.cumsum(0) - sp < top_p # 累计概率"在加上自己之前"还没

```

```

到 P 的都保留
99         sp = sp * keep
100         probs = torch.zeros_like(probs).scatter(0, si, sp)
101         probs = probs / probs.sum()
102         nxt = int(torch.multinomial(probs, 1, generator=g))
103         ids.append(nxt)
104         logits = model(torch.tensor([[nxt]], device=device), cache=cache,
start_pos=len(ids) - 1)[0, -1]
105     return ids
106
107
108 def main():
109     ap = argparse.ArgumentParser()
110     ap.add_argument("--prompt", default="KV Cache 的")
111     ap.add_argument("--n", type=int, default=400, help="生成多少个字符")
112     ap.add_argument("--device", default="cuda" if torch.cuda.is_available() else "
cpu")
113     args = ap.parse_args()
114     dev = args.device
115
116     model, chars, stoi = load(dev)
117     cfg = model.cfg
118     decode = lambda ids: "".join(chars[i] for i in ids)
119     prompt_ids = [stoi[c] for c in args.prompt if c in stoi]
120     n = min(args.n, cfg.max_seq_len - len(prompt_ids))
121     print(f"[决策] 已加载模型: {model.num_params() / 1e6:.2f}M 参数 | 设备 {dev} | 提示词 {args.prompt!r}")
122     generate_cached(model, prompt_ids, 8, dev)    # 预热: 第一次调用 CUDA kernel 有初始化开销, 不计入计时
123
124     # -----
125     print("\\n" + "=" * 76)
126     print("[实验] 实验 1: 朴素自回归 vs KV Cache —— 输出必须完全一致")
127     print("=" * 76)
128     ids_a, logits_a, times_a = generate_naive(model, prompt_ids, n, dev)
129     ids_b, logits_b, times_b, cache = generate_cached(model, prompt_ids, n, dev)
130     same = ids_a == ids_b
131     max_diff = max((a - b).abs().max().item() for a, b in zip(logits_a, logits_b))
132     print(f"    生成 {n} 个字符, 两种方式 token 序列{'完全相同 [OK]' if same else '不同 [X]'}; 各步 logits 最大差值 {max_diff:.2e}")
133     assert same and max_diff < 1e-3, 'KV Cache 与朴素生成结果不一致'
134     print("    (差值不为 0 是因为矩阵形状不同时浮点加法的顺序不同, 属于正常的数值噪声)")
135     print("\\n    Greedy 生成结果 (前 150 字) :")
136     print("    " + decode(ids_b)[:150].replace("\\n", "\\n    "))
137
138     # -----
139     print("\\n" + "=" * 76)
140     print("[实验] 实验 2: 用真实缓存张量验证第 6 篇的 KV Cache 显存公式")

```

```

141 print("=" * 76)
142 tokens_cached = cache[0]["k"].shape[2]
143 actual = sum(t.numel() * t.element_size() for layer in cache for t in layer.
144 values())
145 elem = cache[0]["k"].element_size()
146 formula = 2 * cfg.n_layer * cfg.n_kv_head * cfg.head_dim * elem *
147 tokens_cached
148 print(f"    缓存中 token 数 = {tokens_cached}, 每层 K 的形状 = {list(cache[0]['k']
149 '.shape)} ([batch, n_kv_head, tokens, head_dim])")
150 print(f"    公式  $2 \times L(\text{cfg.n\_layer}) \times n_{\text{kv}}(\text{cfg.n\_kv\_head}) \times d_{\text{head}}(\text{cfg.}$ 
151 head_dim)  $\times \{\text{elem}\} \times \{\text{tokens\_cached}\} = \{\text{formula:},\}$  字节")
152 print(f"    实际缓存张量占用                                = {actual:}, 字节
153 {'[OK]' if formula == actual else '[X]'}")
154 assert formula == actual, 'KV Cache 显存公式与实际不符'
155 mha = formula * cfg.n_head // cfg.n_kv_head
156 print(f"    如果不用 GQA (MHA, {cfg.n_head} 个 KV 头), 会是 {mha:}, 字节, 是现在的 {
157 cfg.n_head // cfg.n_kv_head} 倍")
158
159 # -----
160 print("\n" + "=" * 76)
161 print("[实验] 实验 3: 每一步的耗时 (毫秒)")
162 print("=" * 76)
163 results = {dev: (times_a, times_b)}
164 if dev != "cpu":
165     # 再在 CPU 上跑一遍: CPU 没有"kernel 启动开销被 GPU 并行掩盖"的问题, 重算的代价会直
166     接暴露出来
167     model_cpu, _, _ = load("cpu")
168     _, _, ta = generate_naive(model_cpu, prompt_ids, n, "cpu")
169     _, _, tb, _ = generate_cached(model_cpu, prompt_ids, n, "cpu")
170     results["cpu"] = (ta, tb)
171 header = " | ".join(f"{'朴素(' + d + ')':>12} | {'Cache(' + d + ')':>12}" for
172 d in results)
173 print(f"    {'第几步':>6} | {'上下文长度':>8} | {header}")
174 for i in [1, n // 4, n // 2, 3 * n // 4, n - 1]:
175     row = " | ".join(f"{ta[i] * 1e3:12.2f} | {tb[i] * 1e3:12.2f}" for ta, tb
176 in results.values())
177     print(f"    {i:>6} | {len(prompt_ids) + i:>8} | {row}")
178 for d, (ta, tb) in results.items():
179     print(f"    [{d}] 总耗时: 朴素 {sum(ta) * 1e3:.0f} ms vs KV Cache {sum(tb) *
180 1e3:.0f} ms")
181 print("    --> 解读: ")
182 print("    - CPU 上: 朴素方式每步耗时随上下文线性增长 (每步都重算整段), KV Cache 基本
183 持平 —— 第 6 篇的结论。")
184 print("    - GPU 上: 两者可能差不多! 因为这个模型只有几 MB, 每步真正的计算只要几微秒,
185 耗时几乎全是")
186 print("    Python 与 kernel 启动的固定开销 (第 8 篇引用的 Horace He 文章称之为
187 overhead), 多算一些也看不出来。")
188 print("    模型越大、上下文越长, 计算才会超过固定开销; Lab 07b 在真实的 0.5B 模型上

```

```

176     会看到明显差距。")
177     print("          这也是 vLLM 等引擎要用 CUDA Graph 把整步 Decode 录制成一次启动的原因。")
178     print("\n" + "=" * 76)
179     print("[实验] 实验 4: 采样策略对比 (第 7 篇), 每种生成 80 个字符")
180     print("=" * 76)
181     for name, kw in [("Greedy (T=0)", dict(temperature=0)),
182                     ("T=0.8, Top-P=0.9", dict(temperature=0.8, top_p=0.9)),
183                     ("T=1.5 (高温)", dict(temperature=1.5)),
184                     ("T=3.0 (极高温)", dict(temperature=3.0))]:
185         out = decode(sample(model, prompt_ids, 80, dev, **kw))
186         print(f"    [{name:<16}] {out.replace(chr(10), ' ')}")
187     print("    --> 温度越高越'有创意', 极高温时退化成接近均匀的随机字符 (第 7 篇定理 2)。")
188
189
190
191 if __name__ == "__main__":
192     main()

```

第 18 章 KV Cache 性能基准测试与吞吐加速比实测

18.1 Lab 06: KV Cache 原理与自回归解码基准测试

18.2 [思考] 为什么需要 KV Cache?

在大语言模型 (如 GPT、Llama、Qwen 等 Decoder-only 架构) 生成文本时, 是逐字 (Token-by-Token) 生成的:

```
Prompt: "人工智能的未来是"  
Step 1 -> "星"  
Step 2 -> "辰"  
Step 3 -> "大"  
Step 4 -> "海"
```

在标准的 Self-Attention 计算中:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

18.2.1 朴素实现的问题 (无 KV Cache):

生成第 t 个 token 时, 把前 $t-1$ 个 token 连同最新 token 一起重新送入模型:

- 重新计算前 $t-1$ 个 token 的 Q, K, V 投影矩阵。
- 但事实上, 历史 token 的 Key 和 Value 向量是固定不变的!
- 每次生成都会重复计算历史所有 token 的 K 和 V , 导致总计算复杂度高达 $O(N^2)$, 生成越到后面越慢。

18.2.2 KV Cache 的优化思路:

- 在 **Prefill 阶段** (首字), 计算出 Prompt 中所有 token 的 K 和 V , 并保存在显存中 (KV Cache)。
- 在 **Decode 阶段** (后续生成), 每次只输入最新的 1 个 token:
- 只对最新 token 计算 $q_{\text{new}}, k_{\text{new}}, v_{\text{new}}$ 。
- 把 $k_{\text{new}}, v_{\text{new}}$ 拼接到缓存列表中: $K_{\text{all}} = [K_{\text{cache}}, k_{\text{new}}]$ 。

- 用单个查询向量 q_{new} 与整个 K_{all} 做点积得到注意力分数，加权聚合 V_{all} 。
- 这样，单步生成的计算复杂度从 $O(t)$ 矩阵乘降为向量-矩阵乘，避免了大量冗余算力消耗！

18.3 [执行] 运行测试

本目录下提供了 `kv_cache_benchmark.py`，演示：

1. 朴素生成 vs KV Cache 生成的数学等价性（验证输出完全一致）。
2. 每步生成耗时随序列长度增加的变化趋势对比。

```
1 python3 lab06_kv_cache/kv_cache_benchmark.py
```

(本脚本自适应纯 *Python* 或 *NumPy*，即使暂未安装任何第三方库也能直接运行！)

18.4 [链接] 继续深入

本模块用的是随机权重的单层注意力。想在**真正训练过的模型**上验证同样的结论（输出逐 token 一致、缓存字节数与公式逐字节相等、每步耗时对比），请做：

- **Lab 05**：自己训练的迷你 Llama (`generate.py`)；
- **Lab 07b**：真实的 Qwen2.5-0.5B，上下文越长 KV Cache 的优势越大（2048 时约快一个数量级）。

注意：KV Cache 消除的是对历史 token 重复做投影 / FFN 的冗余，**注意力本身每步仍要读完整个缓存**，长上下文时这会成为新的瓶颈（第 6 篇 §3 的补充说明、第 8 篇 §4.3）。

18.5 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码，重点实现细节如下：

18.5.1 源码实现：kv_cache_benchmark.py

Listing 18.1: kv_cache_benchmark.py

```
1 """
2 Lab 06: KV Cache 原理与基准测试仿真（零外部依赖，自动适配 Pure Python / NumPy）
3
4 本脚本直观演示：
5 1. 朴素自回归解码（无 KV Cache）：每次重新计算前文所有 token 的 K, V 投影。
6 2. 缓存自回归解码（有 KV Cache）：每次只计算新 token 的 Q, K, V，并追加到缓存中。
```

```

7 3. 验证两种方式输出完全等价，并对比推理耗时与计算量差异。
8 """
9
10 import time
11 import math
12 import random
13
14 try:
15     import numpy as np
16     HAVE_NUMPY = True
17 except ImportError:
18     HAVE_NUMPY = False
19
20
21 # =====
22 # 矩阵运算轻量封装（优先 NumPy，无环境时回退纯 Python）
23 # =====
24 class MatrixOps:
25     @staticmethod
26     def random_matrix(rows, cols, seed=42):
27         if HAVE_NUMPY:
28             np.random.seed(seed)
29             return np.random.randn(rows, cols) * 0.05
30         else:
31             rng = random.Random(seed)
32             return [[rng.gauss(0, 0.05) for _ in range(cols)] for _ in range(rows)]
33
34     @staticmethod
35     def matmul(A, B):
36         if HAVE_NUMPY:
37             return np.dot(A, B)
38         else:
39             n, m = len(A), len(A[0])
40             p = len(B[0])
41             res = [[0.0] * p for _ in range(n)]
42             for i in range(n):
43                 for k in range(m):
44                     aik = A[i][k]
45                     for j in range(p):
46                         res[i][j] += aik * B[k][j]
47             return res
48
49     @staticmethod
50     def transpose(A):
51         if HAVE_NUMPY:
52             return A.T
53         else:

```

```

54         return [[A[i][j] for i in range(len(A))] for j in range(len(A[0]))]
55
56     @staticmethod
57     def concat_rows(A, B):
58         if HAVE_NUMPY:
59             return np.concatenate([A, B], axis=0)
60         else:
61             return [row[:] for row in A] + [row[:] for row in B]
62
63     @staticmethod
64     def softmax_row(row):
65         max_val = max(row) if not HAVE_NUMPY else float(np.max(row))
66         if HAVE_NUMPY:
67             exp_vals = np.exp(row - max_val)
68             return exp_vals / np.sum(exp_vals)
69         else:
70             exp_vals = [math.exp(v - max_val) for v in row]
71             sum_exp = sum(exp_vals)
72             return [v / sum_exp for v in exp_vals]
73
74     @staticmethod
75     def max_diff(A, B):
76         if HAVE_NUMPY:
77             return float(np.max(np.abs(A - B)))
78         else:
79             diff = 0.0
80             for i in range(len(A)):
81                 for j in range(len(A[0])):
82                     diff = max(diff, abs(A[i][j] - B[i][j]))
83             return diff
84
85
86     class MinimalAttention:
87         """自注意力机制：分别实现朴素全量计算与 KV Cache 增量计算"""
88         def __init__(self, d_model=128):
89             self.d_model = d_model
90             self.w_q = MatrixOps.random_matrix(d_model, d_model, seed=10)
91             self.w_k = MatrixOps.random_matrix(d_model, d_model, seed=20)
92             self.w_v = MatrixOps.random_matrix(d_model, d_model, seed=30)
93             self.w_o = MatrixOps.random_matrix(d_model, d_model, seed=40)
94
95         def forward_naive(self, seq_tokens):
96             """
97             无 KV Cache：输入包含前缀在内的完整历史 [seq_len, d_model]
98             仅取最后一个 token 的预测结果
99             """
100             # 全量重算全部历史 token 的 Q, K, V
101             Q = MatrixOps.matmul(seq_tokens, self.w_q)

```

```

102     K = MatrixOps.matmul(seq_tokens, self.w_k)
103     V = MatrixOps.matmul(seq_tokens, self.w_v)
104
105     # 只提取最后一个位置的查询向量 q_last: [1, d_model]
106     q_last = Q[-1:]
107
108     # 计算注意力分数: q_last 与所有历史 token 的 K 转置相乘 -> [1, seq_len]
109     K_T = MatrixOps.transpose(K)
110     scores_mat = MatrixOps.matmul(q_last, K_T)
111
112     scale = math.sqrt(self.d_model)
113     if HAVE_NUMPY:
114         scores = scores_mat / scale
115         attn_weights = MatrixOps.softmax_row(scores[0])
116         context = np.dot(attn_weights.reshape(1, -1), V)
117     else:
118         scores = [v / scale for v in scores_mat[0]]
119         attn_weights = MatrixOps.softmax_row(scores)
120         context = MatrixOps.matmul([attn_weights], V)
121
122     output = MatrixOps.matmul(context, self.w_o)
123     return output
124
125 def forward_with_cache(self, new_token, kv_cache=None):
126     """
127     有 KV Cache:
128     - new_token: [1, d_model] 仅输入当前步最新生成的这 1 个 token
129     - kv_cache: (K_past, V_past) 保存的历史 K 和 V
130     """
131     # 仅对新 token 计算单向量的 q, k, v
132     q_new = MatrixOps.matmul(new_token, self.w_q)
133     k_new = MatrixOps.matmul(new_token, self.w_k)
134     v_new = MatrixOps.matmul(new_token, self.w_v)
135
136     # 拼接更新缓存
137     if kv_cache is None or kv_cache[0] is None:
138         k_all = k_new
139         v_all = v_new
140     else:
141         k_past, v_past = kv_cache
142         k_all = MatrixOps.concat_rows(k_past, k_new)
143         v_all = MatrixOps.concat_rows(v_past, v_new)
144
145     new_kv_cache = (k_all, v_all)
146
147     # 注意力计算: q_new (1, d_model) 与 k_all (seq_len, d_model)
148     K_T = MatrixOps.transpose(k_all)
149     scores_mat = MatrixOps.matmul(q_new, K_T)

```

```

150
151     scale = math.sqrt(self.d_model)
152     if HAVE_NUMPY:
153         scores = scores_mat / scale
154         attn_weights = MatrixOps.softmax_row(scores[0])
155         context = np.dot(attn_weights.reshape(1, -1), v_all)
156     else:
157         scores = [v / scale for v in scores_mat[0]]
158         attn_weights = MatrixOps.softmax_row(scores)
159         context = MatrixOps.matmul([attn_weights], v_all)
160
161     output = MatrixOps.matmul(context, self.w_o)
162     return output, new_kv_cache
163
164
165 def run_benchmark():
166     engine_name = "NumPy 加速矩阵运算" if HAVE_NUMPY else "Python 原生标准库运算 (零外
167     部依赖)"
168     print("=====")
169     print("[起步] KV Cache 原理与自回归基准测试")
170     print(f"[代码] 计算引擎: {engine_name}")
171     print("=====\\n")
172
173     d_model = 128
174     prompt_len = 32
175     decode_steps = 40
176
177     layer = MinimalAttention(d_model=d_model)
178     prompt = MatrixOps.random_matrix(prompt_len, d_model, seed=100)
179
180     print(f"[重点] 测试配置: 隐藏层维度={d_model}, Prefill Prompt 长度={prompt_len},
181     Decode 生成步数={decode_steps}\\n")
182
183     # 1. 验证数学等价性
184     print("[观察] 阶段 1: 验证输出数值等价性...")
185     kv_cache = None
186     # Prefill: 灌入 Prompt 构建初始缓存
187     for i in range(prompt_len):
188         token_i = prompt[i:i+1]
189         _, kv_cache = layer.forward_with_cache(token_i, kv_cache)
190
191     # 模拟下一 token 输入
192     next_input = MatrixOps.random_matrix(1, d_model, seed=200)
193
194     # 无缓存方式预测
195     full_seq = MatrixOps.concat_rows(prompt, next_input)
196     out_naive = layer.forward_naive(full_seq)

```

```

196 # 有缓存方式预测
197 out_cache, _ = layer.forward_with_cache(next_input, kv_cache)
198
199 diff = MatrixOps.max_diff(out_naive, out_cache)
200 print(f"    最大绝对误差 (Max Absolute Diff): {diff:.8e}")
201 assert diff < 1e-6, f"有/无 KV Cache 输出不一致, 最大误差 {diff}"
202 print("    [OK] 校验通过: 有/无 KV Cache 输出结果数值完全一致! \n")
203
204 # 2. 耗时基准测试: 两条路径从同一个 prompt 出发、生成同一条序列, 只对 Decode 部分计时
205 print(f"[计时] 阶段 2: 耗时测试 (Prefill 后自回归生成 {decode_steps} 步, 两边都只计 Decode 耗时)...")
206
207 # 无 Cache: 每一步都把完整历史重新送进去
208 tokens_history = [prompt[i:i+1] for i in range(prompt_len)]
209 naive_outs, naive_times = [], []
210 t0 = time.perf_counter()
211 for step in range(decode_steps):
212     st = time.perf_counter()
213     seq = tokens_history[0]
214     for t in tokens_history[1:]:
215         seq = MatrixOps.concat_rows(seq, t)
216     out = layer.forward_naive(seq)
217     tokens_history.append(out)
218     naive_outs.append(out)
219     naive_times.append(time.perf_counter() - st)
220 naive_total = time.perf_counter() - t0
221
222 # 有 Cache: 先 Prefill (不计时间), Prefill 最后一个位置的输出就是第 1 个生成结果
223 kv_cache, out = None, None
224 for i in range(prompt_len):
225     out, kv_cache = layer.forward_with_cache(prompt[i:i+1], kv_cache)
226 cache_outs, cache_times = [out], [0.0] # 第 1 步的输出来自 Prefill, Decode 耗时
记为 0
227 t0 = time.perf_counter()
228 for step in range(1, decode_steps):
229     st = time.perf_counter()
230     out, kv_cache = layer.forward_with_cache(out, kv_cache) # 只送入上一步生成
的 1 个 token
231     cache_outs.append(out)
232     cache_times.append(time.perf_counter() - st)
233 cache_total = time.perf_counter() - t0
234
235 worst = max(MatrixOps.max_diff(a, b) for a, b in zip(naive_outs, cache_outs))
236 assert worst < 1e-6, f"两条路径生成的序列不一致, 最大误差 {worst}"
237 print(f"    [OK] 两条路径逐步生成的 {decode_steps} 个输出全部一致 (最大误差 {worst:.1e}
)}) ")
238 print(f"    (注: 朴素路径的第 1 步本身就是一次完整 Prefill, 所以两边的计时范围略有不同,
差距被略微高估) ")

```

```

239
240 # 打印对比结果
241 print(f"\n[图表] 汇总结果:")
242 print(f"    - 无 KV Cache 累计生成耗时 : {naive_total * 1000:.2f} ms")
243 print(f"    - 有 KV Cache 累计生成耗时 : {cache_total * 1000:.2f} ms")
244 speedup = naive_total / cache_total if cache_total > 0 else 1.0
245 print(f"    - [起步] 加速比 (Speedup)      : {speedup:.2f}x\n")
246
247 print(f"[增长] 步进耗时采样 (毫秒):")
248 sample_indices = [0, decode_steps // 4, decode_steps // 2, decode_steps - 1]
249 print(f"    {'步数 (Step)':<14} | {'无 Cache 耗时 (ms)':<22} | {'有 Cache 耗时 (ms)':<22}")
250 print(f"    {'-'*14}-+-'*22}-+-'*22)")
251 for idx in sample_indices:
252     print(f"        Step {idx+1:<9} | {naive_times[idx]*1000:<22.3f} | {cache_times[idx]*1000:<22.3f}")
253
254 print("\n[思考] 深入思考与核心结论:")
255 print("    1. 【计算量差异】：无 Cache 情况下，每生成 1 个新 token，都需要重新计算前面所有 token 的 QKV 投影矩阵。")
256 print("        在长文本场景下，计算复杂度高达  $O(N^2)$ 。有 Cache 后，QKV 投影只需计算当前这 1 个 token，复杂度降为  $O(N)$ 。")
257 print("    2. 【显存代价】：虽然省去了重算，但所有历史 token 的 K 和 V 矩阵必须常驻显存。")
258 print("        在高并发、多轮对话场景下，KV Cache 迅速吞噬显存，这正是 vLLM 提出 PagedAttention 与连续批处理要解决的痛点！")
259
260
261 if __name__ == "__main__":
262     run_benchmark()

```

第 19 章 工业级文本采样策略实操 (Top-K / Top-P / Min-P)

19.1 Lab 07a: 采样策略显微镜

核心数学定理推导

对应理论：第 7 篇：Logits、温度系数极限证明与采样算法依赖：纯 Python 标准库。

19.2 运行

```
1 python3 lab07a_sampling/sampling_strategies.py
```

用一组固定的 Logits，逐一对比：原始分布、低温 ($T=0.2$)、高温 ($T=2.0$)、Top-K、Top-P、重复惩罚，打印每种策略处理后的概率分布条形图。

19.3 速查

模型 LM Head 输出的不是概率，而是一堆未经归一化的实数值，称为 **Logits** (例如 $[2.1, -0.5, 8.4, 0.1]$)。

- 贪婪搜索 (Greedy Search):

$$\text{next_token} = \arg \max(\text{logits})$$

直接选得分最高的那一个 (确定性强，但容易重复或死板)。

- 温度系数 (Temperature T):

$$\text{probs} = \text{softmax}\left(\frac{\text{logits}}{T}\right)$$

- $T \rightarrow 0$: 分布极度尖锐，退化为贪婪搜索;
- $T > 1$: 分布平缓，低分词也有机会被选中，回答更具创意但更容易胡说八道;
- Top-P (核采样 Nucleus Sampling):

按概率从高到低排序累加，只保留累计概率达到 P (如 0.9) 的头部候选词，截断长尾无意义低概率词，在多样性与合理性之间取得最佳平衡。

19.4 下一步

- 在真正训练过的模型上看采样效果：Lab 05 的 `generate.py` 实验 4；
- 手写重复惩罚并与 Hugging Face 官方实现逐 token 对拍：Lab 07b 实验 5。

19.5 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码，重点实现细节如下：

19.5.1 源码实现：`sampling_strategies.py`

Listing 19.1: `sampling_strategies.py`

```
1  """
2  Lab 07a: 采样策略 (Sampling Strategies) 微观实验
3
4  深入拆解模型生成下一个 Token 时的核心采样算法：
5  （贪婪搜索 = 温度趋近 0 的极限，本脚本用  $T \rightarrow 0$  演示，不单独实现）
6  1. 温度系数调节 (Temperature Scaling)
7  3. Top-K 截断采样
8  4. Top-P (核采样 Nucleus Sampling)
9  5. 惩罚项机制 (Repetition Penalty 重复惩罚)
10 """
11
12 import math
13 import random
14
15
16 def softmax(logits):
17     max_l = max(logits)
18     exp_l = [math.exp(x - max_l) for x in logits]
19     sum_e = sum(exp_l)
20     return [e / sum_e for e in exp_l]
21
22
23 def print_distribution_table(vocab_words, probs, title):
24     print(f"\n[图表] {title}")
25     # 按概率从大到小排序
26     ranked = sorted(zip(vocab_words, probs), key=lambda x: x[1], reverse=True)
27     for word, p in ranked:
28         bar = "█" * int(p * 40)
29         print(f" {word:<6} | {p*100:5.2f}% | {bar}")
30
31
32 # =====
33 # 1. 温度系数 (Temperature)
```

```

34 # =====
35 def apply_temperature(logits, temperature=1.0):
36     """
37     Temperature 调整:  $\text{logits\_new} = \text{logits} / T$ 
38     -  $T < 1.0$  (如 0.2): 分布尖锐化, 增强确定性、严肃性
39     -  $T = 1.0$ : 原始自然分布
40     -  $T > 1.0$  (如 1.8): 分布均匀化, 增强创造力、不可预测性
41     """
42     if temperature <= 1e-4:
43         # 退化为贪婪模式
44         max_idx = logits.index(max(logits))
45         return [1.0 if i == max_idx else 0.0 for i in range(len(logits))]
46
47     scaled_logits = [l / temperature for l in logits]
48     return softmax(scaled_logits)
49
50
51 # =====
52 # 2. Top-K 采样
53 # =====
54 def apply_top_k(logits, k=3):
55     """
56     仅保留概率/打分最高的前  $K$  个 Token, 其余位置全部置为  $-\infty$ 
57     """
58     # 按分数排序后取前  $K$  个【下标】。不能用"分数  $\geq$  第  $K$  大的值"做阈值:
59     # 有并列时会多选, 例如  $[1, 1, 1, 0]$  取  $K=2$  会留下 3 个。并列时按下标先后取 (与 torch.topk 的常见行为一致)。
60     # 先钳制  $K$ :  $k \leq 0$  时如果不处理,  $[:0]$  会返回空集、所有位置都是  $-1e9$ ,
61     # softmax 之后变成"均匀分布"——一个静默的错误。合法范围是  $[1, \text{len}(\text{logits})]$ 。
62     k = max(1, min(int(k), len(logits)))
63     keep = set(sorted(range(len(logits)), key=lambda i: -logits[i])[:k])
64     filtered_logits = [l if i in keep else -1e9 for i, l in enumerate(logits)]
65     return softmax(filtered_logits)
66
67
68 # =====
69 # 3. Top-P (核采样 Nucleus Sampling)
70 # =====
71 def apply_top_p(logits, p=0.85):
72     """
73     将候选词按概率从大到小排序, 累加其概率值。
74     一旦累计概率超过阈值  $P$ , 立即截断后续的长尾无意义词, 并在保留集合中重新归一化。
75     """
76     raw_probs = softmax(logits)
77     # 带索引排序
78     indexed_probs = sorted(enumerate(raw_probs), key=lambda x: x[1], reverse=True)
79
80     cum_sum = 0.0

```

```

81     keep_indices = set()
82     for idx, prob in indexed_probs:
83         keep_indices.add(idx)
84         cum_sum += prob
85         if cum_sum >= p:
86             break
87
88     # 构建截断后的 logits
89     filtered_logits = []
90     for i, l in enumerate(logits):
91         if i in keep_indices:
92             filtered_logits.append(l)
93         else:
94             filtered_logits.append(-1e9)
95
96     return softmax(filtered_logits)
97
98
99 # =====
100 # 4. 重复惩罚 (Repetition Penalty)
101 # =====
102 def apply_repetition_penalty(logits, past_token_indices, penalty=1.2):
103     """
104     对已经在前文中出现过的 Token 施加打分惩罚，降低连续复读概率
105     - 若 logit > 0: logit = logit / penalty
106     - 若 logit < 0: logit = logit * penalty
107     """
108     new_logits = list(logits)
109     for idx in past_token_indices:
110         l = new_logits[idx]
111         if l > 0:
112             new_logits[idx] = l / penalty
113         else:
114             new_logits[idx] = l * penalty
115     return new_logits
116
117
118 def run_experiment():
119     print("=" * 72)
120     print("[实验] 大模型采样算法 (Sampling Strategies) 微观数值对比")
121     print("=" * 72)
122
123     # 假定模型输出的一组候选词未归一化 Logits
124     words = ["北京", "上海", "广州", "深圳", "月球", "香蕉"]
125     # 模型认为"北京"最可能，但"月球"和"香蕉"也有微小的可能噪音
126     logits = [ 3.2,    2.8,    1.5,    1.2,    -1.0,    -2.5 ]
127
128     # 0. 贪婪搜索：直接取 argmax，不涉及概率

```

```

129 greedy = max(range(len(logits)), key=lambda i: logits[i])
130 print(f"\n[图表] 0. 贪婪搜索 (Greedy): argmax(logits) = '{words[greedy]}'，每次都
    选它，结果完全确定")
131
132 # 1. 原始 Softmax 分布
133 p_orig = softmax(logits)
134 print_distribution_table(words, p_orig, "1. 原始分布 (Temperature = 1.0, 默认自
    然状态)")
135
136 # 2. 低温 (T = 0.2)
137 p_cold = apply_temperature(logits, temperature=0.2)
138 print_distribution_table(words, p_cold, "2. 低温严肃模式 (Temperature = 0.2 ->
    头部概率暴涨)")
139
140 # 3. 高温 (T = 2.0)
141 p_hot = apply_temperature(logits, temperature=2.0)
142 print_distribution_table(words, p_hot, "3. 高温创造模式 (Temperature = 2.0 -> 分
    布平坦, 冷门词被激发)")
143
144 # 4. Top-K (K=2)
145 p_topk = apply_top_k(logits, k=2)
146 print_distribution_table(words, p_topk, "4. Top-K 截断 (K = 2 -> 仅保留最高前两
    名, 其余全部剔除)")
147
148 # 5. Top-P (P=0.85)
149 p_topp = apply_top_p(logits, p=0.85)
150 print_distribution_table(words, p_topp, "5. Top-P 核采样 (P = 0.85 -> 动态累加截
    断长尾有害词)")
151
152 # 6. 重复惩罚测试 (假设模型刚说完"北京", 惩罚已出现的词)
153 print("\n" + "=" * 72)
154 print("6. 重复惩罚机制 (Repetition Penalty = 1.5):")
155 penalized_logits = apply_repetition_penalty(logits, past_token_indices=[0],
    penalty=1.5)
156 p_penalized = softmax(penalized_logits)
157 print(f"    对历史词 '北京' 实施 1.5 倍惩罚后:")
158 print_distribution_table(words, p_penalized, "施加惩罚后的分布 (北京的概率被主动压
    低, 避免复读机现象)")
159 print("=" * 72)
160
161
162 def self_check():
163     """自检: 分布合法、Top-K 恰好保留 K 个 (含并列与 k<=0)、Top-P 保留的是概率最高的若干个
        """
164     for probs in [apply_temperature([3.2, 2.8, 1.5], t) for t in (0.2, 1.0, 2.0)]
        + [apply_top_p([3.2, 2.8, 1.5, -1.0], 0.85)]:
165         assert abs(sum(probs) - 1) < 1e-9 and min(probs) >= 0, "输出必须是一个合法分
            布"

```

```

166
167 # --- Top-K ---
168 assert sum(p > 1e-6 for p in apply_top_k([1.0, 1.0, 1.0, 0.0], k=2)) == 2, "并
    列时 Top-K 不应多选"
169 assert sum(p > 1e-6 for p in apply_top_k([3.2, 2.8, 1.5, 1.2, -1.0], k=3)) ==
    3
170 assert sum(p > 1e-6 for p in apply_top_k([3.2, 2.8, 1.5], k=99)) == 3, "k 超过
    词表时应退化为不截断"
171 # k<=0 不钳制的话, [:0] 会让所有位置都是 -1e9, softmax 后变成均匀分布 —— 静默的错误
172 for bad_k in (0, -1):
173     got = apply_top_k([3.2, 2.8, 1.5], k=bad_k)
174     assert abs(got[0] - 1.0) < 1e-9, f"k={bad_k} 应钳制为 1 (只留最大值), 而不是变
        成均匀分布"
175 # 恰好保留 K 个, 且保留的必须是分数最高的 K 个 (不能是随便 K 个)
176 kept = {i for i, p in enumerate(apply_top_k([1.0, 5.0, 2.0, 4.0, 3.0], k=2))
    if p > 1e-6}
177 assert kept == {1, 3}, f"Top-K 保留的应该是分数最高的两个下标 {{1,3}}, 实际 {kept}"
178
179 # --- Top-P: 保留的必须是"概率从大到小累加、刚好越过 P"的那个前缀 ---
180 lg = [3.2, 2.8, 1.5, 1.2, -1.0]
181 probs = softmax(lg)
182 order = sorted(range(len(lg)), key=lambda i: -probs[i])
183 for p in (0.3, 0.5, 0.85, 0.95):
184     kept = [i for i, v in enumerate(apply_top_p(lg, p)) if v > 1e-9]
185     cum, expect = 0.0, []
186     for i in order:
187         expect.append(i)
188         cum += probs[i]
189         if cum >= p:
190             break
191     assert sorted(kept) == sorted(expect), f"p={p}: Top-P 应保留 {sorted(expect)
        }, 实际 {sorted(kept)}"
192
193
194 if __name__ == "__main__":
195     self_check()
196     run_experiment()

```

第 20 章 纯 Python 零依赖 Qwen 模型真实权重推理

20.1 Lab 07b: 亲手实现 Qwen2.5 的推理，并与官方逐位对拍

核心导读与背景

这是整条学习路线里”从用库的人变成懂原理的人”的关键一步。

对应理论：第 4、5、6、7、8 篇全部。前置：完成 Lab 05（你已经在自己训练的模型上写过同样的结构）。依赖：PyTorch、transformers、safetensors、huggingface_hub（见根目录 requirements.txt）。第一次运行会下载约 1GB 权重，国内网络可先 export HF_ENDPOINT=https://hf-mirror.com。

20.2 规则

transformers 只允许用来做两件事：分词，以及作为标准答案和我们对拍。模型的前向传播、RoPE、GQA、KV Cache、重复惩罚全部自己写，直接读取 model.safetensors 里的原始张量。核心的 QwenFromScratch.forward（含 RMSNorm / RoPE / GQA 注意力 / SwiGLU / KV Cache）约 80 行。

20.3 运行

```
1 .venv/bin/python lab07b_qwen_from_scratch/qwen_from_scratch.py
```

20.4 本机（RTX 5060 Ti）一次运行的关键结果

实验 1~5 的数值是确定性的，并且由脚本断言；实验 6 是计时。

核心导读与背景

[计时] 实验 6 的计时数字是本机某一次运行的量级示例，前提是显卡空闲、已预热；GPU 降频或有其它负载时可能慢 40% 以上。请看趋势和比例，以你自己运行的输出为准。

实验	结果
1. 读 config.json	GQA 分组 7 (14 个 Q 头 / 2 个 KV 头), 每 token KV 仅 12 KB
2. 参数分布	词嵌入 28% 、FFN 64%、注意力只有 9%——小模型的词表开销惊人, 所以共享 LM Head
3. BPE 分词	”大模型推理的瓶颈是显存带宽。” → 11 个 token: 大/模型/推理/的/瓶颈/是/显/存/带/宽/。
4. logits 对拍 (FP32)	最大误差 1.5e-5 , 每个位置 top-1 全部一致
5. 生成对拍	纯贪婪逐 token 一致; 并且发现 HF 的 <code>generate(do_sample=False)</code> 偷偷使用了 <code>generation_config.json</code> 里的 <code>repetition_penalty=1.1</code> , 我们按第 7 篇公式手写惩罚后再次逐 token 一致
6. 性能 (BF16)	Prefill 1024 token $\approx$ 80 ms (约 1.25 万 token/s); Decode 约 20 ms/token, 只达到带宽下界 2.6 ms 的 一成左右 (本机多次运行在 7%~14% 之间波动, 取决于 GPU 频率与负载)
6. 朴素 vs Cache	上下文越长差距越大: 128 时两者接近 (约 0.7~1.7x, 此时单步耗时被 kernel 启动等固定开销主导, 谁快取决于当时的噪声), 2048 时约 9~11x

20.4.1 为什么 Decode 只达到理论上限的一成左右?

权重只有 1GB, 按带宽算每 token 只要 2.6ms, 但我们的实现每一层要启动十几个小 kernel (24 层就是几百个), 每个 kernel 启动都有微秒级开销, 逐元素算子还在反复读写显存, GQA 的 `repeat_interleave` 甚至把 KV 复制了 7 份。**这九成左右的差距, 就是 vLLM / llama.cpp / TensorRT-LLM 这些推理引擎存在的全部理由**: 算子融合、CUDA Graph、专用的 GEMV 与注意力 kernel (不复制 KV 的 GQA)。

20.5 动手练习 (按难度递增)

1. 把 `repeat_interleave` 去掉: 把 Q reshape 成 `[B, n_kv, group, T, hd]` 直接和 `[B, n_kv, 1, S, hd]` 的 K 做广播矩阵乘, 测 Decode 耗时变化。(第 8 篇 §4.3 说明了为什么这很重要)
2. 用 `torch.nn.functional.scaled_dot_product_attention` 替换手写的注意力, 确认结果不变, 比较 Prefill 耗时 (它内部会调用 `FlashAttention`, 见第 9 篇)。
3. 加入第 7 篇的 Temperature + Top-P 采样, 并用 `torch.manual_seed` 与 HF 的 `generate(do_sample=..., ...)` 对比分布是否一致 (提示: 逐 token 对拍需要完全相同的随机数使用顺序, 比较难; 改为比较多次采样的统计分布)。
4. 支持 `batch > 1`: 多个不同长度的请求一起 Decode, 需要 padding 与注意力掩码。体会为什么这件事很麻烦——这就是 Lab 12a (连续批处理) 和 Lab 12b (PagedAttention) 要解决的问题。
5. (进阶)参考 [gpt-fast](#) 用 `torch.compile(mode="reduce-overhead")` (内部使用 CUDA

Graph) 加速 Decode, 看能逼近带宽下界的多少。

6. (进阶) 把权重换成 Lab 11 里的分组 INT4 量化再反量化回 BF16, 测量 PPL 与生成质量的变化 (第 11 篇)。

20.6 学完之后

你已经能读懂任何 Llama 系模型的 `modeling_*.py`, 也知道性能差距在哪。下一步:

- 第 9 篇: FlashAttention + Lab 09
- 第 12 篇: 推理服务系统 + Lab 12b

20.7 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码, 重点实现细节如下:

20.7.1 源码实现: `qwen_from_scratch.py`

Listing 20.1: `qwen_from_scratch.py`

```
1 """
2 Lab 07b: 不用 transformers 的模型类, 只拿原始权重张量, 亲手实现 Qwen2.5 的完整推理
3
4 这是整条学习路线里"从使用者变成理解者"的关键一步:
5   - transformers 只用来做两件事: (1) 分词器 (2) 作为"标准答案"和我们对拍
6   - 前向传播、RoPE、GQA、KV Cache、采样全部自己写, 每一行都能对应到教程的公式
7
8 实验清单:
9   1. 读 config.json, 逐字段解释 (第 5 篇的自测题)
10  2. 读 safetensors, 看清每一层有哪些权重、形状是什么、参数都花在哪了
11  3. 看 BPE 分词器怎么切中文 (第 5 篇 §2)
12  4. 手写前向传播, 与 HF 官方实现逐位对拍 logits (FP32 下误差应 < 1e-3)
13  5. 手写 KV Cache 贪婪生成, 与 HF generate() 逐 token 对拍
14  6. 性能: Prefill 吞吐、Decode 每 token 耗时, 与第 8 篇的带宽下界对比; 朴素 vs KV Cache
15
16 运行:
17   .venv/bin/python lab07b_qwen_from_scratch/qwen_from_scratch.py
18 第一次运行会从 Hugging Face 下载 Qwen/Qwen2.5-0.5B-Instruct (约 1 GB)。
19 国内网络可先设置镜像: export HF_ENDPOINT=https://hf-mirror.com
20 """
21
22 import argparse
23 import json
24 import math
25 import os
```

```

26 import time
27
28 import torch
29 import torch.nn.functional as F
30 from safetensors.torch import load_file
31
32 MODEL_ID = "Qwen/Qwen2.5-0.5B-Instruct"
33
34
35 def title(s):
36     print("\n" + "=" * 80)
37     print(f"[实验] {s}")
38     print("=" * 80)
39
40
41 # =====
42 # 实验 1 & 2: 配置与权重
43 # =====
44 CONFIG_NOTES = {
45     "hidden_size": "d_model, 残差流的宽度",
46     "num_hidden_layers": "Transformer 层数 L",
47     "num_attention_heads": "Q 头数 n_h",
48     "num_key_value_heads": "KV 头数 n_kv (< n_h 说明是 GQA, 第 5 篇 §5.5) ",
49     "intermediate_size": "SwiGLU 中间维度 (第 5 篇 §5.3) ",
50     "vocab_size": "词表大小 V (第 5 篇 §2) ",
51     "rope_theta": "RoPE 底数 b, 越大越适合长上下文 (第 5 篇 §5.4) ",
52     "rms_norm_eps": "RMSNorm 分母里的  $\epsilon$ ",
53     "tie_word_embeddings": "词嵌入与 LM Head 共享同一个矩阵 (第 5 篇 §5.7) ",
54     "max_position_embeddings": "设计支持的最长上下文",
55     "torch_dtype": "权重存储精度 (第 8 篇 §6) ",
56     "hidden_act": "SwiGLU 门控用的激活函数",
57 }
58
59
60 def expl_config(model_dir):
61     title("实验 1: 逐字段读懂 config.json")
62     cfg = json.load(open(os.path.join(model_dir, "config.json")))
63     for k, note in CONFIG_NOTES.items():
64         print(f"    {k:<26} = {str(cfg[k]):<12} ← {note}")
65     hd = cfg["hidden_size"] // cfg["num_attention_heads"]
66     kv_per_tok = 2 * cfg["num_hidden_layers"] * cfg["num_key_value_heads"] * hd *
67     2
68     print(f"\n    推导量: head_dim = {cfg['hidden_size']} / {cfg['num_attention_heads']} = {hd}; "
69           f"GQA 分组 = {cfg['num_attention_heads']} // {cfg['num_key_value_heads']}")
70     print(f"    BF16 每 token KV Cache = 2 × {cfg['num_hidden_layers']} × {cfg['num_key_value_heads']} × {hd} × 2B = {kv_per_tok:,} 字节 ≈ {kv_per_tok /

```

```

1024:.0f} KB")
70     print(f"    → 在 16GB 显卡上, 扣掉 1GB 权重后, 理论上能缓存约 {14 * 1024**3 //
kv_per_tok / 1e3:.0f}K 个 token")
71     return cfg
72
73
74 def exp2_weights(model_dir, cfg):
75     title("实验 2: 打开 safetensors, 看看参数都花在哪了")
76     W = load_file(os.path.join(model_dir, "model.safetensors"))
77     total = sum(t.numel() for t in W.values())
78     print(f"    共 {len(W)} 个张量, {total / 1e6:.1f}M 参数, 存储类型 {next(iter(W.
values()))}.dtype}")
79     print("\n    第 0 层的全部权重 (注意 nn.Linear 的权重形状是 [out, in]) :")
80     for name, t in W.items():
81         if name.startswith("model.layers.0."):
82             print(f"        {name[len('model.layers.0.'):]:<34} {str(list(t.shape))
:<14} {t.numel() / 1e6:6.2f}M")
83     emb = W["model.embed_tokens.weight"].numel()
84     L = cfg["num_hidden_layers"]
85     attn = sum(t.numel() for n, t in W.items() if ".self_attn." in n)
86     mlp = sum(t.numel() for n, t in W.items() if ".mlp." in n)
87     print(f"\n    参数分布: 词嵌入 {emb / 1e6:.1f}M ({emb / total:.0%}) | 注意力 {attn
/ 1e6:.1f}M ({attn / total:.0%}) | "
88         f"FFN {mlp / 1e6:.1f}M ({mlp / total:.0%}) | 其余 {(total - emb - attn -
mlp) / 1e6:.2f}M")
89     print(f"    --> 小模型里词表矩阵占了 {emb / total:.0%}! 这就是为什么 0.5B 模型要让 LM
Head 与词嵌入共享权重。")
90     print(f"    注意力只占 {attn / total:.0%}: GQA 让 K/V 投影矩阵很小 ({cfg['
num_key_value_heads']} 个头 vs Q 的 {cfg['num_attention_heads']} 个)。")
91     if "lm_head.weight" not in W:
92         print("    (文件里没有 lm_head.weight — 因为 tie_word_embeddings=true, 推理
时直接复用 embed_tokens)")
93     return W
94
95
96 # =====
97 # 手写 Qwen2 的前向传播 (本文件的核心, 约 80 行)
98 # =====
99 class QwenFromScratch:
100     def __init__(self, W, cfg, device, dtype):
101         self.cfg = cfg
102         self.L = cfg["num_hidden_layers"]
103         self.n_h = cfg["num_attention_heads"]
104         self.n_kv = cfg["num_key_value_heads"]
105         self.hd = cfg["hidden_size"] // self.n_h
106         self.eps = cfg["rms_norm_eps"]
107         self.dtype, self.device = dtype, device
108         self.W = {k: v.to(device=device, dtype=dtype) for k, v in W.items()}

```

```

109         if "lm_head.weight" not in self.W:
110             self.W["lm_head.weight"] = self.W["model.embed_tokens.weight"] # 权重共享
111             # RoPE 频率表:  $\theta_i = \text{base}^{(-2i/d)}$ , 算到 32K 位置
112             inv_freq = 1.0 / (cfg["rope_theta"] ** (torch.arange(0, self.hd, 2, device=device).float() / self.hd))
113             ang = torch.outer(torch.arange(32768, device=device).float(), inv_freq)
114             # [pos, hd/2]
115             self.cos, self.sin = ang.cos(), ang.sin()
116
117         def rms_norm(self, x, w):
118             xf = x.float()
119             xf = xf * torch.rsqrt(xf.pow(2).mean(-1, keepdim=True) + self.eps)
120             return xf.to(self.dtype) * w
121
122         def rope(self, x, pos0):
123             """x: [B, H, T, hd]; 前半与后半配对旋转 (与 HF 的 rotate_half 等价)"""
124             T = x.shape[2]
125             cos = self.cos[pos0:pos0 + T].to(x.dtype)
126             sin = self.sin[pos0:pos0 + T].to(x.dtype)
127             x1, x2 = x[..., : self.hd // 2], x[..., self.hd // 2:]
128             return torch.cat([x1 * cos - x2 * sin, x1 * sin + x2 * cos], dim=-1)
129
130         def new_cache(self, max_len):
131             """预分配 KV Cache: 每层一块 [1, n_kv, max_len, hd], 按位置写入 (比 torch.cat 反复拼接更接近真实引擎)"""
132             shape = (1, self.n_kv, max_len, self.hd)
133             return [(torch.empty(shape, device=self.device, dtype=self.dtype),
134                     torch.empty(shape, device=self.device, dtype=self.dtype)) for _
135                     in range(self.L)]
136
137     @torch.inference_mode()
138     def forward(self, ids, cache=None, pos0=0):
139         """ids: [1, T]; cache 为 None 时做不带缓存的完整前向; 返回 logits [1, T, V]"""
140         W, P = self.W, "model.layers."
141         B, T = ids.shape
142         x = F.embedding(ids, W["model.embed_tokens.weight"])
143         # [B, T, d]
144         for i in range(self.L):
145             p = f"{P}{i}."
146             # ----- 注意力子层 -----
147             h = self.rms_norm(x, W[p + "input_layernorm.weight"])
148             q = F.linear(h, W[p + "self_attn.q_proj.weight"], W[p + "self_attn.q_proj.bias"]) # Qwen2 的 QKV 带 bias
149             k = F.linear(h, W[p + "self_attn.k_proj.weight"], W[p + "self_attn.k_proj.bias"])
150             v = F.linear(h, W[p + "self_attn.v_proj.weight"], W[p + "self_attn.v_proj.bias"])

```

```

148         q = q.view(B, T, self.n_h, self.hd).transpose(1, 2)
149         # [B, n_h, T, hd]
150         k = k.view(B, T, self.n_kv, self.hd).transpose(1, 2)
151         # [B, n_kv, T, hd]
152         v = v.view(B, T, self.n_kv, self.hd).transpose(1, 2)
153         q, k = self.rope(q, pos0), self.rope(k, pos0)
154         # K 旋转后再进缓存
155
156         if cache is not None:
157             kc, vc = cache[i]
158             kc[:, :, pos0:pos0 + T] = k
159             vc[:, :, pos0:pos0 + T] = v
160             k, v = kc[:, :, :pos0 + T], vc[:, :, :pos0 + T]
161             S = k.shape[2]
162
163         rep = self.n_h // self.n_kv
164         # GQA 展开
165         k = k.repeat_interleave(rep, dim=1)
166         v = v.repeat_interleave(rep, dim=1)
167         scores = (q @ k.transpose(-1, -2)) / math.sqrt(self.hd)
168         # [B, n_h, T, S]
169         if T > 1: # Decode (T=1) 时新 token 能看到全部历史, 不需要掩码
170             q_pos = torch.arange(pos0, pos0 + T, device=ids.device)[:, None]
171             k_pos = torch.arange(S, device=ids.device)[None, :]
172             scores = scores.masked_fill(k_pos > q_pos, float("-inf"))
173             attn = F.softmax(scores.float(), dim=-1).to(self.dtype) @ v
174         # [B, n_h, T, hd]
175         attn = attn.transpose(1, 2).reshape(B, T, self.n_h * self.hd)
176         x = x + F.linear(attn, W[p + "self_attn.o_proj.weight"])
177         # 残差 1
178         # ----- SwiGLU 子层 -----
179         h = self.rms_norm(x, W[p + "post_attention_layernorm.weight"])
180         gate = F.linear(h, W[p + "mlp.gate_proj.weight"])
181         up = F.linear(h, W[p + "mlp.up_proj.weight"])
182         x = x + F.linear(F.silu(gate) * up, W[p + "mlp.down_proj.weight"])
183         # 残差 2
184         x = self.rms_norm(x, W["model.norm.weight"])
185         return F.linear(x, W["lm_head.weight"])
186         # [B, T, V]
187
188 @torch.inference_mode()
189 def generate(self, ids, max_new, eos_ids=(), use_cache=True,
190             repetition_penalty=1.0):
191     """贪婪解码。返回 (新 token 列表, prefill 秒数, 每步 decode 秒数列表)
192     repetition_penalty: 第 7 篇 §5 的公式, 对【提示词 + 已生成】里出现过的 token: 正
193     logit 除以  $\theta$ , 负 logit 乘以  $\theta$ """
194     ids = ids.to(self.device)
195     out, dec_times = [], []

```

```

185     sync = torch.cuda.synchronize if self.device.startswith("cuda") else (
186         lambda: None)
187     cache = self.new_cache(ids.shape[1] + max_new) if use_cache else None
188
189     sync(); t0 = time.perf_counter()
190     logits = self.forward(ids, cache, 0)[: , -1]
191     # Prefill
192     sync(); t_prefill = time.perf_counter() - t0
193     seq = ids
194     seen = ids[0].tolist()
195     for step in range(max_new):
196         if repetition_penalty != 1.0:
197             idx = torch.tensor(sorted(set(seen)), device=self.device)
198             sc = logits[0, idx]
199             logits[0, idx] = torch.where(sc > 0, sc / repetition_penalty, sc *
200 repetition_penalty)
201             nxt = logits.argmax(-1, keepdim=True)
202             # [1, 1]
203             seen.append(int(nxt))
204             out.append(int(nxt))
205             if int(nxt) in eos_ids:
206                 break
207             sync(); t0 = time.perf_counter()
208             if use_cache:
209                 logits = self.forward(nxt, cache, ids.shape[1] + step)[: , -1]
210             # 只送 1 个 token
211             else:
212                 seq = torch.cat([seq, nxt], dim=1)
213                 logits = self.forward(seq)[: , -1]
214             # 整段重算
215             sync(); dec_times.append(time.perf_counter() - t0)
216     return out, t_prefill, dec_times
217
218 # =====
219 # 实验 3 ~ 6
220 # =====
221 def exp3_tokenizer(tok):
222     title("实验 3: BPE 分词器是怎么切中文的 (第 5 篇 §2) ")
223     for text in ["大模型推理的瓶颈是显存带宽。", "KV Cache", "Transformer", "饕餮"]:
224         ids = tok.encode(text)
225         pieces = [tok.decode([i]) for i in ids]
226         print(f"    {text!r:<22} → {len(ids):2d} 个 token: {pieces}")
227     print("    --> 常见词整体成一个 token, 罕见字被拆成多个字节级片段 (解码单个片段可能显示为
228 乱码 ☹️)。")
229
230 def exp4_logits_match(model_dir, tok, W, cfg, device):

```

```

226 title("实验 4: 手写前向 vs HF 官方实现 —— 逐位对拍 logits (FP32)")
227 from transformers import AutoModelForCausalLM
228 ref = AutoModelForCausalLM.from_pretrained(model_dir, dtype=torch.float32,
229 attn_implementation="eager").to(device).eval()
230 mine = QwenFromScratch(W, cfg, device, torch.float32)
231 ids = torch.tensor([tok.encode("大模型推理的本质是自回归地预测下一个 token, ")]),
232 device=device)
233 with torch.inference_mode():
234     ref_logits = ref(ids).logits
235 my_logits = mine.forward(ids)
236 diff = (ref_logits - my_logits).abs().max().item()
237 same_top1 = bool((ref_logits.argmax(-1) == my_logits.argmax(-1)).all())
238 print(f"    输入 {ids.shape[1]} 个 token, logits 形状 {list(my_logits.shape)}")
239 print(f"    最大绝对误差 = {diff:.2e}, 每个位置的 top-1 预测{'全部一致 [OK]' if
240 same_top1 else '存在不一致 [X]'}")
241 assert diff < 1e-3 and same_top1, '手写前向与 HF 官方不一致'
242 print("    --> 误差只来自浮点运算顺序不同。你写的每一行 (RMSNorm、RoPE、GQA、bias、
243 SwiGLU、权重共享) 都和官方一致。")
244 return ref, mine
245
246 def exp5_generate_match(tok, ref, mine, device):
247     title("实验 5: 手写 KV Cache 贪婪生成 vs HF generate() —— 逐 token 对拍")
248     msgs = [{"role": "user", "content": "用两句话解释什么是 KV Cache。"}]
249     text = tok.apply_chat_template(msgs, tokenize=False, add_generation_prompt=
250 True)
251     ids = torch.tensor([tok.encode(text)], device=device)
252     eos = {tok.convert_tokens_to_ids("<|im_end|>"), tok.eos_token_id}
253     print(f"    提示词 (套用聊天模板后 {ids.shape[1]} 个 token) : {msgs[0]['content']}")
254 )
255     print(f"    聊天模板展开后的真实输入: {text!r}")
256
257 def compare(tag, ref_kwargs, my_penalty):
258     with torch.inference_mode():
259         ref_out = ref.generate(ids, max_new_tokens=60, do_sample=False, **
260 ref_kwargs)[0, ids.shape[1]:].tolist()
261         my_out, _, _ = mine.generate(ids, 60, eos_ids=eos, repetition_penalty=
262 my_penalty)
263         ref_out = [t for t in ref_out if t != tok.pad_token_id]
264         print(f"\n    [{tag}] ")
265         print(f"    我的实现: {tok.decode(my_out, skip_special_tokens=True)}")
266         print(f"    HF 官方 : {tok.decode(ref_out, skip_special_tokens=True)}")
267         print(f"    {len(my_out)} vs {len(ref_out)} 个 token, {'逐 token 完全一致 [OK]
268 }' if my_out == ref_out else '不一致 [X]'}")
269         assert my_out == ref_out, f'{tag}: 生成结果与 HF 官方不一致'
270
271 compare("纯贪婪: 显式关闭重复惩罚", dict(repetition_penalty=1.0), 1.0)
272 compare("HF 默认行为: generation_config.json 里藏着 repetition_penalty=1.1",

```

```

dict(), 1.1)
265 print("\n    --> 两个教训:")
266 print("        1. 只写 do_sample=False 时, HF 仍然会读取模型自带的 generation_config
.json, 偷偷套用 repetition_penalty=1.1。")
267 print("        '用库'时这类隐藏默认值随处可见; 自己实现一遍才知道每个输出到底是怎么来的。
")
268 print("        2. 我们按第 7 篇 §5 的公式手写重复惩罚后, 与官方再次逐 token 一致 —— 公
式就是全部真相。")
269 return ids
270
271
272 def exp6_perf(W, cfg, tok, device, bw_gbs):
273     title("实验 6: 性能 —— Prefill 吞吐、Decode 耗时与第 8 篇的带宽下界 (BF16)")
274     mine = QwenFromScratch(W, cfg, device, torch.bfloat16)
275     n_params = sum(t.numel() for t in W.values())
276     w_bytes = n_params * 2
277
278     # Prefill: 一次性处理长 prompt
279     long_ids = torch.randint(0, 150000, (1, 1024), device=device)
280     mine.generate(long_ids[:, :32], 4) # 预热
281     _, t_pf, dec = mine.generate(long_ids, 32)
282     flops = 2 * n_params * 1024
283     print(f"    Prefill 1024 个 token: {t_pf * 1e3:.1f} ms → {1024 / t_pf:,.0f}
token/s, "
f"约 {flops / t_pf / 1e12:.1f} TFLOP/s (线性层部分, 按 2N FLOPs/token 估算)
")
284
285     per_tok = sum(dec) / len(dec)
286     bound = w_bytes / (bw_gbs * 1e9)
287     print(f"    Decode (上下文 ~1K) : {per_tok * 1e3:.2f} ms/token → {1 / per_tok:,.0
f} token/s")
288     print(f"    带宽下界: 权重 {w_bytes / 1e9:.2f} GB / {bw_gbs:.0f} GB/s = {bound *
1e3:.2f} ms/token ({1 / bound:.0f} token/s 上限) ")
289     print(f"    --> 我们只达到了上限的 {bound / per_tok:.0%}。差距来自: 24 层 × 每层十几个
小 kernel 的启动开销、")
290     print("        未融合的逐元素算子反复读写显存、repeat_interleave 复制 KV 等。vLLM /
llama.cpp 用算子融合、")
291     print("        CUDA Graph、专用 GEMV 与注意力 kernel 把这些开销吃掉——这正是推理引擎存
在的意义 (capstone_engines_practice) 。")
292
293     print("\n    朴素 (每步重算整段) vs KV Cache, 在不同上下文长度下每生成 1 个 token 的耗时
:")
294     print(f"        {'上下文':>8} | {'朴素 ms':>9} | {'KV Cache ms':>12} | {'倍数':>6}")
295     for ctx in [128, 512, 1024, 2048]:
296         ids = torch.randint(0, 150000, (1, ctx), device=device)
297         _, _, d_naive = mine.generate(ids, 6, use_cache=False)
298         _, _, d_cache = mine.generate(ids, 6, use_cache=True)
299         a, b = sum(d_naive[1:]) / len(d_naive[1:]), sum(d_cache[1:]) / len(d_cache
[1:])

```

```

300     print(f"    {ctx:>8} | {a * 1e3:9.2f} | {b * 1e3:12.2f} | {a / b:5.1f}x")
301     print("    --> 与 Lab 05 的迷你模型不同，真实模型上重算的代价在 GPU 上也清清楚楚：上下文
      越长差距越大。")
302
303
304 def main():
305     ap = argparse.ArgumentParser()
306     ap.add_argument("--device", default="cuda" if torch.cuda.is_available() else "
      cpu")
307     ap.add_argument("--bandwidth", type=float, default=380.0, help="你的显卡实测带宽
      GB/s (见 measure_gpu_roofline.py) ")
308     args = ap.parse_args()
309     torch.backends.cuda.matmul.allow_tf32 = False
310
311     from huggingface_hub import snapshot_download
312     from transformers import AutoTokenizer
313     model_dir = snapshot_download(MODEL_ID, allow_patterns=["*.json", "*.
      safetensors", "*.txt"])
314     tok = AutoTokenizer.from_pretrained(model_dir)
315
316     cfg = expl_config(model_dir)
317     W = exp2_weights(model_dir, cfg)
318     exp3_tokenizer(tok)
319     ref, mine = exp4_logits_match(model_dir, tok, W, cfg, args.device)
320     exp5_generate_match(tok, ref, mine, args.device)
321     del ref, mine
322     if args.device.startswith("cuda"):
323         torch.cuda.empty_cache()
324         exp6_perf(W, cfg, tok, args.device, args.bandwidth)
325
326
327 if __name__ == "__main__":
328     main()

```

第 21 章 显存占用严谨核算与 GPU 算力 Roofline 实测

21.1 Lab 08：显存开销与 Roofline 性能模型

在进行大模型推理系统设计或显卡选型时，有两大黄金法则：

1. **显存容量法则**：决定了你「能不能跑得下」（模型参数 + KV Cache + 运行时开销 $\leq$ 显存容量）。
2. **Roofline 性能模型与访存带宽法则**：决定了你「能跑多快」（自回归解码受限于 GPU 显存带宽）。

21.2 [推导] 1. 显存开销组成详解

大模型在推理运行时的显存占用主要由以下四部分组成：

$$\text{Total Memory} = \text{Mem}_{\text{weights}} + \text{Mem}_{\text{KV_Cache}} + \text{Mem}_{\text{activation}} + \text{Mem}_{\text{CUDA_overhead}}$$

21.2.1 (1) 模型权重 (Model Weights)

- **FP16 / BF16**: 每个参数占用 **2 字节 (Bytes)**。
- 例如 7B 模型需要约 $7 \times 2 = 14$ GB。
- **INT8 / FP8**: 每个参数占用 **1 字节** (7B 约 7 GB)。
- **INT4 / AWQ / GPTQ**: 每个参数占用 **0.5 字节** (7B 约 3.5 GB)。

21.2.2 (2) KV Cache 显存

这是动态显存开销的最大头，随并发请求数和序列长度成正比膨胀：对于采用分组查询注意力（GQA, Grouped-Query Attention）的模型：

- 层数 L
- KV 头数 $n_{\text{kv_heads}}$ ，单头维度 d_{head}
- 并发请求数 B (Batch Size)

- 序列总长度 S (Prompt 长度 + 已生成长度)
- 存储精度字节数 b (FP16 为 2)

$$\text{KV Cache (Bytes)} = 2 \times L \times (n_{\text{kv_heads}} \times d_{\text{head}}) \times S \times B \times b$$

核心直觉与思考

[思考] **MHA vs GQA 的显存革命**: - 早期模型 (如 LLaMA-1 65B) 使用 Multi-Head Attention (MHA), KV 头数等于 Query 头数 (例如 64 头)。- 现代模型 (如 Llama-3、Qwen2.5) 几乎全部采用 GQA (例如 8 个 KV 头), KV Cache 显存直接暴降为原来的 $1/4 \sim 1/8$!

21.2.3 (3) 激活值 (Activations)

在 Prefill 阶段较大 (因为输入序列长), 但在 Decode 阶段因为一次只输入 1 个 token, 激活值极小 (通常仅几十 MB 到几百 MB)。

21.2.4 (4) CUDA 上下文与运行时开销

PyTorch / CUDA runtime、NCCL 通信缓冲区等, 通常占用约 1 ~ 2 GB。

21.3 [高速] 2. Roofline 模型与为什么 Decode 是访存受限?

21.3.1 计算密度 (Arithmetic Intensity)

$$\text{Arithmetic Intensity (AI)} = \frac{\text{计算量 (FLOPs)}}{\text{访存数据量 (Bytes)}}$$

GPU 的极限速度受制于:

- 最大算力 (TFLOPs)
- 显存带宽 (GB/s)

两者的交界点为硬件的转折算术密度 $AI_{\text{roof}} = \frac{\text{Peak FLOPs}}{\text{Peak Bandwidth}}$ 。

阶段	输入规模	计算量	访存量	算术强度 AI	性能瓶颈
Prefill 阶段	N 个 token 并行	$O(N \cdot P)$ 大矩阵乘法	读取一次模型权重 P	很高 (> 100)	算力受限 (Compute-Bound) , GPU 计算核心满载
Decode 阶段	每次仅 1 个 token	仅对 1 个 token 做矩阵-向量乘	必须把全部模型权重和全部历史 KV Cache 从显存重新读一遍!	极低 ($\approx 1 \sim 2$)	访存受限 (Memory-Bound) , 算力被闲置, 卡在显存带宽搬运上

21.3.2 单 Token 解码耗时下界理论估算

在 Batch Size = 1 时, 生成 1 个 token 必须把整个模型权重读取一遍:

$$T_{\text{decode_min}} \approx \frac{\text{模型权重体积 (Bytes)}}{\text{显存有效读取带宽 (Bytes/s)}}$$

核心导读与背景

例如: 在显存带宽为 448 GB/s 的显卡上运行 14 GB 的 7B FP16 模型: 每生成 1 个 token 至少需要 $14 \text{ GB} / 448 \text{ GB/s} \approx 31.25 \text{ ms}$ (即吞吐上限约为 32 tokens/s)。如果通过 INT4 量化将权重压缩为 3.5 GB, 理论下界降为 $3.5 / 448 \approx 7.8 \text{ ms}$ (吞吐暴涨到 128 tokens/s)! 这就是量化在推理阶段能带来巨大速度提升的根本原因!

[注意] 上面的 448 GB/s 是**厂商标称值**, 实际达不到 (本机实测约 380 GB/s, 见下面”一次实测结果”)。用实测值算, 同一个例子的下界是 $14 / 380 \approx 36.8 \text{ ms}$ 而不是 31.25 ms —— 标称值会乐观约 18%。memory_calculator.py 现在两种都打印, 默认用实测值。

核心导读与背景

单位口径: 本节算例用**十进制 GB** ($7\text{B} \times 2 \text{ 字节} = 14 \times 10^9 \text{ B} = 14 \text{ GB}$), 因为要和 GB/s 相除。而 memory_calculator.py 的权重/KV 打印用 **GiB** (除以 1024^3), 因为要和显卡容量比较。同一个 7B FP16 模型: $14 \text{ GB} = 13.04 \text{ GiB}$, **两个都对, 差 7.4%**。混用是这一行最常见的错误, 脚本里专门有一段注释说明。

21.4 [工具] 显存与性能计算工具使用

运行 memory_calculator.py:

```
1 python3 lab08_memory_and_roofline/memory_calculator.py
```

可评估主流模型（Llama-3-8B/70B、Qwen-2.5-7B/72B 等）在各类 GPU（如 RTX 5060 Ti、RTX 4090、A100）下的显存与并发上限。

21.5 [实验] 在你自己的显卡上实测 Roofline（需要 PyTorch）

理论推导见 第 8 篇：硬件视角。运行：

```
1 .venv/bin/python lab08_memory_and_roofline/measure_gpu_roofline.py
```

它会测出你这张卡的真实带宽 β 、BF16 峰值 π 、拐点 $AI^* = \pi/\beta$ ，并验证两个关键推导：

- 1. **Decode 线性层的算术强度 $\approx$ batch size**：batch 从 1 到 16，耗时几乎不变；
- 2. **Decode 注意力的算术强度 $\approx$ GQA 分组大小**：与 batch 无关，所以 GQA/MLA 不仅省显存，还直接提速。

本机（RTX 5060 Ti 16GB）的一次实测结果：

核心导读与背景

[计时] 以下计时类数字是本机某一次运行的量级示例，前提是显卡空闲、已预热；GPU 降频或有其它负载时可能慢 40% 以上。请看趋势和比例，以你自己运行的输出为准。例如有评审在并发负载下测到过 217~234 GB/s 的带宽。

指标	厂商标称	实测
显存带宽	448 GB/s	≈ 380 GB/s（拷贝）， ≈ 400 GB/s（GEMV 读权重）
BF16 Tensor 稠密算力	宣传口径更高	≈ 48 TFLOP/s
拐点 AI^*	—	≈ 130 FLOP/Byte：batch < 100 左右的 Decode 都是访存受限

避坑指南与核心警示

[注意] 实测中还会看到 batch = 32/64 的线性层比 batch = 128 还慢——这是 cuBLAS 对这些形状选中了低效 kernel，是真实存在的现象，不是噪声。它提醒你：**Roofline 是上界，实现质量决定你离上界有多远。**

21.6 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码，重点实现细节如下：

21.6.1 源码实现：memory_calculator.py

Listing 21.1: memory_calculator.py

```
1  """
2  Lab 08: 显存占用与 Roofline 解码速率理论计算工具 (无需第三方依赖)
3
4  支持主流大模型 (Llama-3、Qwen-2.5、DeepSeek 等) 在不同量化精度 (FP16/INT8/INT4) 下:
5  1. 权重显存与每 Token KV Cache 显存测算
6  2. 指定 GPU 显存容量下的最大并发数 (Max Batch Size) 与最大上下文推演
7  3. 基于显存带宽 (Memory Bandwidth) 评估 Decode 阶段单 Token 理论生成耗时下界
8  """
9
10 # =====
11 # 典型模型架构参数库
12 # =====
13 MODELS = {
14     "Llama-3-8B": {
15         "params_b": 8.03,
16         "layers": 32,
17         "q_heads": 32,
18         "kv_heads": 8,          # GQA 组数 (8 vs 32)
19         "head_dim": 128,
20         "hidden_size": 4096,
21         "vocab_size": 128256,
22     },
23     "Llama-3-70B": {
24         "params_b": 70.6,
25         "layers": 80,
26         "q_heads": 64,
27         "kv_heads": 8,          # GQA 组数 (8 vs 64, KV 压缩 8 倍!)
28         "head_dim": 128,
29         "hidden_size": 8192,
30         "vocab_size": 128256,
31     },
32     "Qwen2.5-7B": {
33         "params_b": 7.61,
34         "layers": 28,
35         "q_heads": 28,
36         "kv_heads": 4,          # GQA
37         "head_dim": 128,
38         "hidden_size": 3584,
39         # 注意: Qwen2.5 小尺寸 (0.5B/1.5B/3B) 词表是 151936 且与 LM Head 共享权重,
40         # 7B 及以上才是 152064 且不共享。这里两个都是 7B+ 的配置 (第 5 篇 §5.2)。
41         "vocab_size": 152064,
42     },
43     "Qwen2.5-72B": {
44         "params_b": 72.7,
```

```

45     "layers": 80,
46     "q_heads": 64,
47     "kv_heads": 8,          # GQA
48     "head_dim": 128,
49     "hidden_size": 8192,
50     "vocab_size": 152064,
51 }
52 }
53
54 # =====
55 # 典型显卡硬件规格库
56 # 注意: bandwidth_gbs 用的是  $10^9$  字节/秒 (厂商口径), fp16_tflops 是厂商标称的 Tensor
    Core 稠密峰值,
57 # GeForce 消费卡在 FP32 累加模式下通常只有标称值的一半, 且实际可达带宽一般只有标称的 80%~90%
    。
58 # 以 lab08_memory_and_roofline/measure_gpu_roofline.py 在你机器上的实测值为准。
59 # =====
60 GPUS = {
61     "RTX 5060 Ti (16GB) [本机配置]": {
62         "vram_gb": 16.0,
63         "bandwidth_gbs": 448.0,          # 标称值 (厂商标注的 448 GB/s)
64         "bandwidth_gbs_measured": 380.0, # measure_gpu_roofline.py 实测拷贝带宽
65         "fp16_tflops": 48.0             # 本机实测 BF16 稠密矩阵乘约 48 TFLOP/s (
        FP32 累加)
66     },
67     "RTX 4090 (24GB)": {
68         "vram_gb": 24.0,
69         "bandwidth_gbs": 1008.0,
70         "fp16_tflops": 165.0          # FP32 累加口径 (FP16 累加口径为 330)
71     },
72     "NVIDIA A100-SXM4 (80GB)": {
73         "vram_gb": 80.0,
74         "bandwidth_gbs": 2039.0,
75         "fp16_tflops": 312.0
76     },
77     "NVIDIA H100-SXM5 (80GB)": {
78         "vram_gb": 80.0,
79         "bandwidth_gbs": 3350.0,
80         "fp16_tflops": 989.0
81     }
82 }
83
84
85 def calculate_kv_cache_per_token_bytes(model_cfg, kv_bits=16):
86     """
87     计算单个 Token 产生的 Key 和 Value 向量占用的总字节数
88     KV Cache bytes/token = 2 (K & V) * layers * (kv_heads * head_dim) * (kv_bits /
        8)

```

```

89     """
90     bytes_per_elem = kv_bits / 8.0
91     layers = model_cfg["layers"]
92     kv_heads = model_cfg["kv_heads"]
93     head_dim = model_cfg["head_dim"]
94     return 2 * layers * (kv_heads * head_dim) * bytes_per_elem
95
96
97 def calculate_weight_memory_gb(params_b, weight_bits=16):
98     """
99     计算模型权重显存占用，单位 GiB（除以  $1024^3$ ）。
100
101     注意这里的口径：参数量按十进制算（ $1B = 10^9$  个参数），存储按二进制换算（ $1024^3$ ）。
102     所以  $1B$  参数在  $16\text{-bit}$  下是  $1.86\text{ GiB}$ ，不是“ $2GB$ ”。
103     这么选是为了和 GPU 的 vram_gb（ $16GB$  卡 =  $16\text{ GiB}$  容量）能直接比较 ——
104     上面那段 Roofline 计算用的是十进制  $GB$ （bytes / 1e9），因为要和  $GB/s$  相除，两处故意不一致，
105     下面的打印会把单位写清楚。
106     """
107     bytes_per_param = weight_bits / 8.0
108     total_bytes = params_b * (10**9) * bytes_per_param
109     return total_bytes / (1024**3)
110
111
112 def print_model_analysis(model_name, gpu_name, context_len=4096, batch_size=4):
113     model = MODELS[model_name]
114     gpu = GPUS[gpu_name]
115
116     print("=" * 72)
117     print(f"[图表] 模型推理性能评估: {model_name} on {gpu_name}")
118     print("=" * 72)
119
120     # 1. 权重占用
121     w_fp16 = calculate_weight_memory_gb(model["params_b"], 16)
122     w_int8 = calculate_weight_memory_gb(model["params_b"], 8)
123     w_int4 = calculate_weight_memory_gb(model["params_b"], 4)
124
125     print(f"[重点] 模型基本参数: {model['params_b']}B 参数 | {model['layers']} 层 | Q  
头={model['q_heads']} | KV头={model['kv_heads']}")
126     print(f"[重点] 权重显存需求 (GiB = bytes/1024³, 和显卡容量同口径) :")
127     print(f"    - FP16 (16-bit) : {w_fp16:6.2f} GiB")
128     print(f"    - INT8 / FP8      : {w_int8:6.2f} GiB")
129     print(f"    - INT4 / AWQ      : {w_int4:6.2f} GiB")
130
131     # 2. KV Cache 计算
132     kv_fp16_bytes = calculate_kv_cache_per_token_bytes(model, 16)
133     kv_int8_bytes = calculate_kv_cache_per_token_bytes(model, 8)

```

```

134 kv_int4_bytes = calculate_kv_cache_per_token_bytes(model, 4)
135
136 print(f"\n[重点] 单 Token KV Cache 显存:")
137 print(f"    - FP16 KV Cache : {kv_fp16_bytes / 1024:6.2f} KB/token")
138 print(f"    - INT8 KV Cache : {kv_int8_bytes / 1024:6.2f} KB/token")
139 print(f"    - INT4 KV Cache : {kv_int4_bytes / 1024:6.2f} KB/token")
140
141 # 3. 指定场景显存拆解 (如 Batch=4, Context=4096)
142 print(f"\n[重点] 场景评估 (并发数={batch_size}, 序列长度={context_len} tokens):")
143 total_tokens = batch_size * context_len
144 total_kv_fp16_gb = (total_tokens * kv_fp16_bytes) / (1024**3)
145 cuda_overhead_gb = 1.2 # 运行时驱动与环境开销
146
147 total_mem_fp16 = w_fp16 + total_kv_fp16_gb + cuda_overhead_gb
148 total_mem_int4 = w_int4 + total_kv_fp16_gb + cuda_overhead_gb
149
150 print(f"    - 全量 Token 总数    : {total_tokens} tokens")
151 print(f"    - KV Cache 总占用    : {total_kv_fp16_gb:6.2f} GiB (FP16)")
152 print(f"    - FP16 总显存需求    : {total_mem_fp16:6.2f} GiB (权重 {w_fp16:.1f} + KV {total_kv_fp16_gb:.1f} + 基础开销 {cuda_overhead_gb:.1f})")
153 print(f"    - INT4 权重总显存    : {total_mem_int4:6.2f} GiB (权重 {w_int4:.1f} + KV {total_kv_fp16_gb:.1f} + 基础开销 {cuda_overhead_gb:.1f})")
154
155 vram_avail = gpu["vram_gb"]
156 print(f"\n[目标] 硬件显存匹配状态 ({gpu_name} - {vram_avail:.0f} GiB):")
157 if total_mem_fp16 <= vram_avail:
158     print(f"    [OK] FP16 原生精度: 显存放得下 (剩余 {vram_avail - total_mem_fp16:.2f} GiB) ")
159 elif total_mem_int4 <= vram_avail:
160     print(f"    [注意] FP16 显存不足 (超标 {total_mem_fp16 - vram_avail:.2f} GiB) , 但 INT4/AWQ 量化后放得下 (剩余 {vram_avail - total_mem_int4:.2f} GiB)")
161 else:
162     print(f"    [X] INT4 量化后仍超出容量 (超标 {total_mem_int4 - vram_avail:.2f} GiB), 需多卡张量并行 (TP) 或 Offload")
163 print(f"    [注意] 这只是【静态显存账本】。真实部署还要算上激活、CUDA Context、碎片, "
164       f"1.2 GiB 的开销常数很乐观——贴着容量上限的配置 (比如 7B FP16 放在 16 GiB 上) 实际常常 OOM。")
165
166 # 4. Roofline 显存带宽解码速率下界 (Batch Size = 1)
167 # 注意单位: 权重按字节算, 带宽是 10^9 B/s, 不要把 GiB 和 GB/s 直接相除 (会差 7.4%)
168 # [注意] 这里必须用【实测】带宽, 不能用标称值: 标称是硬件上界, 实际达不到。
169 # lab07b 的手写实现也默认用 380 GB/s, 两个 lab 要同口径才能互相印证。
170 bw = gpu.get("bandwidth_gbs_measured", gpu["bandwidth_gbs"])
171 bw_nominal = gpu["bandwidth_gbs"]
172 w_fp16_bytes = model["params_b"] * 1e9 * 2
173 w_int4_bytes = model["params_b"] * 1e9 * 0.5
174 # 理论单 token 最低耗时 (ms) = 模型权重读取时间
175 t_fp16_ms = w_fp16_bytes / (bw * 1e9) * 1000.0

```

```

176 t_int4_ms = w_int4_bytes / (bw * 1e9) * 1000.0
177 speed_fp16 = 1000.0 / t_fp16_ms if t_fp16_ms > 0 else 0
178 speed_int4 = 1000.0 / t_int4_ms if t_int4_ms > 0 else 0
179
180 print(f"\n[加速] Decode 阶段理论单流最大生成速度")
181 print(f"      (用实测带宽 {bw:.0f} GB/s 计算; 标称 {bw_nominal:.0f} GB/s 只是硬件上
182      界, "
183      f"用它算会乐观 {(bw_nominal / bw - 1) * 100:.0f}%) ")
184 print(f"      - FP16 解码极限 : {t_fp16_ms:5.1f} ms/token -> 上限约 {speed_fp16:5.1
185 f} tokens/s")
186 print(f"      - INT4 解码极限 : {t_int4_ms:5.1f} ms/token -> 上限约 {speed_int4:5.1
187 f} tokens/s (加速 {t_fp16_ms/t_int4_ms:.1f}x)")
188
189 # 5. 更完整的 Decode 下界: 每一步不仅要读一遍权重, 还要读一遍【所有请求的全部 KV Cache】
190 # 一步耗时 >= (权重字节 + B * S * 每token KV字节) / 带宽, 这一步产出 B 个 token
191 kv_bytes_all = total_tokens * kv_fp16_bytes
192 t_step_ms = (w_fp16_bytes + kv_bytes_all) / (bw * 1e9) * 1000.0
193 print(f"\n[加速] 场景 (B={batch_size}, S={context_len}) 下 FP16 每一步 Decode 的
194 带宽下界 (权重 + 全部 KV 各读一遍):")
195 print(f"      - 每步至少 {t_step_ms:5.1f} ms, 其中读 KV 占 {kv_bytes_all / (
196 w_fp16_bytes + kv_bytes_all) * 100:4.1f}%")
197 print(f"      - 系统总吞吐上限约 {batch_size * 1000.0 / t_step_ms:6.1f} tokens/s (每
198 个请求各 {1000.0 / t_step_ms:5.1f} tokens/s)")
199 print(f"      -> Batch 越大, 同一次权重读取被越多请求分摊, 总吞吐越高; 但上下文越长, KV 读
200 取越会成为新瓶颈。")
201 print("=" * 72 + "\n")
202
203 if __name__ == "__main__":
204     # 自检: Llama-3-8B 每 token KV = 2 × 32 层 × 8 头 × 128 维 × 2 字节 = 128 KB (第
205     6 篇、第 5 篇的算例)
206     assert calculate_kv_cache_per_token_bytes(MODELS["Llama-3-8B"], 16) == 131072
207     # 演示本机 RTX 5060 Ti 16GB 跑 Llama-3-8B 和 Qwen2.5-7B
208     print_model_analysis("Llama-3-8B", "RTX 5060 Ti (16GB) [本机配置]", context_len
209     =2048, batch_size=2)
210     print_model_analysis("Qwen2.5-7B", "RTX 5060 Ti (16GB) [本机配置]", context_len
211     =4096, batch_size=1)
212     # 演示大模型 70B
213     print_model_analysis("Llama-3-70B", "NVIDIA A100-SXM4 (80GB)", context_len
214     =4096, batch_size=4)

```

21.6.2 源码实现: measure_gpu_roofline.py

Listing 21.2: measure_gpu_roofline.py

```

1 """
2 Lab 08 - 实测脚本: 在你自己的 GPU 上测出 Roofline, 并验证第 8 篇的三个推导 (需要 PyTorch +

```

```

    CUDA)
3
4 对应教程: zero_to_hero_tutorial/08_硬件视角: GPU存储层级、FLOPs、算术强度与浮点格式.md
5
6 实验清单:
7     1. 显卡基本信息 (SM 数、显存、L2 大小)
8     2. 实测显存带宽  $\beta$  (大张量拷贝)
9     3. 实测 BF16 Tensor Core 峰值算力  $\pi$  (大方阵乘), 得到你这张卡的拐点  $AI^* = \pi / \beta$ 
10    4. 【验证推导 1】Decode 形状的线性层  $[B, K] \times [K, N]$ : batch 从 1 涨到 1024,
11        耗时在  $B$  远小于拐点时几乎不变  $\rightarrow$  算术强度  $\approx B$ 
12    5. 【验证推导 2】Decode 注意力: GQA 分组大小  $g$  从 1 涨到 8, 读取的 KV 字节不变、FLOPs 涨
13         $g$  倍,
14        耗时几乎不变  $\rightarrow$  算术强度  $\approx g$ , 与 batch 无关
15    6. 【验证第 8 篇 §6】BF16 / FP16 的精度与范围陷阱 (CPU 即可运行)
16
17 运行:
18     source .venv/bin/activate
19     python lab08_memory_and_roofline/measure_gpu_roofline.py
20
21 import torch
22
23
24 def title(s):
25     print("\n" + "=" * 78)
26     print(f"[实验] {s}")
27     print("=" * 78)
28
29
30 def cuda_time_ms(fn, warmup=5, iters=20):
31     """用 CUDA Event 计时 (GPU 是异步执行的, 用 time.time() 会测到错误的时间) """
32     for _ in range(warmup):
33         fn()
34     torch.cuda.synchronize()
35     start = torch.cuda.Event(enable_timing=True)
36     end = torch.cuda.Event(enable_timing=True)
37     start.record()
38     for _ in range(iters):
39         fn()
40     end.record()
41     torch.cuda.synchronize()
42     return start.elapsed_time(end) / iters
43
44
45 def expl_device_info():
46     title("实验 1: 显卡基本信息")
47     p = torch.cuda.get_device_properties(0)
48     print(f"    名称          : {p.name}")

```

```

49     print(f"    计算能力          : sm_{p.major}{p.minor}")
50     print(f"    SM 数量          : {p.multi_processor_count}")
51     print(f"    显存容量          : {p.total_memory / 1024**3:.1f} GiB")
52     l2 = getattr(p, "L2_cache_size", None)
53     if l2:
54         print(f"    L2 缓存          : {l2 / 1024**2:.0f} MiB (测带宽时张量必须远大于它,
55 否则测到的是 L2 带宽)")
56     return l2 or 0
57
58 def exp2_bandwidth(l2_bytes):
59     title("实验 2: 实测显存带宽  $\beta$ ")
60     n_bytes = max(1 << 30, 16 * l2_bytes) # 至少 1 GiB, 远大于 L2
61     x = torch.empty(n_bytes // 2, dtype=torch.bfloat16, device="cuda").uniform_()
62     y = torch.empty_like(x)
63     t_copy = cuda_time_ms(lambda: y.copy_(x))
64     bw_copy = 2 * n_bytes / (t_copy / 1e3) / 1e9 # 读一遍 + 写一遍
65     print(f"    拷贝 {n_bytes / 1024**3:.1f} GiB: {t_copy:.2f} ms → 有效带宽 {bw_copy
66 :.0f} GB/s (读+写)")
67     del x, y
68     torch.cuda.empty_cache()
69     return bw_copy
70
71 def exp3_peak_flops():
72     title("实验 3: 实测 BF16 矩阵乘峰值算力  $\pi$ ")
73     best = 0.0
74     for n in [1024, 2048, 4096, 8192]:
75         a = torch.randn(n, n, dtype=torch.bfloat16, device="cuda")
76         b = torch.randn(n, n, dtype=torch.bfloat16, device="cuda")
77         t = cuda_time_ms(lambda: a @ b, iters=10)
78         tflops = 2 * n ** 3 / (t / 1e3) / 1e12
79         best = max(best, tflops)
80         ai = 2 * n ** 3 / (2 * 3 * n * n)
81         print(f"    [{n}x{n}] @ [{n}x{n}] : {t:7.3f} ms → {tflops:6.1f} TFL0P/s (
82 AI = {ai:6.0f})")
83     del a, b
84     torch.cuda.empty_cache()
85     return best
86
87 def exp4_decode_linear(bw, peak):
88     title("实验 4: 验证'Decode 线性层的算术强度  $\approx$  batch size'")
89     K, N = 4096, 14336 # Llama-3-8B 的 FFN up/gate 投影形状, BF16 权重约 117 MB (远
90 大于 L2)
91     # nn.Linear 把权重存成 [out, in], 前向是  $x @ W^T$ , 这里用同样的布局
92     W = torch.randn(N, K, dtype=torch.bfloat16, device="cuda")
93     w_bytes = K * N * 2

```

```

93     ridge = peak * 1e12 / (bw * 1e9)
94     print(f"    权重 W: [{K}, {N}] BF16 = {w_bytes / 1024**2:.0f} MiB | 你这张卡的实测
95     拐点 AI* ≈ {ridge:.0f}")
96     print(f"    {'batch B':>8} | {'耗时 ms':>8} | {'理论 AI':>8} | {'实际 TFLOP/s
97     ':>12} | {'等效权重带宽 GB/s':>18}")
98     times = {}
99     for B in [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]:
100         X = torch.randn(B, K, dtype=torch.bfloat16, device="cuda")
101         t = cuda_time_ms(lambda: torch.nn.functional.linear(X, W), iters=50)
102         times[B] = t
103         ai = 2 * B * K * N / (2 * (B * K + K * N + B * N))
104         tflops = 2 * B * K * N / (t / 1e3) / 1e12
105         print(f"    {B:>8} | {t:8.3f} | {ai:8.1f} | {tflops:12.2f} | {w_bytes / (t
106         / 1e3) / 1e9:18.0f}")
107     print(f"    --> B 远小于拐点时, 耗时几乎不随 B 变化 (都在搬同一份权重); B 接近拐点 {
108     ridge:.0f} 后才开始线性增长。")
109     print(f"    所以 Decode 时加大 batch 几乎是'免费'提升吞吐 —— 这就是连续批处理 (Lab
110     12a) 的物理基础。")
111     slow = [B for B in times if any(times[B] > 1.3 * times[B2] for B2 in times if
112     B2 > B)]
113     if slow:
114         print(f"    [注意] 注意 B = {slow} 这几档比更大的 batch 还慢: 这不是测量噪声, 而是
115         cuBLAS 的启发式规则对这些形状")
116         print("    选中了效率较低的 kernel。Roofline 只给出上界, 库实现不一定能贴着它走。
117         ")
118         print("    推理引擎因此会对常用形状做调优、用 CUDA Graph 把 batch 补齐到固定档
119         位、甚至手写 GEMV kernel。")
120     del W
121     torch.cuda.empty_cache()
122
123 def exp5_decode_attention():
124     title("实验 5: 验证'Decode 注意力的算术强度 ≈ GQA 分组大小 g, 与 batch 无关'")
125     B, S, n_kv, d = 16, 4096, 8, 128 # 16 个请求, 每个上下文 4096, 8 个 KV 头
126     K = torch.randn(B, n_kv, S, d, dtype=torch.bfloat16, device="cuda")
127     V = torch.randn(B, n_kv, S, d, dtype=torch.bfloat16, device="cuda")
128     kv_bytes = 2 * K.numel() * 2
129     print(f"    {B} 个请求 × 上下文 {S} × {n_kv} 个 KV 头 × d={d}: KV Cache 共 {
130     kv_bytes / 1024**2:.0f} MiB (每步必须全部读一遍)")
131     print(f"    {'每组 Q 头数 g':>12} | {'Q 头总数':>8} | {'耗时 ms':>8} | {'理论 AI
132     ':>8} | {'等效 KV 带宽 GB/s':>18}")
133     for g in [1, 2, 4, 8]:
134         q = torch.randn(B, n_kv, g, d, dtype=torch.bfloat16, device="cuda") # 每
135         个 KV 头被 g 个 Q 头共享
136
137         def attn():
138             s = (q @ K.transpose(-1, -2)) * (d ** -0.5) # [B, n_kv, g, S]
139             p = torch.softmax(s.float(), dim=-1).to(torch.bfloat16)

```

```

129         return p @ V # [B, n_kv, g, d]
130     t = cuda_time_ms(attn)
131     print(f"    {g:>12} | {g * n_kv:>8} | {t:8.3f} | {g:8d} | {kv_bytes / (t /
132     le3) / le9:18.0f}")
133     print("    --> g 从 1 到 8, FLOPs 涨了 8 倍, 耗时却基本不变 (瓶颈是读 KV, 而 KV 字节数
134     不变)。")
135     print("    MHA 相当于 g=1, 把同样的 KV 读取只用了一次; GQA/MLA 让同一份 KV 被多个 Q
136     头复用 → Decode 更快。")
137     print("    (这里没有做算子融合, softmax 等中间结果也会读写显存, 所以带宽数字偏低;
138     FlashDecoding 会把它们融合掉。)")
139     del K, V
140     torch.cuda.empty_cache()
141
142 def exp6_float_pitfalls():
143     title("实验 6: 浮点格式的范围与精度陷阱 (第 8 篇 §6)")
144     for dt in [torch.float32, torch.float16, torch.bfloat16]:
145         fi = torch.finfo(dt)
146         print(f"    {str(dt):<16}: 最大值 {fi.max:.3e} | 机器 ε {fi.eps:.3e}")
147
148     print("\n    (a) 范围: 70000 能表示吗? ")
149     print(f"    FP16: {torch.tensor(70000.0, dtype=torch.float16).item()} ←
150     超过 65504, 溢出成 inf")
151     print(f"    BF16: {torch.tensor(70000.0, dtype=torch.bfloat16).item()} ←
152     能表示, 但只有约 3 位有效数字")
153
154     print("\n    (b) 精度: 256 + 1 = ?")
155     print(f"    BF16: {(torch.tensor(256.0, dtype=torch.bfloat16) + 1).item()}
156     ← 在 256 附近 BF16 的间隔是 2, +1 被吞掉")
157     print(f"    FP16: {(torch.tensor(256.0, dtype=torch.float16) + 1).item()}")
158
159     print("\n    (c) 在 BF16 里逐个累加 1000 个 1.0:")
160     s = torch.tensor(0.0, dtype=torch.bfloat16)
161     one = torch.tensor(1.0, dtype=torch.bfloat16)
162     for _ in range(1000):
163         s = s + one
164     print(f"    BF16 累加器结果 = {s.item()} (正确答案 1000)")
165     print(f"    FP32 累加器结果 = {torch.ones(1000, dtype=torch.bfloat16).float
166     ().sum().item()}")
167     print("    --> 所以 Tensor Core 做 BF16 矩阵乘时内部用 FP32 累加; Softmax / RMSNorm
168     的求和也要先转 FP32。")
169
170 def main():
171     exp6_float_pitfalls()
172     if not torch.cuda.is_available():
173         print("\n[注意] 没有检测到 CUDA, 实验 1~5 需要 NVIDIA GPU。RTX 50 系列请安装
174         cu128 版本的 PyTorch (见 requirements.txt)。")

```

```

167         return
168     torch.backends.cuda.matmul.allow_tf32 = False
169     l2 = exp1_device_info()
170     bw = exp2_bandwidth(l2)
171     peak = exp3_peak_flops()
172     title("小结: 你这张卡的 Roofline")
173     print(f"    实测带宽  $\beta \approx \{bw:.0f\}$  GB/s | 实测 BF16 峰值  $\pi \approx \{peak:.1f\}$  TFLOP/s |
拐点  $AI^* = \pi/\beta \approx \{peak * 1e12 / (bw * 1e9):.0f\}$  FLOP/Byte")
174     print(f"    BF16 Qwen2.5-7B (约 15.2 GB 权重) batch=1 Decode 上限  $\approx \{bw / 15.2:.0
f\}$  token/s")
175     exp4_decode_linear(bw, peak)
176     exp5_decode_attention()
177
178
179 if __name__ == "__main__":
180     main()

```

第 22 章 FlashAttention 算法仿真与 CUDA Softmax 算子

22.1 Lab 09: FlashAttention 与 FlashDecoding

核心数学定理推导

对应理论：第 9 篇：FlashAttention——Online Softmax 的严格推导与 IO 复杂度前置：第 8 篇（存储层级、算子融合）。依赖：NumPy；实验 5 可选 PyTorch + GPU。

22.2 运行

```
1 .venv/bin/python lab09_flash_attention/flash_attention_numpy.py
```

实验	验证的结论
1	Online Softmax 一遍扫描 = 三遍扫描；打印每一步的修正因子 $e^{m_{old}-m_{new}}$
2	任意块大小的 FlashAttention（含因果掩码、整块跳过）与朴素注意力误差 $\sim 1e-15$
3	访存量模型（元素个数口径）：渐近节省倍数 $= 2B_r/d = M/(2d^2)$ ，与论文的 M/d^2 同量级，差的 2 是记账口径；小 N 时因尾块占比大还到不了这个常数；随 N 增长真正的收益是 $O(N^2) \rightarrow O(N)$ 的显存
4	FlashDecoding：KV 切 1/4/16/64 段分别计算再合并，结果一致；合并满足结合律
5	GPU 实测：朴素注意力 vs F.scaled_dot_product_attention

22.3 本机（RTX 5060 Ti）实验 5 结果

核心导读与背景

[计时] 以下计时类数字是本机某一次运行的量级示例，前提是显卡空闲、已预热；GPU 降频或有其它负载时可能慢 40% 以上。请看趋势和比例，以你自己运行的输出为准。

N	朴素	朴素额外显存	SDPA	SDPA 额外显存
1024	0.67 ms	88 MB	0.12 ms	2 MB
4096	14.4 ms	1.3 GB	0.91 ms	8 MB
8192	57.3 ms	5.1 GB	3.3 ms	16 MB
16384	2309 ms	20 GB	12.6 ms	33 MB

最后一行的朴素实现需要 20GB，超过了 16GB 显存却没有报 OOM，而是慢了 180 倍：Windows/WSL 的驱动把溢出部分放进了**系统内存**，通过 PCIe 访问。这是一个很好的提醒——”能跑”不等于”跑在显存里”。

22.4 [附件] 附录：真机 CUDA 版（强烈建议做）

上面的实验全部是 NumPy 仿真——它证明**数学**是对的,但访存量都是算出来的。**cuda_softmax/**用**真正的 CUDA C++ 把同一套 online softmax 写成 kernel**，在你的显卡上量出带宽：

实现	DRAM 搬运	耗时	占理论上界	相对
三段式（= 本实验 §1 的朴素版）	4 次	0.694 ms	51%	1.00x
单 kernel + online softmax	2 次	0.359 ms	97~98%	1.9x
同上 + 行缓存进 shared	2 次	0.360 ms	97~98%	1.9x

三个实现都贴着 **380 GB/s 的 Roofline**，差别只在搬了多少字节。里面还有一个反直觉结果：**行缓存进 shared 的版本在 $N=8192$ 时反而崩到 74%**——因为 34 KB shared/block 把每个 SM 的常驻 block 数从 6 压到 2。这正是 **FlashAttention 要做分块而不是缓存整行的原因**。

```
1 .venv/bin/python lab09_flash_attention/cuda_softmax/run.py
```

不需要装 CUDA toolkit(用的是 PyTorch 自带的 NVRTC)。详见 **cuda_softmax/README.md**。

22.5 动手练习

1. 在 **flash_attention** 里加上对 GQA 的支持：多个 Q 头共享同一组 K/V 块时，同一个 K/V 块读一次就能服务多个头（对照第 8 篇 §4.3）。
2. 实现 FlashAttention-1 的循环顺序（外层 K/V、内层 Q），需要把每个 Q 块的 m, ℓ, acc 写回”显存”再读出来；统计访存量，理解 FlashAttention-2 为什么交换循环。
3. 把实验 4 的 **partial_attention** 改成读取**不连续**的 KV 块（给一个 block table，见 Lab 12b），这就是 PagedAttention kernel 的核心逻辑。
4. (进阶) 在 **cuda_softmax/** 里把行缓存版改成**真正的分块**（只缓存 BLOCK 个元素、循环推进），验证占用率恢复后带宽回到 97~98%——做完这题你就理解了 tiling 是怎么来的。
5. (进阶) 跟着 [Triton Fused Attention 教程](#) 写一个真正在 GPU 上运行的 FlashAttention 前向，与 SDPA 对比速度。

22.6 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码，重点实现细节如下：

22.6.1 源码实现：flash_attention_numpy.py

Listing 22.1: flash_attention_numpy.py

```
1  """
2  Lab 09: 用 NumPy 从零实现 Online Softmax、FlashAttention 与 FlashDecoding, 并逐一验证
        与朴素注意力完全相等
3
4  对应教程: zero_to_hero_tutorial/09_FlashAttention: OnlineSoftmax的严格推导与IO复杂度.
        md
5
6  实验清单:
7      1. Online Softmax: 一遍扫描得到的 (m, l) 与三遍扫描完全一致; 逐步打印修正因子
8      2. 分块 FlashAttention (含因果掩码与整块跳过) vs 朴素注意力: 误差 ~1e-15
9      3. 显存访问量模型: 朴素  $\theta(Nd + N^2)$  vs FlashAttention  $\theta(N^2d^2/M)$ 
10     4. FlashDecoding: 把 KV 切成若干段各自计算, 再用合并公式合并, 结果与整体计算一致; 合并顺序无
        关 (结合律)
11     5. (可选, 需要 PyTorch + GPU) 朴素注意力 vs F.scaled_dot_product_attention 的真实显
        存与耗时
12
13  运行:
14      .venv/bin/python lab09_flash_attention/flash_attention_numpy.py
15  """
16
17  import math
18
19  import numpy as np
20
21
22  def title(s):
23      print("\n" + "=" * 78)
24      print(f"[实验] {s}")
25      print("=" * 78)
26
27
28  def naive_attention(Q, K, V, causal=False):
29      """朴素实现: 显式构造  $N \times N$  的 S 和 P"""
30      S = Q @ K.T / math.sqrt(Q.shape[1])
31      if causal:
32          S = S + np.triu(np.full(S.shape, -np.inf), k=1)
33      S = S - S.max(axis=1, keepdims=True)
34      P = np.exp(S)
35      P = P / P.sum(axis=1, keepdims=True)
36      return P @ V
37
38
39  # =====
40  # 实验 1: Online Softmax
```

```

41 # =====
42 def expl_online_softmax():
43     title("实验 1: Online Softmax —— 一遍扫描求出最大值 m 与归一化因子 l")
44     x = np.array([1.0, 3.0, 2.0, 8.0, 5.0, 9.0, 4.0])
45     m, l = -np.inf, 0.0
46     print(f"    输入 x = {x.tolist()}")
47     print(f"    {'j':>3} | {'x_j':>5} | {'m_j':>5} | {'修正因子 e^(m_old - m_new) '}>26} | {'l_j':>10}")
48     for j, xj in enumerate(x, 1):
49         m_new = max(m, xj)
50         corr = math.exp(m - m_new) if m > -np.inf else 0.0
51         l = l * corr + math.exp(xj - m_new)
52         m = m_new
53         print(f"    {'j':>3} | {'xj':>5.1f} | {'m':>5.1f} | {'corr':>26.6f} | {'l':>10.6f}")
54     m_ref = x.max()
55     l_ref = np.exp(x - m_ref).sum()
56     print(f"    三遍扫描的标准答案: m = {m_ref}, l = {l_ref:.6f} → 一致 [OK]" if abs(
57         l - l_ref) < 1e-12 else "    [X] 不一致")
58     assert abs(l - l_ref) < 1e-12 and m == m_ref
59     print(f"    --> 每当出现新的最大值 (第 2、4、6 步), 之前的和就乘上一个 ≤ 1 的修正因子, 换算到新基准。")
60
61 # =====
62 # 实验 2: 分块 FlashAttention
63 # =====
64 def flash_attention(Q, K, V, Br=16, Bc=16, causal=False, stats=None):
65     """
66     FlashAttention-2 的循环顺序 (外层 Q 块, 内层 K/V 块)。
67     所有 Br×Bc 的中间矩阵都是局部变量 (对应 GPU 上的 SRAM / 寄存器), 从不构造 N×N 矩阵。
68     stats: 传入 dict 时, 统计"从显存读写的元素个数"和"跳过的块数"
69     """
70     N, d = Q.shape
71     O = np.zeros_like(Q)
72     scale = 1.0 / math.sqrt(d)
73     for i0 in range(0, N, Br):
74         Qi = Q[i0:i0 + Br]                                # 读 Q 块
75         br = Qi.shape[0]
76         m = np.full(br, -np.inf)
77         l = np.zeros(br)
78         acc = np.zeros((br, d))
79         if stats is not None:
80             stats["hbm"] += Qi.size
81         for j0 in range(0, N, Bc):
82             if causal and j0 > i0 + br - 1:                # 整块都在未来 → 直接跳过 (因果注意力
83                 # 省一半计算)
84             if stats is not None:
85                 stats["skipped"] += 1

```

```

85         continue
86         Kj, Vj = K[j0:j0 + Bc], V[j0:j0 + Bc] # 读 K、V 块
87         if stats is not None:
88             stats["hbm"] += Kj.size + Vj.size
89             stats["blocks"] += 1
90         S = Qi @ Kj.T * scale # [br, bc], 只存在"片上"
91         if causal:
92             q_pos = np.arange(i0, i0 + br)[: , None]
93             k_pos = np.arange(j0, j0 + Kj.shape[0])[None, :]
94             S = np.where(k_pos > q_pos, -np.inf, S)
95             m_new = np.maximum(m, S.max(axis=1))
96             P = np.exp(S - m_new[: , None])
97             corr = np.exp(m - m_new) # 修正因子
98             l = l * corr + P.sum(axis=1)
99             acc = acc * corr[: , None] + P @ Vj
100             m = m_new
101             O[i0:i0 + br] = acc / l[: , None] # 写 O 块
102             if stats is not None:
103                 stats["hbm"] += Qi.size
104         return O
105
106
107 def exp2_flash_vs_naive():
108     title("实验 2: 分块 FlashAttention vs 朴素注意力 (数学上严格相等)")
109     rng = np.random.default_rng(0)
110     N, d = 256, 64
111     Q, K, V = (rng.standard_normal((N, d)) for _ in range(3))
112     for causal in [False, True]:
113         ref = naive_attention(Q, K, V, causal)
114         for Br, Bc in [(16, 16), (32, 64), (64, 32), (256, 256)]:
115             st = {"hbm": 0, "blocks": 0, "skipped": 0}
116             out = flash_attention(Q, K, V, Br, Bc, causal, st)
117             print(f"    {'因果' if causal else '双向'} | 块大小 Br={Br:<3} Bc={Bc:<3}
118                   | 最大误差 {np.abs(out - ref).max():.1e} | "
119                   f"计算 {st['blocks']:3d} 块, 跳过 {st['skipped']:3d} 块")
120             assert np.abs(out - ref).max() < 1e-10, 'FlashAttention 与朴素注意力不一致'
121         print("    -> 无论块怎么切, 结果都与朴素实现相等 (误差是 1e-15 的浮点舍入); 因果注意力约
122               一半的块被整块跳过。")
123
124 # =====
125 # 实验 3: 显存访问量模型
126 # =====
127 def exp3_io_model():
128     title("实验 3: 显存访问量 (元素个数) —— 朴素 vs FlashAttention")
129     sram_elems = 100 * 1024 // 2 # 约 100 KB 片上 SRAM, BF16
130     print("    假设片上 SRAM ≈ 100 KB. 朴素 = 读 Q,K,V + 写 O + (写 S、读 S、写 P、读 P)

```

```

; ")
130 print("    Flash = 读 Q + 写 O + 每个 Q 块都把 K、V 完整读一遍 (共 ceil(N/Br) 遍) ")
131 print("    本文件用的是【元素个数】口径, 不是字节。")
132 for d in [64, 128]:
133     Br = max(16, sram_elems // (4 * d) // 16 * 16)    # Q、K、V、O 四个块大致放得
下
134     # 本模型: 朴素 =  $4Nd + 4N^2$ , Flash =  $2Nd + 2N^2d/Br$ 
135     #  $N \rightarrow \infty$  时比值  $\rightarrow 2 \cdot Br/d$ , 代入  $Br \approx M/(4d)$  得  $M/(2d^2)$ 。
136     ratio_pred = 2 * Br / d
137     print(f"\n    头维度 d = {d}  $\rightarrow$  块大小约 Br = Bc = {Br} ( $Br \approx M/(4d)$ ), M = {
sram_elems} 元素) ")
138     print(f"    本模型的渐近节省倍数 =  $2 \cdot Br/d = \{ratio\_pred:.1f\}$ , "
139           f"等价地  $M/(2d^2) = \{sram\_elems / (2 * d * d):.1f\}$ ")
140     print(f"    论文写的是  $M/d^2 = \{sram\_elems / (d * d):.1f\}$  —— 是同一个量级, 差的
这个 2 来自记账口径, ")
141     print(f"    不是矛盾: 本文的朴素项数了 4 趟  $N^2$  (写 S/读 S/写 P/读 P), 论文的  $\Theta(Nd
+ N^2)$  只按元素数算一趟。")
142     print(f"    {'序列长度 N':>10} | {'朴素':>10} | {'Flash':>10} | {'节省':>6} |
{'朴素需要的  $N^2$  额外显存 (1 个头)':>28}")
143     ratios = []
144     for N in [1024, 4096, 16384, 65536]:
145         naive = 4 * N * d + 4 * N * N
146         n_kv_passes = -( -N // Br)                                # ceil, 不是 floor: 最后那
个不满的块也要读
147         flash = 2 * N * d + n_kv_passes * 2 * N * d
148         ratios.append(naive / flash)
149         print(f"    {N:>10} | {naive:>10.2e} | {flash:>10.2e} | {naive / flash
:5.1f}x | {2 * N * N * 2 / 1024**2:>25.0f} MB")
150         # 断言: 比值单调趋近  $2 \cdot Br/d$ , 且永远不超过它 (尾块只会让节省变少, 不会变多)
151         assert all(ratios[i] <= ratios[i + 1] + 1e-9 for i in range(len(ratios) -
1)), \
152             f"节省倍数应随 N 单调上升 (尾块占比下降), 实际 {ratios}"
153         assert ratios[-1] <= ratio_pred + 1e-9, \
154             f"节省倍数 {ratios[-1]:.2f} 不该超过渐近值 {ratio_pred:.2f} (ceil 只可能让
它更小) "
155         assert ratios[-1] > ratio_pred * 0.95, \
156             f"N=65536 时应已接近渐近值 {ratio_pred:.2f}, 实际 {ratios[-1]:.2f}"
157     print("\n    --> 注意: 访存节省倍数是一个【与 N 无关的常数】 (d 越小、SRAM 越大, 省得越
多), 并不会随 N 无限增大。")
158     print(f"    但小 N 时它还到不了那个常数 (尾块占比大); 本例要到  $N \geq 16K$  才收敛到 {
ratio_pred:.1f}x。")
159     print("    随 N 增长的真实收益是【额外显存从  $O(N^2)$  降到  $O(N)$ 】—— 没有它, 长上下文根
本放不进显存。")
160     print("    再加上多个 kernel 融合成一个 (省掉启动开销与 FP32 中间结果), 实际加速往往
比这个模型更大, 见实验 5。")
161     print("    【注意】这条结论在真机上不总是成立: 把整行缓存进 shared 会压低占用率, N 一大
反而变慢 ——")
162     print("    见 cuda_softmax/README.md 里 N=8192 的实测 (同一个道理也是

```

```

FlashAttention 必须分块的原因) 。")
163
164
165 # =====
166 # 实验 4: FlashDecoding
167 # =====
168 def partial_attention(q, K, V):
169     """对一段 KV 计算部分结果 (m, l, acc), q: [d]"""
170     s = K @ q / math.sqrt(q.shape[0])
171     m = s.max()
172     p = np.exp(s - m)
173     return m, p.sum(), p @ V
174
175
176 def merge(parts):
177     """第 9 篇 §4.2 的合并公式"""
178     m = max(p[0] for p in parts)
179     l = sum(p[1] * math.exp(p[0] - m) for p in parts)
180     acc = sum(p[2] * math.exp(p[0] - m) for p in parts)
181     return m, l, acc
182
183
184 def exp4_flash_decoding():
185     title("实验 4: FlashDecoding —— 切分 KV 并行计算, 再合并")
186     rng = np.random.default_rng(1)
187     S, d = 8192, 128
188     q = rng.standard_normal(d)
189     K, V = rng.standard_normal((S, d)), rng.standard_normal((S, d))
190     ref = naive_attention(q[None, :], K, V)[0]
191     for n_split in [1, 4, 16, 64]:
192         chunks = np.array_split(np.arange(S), n_split)
193         parts = [partial_attention(q, K[c], V[c]) for c in chunks] # 在 GPU 上,
194         # 这些段分给不同的 SM 同时算
195         m, l, acc = merge(parts)
196         print(f"    切成 {n_split}>2} 段并行 → 合并后最大误差 {np.abs(acc / l - ref).max():.1e}")
197         assert np.abs(acc / l - ref).max() < 1e-10
198         parts = [partial_attention(q, K[c], V[c]) for c in np.array_split(np.arange(S), 8)]
199         a = merge([merge(parts[:3]), merge(parts[3:])])
200         b = merge([merge(parts[:2]), merge(parts[1:2])])
201         print(f"    结合律: 两种不同的分组合并顺序, 结果差 {np.abs(a[2] / a[1] - b[2] / b[1]).max():.1e}")
202         assert np.abs(a[2] / a[1] - b[2] / b[1]).max() < 1e-12
203         print(f"    --> 合并满足结合律, 所以可以任意切分、任意顺序合并 —— 这也是 PagedAttention 和跨卡 Ring Attention 的基础。")
204

```

```

205 # =====
206 # 实验 5 (可选) : GPU 上的真实差距
207 # =====
208 def exp5_gpu():
209     title("实验 5 (可选) : GPU 上朴素注意力 vs PyTorch SDPA (内部调用 FlashAttention 类
210         kernel) ")
211     try:
212         import torch
213         import torch.nn.functional as F
214     except ImportError:
215         print(" 未安装 PyTorch, 跳过。")
216         return
217     if not torch.cuda.is_available():
218         print(" 没有 CUDA, 跳过。")
219         return
220
221     def naive(q, k, v):
222         s = (q @ k.transpose(-1, -2)) / math.sqrt(q.shape[-1])
223         n = s.shape[-1]
224         s = s.masked_fill(torch.ones(n, n, dtype=torch.bool, device=s.device).triu(
225             1), float("-inf"))
226         return torch.softmax(s.float(), dim=-1).to(q.dtype) @ v
227
228     def measure(fn, *a):
229         torch.cuda.synchronize()
230         torch.cuda.reset_peak_memory_stats()
231         base = torch.cuda.memory_allocated()
232         for _ in range(2):
233             fn(*a)
234         torch.cuda.synchronize()
235         t0 = torch.cuda.Event(enable_timing=True); t1 = torch.cuda.Event(
236             enable_timing=True)
237         t0.record()
238         for _ in range(5):
239             fn(*a)
240         t1.record(); torch.cuda.synchronize()
241         return t0.elapsed_time(t1) / 5, (torch.cuda.max_memory_allocated() - base)
242             / 1024**2
243
244     H, d = 8, 128
245     print(f" 单层、{H} 个头、d={d}、BF16、因果注意力")
246     print(f" {'N':>6} | {'朴素 ms':>8} | {'朴素额外显存':>12} | {'SDPA ms':>8} |
247         {'SDPA 额外显存':>13}")
248     for N in [1024, 4096, 8192, 16384]:
249         q, k, v = (torch.randn(1, H, N, d, device="cuda", dtype=torch.bfloat16)
250             for _ in range(3))
251         sdpa = lambda q, k, v: F.scaled_dot_product_attention(q, k, v, is_causal=
252             True)

```

```

246     ts, ms = measure(sdpa, q, k, v)
247     try:
248         tn, mn = measure(naive, q, k, v)
249         naive_str = f"{tn:8.2f} | {mn:9.0f} MB"
250     except torch.OutOfMemoryError:
251         naive_str = f"'OOM':>8} | {'显存不足':>10}"
252         torch.cuda.empty_cache()
253     print(f"    {N:>6} | {naive_str} | {ts:8.2f} | {ms:10.0f} MB")
254     del q, k, v
255     torch.cuda.empty_cache()
256     print("    --> 朴素实现的额外显存随  $N^2$  增长，SDPA 基本只和输出一样大；速度差距也随  $N$  拉
    大。")
257     print("    朴素实现还有好几个  $N^2$  的中间张量（FP32 的 softmax、掩码矩阵），所以实际加
    速比实验 3 的访存模型更大。")
258     total = torch.cuda.get_device_properties(0).total_memory / 1024**2
259     print(f"    [注意] 如果某一行朴素实现的'额外显存'超过了你的显存容量（{total:.0f} MB）却
    没有 OOM、而是慢了上百倍：")
260     print("    这是 Windows / WSL 的显卡驱动把放不下的部分挪到了系统内存（system
    fallback），经过 PCIe 访问，")
261     print("    带宽只有显存的几十分之一。在 Linux 原生驱动或数据中心卡上，这里会直接 OOM。
    ")
262
263
264 if __name__ == "__main__":
265     exp1_online_softmax()
266     exp2_flash_vs_naive()
267     exp3_io_model()
268     exp4_flash_decoding()
269     exp5_gpu()

```

第 23 章 投机解码仿真实现与无偏性实证校验

23.1 Lab 10: 投机采样 (Speculative Decoding) 原理与算法仿真

23.2 为什么需要投机采样?

在 Lab 08 中我们已经知道:

- 自回归 Decode 阶段是访存带宽受限 (Memory-Bound) 的。
- 大模型每生成 1 个 token, 就必须把整个模型的权重 (数十 GB) 完整搬运一次, 因此单步延迟很难降低。
- 但如果同时输入 K 个 token (如 Prefill 阶段), GPU 的算力利用率大幅提高, 耗时几乎与输入 1 个 token 相同!

投机采样 (Speculative Decoding) 的洞察:

核心导读与背景

”与其每次花很长时间让昂贵的大模型只猜 1 个词, 不如让一个廉价的小模型快速一口气猜 K 个词, 再让大模型做一次并行前向传播进行批量验证!”

23.3 核心算法流程

假设目标大模型为 M_{target} , 轻量草稿模型为 M_{draft} (参数量通常仅为大模型的 $1/10 \sim 1/5$):

23.3.1 严格的数学无偏性保证 (Unbiased Sampling)

投机采样最迷人的特点是: **完全无损 (Lossless)**。通过巧妙的拒绝采样概率公式:

$$\alpha = \min \left(1, \frac{P_{\text{target}}(x)}{P_{\text{draft}}(x)} \right)$$

- 若以概率 α 接受草稿 token x ;
- 若拒绝, 则根据校正后的残差分布采样:

$$P_{\text{residual}}(x) = \frac{\max(0, P_{\text{target}}(x) - P_{\text{draft}}(x))}{\sum \max(0, P_{\text{target}}(x') - P_{\text{draft}}(x'))}$$

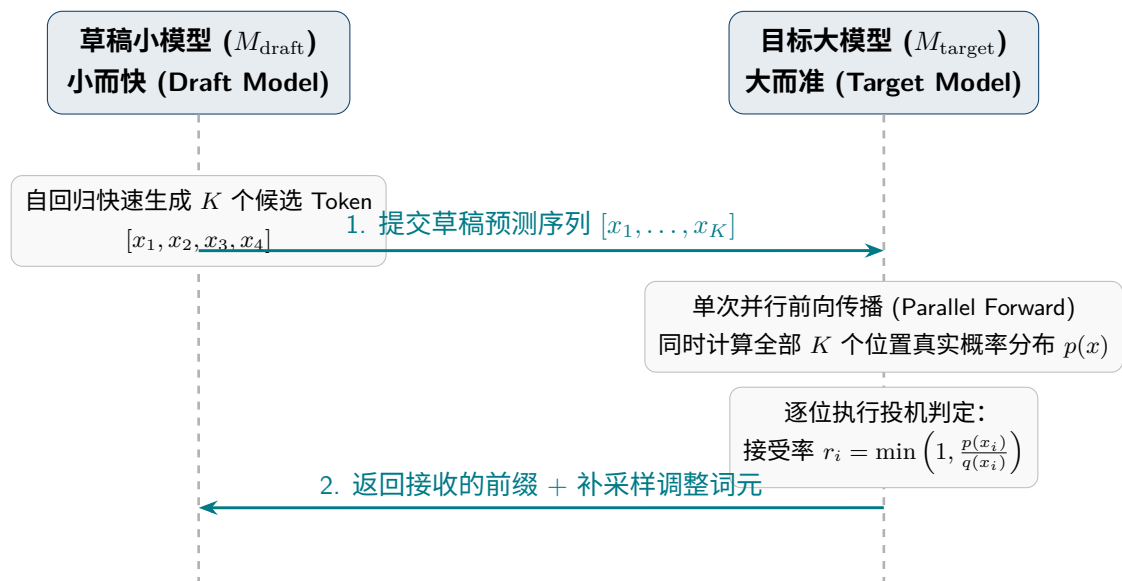


图 23.1: 投机采样 (Speculative Decoding) 草稿与目标模型协同序列交互

数学上严格证明：无论草稿模型多弱，最终生成的文本统计分布与单独用大模型生成 100% 完全一致！（草稿模型越弱，只会让接受率变低、加速比变小，而不会改变输出分布。）

核心数学定理推导

[几何] 证明在哪？完整的逐行证明见 第 10 篇：投机解码；用蒙特卡洛实验”亲眼验证”这个定理，请运行 `verify_unbiasedness.py`。

[注意] 注意：本目录的 `toy_speculative_decoding.py` 里，接收/拒绝只用了 被采样 token 的一个概率值，所以是正确的；但 $p_{\text{target}} / p_{\text{draft}}$ 必须用同一个前缀下的两个分布——这一点在真实实现里最容易写错。

23.4 [执行] 运行仿真实验

运行 `toy_speculative_decoding.py`:

```
1 python3 lab10_speculative_decoding/toy_speculative_decoding.py
```

本脚本模拟了草稿模型与目标模型的联合采样过程，展示每轮投机接受长度、大模型调用次数节省比例及端到端加速比。

```
1 python3 lab10_speculative_decoding/verify_unbiasedness.py
```

对同一个前缀重复做 20 万次”投机采样的一步”，统计输出 token 的经验分布，与目标模型分布逐项比较（总变差距离应趋近 0），并给出理论接受率 $\sum_x \min(p, q)$ 与期望加速比公式的实测验证。还会演示两种常见错误写法（拒绝后直接从 p 或 q 重采样）如何让分布产生 11%~17% 的总变差偏差。

23.5 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码，重点实现细节如下：

23.5.1 源码实现：toy_speculative_decoding.py

Listing 23.1: toy_speculative_decoding.py

```
1  """
2  Lab 10: 投机采样 (Speculative Decoding) 核心逻辑算法仿真
3
4  实现无偏差投机采样 (Lossless Speculative Sampling):
5  1. 草稿模型 (Draft Model) 连续自回归推测 K 个候选 Token
6  2. 目标大模型 (Target Model) 并行前向传播一次，获取各位置的条件分布
7  3. 执行严格的概率接收/拒绝准则 (Leviathan et al.)
8  4. 统计接受率 (Acceptance Rate)、单步有效产出 (Tokens per Step) 与加速比
9  """
10
11 import random
12 import time
13 import math
14
15
16 class ToyLanguageModel:
17     """模拟语言模型，输出词表上的概率分布"""
18     def __init__(self, vocab_size=1000, forward_latency_ms=10.0, seed=42):
19         self.vocab_size = vocab_size
20         self.latency_s = forward_latency_ms / 1000.0
21         self.rng = random.Random(seed)
22
23     def sample_distribution(self, context_tokens):
24         """模拟针对当前上下文输出的下一个 Token 概率分布 (Dirichlet/Softmax)"""
25         # 利用上下文 hash 保证相同前缀产生相同的确定性先验，加上随机噪声
26         ctx_hash = sum(context_tokens) % 10007
27         raw_logits = [(math.sin(i * 0.1 + ctx_hash) + 1.0) for i in range(self.vocab_size)]
28
29         # 针对前几个词加大权重，模拟自然语言的长尾分布
30         for i in range(min(10, self.vocab_size)):
31             raw_logits[i] *= 2.5
32
33         sum_l = sum(raw_logits)
34         return [l / sum_l for l in raw_logits]
35
36     def sample_next_token(self, probab_dist):
37         """根据分布依概率采样一个 Token"""
38         r = self.rng.random() # 用本模型自己的、已设种子的随机数生成器，保证结果
可复现
```

```

39     cumulative = 0.0
40     for idx, p in enumerate(prob_dist):
41         cumulative += p
42         if r <= cumulative:
43             return idx
44     return len(prob_dist) - 1
45
46
47 def speculative_decoding_simulation(
48     target_model_latency=30.0, # 大模型单次前向 30ms (如 70B)
49     draft_model_latency=3.0,   # 小模型单次前向 3ms (如 7B 或专门的草稿头)
50     gamma=4,                  # 每轮草稿模型推测 token 数 K=4
51     total_tokens_to_generate=50,
52     similarity=0.75            # 草稿模型与目标模型分布的相似度/拟合程度
53 ):
54     print("=" * 68)
55     print("[起步] 投机采样 (Speculative Decoding) 端到端仿真")
56     print(f"[重点] 参数配置: 大模型耗时={target_model_latency}ms | 草稿模型耗时={draft_model_latency}ms")
57     print(f"[重点] 投机窗口大小 K={gamma} | 目标生成长度={total_tokens_to_generate} | 模型相似度={similarity*100:.0f}%")
58     print("=" * 68 + "\n")
59
60     vocab_size = 200
61     target_lm = ToyLanguageModel(vocab_size=vocab_size, forward_latency_ms=target_model_latency, seed=1)
62
63     generated_tokens = [1, 2, 3] # 初始 Prompt
64
65     round_count = 0
66     total_accepted_tokens = 0
67     simulated_wall_time = 0.0
68     tokens_produced = 0
69
70     print("开始投机解码迭代...\n")
71
72     while tokens_produced < total_tokens_to_generate:
73         round_count += 1
74
75         # -----
76         # 阶段 1: 草稿模型连续自回归预测 gamma 个 token
77         # -----
78         draft_tokens = []
79         draft_probs = []
80         curr_draft_context = list(generated_tokens)
81
82         for _ in range(gamma):
83             # 草稿模型前向

```

```

84         simulated_wall_time += draft_model_latency
85
86     # 模拟草稿分布（以大模型为基准，加入适度偏置模拟小模型的精度损失）
87     true_dist = target_lm.sample_distribution(curr_draft_context)
88     draft_dist = []
89     for p in true_dist:
90         # 混合真实分布与均匀分布
91         p_draft = similarity * p + (1.0 - similarity) * (1.0 / vocab_size)
92         draft_dist.append(p_draft)
93
94     s = sum(draft_dist)
95     draft_dist = [p / s for p in draft_dist]
96
97     next_t = target_lm.sample_next_token(draft_dist)
98     draft_tokens.append(next_t)
99     draft_probs.append(draft_dist[next_t])
100     curr_draft_context.append(next_t)
101
102     # -----
103     # 阶段 2：目标大模型单次前向，并行验证所有 gamma 个 token
104     # -----
105     # 注意：这里只花费 1 次大模型前向的时间！
106     simulated_wall_time += target_model_latency
107
108     # 逐个位置进行接收/拒绝检验 (Leviathan et al. 准则)
109     accepted_this_round = []
110     curr_verify_context = list(generated_tokens)
111
112     all_accepted = True
113     bonus_token = None
114
115     for i in range(gamma):
116         cand_token = draft_tokens[i]
117         p_draft = draft_probs[i]
118
119         # 大模型计算该上下文下的真实条件概率
120         target_dist = target_lm.sample_distribution(curr_verify_context)
121         p_target = target_dist[cand_token]
122
123         # 接收概率  $\alpha = \min(1, p_{\text{target}} / p_{\text{draft}})$ 
124         alpha = min(1.0, p_target / p_draft if p_draft > 0 else 1.0)
125
126         if target_lm.rng.random() < alpha:
127             # 接受该候选词
128             accepted_this_round.append(cand_token)
129             curr_verify_context.append(cand_token)
130         else:
131             # 拒绝该候选词，并根据残差分布从大模型无偏重采样一个替代词

```

```

132         all_accepted = False
133         # 残差分布:  $\max(0, p_{\text{target}} - p_{\text{draft}})$ 
134         residual = [max(0.0, target_dist[v] - (similarity * target_dist[v]
+ (1 - similarity)/vocab_size)) for v in range(vocab_size)]
135         sum_res = sum(residual)
136         if sum_res > 1e-9:
137             norm_residual = [r / sum_res for r in residual]
138             replacement_token = target_lm.sample_next_token(norm_residual)
139         else:
140             replacement_token = target_lm.sample_next_token(target_dist)
141
142         accepted_this_round.append(replacement_token)
143         break
144
145     # 如果全部  $\gamma$  个词都被接受, 大模型可以“免费”多采样 1 个新词!
146     if all_accepted:
147         final_dist = target_lm.sample_distribution(curr_verify_context)
148         bonus_token = target_lm.sample_next_token(final_dist)
149         accepted_this_round.append(bonus_token)
150
151     # 更新已生成序列
152     generated_tokens.extend(accepted_this_round)
153     tokens_produced += len(accepted_this_round)
154     total_accepted_tokens += len(accepted_this_round)
155
156     print(f"Round {round_count:2d}: 草稿推测 {gamma} 词 -> 成功采纳 {len(
accepted_this_round)} 词 (Tokens: {accepted_this_round})")
157
158     # -----
159     # 阶段 3: 对比基线 (无投机采样, 单独用大模型逐字生成)
160     # -----
161     baseline_wall_time = tokens_produced * target_model_latency
162     avg_tokens_per_round = total_accepted_tokens / round_count
163     speedup = baseline_wall_time / simulated_wall_time
164
165     print("\n" + "=" * 68)
166     print("[图表] 仿真测试统计结果")
167     print("=" * 68)
168     print(f"    - 总生成 Token 数量          : {tokens_produced}")
169     print(f"    - 大模型总前向次数 (Rounds) : {round_count} 次 (若无投机需要 {
tokens_produced} 次)")
170     print(f"    - 平均每轮产生 Token 数      : {avg_tokens_per_round:.2f} tokens/step")
171     print(f"    - 基线耗时 (仅大模型生成)    : {baseline_wall_time:.1f} ms")
172     print(f"    - 投机采样累计耗时          : {simulated_wall_time:.1f} ms")
173     print(f"    - [起步] 端到端真实加速比      : {speedup:.2f}x")
174     print("=" * 68)
175     print("[思考] 结论:")
176     print("    投机采样的加速比核心取决于: ")

```

```

177     print("    1. 草稿模型的接受率 (Acceptance Rate); ")
178     print("    2. 草稿模型与大模型的相对耗时比 (越轻量越好)。")
179     print("    本仿真的数字只取决于上面三个假设参数 (耗时、K、相似度)，换一组参数结果会完全不同; ")
180     print("    真实框架里的加速比要实测 (见 capstone_engines_practice 任务 5)，并可用第 10 篇的期望加速比公式预测。")
181     assert tokens_produced >= total_tokens_to_generate and round_count <
        tokens_produced, "每轮应至少产出 1 个 token"
182
183
184 if __name__ == "__main__":
185     speculative_decoding_simulation()

```

23.5.2 源码实现：verify_unbiasedness.py

Listing 23.2: verify_unbiasedness.py

```

1  """
2  Lab 10 - 验证脚本：用蒙特卡洛实验"亲眼看到"投机采样的无偏性定理与期望加速比公式（零外部依赖）
3
4  对应教程：zero_to_hero_tutorial/10_投机解码：无偏性证明与期望加速比.md
5
6  实验清单：
7      1. 单 token: 重复 20 万次"草稿采样 → 接受/拒绝 → 残差重采样"，经验分布 = 目标分布  $p$ （而不是草稿分布  $q$ ）
8      2. 草稿再差也无偏： $q$  与  $p$  几乎相反时，输出分布仍然是  $p$ ，只是接受率很低；实测接受率 =  $\sum \min(p, q)$ 
9      3. 两种常见错误写法会产生偏差：(a) 拒绝后直接从  $p$  重采样 (b) 拒绝后从  $q$  重采样
10     4. 多 token:  $K$  个草稿一轮验证，实测每轮产出 =  $(1 - \alpha^{(K+1)}) / (1 - \alpha)$ ；扫描最优  $K$  与加速比
11 """
12
13 import random
14
15 N_SAMPLES = 200_000
16
17
18 def sample(dist, rng):
19     r = rng.random()
20     c = 0.0
21     for i, p in enumerate(dist):
22         c += p
23         if r < c:
24             return i
25     return len(dist) - 1
26
27

```

```

28 def normalize(v):
29     s = sum(v)
30     return [x / s for x in v]
31
32
33 def tv(a, b):
34     """总变差距离  $TV(a, b) = 0.5 \sum |a - b|$ """
35     return 0.5 * sum(abs(x - y) for x, y in zip(a, b))
36
37
38 def speculative_step(p, q, rng, mode="correct"):
39     """返回 (输出 token, 是否接受了草稿)"""
40     x = sample(q, rng)
41     if rng.random() < min(1.0, p[x] / q[x]):
42         return x, True
43     if mode == "correct":
44         residual = [max(0.0, pi - qi) for pi, qi in zip(p, q)]
45         return sample(normalize(residual), rng), False
46     if mode == "resample_p": # 常见错误 1: 拒绝后直接从目标分布 p 采样
47         return sample(p, rng), False
48     if mode == "resample_q": # 常见错误 2: 拒绝后再从草稿分布 q 采样
49         return sample(q, rng), False
50     raise ValueError(mode)
51
52
53 def empirical(p, q, mode, seed):
54     rng = random.Random(seed)
55     counts = [0] * len(p)
56     acc = 0
57     for _ in range(N_SAMPLES):
58         x, a = speculative_step(p, q, rng, mode)
59         counts[x] += 1
60         acc += a
61     return [c / N_SAMPLES for c in counts], acc / N_SAMPLES
62
63
64 def show(name, dist):
65     print(f" {name:<22}: " + " ".join(f"{x:.3f}" for x in dist))
66
67
68 def title(s):
69     print("\n" + "=" * 78)
70     print(f"[实验] {s}")
71     print("=" * 78)
72
73
74 def expl():
75     title(f"实验 1: 单 token 投机采样的输出分布 (重复 {N_SAMPLES:,} 次)")

```

```

76     p = [0.40, 0.25, 0.15, 0.10, 0.05, 0.03, 0.02]      # 目标（大）模型
77     q = [0.20, 0.35, 0.10, 0.20, 0.05, 0.05, 0.05]      # 草稿（小）模型
78     emp, acc = empirical(p, q, "correct", 0)
79     show("目标分布 p", p)
80     show("草稿分布 q", q)
81     show("投机采样实测输出", emp)
82     alpha = sum(min(a, b) for a, b in zip(p, q))
83     print(f"    TV(实测, p) = {tv(emp, p):.4f}    TV(实测, q) = {tv(emp, q):.4f}")
84     assert tv(emp, p) < 0.01 and abs(acc - alpha) < 0.01, '投机采样输出分布应等于 p'
85     print(f"    接受率: 实测 {acc:.4f} | 理论  $\sum \min(p, q) = \{\alpha:.4f\} = 1 - TV(p, q) =$ 
86           {1 - tv(p, q):.4f}")
87     print("    --> 输出分布贴合 p (TV 只剩约 1/√N 量级的采样噪声), 与 q 明显不同 —— 定理成
88           立。")
89
90 def exp2():
91     title("实验 2: 草稿模型再差, 结果依然无偏 (只是慢)")
92     p = [0.70, 0.20, 0.05, 0.03, 0.02]
93     q = [0.02, 0.03, 0.05, 0.20, 0.70]                  # 几乎与 p 相反
94     emp, acc = empirical(p, q, "correct", 1)
95     show("目标分布 p", p)
96     show("草稿分布 q (很烂)", q)
97     show("投机采样实测输出", emp)
98     print(f"    TV(实测, p) = {tv(emp, p):.4f}; 接受率实测 {acc:.3f}, 理论 {sum(min(a,
99           b) for a, b in zip(p, q)):.3f}")
100     assert tv(emp, p) < 0.01
101     print("    --> 草稿的质量只影响速度 (接受率), 不影响正确性。")
102
103 def exp3():
104     title("实验 3: 两种常见的错误实现 —— 分布出现系统性偏差")
105     p = [0.40, 0.25, 0.15, 0.10, 0.05, 0.03, 0.02]
106     q = [0.20, 0.35, 0.10, 0.20, 0.05, 0.05, 0.05]
107     show("目标分布 p", p)
108     for mode, name in [("correct", "[OK] 残差分布重采样"), ("resample_p", "[X] 拒绝后
109           从 p 采样"), ("resample_q", "[X] 拒绝后从 q 采样")]:
110         emp, _ = empirical(p, q, mode, 2)
111         show(name, emp)
112         print(f"    {'':<22} TV(实测, p) = {tv(emp, p):.4f}")
113         assert (tv(emp, p) < 0.01) == (mode == 'correct'), f'{mode} 的偏差不符合预期'
114
115     print("    --> '拒绝后从 p 采样'看起来很合理, 其实会让 q 偏高的 token 被双重计入。")
116     print("    只有第 10 篇推导出的残差分布  $\max(0, p - q) / Z$  能恰好补齐缺口。")
117
118 def exp4():
119     title("实验 4: K 个草稿一轮 —— 期望产出公式与最优 K")
120     # 为了让每个位置的接受率相同 (公式的前提), 这里每个位置都使用同一对 (p, q)

```

```

119 p = [0.40, 0.25, 0.15, 0.10, 0.05, 0.03, 0.02]
120 q = normalize([0.6 * a + 0.4 / len(p) for a in p]) # 一个"还凑合"的草稿模型
121 alpha = sum(min(a, b) for a, b in zip(p, q))
122 rng = random.Random(3)
123 c = 0.1 # 草稿模型单步耗时 = 大模型的
10%
124 print(f" 每个位置接受率  $\alpha$  = {alpha:.3f}, 草稿/目标耗时比 c = {c}")
125 print(f"  {'K':>3} | {'实测每轮产出':>12} | {'公式 (1- $\alpha^{K+1})/(1-\alpha)$ ':>24} | {'
加速比 E/(Kc+1)':>16}")
126 best = (0, 0)
127 for K in [1, 2, 3, 4, 5, 6, 8, 12, 16]:
128     rounds, produced = 20_000, 0
129     for _ in range(rounds):
130         n = 0
131         for _ in range(K):
132             x, ok = speculative_step(p, q, rng)
133             n += 1
134             if not ok:
135                 break
136         else:
137             n += 1 # 全部接受 → 免费多采一个
138         produced += n
139     e_emp = produced / rounds
140     e_th = (1 - alpha ** (K + 1)) / (1 - alpha)
141     sp = e_th / (K * c + 1)
142     best = max(best, (sp, K))
143     print(f"  {'K':>3} | {'e_emp':12.3f} | {'e_th':24.3f} | {'sp':16.2f}x")
144     assert abs(e_emp - e_th) / e_th < 0.03, '实测每轮产出应与公式吻合'
145     print(f"  --> 实测与公式吻合。K 太大时  $\alpha^i$  衰减、草稿成本却线性增加, 最优  $K \approx \{best[1]\}$  (加速 {best[0]:.2f}x) 。")
146     print("      注意这里假设'大模型验证 K 个 token 与验证 1 个一样快'——只在小 batch 的访
存受限区成立 (第 8 篇) 。")
147
148
149 if __name__ == "__main__":
150     exp1()
151     exp2()
152     exp3()
153     exp4()

```

第 24 章 工业量化机制 (RTN / SmoothQuant / AWQ / GPTQ) 实战

24.1 Lab 11: 模型量化技术 (INT8 / INT4 / FP8, AWQ, GPTQ)

24.2 [目标] 什么是量化? 为什么对推理如此重要?

在深度学习推理中, 量化 (Quantization) 是将高精度浮点数 (如 FP32 / FP16) 映射到低位宽定点数或微缩浮点数 (如 INT8 / INT4 / FP8) 的技术。

对于大模型推理, 量化带来双重巨大收益:

1. **显存容量降低**: 70B 模型从 FP16 的 140GB 显存, 经 INT4 压缩后仅需约 35GB, 可以在单张或双张消费级显卡上加载。
2. **生成速度翻倍**: 由于 Decode 阶段瓶颈在于显存带宽, 将权重体积压缩至 1/4 后, 显存读取时间直接缩短至 1/4, 理论生成速度可提升至接近 4 倍!

24.3 [几何] 核心量化数学公式

24.3.1 对称量化 (Symmetric Quantization)

将浮点零点映射为定点零点 (即零偏置 $Z = 0$):

- 缩放因子 (Scale):

$$S = \frac{\max(|X|)}{2^{b-1} - 1}$$

- 量化过程:

$$q = \text{clip} \left(\text{round} \left(\frac{X}{S} \right), -2^{b-1} + 1, 2^{b-1} - 1 \right)$$

- 反量化 (Dequantization):

$$\hat{X} = q \times S$$

24.3.2 非对称量化 (Asymmetric / Affine Quantization)

当数据分布具有明显单边偏置时，引入零点 Zero-point (Z):

- 缩放因子与零点:

$$S = \frac{X_{\max} - X_{\min}}{2^b - 1}$$

$$Z = \text{round}\left(-\frac{X_{\min}}{S}\right)$$

- 量化与反量化:

$$q = \text{clip}\left(\text{round}\left(\frac{X}{S}\right) + Z, 0, 2^b - 1\right)$$

$$\hat{X} = (q - Z) \times S$$

24.4 [模块] 量化粒度 (Granularity)

粒度模式	描述	硬件友好度	精度保持度
Per-Tensor	整个张量共享一个 Scale	最高 (计算开销最小)	最差 (易受离群异常值干扰)
Per-Channel	矩阵的每个输出通道拥有独立的 Scale	高	较好
Per-Group (分组量化)	连续每 32 / 64 / 128 个权重组成一组独立量化 (AWQ/GPTQ 标配; 本仓库脚本用 group_size=32 演示)	中等	最好 (极大减小离群值污染范围)

24.5 [思考] 工业级算法速览

1. W4A16 (Weight-Only Quantization):

- 权重存为 INT4，计算时在寄存器/SRAM 实时反量化为 FP16 与激活值做运算。
- 代表: AWQ (Activation-aware Weight Quantization) (保护 1% 关键特征通道) 和 GPTQ (二阶 Hessian 误差补偿)。原理与实现见下方“进阶”。

1. **W8A8 (Weight & Activation Quantization):**

- 权重与激活值均量化为 INT8 或 FP8，直接使用硬件 INT8/FP8 Tensor Core 进行矩阵乘法，Prefill 和 Decode 均获加速。
- 代表：**SmoothQuant**（通过数学等价变换将激活值的离群峰值转移给权重）。

1. **FP8 量化（现代 GPU 标配）：**

- Hopper/Ada/Blackwell 架构原生支持 E4M3 和 E5M2 两种 FP8 格式，几乎无损兼顾训练与推理。

24.6 [执行] 运行量化实验

运行 `quant_basics.py`:

```
1 python3 lab11_quantization/quant_basics.py
```

对比对称量化、非对称量化以及 Group-wise INT4 分组量化的重构误差（MSE）与信噪比（SNR）。

24.7 [实验] 进阶：从零实现 SmoothQuant / AWQ / GPTQ

原理推导见 第 11 篇：量化。

```
1 .venv/bin/python lab11_quantization/advanced_quant_numpy.py
```

实验	本机结果	说明
每多 1 位 SNR +6 dB	INT7→INT8 +6.1 dB	误差方差 $S^2/12$
激活离群值	6 个离群通道让普通通道 INT8 SNR 只剩 10 dB	LLM.int8() 的发现
SmoothQuant	W8A8 输出 SNR 20.2 → 32.4 dB ($\alpha=0.5$)	恒等变换，零推理开销
AWQ	W4 分组量化输出 SNR +2.9 dB，关键通道误差降为 1/2.6	放大关键通道再量化
GPTQ	输出 SNR 18.2 → 29.6 dB ；但权重本身的误差反而更大	优化 $\ WX - \hat{W}X\ $ 而非 $\ W - \hat{W}\ $ ；输入不相关时毫无收益

动手练习

1. 把实验 5 的 GPTQ 改成分组量化（每 128 列一组各自一个 scale），与 AWQ 在同一份数据上比较。

2. 在 Lab 07b 的 Qwen2.5-0.5B 上，对所有线性层做 W4 分组 RTN 量化（反量化回 BF16 再计算），用一段文本测 PPL 变化；再换成你实现的 GPTQ，看能挽回多少。
3. 用第 8 篇的带宽公式估算：Qwen2.5-7B 在 5060 Ti 上从 BF16 换成 W4A16，Decode 理论上限从多少 token/s 变成多少？为什么 Prefill 不会同样变快？

24.8 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码，重点实现细节如下：

24.8.1 源码实现：quant_basics.py

Listing 24.1: quant_basics.py

```
1  """
2  Lab 11: 模型量化原理与数值精度仿真（零外部依赖）
3
4  本脚本深入演示：
5  1. 对称 INT8 量化 (Symmetric Quantization)
6  2. 非对称 INT8 量化 (Asymmetric Quantization)
7  3. 工业级分组 INT4 量化 (Group-wise INT4, 类似 AWQ/GPTQ 基础)
8  4. 离群值 (Outliers) 敏感性测试与误差 (MSE / SNR) 分析
9  """
10
11 import math
12 import random
13
14
15 def mean_squared_error(orig, dequant):
16     total = sum((a - b) ** 2 for a, b in zip(orig, dequant))
17     return total / len(orig)
18
19
20 def signal_to_noise_ratio(orig, dequant):
21     signal_pwr = sum(a ** 2 for a in orig) / len(orig)
22     noise_pwr = sum((a - b) ** 2 for a, b in zip(orig, dequant)) / len(orig)
23     if noise_pwr < 1e-12:
24         return float('inf')
25     return 10.0 * math.log10(signal_pwr / noise_pwr)
26
27
28 # =====
29 # 1. 对称 INT8 量化 (Zero-point = 0)
30 # =====
31 def quantize_symmetric_int8(tensor):
32     """
33     对称量化映射到 [-127, 127]
```

```

34     Scale = max(|tensor|) / 127
35     """
36     max_abs = max(abs(x) for x in tensor)
37     if max_abs == 0:
38         scale = 1.0
39     else:
40         scale = max_abs / 127.0
41
42     # 量化到 INT8
43     q_tensor = []
44     for x in tensor:
45         q = round(x / scale)
46         q = max(-127, min(127, q))
47         q_tensor.append(int(q))
48
49     # 反量化
50     dequant = [q * scale for q in q_tensor]
51     return q_tensor, scale, dequant
52
53
54 # =====
55 # 2. 非对称 INT8 量化 (Zero-point != 0)
56 # =====
57 def quantize_asymmetric_int8(tensor):
58     """
59     非对称量化映射到 [0, 255]
60     Scale = (max - min) / 255
61     Zero_point = round(-min / scale)
62     """
63     t_min = min(tensor)
64     t_max = max(tensor)
65     if t_max == t_min:
66         scale = 1.0
67         zp = 0
68     else:
69         scale = (t_max - t_min) / 255.0
70         zp = round(-t_min / scale)
71         zp = max(0, min(255, zp))
72
73     q_tensor = []
74     for x in tensor:
75         q = round(x / scale) + zp
76         q = max(0, min(255, q))
77         q_tensor.append(int(q))
78
79     # 反量化
80     dequant = [(q - zp) * scale for q in q_tensor]
81     return q_tensor, scale, zp, dequant

```

```

82
83
84 # =====
85 # 3. 分组 INT4 量化 (Group-wise INT4)
86 # =====
87 def quantize_groupwise_int4(tensor, group_size=32):
88     """
89     将一维张量切分为长度为 group_size 的小块，每块独立进行对称 INT4 量化 [-7, 7]
90     这是现代 AWQ、GPTQ 维持超高精度的核心基石。
91     """
92     dequant_all = []
93     scales = []
94
95     num_elements = len(tensor)
96     for i in range(0, num_elements, group_size):
97         chunk = tensor[i: i + group_size]
98         max_abs = max(abs(x) for x in chunk)
99         scale = (max_abs / 7.0) if max_abs > 0 else 1.0
100         scales.append(scale)
101
102         # 量化与反量化当前 group
103         for x in chunk:
104             q = round(x / scale)
105             q = max(-7, min(7, q))
106             dequant_all.append(q * scale)
107
108     return dequant_all, scales
109
110
111 def run_experiment():
112     print("=" * 70)
113     print("[起步] 模型量化算法与误差评测 (Quantization Benchmark)")
114     print("=" * 70)
115
116     # 模拟一个 Transformer 层的权重分布 (正态分布 + 少量极值离群点)
117     random.seed(42)
118     n_params = 1024
119     normal_weights = [random.gauss(0, 0.05) for _ in range(n_params)]
120
121     # 场景 A: 无极端离群值的标准分布
122     print("\n【场景 A】: 标准均匀高斯权重 (无显著离群点)")
123     print("-" * 70)
124
125     # 对称 INT8
126     _, _, deq_sym8 = quantize_symmetric_int8(normal_weights)
127     mse_sym8 = mean_squared_error(normal_weights, deq_sym8)
128     snr_sym8 = signal_to_noise_ratio(normal_weights, deq_sym8)
129

```

```

130 # 非对称 INT8
131 _, _, _, deq_asym8 = quantize_asymmetric_int8(normal_weights)
132 mse_asym8 = mean_squared_error(normal_weights, deq_asym8)
133 snr_asym8 = signal_to_noise_ratio(normal_weights, deq_asym8)
134
135 # 分组 INT4 (group_size=32)
136 deq_grp4, _ = quantize_groupwise_int4(normal_weights, group_size=32)
137 mse_grp4 = mean_squared_error(normal_weights, deq_grp4)
138 snr_grp4 = signal_to_noise_ratio(normal_weights, deq_grp4)
139
140 print(f"1. 对称 INT8      : MSE = {mse_sym8:.8e} | SNR = {snr_sym8:6.2f} dB (显
141 存压缩 50%)")
142 print(f"2. 非对称 INT8   : MSE = {mse_asym8:.8e} | SNR = {snr_asym8:6.2f} dB (
143 显存压缩 50%)")
144 print(f"3. 分组 INT4(G32): MSE = {mse_grp4:.8e} | SNR = {snr_grp4:6.2f} dB (显
145 存压缩 75%)")
146
147 # 场景 B: 含有离群点 (Outliers) 的权重
148 print("\n【场景 B】: 注入 2 个大模型中常见的显著离群点 (Outliers: 绝对值放大 50 倍)")
149 print("-" * 70)
150 outlier_weights = list(normal_weights)
151 outlier_weights[10] = 2.5 # 正常值在 0.05 左右, 此处为超大离群值
152 outlier_weights[200] = -2.8
153
154 # 全局对称 INT8
155 _, _, deq_sym8_out = quantize_symmetric_int8(outlier_weights)
156 mse_sym8_out = mean_squared_error(outlier_weights, deq_sym8_out)
157 snr_sym8_out = signal_to_noise_ratio(outlier_weights, deq_sym8_out)
158
159 # 全局非对称 INT8
160 _, _, _, deq_asym8_out = quantize_asymmetric_int8(outlier_weights)
161 mse_asym8_out = mean_squared_error(outlier_weights, deq_asym8_out)
162 snr_asym8_out = signal_to_noise_ratio(outlier_weights, deq_asym8_out)
163
164 # 分组 INT4
165 deq_grp4_out, _ = quantize_groupwise_int4(outlier_weights, group_size=32)
166 mse_grp4_out = mean_squared_error(outlier_weights, deq_grp4_out)
167 snr_grp4_out = signal_to_noise_ratio(outlier_weights, deq_grp4_out)
168
169 # 同为 4 bit 的公平对照: 全局 INT4 (整个张量只有一组 scale)
170 deq_glob4_out, _ = quantize_groupwise_int4(outlier_weights, group_size=len(
171 outlier_weights))
172 snr_glob4_out = signal_to_noise_ratio(outlier_weights, deq_glob4_out)
173 deq_glob4, _ = quantize_groupwise_int4(normal_weights, group_size=len(
174 normal_weights))
175 snr_glob4 = signal_to_noise_ratio(normal_weights, deq_glob4)
176
177 print(f"1. 全局对称 INT8 (Per-Tensor) : MSE = {mse_sym8_out:.8e} | SNR = {

```

```

173     snr_sym8_out:6.2f} dB (比场景 A 下降 {snr_sym8 - snr_sym8_out:5.2f} dB)")
174     print(f"2. 全局非对称 INT8          : MSE = {mse_asym8_out:.8e} | SNR = {
snr_asym8_out:6.2f} dB (比场景 A 下降 {snr_asym8 - snr_asym8_out:5.2f} dB)")
175     print(f"3. 分组 INT4 (Group-wise G32) : MSE = {mse_grp4_out:.8e} | SNR = {
snr_grp4_out:6.2f} dB (比场景 A 下降 {snr_grp4 - snr_grp4_out:5.2f} dB)")
176     print(f"4. 对照: 全局 INT4 (Per-Tensor): SNR 从 {snr_glob4:6.2f} dB 降到 {
snr_glob4_out:6.2f} dB")
177     # 分组量化的优势是"对离群值不敏感", 而不是"4 bit 比 8 bit 更准"
178     assert snr_grp4 - snr_grp4_out < 2 and snr_sym8 - snr_sym8_out > 10, "离群值应
重創全局量化、几乎不影响分组量化"
179     assert snr_grp4_out > snr_glob4_out + 10, "同为 4 bit 时分组应远好于全局"
180
181     print("\n" + "=" * 70)
182     print("[思考] 工业级洞察 (Key Takeaways):")
183     print("    1. 【全局量化的死穴】: 当张量中存在个别极大离群值时, 全局 Per-Tensor 的 Scale
被拉得极大, 导致正常的小数值在量化时被无情舍入为 0, SNR 骤降。")
184     print("    2. 【分组量化 (Group-wise) 的力量】: 将权重按 Group (如 32 或 128) 切分, 离
群值只污染自身所在的小组, 其他组的精度完好无损。")
185     print("        注意比较的口径: 分组 INT4 的绝对 SNR 仍低于全局 INT8 (4 bit 本来就比 8
bit 粗), 它的优势是【几乎不受离群值影响】, ")
186     print("        以及同为 4 bit 时远好于全局 INT4。")
187     print("    3. 这就是为什么现代 AWQ / GPTQ 几乎一律采用 Group-wise INT4: 省 75% 显存,
精度损失通常很小 (但要用 PPL 与下游任务实测, 见第 11 篇)。")
188     print("=" * 70)
189
190 if __name__ == "__main__":
191     run_experiment()

```

24.8.2 源码实现: advanced_quant_numpy.py

Listing 24.2: advanced_quant_numpy.py

```

1  """
2  Lab 11 - 进阶: 用 NumPy 从零实现 SmoothQuant、AWQ、GPTQ, 验证第 11 篇的每个论断
3
4  对应教程: zero_to_hero_tutorial/11_量化: 从每位6dB到SmoothQuant、AWQ与GPTQ.md
5
6  实验清单:
7      1. 每多 1 位, 信噪比 +6 dB (均匀量化误差方差 =  $S^2/12$ )
8      2. 激活离群通道: per-tensor INT8 量化被几个通道毁掉
9      3. SmoothQuant: 严格的数学恒等变换 + 把激活的量化难度转移给权重 (w8a8)
10     4. AWQ: 放大关键输入通道的权重, w4 分组量化的输出误差下降
11     5. GPTQ: 逐列量化 + 用 Hessian 逆补偿误差, 对比直接四舍五入 (RTN)
12
13  运行:
14     .venv/bin/python lab11_quantization/advanced_quant_numpy.py

```

```

15  """
16
17  import numpy as np
18
19  rng = np.random.default_rng(0)
20
21
22  def title(s):
23      print("\n" + "=" * 78)
24      print(f"[实验] {s}")
25      print("=" * 78)
26
27
28  def snr_db(ref, approx):
29      return 10 * np.log10((ref ** 2).mean() / ((ref - approx) ** 2).mean())
30
31
32  def quant_sym(x, bits, axis=None):
33      """对称量化再反量化。axis=None: 整个张量一个 scale; axis=k: 沿第 k 维求 max, 即每个"其
34      它维度的切片"一个 scale"""
35      qmax = 2 ** (bits - 1) - 1
36      amax = np.abs(x).max() if axis is None else np.abs(x).max(axis=axis, keepdims=
37      True)
38      scale = np.maximum(amax, 1e-12) / qmax
39      return np.clip(np.round(x / scale), -qmax, qmax) * scale
40
41  def quant_group(W, bits, group):
42      """W: [C_in, C_out], 沿输入维度每 group 个元素一组 (每个输出列各自分组), 对称量化"""
43      C_in, C_out = W.shape
44      Wg = W.reshape(C_in // group, group, C_out)
45      return quant_sym(Wg, bits, axis=1).reshape(C_in, C_out)
46
47  # =====
48  # 实验 1
49  # =====
50  def exp1_six_db():
51      title("实验 1: 每多 1 位, 信噪比约 +6 dB")
52      x = rng.standard_normal(1_000_000)
53      prev = None
54      for b in range(2, 9):
55          s = snr_db(x, quant_sym(x, b))
56          delta = "" if prev is None else f"({s - prev:.2f} dB)"
57          print(f"INT{b}: SNR = {s:6.2f} dB {delta}")
58          assert prev is None or 5.5 < s - prev < 8, '每位增益应在 6 dB 附近'
59          prev = s
60      print("    --> 位数 +1 → 格宽 S 减半 → 误差方差 S²/12 变为 1/4 → SNR +10 · log10(4)

```

```

≈ 6.02 dB。 \n"
61     "        理论上 INT8→INT4 损失约 24 dB；但那是高分辨率近似，低位宽下反而损失更多
    ____"
62     "本机实测请见上面打印的实际 SQNR 差值（通常 25 dB 左右，低比特那几档每 bit 超过 6
    dB）。")
63
64
65 # =====
66 # 共享的"模拟 LLM 线性层"数据
67 # =====
68 def make_layer(T=512, C_in=1024, C_out=1024, n_outlier=6, outlier_scale=40.0):
69     X = rng.standard_normal((T, C_in))
70     outlier_ch = rng.choice(C_in, n_outlier, replace=False)
71     X[:, outlier_ch] *= outlier_scale      # 少数固定通道在每个 token 上都很大 (LLM.
    int8 的发现)
72     W = rng.standard_normal((C_in, C_out)) * 0.02
73     return X, W, outlier_ch
74
75
76 # =====
77 # 实验 2 & 3
78 # =====
79 def exp2_3_smoothquant():
80     X, W, outlier_ch = make_layer()
81     Y = X @ W
82     title("实验 2: 激活离群通道让 per-tensor 量化失效")
83     ch_max = np.abs(X).max(axis=0)
84     print(f"    {X.shape[1]} 个通道里有 {len(outlier_ch)} 个离群通道: 离群通道最大值 ≈ {
    ch_max[outlier_ch].mean():.0f}, "
85           f"普通通道 ≈ {np.median(ch_max):.1f}")
86     normal = np.setdiff1d(np.arange(X.shape[1]), outlier_ch)
87     Xq = quant_sym(X, 8)
88     print(f"    per-tensor INT8 量化激活: 整体 SNR = {snr_db(X, Xq):.1f} dB, 但普通通道
    SNR 只有 {snr_db(X[:, normal], Xq[:, normal]):.1f} dB")
89     print(f"    (格宽由最大值约 {np.abs(X).max():.0f} 决定, 普通通道的值大多被舍入到了 0
    附近的几格里) ")
90     print(f"    权重分布规整: per-tensor INT8 量化权重 SNR = {snr_db(W, quant_sym(W,
    8)):.1f} dB")
91
92     title("实验 3: SmoothQuant —— 恒等变换  $X \cdot W = (X/s) \cdot (s \cdot W)$ , 把难度从激活转移给权重"
    )
93     base = snr_db(Y, quant_sym(X, 8) @ quant_sym(W, 8))
94     print(f"    直接 W8A8 (均 per-tensor) : 输出 SNR = {base:.1f} dB")
95     for alpha in [0.25, 0.5, 0.75]:
96         s = np.abs(X).max(axis=0) ** alpha / np.abs(W).max(axis=1) ** (1 - alpha)
97         Xs, Ws = X / s, W * s[:, None]
98         exact = np.abs(Xs @ Ws - Y).max()
99         out = snr_db(Y, quant_sym(Xs, 8) @ quant_sym(Ws, 8))

```

```

100     print(f"    α = {alpha:<4}: 未量化时变换前后最大差 {exact:.1e} (严格等价) | W8A8
      输出 SNR = {out:.1f} dB ({out - base:.1f})")
101     assert exact < 1e-10 and out > base, 'SmoothQuant 应严格等价且改善 W8A8'
102     print("    --> α=0.5 附近效果最好: 激活的离群值被压下去, 权重相应变大一点, 两边都好量化。"
      )
103     print("    推理时 1/s 合并进前一层 RMSNorm 的 γ、s 合并进 W, 计算图零改动、零额外开
      销。")
104
105
106 # =====
107 # 实验 4: AWQ
108 # =====
109 def exp4_awq():
110     title("实验 4: AWQ —— 按激活幅度放大关键权重通道, 再做 W4 分组量化")
111     T, C_in, C_out, group = 512, 1024, 512, 128
112     X = rng.standard_normal((T, C_in))
113     salient = rng.choice(C_in, C_in // 100, replace=False) # 约 1% 的输入通道激活很
      大
114     X[:, salient] *= 15.0
115     W = rng.standard_normal((C_in, C_out)) * 0.02
116     Y = X @ W
117
118     rtn = snr_db(Y, X @ quant_group(W, 4, group))
119     print(f"    W4 分组量化 (group={group}), 直接四舍五入 RTN: 输出 SNR = {rtn:.2f} dB"
      )
120     act_mag = np.abs(X).mean(axis=0)
121     best = (rtn, 0.0)
122     for beta in np.linspace(0, 1, 11):
123         s = act_mag ** beta
124         s = s / np.sqrt(s.max() * s.min()) # 归一化, 避免整体缩放
125         Wq = quant_group(W * s[:, None], 4, group) / s[:, None] # 等价于  $Q(w \cdot s) \cdot (x/s)$ 
126         val = snr_db(Y, X @ Wq)
127         best = max(best, (val, beta))
128     print(f"    AWQ 网格搜索 β ∈ [0,1], s = (激活平均幅度)^β: 最优 β = {best[1]:.1f}, 输
      出 SNR = {best[0]:.2f} dB ({best[0] - rtn:.2f} dB)")
129     assert best[0] > rtn, 'AWQ 应改善输出 SNR'
130     s = act_mag ** best[1]; s = s / np.sqrt(s.max() * s.min())
131     err_rtn = np.abs((W - quant_group(W, 4, group))[salient]).mean()
132     err_awq = np.abs((W - quant_group(W * s[:, None], 4, group) / s[:, None])[
      salient]).mean()
133     print(f"    关键通道上的权重误差: RTN {err_rtn:.2e} → AWQ {err_awq:.2e} (这些误差会被
      15 倍的大激活放大, 所以最要紧)")
134     assert err_awq < err_rtn, 'AWQ 应减小关键通道的权重误差'
135     print("    --> 放大 s 倍后, 关键通道相对于格宽更'粗', 舍入误差相对变小; 代价是同组其它权重
      精度略降, 搜索 β 找平衡。")
136
137

```

```

138 # =====
139 # 实验 5: GPTQ
140 # =====
141 def gptq(W, X, bits=4, damp=0.01):
142     """
143     W: [C_out, C_in] (与 nn.Linear 相同), X: [C_in, n] 校准输入。最小化  $||WX - \hat{W}X||^2$ 。
144     每个输出行一个 scale (per-channel), 按输入通道从左到右逐列量化, 并用  $H^{-1}$  把误差摊给
145     还没量化的列。
146     """
147     W = W.copy()
148     C_out, C_in = W.shape
149     qmax = 2 ** (bits - 1) - 1
150     scale = np.abs(W).max(axis=1) / qmax # 用原始权重确定格宽,
151     # 与 RTN 相同
152     H = 2 * X @ X.T
153     H += damp * np.mean(np.diag(H)) * np.eye(C_in) # 阻尼: 保证可逆、数值
154     # 稳定
155     Hinv = np.linalg.inv(H)
156     for j in range(C_in):
157         w = W[:, j]
158         q = np.clip(np.round(w / scale), -qmax, qmax) * scale
159         err = (w - q) / Hinv[j, j]
160         W[:, j] = q
161         W[:, j + 1:] -= np.outer(err, Hinv[j, j + 1:]) # OBS 最优补偿: 让后面
162         # 的列替第 j 列"还债"
163         Hinv -= np.outer(Hinv[:, j], Hinv[j, :]) / Hinv[j, j] # 从问题中移除第 j 列
164         # (高斯消元一步)
165     return W
166
167 def exp5_gptq():
168     title("实验 5: GPTQ vs 直接四舍五入 (RTN), INT4 per-channel")
169     C_in, C_out, n = 256, 256, 4096
170     # 让输入通道之间"相关": 真实网络里的激活有很强的相关结构, 这正是 GPTQ 能补偿误差的前提
171     mix = rng.standard_normal((C_in, 32)) @ rng.standard_normal((32, C_in)) / 6 +
172     np.eye(C_in)
173     X_cal = mix @ rng.standard_normal((C_in, n))
174     X_test = mix @ rng.standard_normal((C_in, n)) # 独立的测试数据, 检验
175     # 是否过拟合校准集
176     W = rng.standard_normal((C_out, C_in)) * 0.02
177     Y = W @ X_test
178
179     qmax = 7
180     scale = np.abs(W).max(axis=1, keepdims=True) / qmax
181     W_rtn = np.clip(np.round(W / scale), -qmax, qmax) * scale
182     W_gptq = gptq(W, X_cal)
183     s_rtn, s_gptq = snr_db(Y, W_rtn @ X_test), snr_db(Y, W_gptq @ X_test)
184     print(f" 权重本身的误差: RTN SNR = {snr_db(W, W_rtn):.2f} dB | GPTQ SNR = {

```

```

snr_db(W, W_gptq):.2f} dB")
179 print(f"    层输出（测试集）：RTN SNR = {s_rtn:.2f} dB | GPTQ SNR = {s_gptq:.2f}
dB (+{s_gptq - s_rtn:.2f} dB)")
180 assert s_gptq > s_rtn + 5, 'GPTQ 应显著优于 RTN'
181 print("    --> 有意思的是：GPTQ 的【权重】误差反而更大（它故意改动了还没量化的权重），但
【输出】误差显著更小。")
182 print("    GPTQ 优化的是  $||WX - \hat{W}X||$ ，不是  $||W - \hat{W}||$ ：输入通道相关时，一系列的舍入误
差可以由相关的列抵消。")
183
184 X_ind = rng.standard_normal((C_in, n))
185 W_g2 = gptq(W, X_ind)
186 X_ind_test = rng.standard_normal((C_in, n))
187 gain = snr_db(W @ X_ind_test, W_g2 @ X_ind_test) - snr_db(W @ X_ind_test,
W_rtn @ X_ind_test)
188 print(f"    对照：若输入通道彼此独立（H 近似对角），GPTQ 相对 RTN 只提升 {gain:+.2f} dB
—— 没有相关性就没有东西可补偿。")
189
190
191 if __name__ == "__main__":
192     exp1_six_db()
193     exp2_3_smoothquant()
194     exp4_awq()
195     exp5_gptq()

```

第 25 章 Continuous Batching (持续批处理) 动态调度仿真

25.1 Lab 12a: 显存调度与连续批处理 (Continuous Batching)

25.2 [施工] 为什么静态批处理 (Static Batching) 在 LLM 中失效?

在传统的计算机视觉或文本分类任务中，模型输入输出维度是固定的，静态批处理 (Static Batching) 非常有效。

但在大语言模型生成场景中，每个请求的：

1. 输入 Prompt 长度极不一致 (有的几十字，有的几千字)。
2. 生成长度 (Output Tokens) 难以事先预知 (有的遇到 <eos> 10 步就结束，有的需要生成 1000 步)。

25.2.1 静态 Batching 的两大致命缺陷：

```
Request 1: [Prompt: 10] [=== Gen: 20 ===] -> 结束! (闲置等待...) [PADDING PADDING PADDING]
Request 2: [Prompt: 30] [===== Gen: 100 =====] -> 最长请求!
Request 3: [Prompt: 5 ] [= Gen: 10 =] -> 结束! (闲置等待...) [PADDING PADDING PADDING PADDING]
```

1. **显存与算力气泡 (Bubble)**：短请求早早结束，但由于必须作为一个 Batch 统一返回，短请求必须不断 padding 空 token 陪跑，浪费大量算力与带宽。
2. **首字延迟排队灾难**：新到达的请求必须等待当前 Batch 中最慢的那一个请求完全结束，才能开始下一批，导致极高的排队排班延迟。

25.3 [加速] 连续批处理 (Continuous Batching / In-flight Batching)

由 OSDI'22 论文 *Orca* 提出，并在 vLLM 和 TensorRT-LLM 中发扬光大：

核心导读与背景

核心思想：调度粒度从「请求级 (Request-level)」细化为「迭代步进级 (Iteration-level)」。

```
Iteration t: [Req 1: decode step 15] [Req 2: decode step 40] [Req 3: decode step 8]
Iteration t+1: [Req 1: 触发 EOS! 退出] [Req 2: decode step 41] [Req 3: decode step 9]
                ↓ (槽位立即释放, 新请求无缝插入!)
Iteration t+2: [Req 4: 快速 Prefill!] [Req 2: decode step 42] [Req 3: decode step 10]
```

- **立即退场**: 任何请求一旦生成结束标记 (EOS) 或达到最大长度, 当步立刻释放槽位与 KV Cache。
- **即时插入**: 等待队列中的新请求无需等待其他长请求完成, 在下一步前向迭代中直接加入当前 Batch 进行 Prefill。
- **吞吐飞跃**: 消除了绝大部分 Padding 气泡, 吞吐量显著提升 (Orca、vLLM 等论文中常见数倍提升, 具体取决于负载)。本仿真会同时打印“Padding 无效计算占比”和“槽位占用率”: 前者对连续批处理按构造恒为 0, 后者才是两种方案可比的指标。

25.4 [文档] 内存革命: PagedAttention 原理

即使有了 Continuous Batching, 连续的内存分配依然会造成严重浪费:

- 在传统显存管理中, 框架必须为请求预先分配一段**连续的固定最大长度显存** (如预留 4096 tokens)。
- 但大多数请求只生成了 100 个 token。vLLM 论文测到 60%~80% 的 KV 显存被浪费——这不只是**内部碎片**, 还包括按 `max_len` 预留的闲置和外部碎片, 三者要分开看 (第 12 篇 §3)。

vLLM 的解决方案: 借鉴操作系统的**虚拟内存分页机制 (Virtual Memory Paging)**:

- 将 KV Cache 切分成固定大小的物理块 (Physical Blocks, 例如每块 16 个 tokens)。
- 物理显存块**无需连续**, 通过一个逻辑块表 (Block Table) 映射。
- 只有当生成了新的 16 个 token 时, 才向物理池申请分配 1 个新块。
- **成效**: 分页把「最后一个块没写满」以外的浪费基本消掉; 单序列的上界是 $15/S$, 序列长度 $S \gtrsim 400$ token 时才低于 4% (短对话达不到, 见第 12 篇 §3); 实际收益是同一张卡能承载成倍增加的并发请求数。

25.5 [执行] 运行仿真实验

运行 `continuous_batching_sim.py`:

```
1 python3 lab12a_continuous_batching/continuous_batching_sim.py
```

对比静态批处理与连续批处理在处理相同真实负载时的总吞吐量 (Throughput)、平均等待延迟 (Latency) 与算力浪费 (Bubble Ratio)。

25.6 [链接] 继续深入

- **为什么”每一步耗时视为常数”这个假设成立?** 本仿真假设无论批里有几个请求, 一步的耗时都一样。这来自第 8 篇的推导: Decode 线性层的算术强度 $\approx$ batch size, batch 远小于拐点时, 一步的耗时由”读一遍权重”决定, 与 batch 几乎无关。见第 8 篇 §4 与 lab08_memory_and_roofline/measure_gpu_roofline.py 实验 4。
- **本仿真没有模拟的:** Prefill 的开销、显存上限、长 Prefill 对 Decode 的干扰。这些在第 12 篇与 Lab 12b (分页 KV Cache、前缀缓存、Chunked Prefill) 中补齐。

25.7 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码, 重点实现细节如下:

25.7.1 源码实现: `continuous_batching_sim.py`

Listing 25.1: `continuous_batching_sim.py`

```
1 """
2 Lab 12a: 静态 Batching vs 连续批处理 (Continuous Batching) 离散事件仿真器 (零外部依赖)
3
4 本脚本直观演示:
5 1. 静态批处理 (Static Batching): 必须等待 Batch 内所有请求全部生成结束, 长尾请求导致严重的
   Padding 气泡与排队延迟。
6 2. 连续批处理 (Continuous Batching / In-flight Batching): 迭代级调度, 短请求一旦结束立即
   释放槽位, 等待中的新请求立刻补位。
7 3. 统计两者的系统吞吐量 (Throughput)、首字排队耗时 (TTFT 排队延迟) 与算力槽位浪费率 (Bubble
   Ratio)。
8 """
9
10 import collections
11
12
13 class Request:
14     def __init__(self, req_id, arrival_time, output_len):
15         self.req_id = req_id
```

```

16         self.arrival_time = arrival_time
17         self.output_len = output_len
18         self.tokens_generated = 0
19         self.start_time = None
20         self.finish_time = None
21
22     def is_finished(self):
23         return self.tokens_generated >= self.output_len
24
25
26 def simulate_static_batching(requests_data, max_batch_size=4):
27     """
28     静态批处理模式：
29     每次必须凑足（或取满）max_batch_size 个请求作为一个固定批次，
30     批次内所有请求必须等待最长请求完成后，才能释放槽位迎接下一批。
31     """
32     queue = [Request(r["id"], r["arrival"], r["output_len"]) for r in
33               requests_data]
34     current_time = 0
35     completed = []
36
37     total_effective_tokens = sum(r.output_len for r in queue)
38     total_slot_steps = 0 # 统计占用的槽位步数
39
40     while queue:
41         # 挑选已到达的请求
42         ready = [r for r in queue if r.arrival_time <= current_time]
43         if not ready:
44             # 快进时间到下一个到达的请求
45             current_time = queue[0].arrival_time
46             ready = [queue[0]]
47
48         batch = ready[:max_batch_size]
49         for r in batch:
50             queue.remove(r)
51             r.start_time = current_time
52
53         # 批次中最长生成长度
54         max_len = max(r.output_len for r in batch)
55
56         # 静态执行 max_len 步
57         total_slot_steps += max_len * len(batch)
58         current_time += max_len
59
60         for r in batch:
61             r.tokens_generated = r.output_len
62             r.finish_time = current_time
63             completed.append(r)

```

```

63
64     makespan = current_time
65     # 算力浪费率: 实际生成的 token 步数 vs 实际占用的槽位步数
66     bubble_ratio = (total_slot_steps - total_effective_tokens) / total_slot_steps
67     if total_slot_steps > 0 else 0
68     return completed, makespan, bubble_ratio
69
70 def simulate_continuous_batching(requests_data, max_batch_size=4):
71     """
72     连续批处理模式 (Continuous Batching / In-flight Batching):
73     每一步迭代 (Iteration) 检查:
74     - 是否有请求生成完毕? 若有, 立即退场!
75     - 是否有空余槽位且等待队列中有新请求已到达? 若有, 立即加入当前步!
76     """
77     pending = collections.deque([Request(r["id"], r["arrival"], r["output_len"])
78     for r in requests_data])
79     active_slots = []
80     completed = []
81     current_time = 0
82
83     total_effective_tokens = sum(r["output_len"] for r in requests_data)
84     total_slot_steps = 0
85
86     while pending or active_slots:
87         # 1. 尝试将已到达的新请求补充进空闲槽位
88         while len(active_slots) < max_batch_size and pending:
89             if pending[0].arrival_time <= current_time:
90                 req = pending.popleft()
91                 if req.start_time is None:
92                     req.start_time = current_time
93                     active_slots.append(req)
94             else:
95                 break
96
97         # 如果槽位全空且没有就绪请求, 快进时间
98         if not active_slots:
99             current_time = pending[0].arrival_time
100             continue
101
102         # 2. 执行单步生成 (Iteration Step)
103         total_slot_steps += len(active_slots)
104         current_time += 1
105
106         # 3. 检查每个槽位的生成进度
107         survivors = []
108         for req in active_slots:
109             req.tokens_generated += 1

```

```

109         if req.is_finished():
110             req.finish_time = current_time
111             completed.append(req)
112         else:
113             survivors.append(req)
114
115     active_slots = survivors
116
117     makespan = current_time
118     bubble_ratio = (total_slot_steps - total_effective_tokens) / total_slot_steps
119     if total_slot_steps > 0 else 0
120     return completed, makespan, bubble_ratio
121
122 def run_simulation():
123     print("=" * 72)
124     print("[起步] 静态批处理 (Static) vs 连续批处理 (Continuous) 调度性能仿真")
125     print("=" * 72)
126
127     # 模拟 10 个具有不同到达时间与输出长度的并发请求
128     # 包含典型长尾负载: 部分请求 5-15 个 token 极快结束, 部分需要 80 个 token
129     requests_workload = [
130         {"id": 1, "arrival": 0, "output_len": 15},
131         {"id": 2, "arrival": 0, "output_len": 70}, # 长请求
132         {"id": 3, "arrival": 0, "output_len": 10},
133         {"id": 4, "arrival": 0, "output_len": 25},
134         {"id": 5, "arrival": 10, "output_len": 80}, # 稍后到达的长请求
135         {"id": 6, "arrival": 12, "output_len": 8},
136         {"id": 7, "arrival": 20, "output_len": 18},
137         {"id": 8, "arrival": 25, "output_len": 12},
138         {"id": 9, "arrival": 30, "output_len": 65},
139         {"id": 10, "arrival": 35, "output_len": 20},
140     ]
141
142     total_tokens = sum(r["output_len"] for r in requests_workload)
143     max_batch = 4
144
145     print(f"[重点] 测试负载: {len(requests_workload)} 个请求 | 总 Token 需求: {total_tokens} | 槽位容量 (Max Batch): {max_batch}")
146     print(f"[重点] 请求分布:")
147     for r in requests_workload:
148         print(f"    - 请求 #{r['id']:2d}: 到达时间 t={r['arrival']:2d} | 目标输出长度 = {r['output_len']:2d} tokens")
149     print("-" * 72)
150
151     # 1. 静态批处理仿真
152     c_static, makespan_static, bubble_static = simulate_static_batching(
        requests_workload, max_batch_size=max_batch)

```

```

153     avg_wait_static = sum(r.start_time - r.arrival_time for r in c_static) / len(
154         c_static)
155     avg_turnaround_static = sum(r.finish_time - r.arrival_time for r in c_static)
156         / len(c_static)
157     throughput_static = total_tokens / makespan_static
158
159     # 2. 连续批处理仿真
160     c_cont, makespan_cont, bubble_cont = simulate_continuous_batching(
161         requests_workload, max_batch_size=max_batch)
162     avg_wait_cont = sum(r.start_time - r.arrival_time for r in c_cont) / len(
163         c_cont)
164     avg_turnaround_cont = sum(r.finish_time - r.arrival_time for r in c_cont) /
165         len(c_cont)
166     throughput_cont = total_tokens / makespan_cont
167
168     # 3. 对比总结
169     print("\n[图表] 调度算法核心指标对比:")
170     print(f"    {'指标项':<24} | {'静态批处理 (Static)':<22} | {'连续批处理 (Continuous)':<22}")
171     print(f"    {'-'*24}+--{'-'*22}+--{'-'*22}")
172     print(f"    {'总处理耗时 (Makespan)':<24} | {makespan_static:<22} 步 | {
173         makespan_cont:<22} 步")
174     print(f"    {'系统总吞吐量 (Throughput)':<24} | {throughput_static:<20.2f} tok/步
175         | {throughput_cont:<20.2f} tok/步")
176     print(f"    {'平均首字排队等待时间':<24} | {avg_wait_static:<22.2f} 步 | {
177         avg_wait_cont:<22.2f} 步")
178     print(f"    {'平均端到端周转耗时':<24} | {avg_turnaround_static:<22.2f} 步 | {
179         avg_turnaround_cont:<22.2f} 步")
180     print(f"    {'Padding 无效计算占比':<24} | {bubble_static * 100:<21.1f}% | {
181         bubble_cont * 100:<21.1f}%")
182     util_static = total_tokens / (max_batch * makespan_static)
183     util_cont = total_tokens / (max_batch * makespan_cont)
184     print(f"    {'槽位占用率 (有效token/容量)':<22} | {util_static * 100:<21.1f}% | {
185         util_cont * 100:<21.1f}%")
186     # [注意] 注意 bubble_cont 恒等于 0.0 —— 它是【构造使然】，不是优化出来的结果：
187     #     上面的 total_slot_steps += len(active_slots) 与每个活跃槽 +1 个 token 是同一
188     #     件事，
189     #     所以 (x - x) / x 永远是 0。拿它当断言等于什么都没断言（这里就不写了）。
190     #     真正可能因调度器写错而失败的，是下面这两条：
191     assert util_cont > util_static, f"连续批处理的槽位占用率 {util_cont:.1f}% 应高于静态
192         批处理的 {util_static:.1f}%"
193     assert makespan_cont < makespan_static, \
194         f"连续批处理的总耗时 {makespan_cont} 应短于静态批处理的 {makespan_static}"
195
196     print("\n[起步] 连续批处理核心收益:")
197     print(f"    - 总耗时缩短 : {(1 - makespan_cont / makespan_static) * 100:.1f}%")
198     print(f"    - 吞吐量提升 : {throughput_cont / throughput_static:.2f}x")
199     print(f"    - 首字等待排队缩短 : {avg_wait_static / avg_wait_cont if

```

```

187     avg_wait_cont > 0 else 999:.1f}x")
188
189     print("\n[思考] 架构剖析:")
190     print(f"    1. 静态批处理中, 短请求做完后必须填充 Padding 陪跑: 本负载下 {bubble_static * 100:.0f}% 的计算花在了 Padding 上。")
191     print(f"    2. 连续批处理的 Padding 无效计算按构造就是 0 (结束的请求当步就离开), 所以这一行的对比只说明'它不陪跑'。")
192     print(f"    更公平的对比是槽位占用率: {util_static * 100:.0f}% → {util_cont * 100:.0f}%。连续批处理也达不到 100%, ")
193     print(f"    剩下的空槽来自'没有新请求可补位' (请求到达得不够快) ——此时空槽不算浪费, 只是负载不满。")
194     print(f"    3. 这正是为什么现代大模型推理服务 (如 vLLM、TGI、TensorRT-LLM) 必须将连续批处理作为默认底座!")
195     print("=" * 72)
196
197 if __name__ == "__main__":
198     run_simulation()

```

第 26 章 PagedAttention (分页显存分配机制) 仿真

26.1 Lab 12b: PagedAttention、写时复制、前缀缓存与 Chunked Prefill

核心导读与背景

对应理论：第 12 篇：推理服务系统前置：第 6 篇、Lab 12a、第 8 篇、第 9 篇（Online Softmax）。依赖：NumPy。

核心导读与背景

[计时] 以下计时类数字是本机某一次运行的量级示例（显卡空闲、已预热）。它是第 8 篇耗时模型的推算，不是真实 kernel 的实测，请看趋势和比例。

26.2 运行

```
1 .venv/bin/python lab12b_paged_attention/paged_kv_cache_sim.py
```

实验	你会看到
1. 分页注意力	三个请求交替写入，物理块号杂乱（如 [49, 26, 32, 50]），逐块 Online Softmax 结果与连续存储误差 $\sim 1e-16$
2. 显存利用率	同样 64K token 的显存：预留 <code>max_len</code> 只能服务 32 个请求（浪费 78%），分页可服务 144 个（浪费 1.7%）
3. 写时复制	并行采样 <code>n=4</code> ：共享提示词块只存一份，只有“共享且未写满”的最后一块被复制，而且只复制 <code>n-1</code> 次（最后一个子序列可原地写入），省 50%
4. 前缀缓存	1000 token 系统提示词：84% 的 token 免去 Prefill；改掉第 1 个 token 后缓存全部失效
5. Chunked Prefill	6000 token 长提示词插队：不切块时所有 Decode 用户卡顿 390ms；切成 256 一块后最长卡顿 22ms（代价是 TTFT 从 390ms 涨到 506ms）

26.3 代码结构

- BlockAllocator: 空闲链表 + 引用计数（ $\approx$ 操作系统的物理页框管理）
- PagedKVCache: 物理 KV 池 [num_blocks, 16, d] + 每个序列一张块表；append 里实现按需分配与写时复制；attention 按块表逐块读取并用 Online Softmax 累加

这就是 vLLM 的 `BlockManager` 与 `PagedAttention` kernel 的核心逻辑，去掉了 GPU 与多层细节。

26.4 动手练习

1. 给 `BlockAllocator` 加上 LRU 淘汰：请求结束后块不立即释放而是进入“可回收缓存”，把实验 4 的前缀缓存和实验 2 的分配器真正接在一起。
2. 实现抢占：显存不足时选择最晚到达的请求，分别实现“换出到 CPU”与“丢弃后重算”，用第 8 篇的耗时模型比较两者代价（提示：PCIe 带宽约 25 GB/s，Prefill 的算力受限耗时见实验 5）。
3. 把块大小改成 1、16、128，观察实验 2 的浪费率与“块表长度”之间的权衡（块太小，块表和 kernel 里的间接寻址开销变大）。
4. （阅读）对照 vLLM 源码中 KV Cache 管理与调度器相关的模块（在官方文档 Design 部分可以找到架构说明），找到与本文件中每个函数对应的地方。

26.5 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码，重点实现细节如下：

26.5.1 源码实现：`paged_kv_cache_sim.py`

Listing 26.1: `paged_kv_cache_sim.py`

```
1  """
2  Lab 12b: PagedAttention、写时复制、前缀缓存与 Chunked Prefill 仿真
3
4  对应教程：zero_to_hero_tutorial/12_推理服务系统：PagedAttention、前缀缓存与调度.md
5
6  实验清单：
7      1. 块分配器 + 块表 + 分页注意力：KV 存在不连续的物理块里，逐块用 Online Softmax 计算，结果
          与连续存储完全一致
8      2. 显存利用率：按 max_len 预留连续显存 vs 分页，在同样的显存预算下能同时服务多少请求
9      3. 并行采样的写时复制 (Copy-on-Write)：共享提示词块，只在写入共享块时才复制
10     4. 前缀缓存（哈希式，类似 vLLM Automatic Prefix Caching）：共享系统提示词时省掉多少
        Prefill
11     5. Chunked Prefill：一个长 Prefill 插入时，正在 Decode 的请求会卡多久；切块后如何改善
12
13  运行（需要 NumPy）：
14      .venv/bin/python lab12b_paged_attention/paged_kv_cache_sim.py
15  """
16
17  import math
```

```

18 import random
19
20 import numpy as np
21
22 BLOCK = 16    # 每个物理块存 16 个 token 的 K/V (vLLM 默认值)
23
24
25 def title(s):
26     print("\n" + "=" * 78)
27     print(f"[实验] {s}")
28     print("=" * 78)
29
30
31 # =====
32 # 核心数据结构
33 # =====
34 class BlockAllocator:
35     """物理块池：空闲链表 + 引用计数（对应操作系统的物理页框管理）"""
36
37     def __init__(self, num_blocks):
38         self.free = list(range(num_blocks))
39         self.ref = [0] * num_blocks
40         self.num_blocks = num_blocks
41
42     def alloc(self):
43         if not self.free:
44             raise MemoryError("没有空闲块 → 调度器需要抢占（换出或重算）某个请求")
45         b = self.free.pop()
46         self.ref[b] = 1
47         return b
48
49     def incref(self, b):
50         self.ref[b] += 1
51
52     def decref(self, b):
53         self.ref[b] -= 1
54         if self.ref[b] == 0:
55             self.free.append(b)
56
57     @property
58     def used(self):
59         return self.num_blocks - len(self.free)
60
61
62 class PagedKVCache:
63     """
64     物理存储:  $K_{pool}/V_{pool}$  形状  $[num\_blocks, BLOCK, d]$ 
65     每个序列一张块表:  $block\_table[seq\_id] = [物理块号, \dots]$ , 第  $i$  个 token 在 (

```

```

        block_table[i // BLOCK], i % BLOCK)
    """
66
67
68     def __init__(self, num_blocks, d):
69         self.alloc = BlockAllocator(num_blocks)
70         self.K = np.zeros((num_blocks, BLOCK, d))
71         self.V = np.zeros((num_blocks, BLOCK, d))
72         self.tables = {}
73         self.lens = {}
74         self.cow_copies = 0
75
76     def new_seq(self, sid):
77         self.tables[sid], self.lens[sid] = [], 0
78
79     def fork(self, parent, child):
80         """并行采样：子序列共享父序列的全部块（只加引用计数，不拷贝数据）"""
81         self.tables[child] = list(self.tables[parent])
82         self.lens[child] = self.lens[parent]
83         for b in self.tables[child]:
84             self.alloc.incref(b)
85
86     def append(self, sid, k, v):
87         table, n = self.tables[sid], self.lens[sid]
88         if n % BLOCK == 0:                                     # 上一块写满了（或还没有块）→ 分配新块
89             table.append(self.alloc.alloc())
90         else:
91             last = table[-1]
92             if self.alloc.ref[last] > 1:                       # 最后一块被别人共享 → 写时复制
93                 new = self.alloc.alloc()
94                 self.K[new], self.V[new] = self.K[last], self.V[last]
95                 self.alloc.decref(last)
96                 table[-1] = new
97                 self.cow_copies += 1
98             b = table[-1]
99             self.K[b, n % BLOCK], self.V[b, n % BLOCK] = k, v
100             self.lens[sid] = n + 1
101
102     def free_seq(self, sid):
103         for b in self.tables.pop(sid):
104             self.alloc.decref(b)
105         self.lens.pop(sid)
106
107     def attention(self, sid, q):
108         """分页注意力：按块表逐块读取 K/V，用 Online Softmax（第 9 篇）累加 —— 从不拼出连续的 K/V"""
109         m, l, acc = -np.inf, 0.0, np.zeros_like(q)
110         n = self.lens[sid]
111         for i, b in enumerate(self.tables[sid]):

```

```

112         cnt = min(BLOCK, n - i * BLOCK)
113         s = self.K[b, :cnt] @ q / math.sqrt(q.shape[0])
114         m_new = max(m, s.max())
115         p = np.exp(s - m_new)
116         corr = math.exp(m - m_new) if m > -np.inf else 0.0
117         l = l * corr + p.sum()
118         acc = acc * corr + p @ self.V[b, :cnt]
119         m = m_new
120     return acc / l
121
122
123 # =====
124 # 实验 1: 分页注意力的正确性
125 # =====
126 def exp1_correctness():
127     title("实验 1: KV 分散在不连续的物理块里, 注意力结果与连续存储完全一致")
128     rng = np.random.default_rng(0)
129     d = 64
130     cache = PagedKVCache(num_blocks=64, d=d)
131     random.seed(0)
132     random.shuffle(cache.alloc.free) # 打乱空闲链表, 让分到的物理块号杂乱无章
133     ks, vs = {}, {}
134     for sid, n in [("A", 50), ("B", 37), ("C", 70)]:
135         cache.new_seq(sid)
136         ks[sid], vs[sid] = [], []
137     for t in range(70): # 三个请求交替写入 (模拟连续批处理)
138         for sid, n in [("A", 50), ("B", 37), ("C", 70)]:
139             if t < n:
140                 k, v = rng.standard_normal(d), rng.standard_normal(d)
141                 cache.append(sid, k, v)
142                 ks[sid].append(k); vs[sid].append(v)
143     for sid in ["A", "B", "C"]:
144         q = rng.standard_normal(d)
145         K, V = np.array(ks[sid]), np.array(vs[sid])
146         s = K @ q / math.sqrt(d)
147         ref = np.exp(s - s.max()) @ V / np.exp(s - s.max()).sum()
148         out = cache.attention(sid, q)
149         print(f"    请求 {sid}: {cache.lens[sid]:2d} 个 token, 块表 = {cache.tables[
150             sid]}, 与连续计算的最大误差 {np.abs(out - ref).max():.1e}")
151         assert np.abs(out - ref).max() < 1e-12
152     print(f"    共使用 {cache.alloc.used} 个物理块; 每个请求只有最后一块可能没写满。")
153
154 # =====
155 # 实验 2: 显存利用率
156 # =====
157 def exp2_utilization():
158     title("实验 2: 同样的显存, 预留连续空间 vs 分页, 能同时服务多少请求? ")

```

```

159 rng = random.Random(42)
160 budget_tokens = 64 * 1024 # KV 显存总共能放 64K 个 token
161 max_len = 2048
162 # 真实负载: 提示词和输出长度都是长尾分布, 大多数请求远小于 max_len
163 reqs = [(int(rng.lognormvariate(5.5, 0.8)), int(rng.lognormvariate(5.0, 0.9)))
164         for _ in range(2000)]
165 reqs = [(min(p, 1500), min(o, max_len - min(p, 1500))) for p, o in reqs]
166 avg_final = sum(p + o for p, o in reqs) / len(reqs)
167
168 static_slots = budget_tokens // max_len
169 # 分页: 每个请求按"当前长度"占块, 这里取它们生命周期中点的平均长度估计稳态占用
170 avg_blocks_mid = sum(math.ceil((p + o) / 2) / BLOCK for p, o in reqs) / len(
171 reqs)
172 paged_slots = int(budget_tokens / BLOCK / avg_blocks_mid)
173 real_tokens_mid = sum(p + o / 2 for p, o in reqs) / len(reqs)
174 waste_paged = 1 - real_tokens_mid / (avg_blocks_mid * BLOCK)
175 waste_static = 1 - real_tokens_mid / max_len
176
177 print(f"    负载: 2000 个请求, 平均最终长度 {avg_final:.0f} token (max_len = {
178 max_len}) ")
179 print(f"    KV 显存预算: {budget_tokens:,} 个 token 的空间")
180 print(f"    {'方案':<26} | {'可同时服务':>10} | {'显存浪费率':>10}")
181 print(f"    {'按 max_len 预留连续空间':<22} | {static_slots:>10} | {waste_static
182 :>9.0%}")
183 print(f"    {'PagedAttention (块=16)':<24} | {paged_slots:>10} | {waste_paged
184 :>9.1%}")
185 # 分页的两个核心收益, 逐条断言:
186 assert paged_slots > static_slots, "分页必须能服务更多并发请求, 否则这个方案没有意义"
187 assert waste_paged < waste_static, "分页的浪费率必须低于按 max_len 预留"
188 # 分页的浪费上界是每序列最后一块没写满 (< 16 token), 不是 0
189 assert waste_paged < 16 / avg_final, \
190     f"分页浪费率 {waste_paged:.2%} 超过了「每序列最后一块没写满」的上界 {16 /
191 avg_final:.2%}, " \
192     f"说明块分配器有别的浪费"
193 print(f"    -> 并发数提升 {paged_slots / static_slots:.1f} 倍。第 8 篇说过 Decode
194 时 batch 越大吞吐越高, ")
195 print("    所以省下的显存会直接变成吞吐。vLLM 论文实测旧系统只有 20%~38% 的 KV 显存真
196 正存了 token。")
197
198 # =====
199 # 实验 3: 写时复制
200 # =====
201 def exp3_copy_on_write():
202     title("实验 3: 并行采样 n=4 —— 引用计数共享提示词块 + 写时复制")
203     rng = np.random.default_rng(1)
204     d, prompt_len, gen_len, n = 8, 100, 30, 4
205     cache = PagedKVCache(num_blocks=256, d=d)

```

```

199 cache.new_seq("p")
200 for _ in range(prompt_len):
201     cache.append("p", rng.standard_normal(d), rng.standard_normal(d))
202 print(f" 提示词 {prompt_len} token 写入后占 {cache.alloc.used} 块 (最后一块只写了
    {prompt_len % BLOCK}/{BLOCK}) ")
203 for i in range(n):
204     cache.fork("p", f"s{i}")
205 cache.free_seq("p")
206 print(f"  fork 出 {n} 个子序列后仍只占 {cache.alloc.used} 块, 每块引用计数 = {
    cache.alloc.ref[cache.tables['s0'][0]]}")
207 for _ in range(gen_len):
208     for i in range(n):
209         cache.append(f"s{i}", rng.standard_normal(d), rng.standard_normal(d))
210 no_share = n * math.ceil((prompt_len + gen_len) / BLOCK)
211 print(f"  各自生成 {gen_len} token 后: 共享方案用了 {cache.alloc.used} 块, 发生写时
    复制 {cache.cow_copies} 次")
212 assert cache.cow_copies == n - 1 and cache.alloc.used < no_share, '应恰好发生 n
    -1 次写时复制'
213 print(f"  不共享时需要 {no_share} 块 → 节省 {1 - cache.alloc.used / no_share
    :.0%}")
214 print(f"  --> 只有那个'被共享且没写满'的最后一块会被复制, 而且只复制 {n - 1} 次而不是 {
    n} 次: ")
215 print(f"  前 {n - 1} 个子序列各复制一份后, 原块的引用计数降到 1, 最后一个子序列可以
    直接原地写入。")
216 print("  前面写满的提示词块始终只存一份。")
217
218
219 # =====
220 # 实验 4: 前缀缓存
221 # =====
222 def exp4_prefix_cache():
223     title("实验 4: 前缀缓存 —— 所有请求共享同一段系统提示词")
224     rng = random.Random(7)
225     system_prompt = [rng.randrange(50000) for _ in range(1000)] # 1000 token 的
    系统提示词
226     cached = {} # 哈希 (前缀的所有
    token) → 物理块号
227
228     def prefill_with_cache(tokens):
229         """逐个"写满的块"算哈希: 键 = 从开头到这一块末尾的全部 token (不能只看本块, 否则位置不
    同的相同内容会误命中) """
230         hit = 0
231         for i in range(len(tokens) // BLOCK):
232             key = hash(tuple(tokens[: (i + 1) * BLOCK]))
233             if key in cached:
234                 hit += BLOCK
235             else:
236                 break # 前缀一旦不同, 后面全部不能复用

```

```

237     for i in range(hit // BLOCK, len(tokens) // BLOCK):
238         cached[hash(tuple(tokens[: (i + 1) * BLOCK]))] = len(cached)
239     return hit
240
241 total, saved = 0, 0
242 for r in range(20):
243     user = [rng.randrange(50000) for _ in range(rng.randint(20, 200))]
244     tokens = system_prompt + user
245     hit = prefill_with_cache(tokens)
246     total += len(tokens)
247     saved += hit
248     if r < 3:
249         print(f"    请求 {r}: 共 {len(tokens)} token, 命中缓存 {hit} token, 只需
Prefill {len(tokens) - hit} token")
250     print(f"    ... 20 个请求合计 {total} token, 其中 {saved} token ({saved / total
:.0%}) 直接复用缓存, 免去 Prefill")
251
252 # 只改开头第 1 个 token (比如提示词里写入了当前时间) 。
253 # 必须【确定性地】换成一个不同的值: 写成 rng.randrange(50000) 的话有 1/50000 的概率
254 # 抽到和原来相同, 于是缓存照样命中, 而打印出来的结论就变成了假的。
255 changed_first = (system_prompt[0] + 1) % 50000
256 assert changed_first != system_prompt[0]
257 hit = prefill_with_cache([changed_first] + system_prompt[1:] + [1, 2, 3])
258 assert hit == 0, f"第 1 个 token 变了之后不该有任何前缀命中, 实际命中 {hit} token"
259 print(f"    [注意] 把系统提示词的第 1 个 token 改掉后: 命中 {hit} token —— 整个前缀缓
存全部失效")
260
261
262 # =====
263 # 实验 5: Chunked Prefill
264 # =====
265 def exp5_chunked_prefill():
266     title("实验 5: Chunked Prefill —— 长 Prefill 插队时, 正在 Decode 的用户会卡多久? ")
267     # 用第 8 篇的耗时模型, 参数取自本机实测 (RTX 5060 Ti)
268     # [注意] 这两个数都是 measure_gpu_roofline.py **已经实测达到**的值, 不要再乘 MFU:
269     # 带宽 380 GB/s (实测拷贝带宽)、算力 48 TFLOP/s (实测 BF16 Tensor 稠密算力) 。
270     # 两侧必须同口径——只给算力降额、访存却用 achieved 值, 比值会偏一倍。
271     # 模型: Qwen2.5-1.5B (BF16 权重约 3.1 GB)
272     W_bytes, n_params = 3.1e9, 1.54e9
273     bw, peak, overhead = 380e9, 48e12, 3e-3 # 实测带宽 / 实测算力 + 3ms
        固定开销
274
275     def step_time(n_tokens):
276         return max(W_bytes / bw, 2 * n_params * n_tokens / peak) + overhead
277
278     decode_batch = 32
279     long_prompt = 6000

```

```

280 print(f"    场景: {decode_batch} 个用户正在 Decode, 此时来了一个 {long_prompt}
token 的长提示词")
281 print(f"    单步耗时模型: max(读权重 {W_bytes / bw * 1e3:.1f} ms, 计算 2N×tokens/算
力) + 固定开销 {overhead * 1e3:.0f} ms")
282 print(f"    纯 Decode 一步 ({decode_batch} token) : {step_time(decode_batch) * 1
e3:.1f} ms")
283 print(f"\n    {'方案':<22} | {'Prefill 分几步':>12} | {'Decode 用户最长卡顿':>18}
| {'新请求 TTFT':>12}")
284 for chunk in [None, 2048, 512, 256]:
285     if chunk is None:
286         steps = [long_prompt]
287         name = "不切块 (一次做完) "
288     else:
289         steps = [min(chunk, long_prompt - i) for i in range(0, long_prompt,
chunk)]
290         name = f"Chunked Prefill {chunk}"
291         times = [step_time(s + decode_batch) for s in steps]
292         print(f"    {name:<20} | {len(steps):>12} | {max(times) * 1e3:>15.1f} ms |
{sum(times) * 1e3:>9.1f} ms")
293         print(f"    --> 不切块时, 32 个用户在这一步里全部卡住几百毫秒 (TPOT 尖刺)。切成小块后, 每
步耗时有上界, ")
294         print(f"    代价是新请求的 TTFT 略增。块越小, Decode 越平滑, 但 Prefill 越慢 (每步都
要重新读一遍权重)。")
295         ridge_tokens = W_bytes / bw * peak / (2 * n_params)
296         # 交叉核对: 这里的拐点必须和 lab08 实测的 AI* 一致 (都是 实测算力 / 实测带宽)
297         ai_star = peak / bw
298         assert 100 < ai_star < 160, \
299             f"AI* = {ai_star:.0f} 与 lab08 在 5060 Ti 上实测的约 130 不符, 检查两侧口径是否
同源"
300         print(f"    另外: AI* = 算力/带宽 = {ai_star:.0f} FLOP/Byte, 与 lab08 实测的约
130 一致。")
301         print(f"    所以这个模型每步 token 数低于约 {ridge_tokens:.0f} 时仍是访存受限 (第 8
篇的拐点), ")
302         print(f"    此时往 Decode 批次里塞进一小段 Prefill 几乎不增加耗时 —— 这就是'两种瓶
颈互补'。")
303         print(f"    真实引擎 (vLLM 的 max_num_batched_tokens) 通常取几百到几千, 在平滑度和
Prefill 效率间折中。")
304
305
306 if __name__ == "__main__":
307     exp1_correctness()
308     exp2_utilization()
309     exp3_copy_on_write()
310     exp4_prefix_cache()
311     exp5_chunked_prefill()

```

第 27 章 多卡张量并行（Tensor Parallelism）切分与 All-Reduce 仿真

27.1 Lab 13：多卡并行推理仿真

核心导读与背景

对应理论：第 13 篇：多卡并行推理——张量并行、流水线并行与专家并行前置：第 3 篇（矩阵乘形状）、第 8 篇（带宽与耗时模型）。依赖：NumPy。不需要真的有多张显卡——用列表里的多个数组模拟多张卡，重点是看清”切法”的数学。

核心导读与背景

[计时] 以下计时类数字是本机某一次运行的量级示例（显卡空闲、已预热）。它是第 8 篇耗时模型的推算，不是真实 kernel 的实测，请看趋势和比例。

27.2 运行

```
1 .venv/bin/python lab13_parallelism/tensor_parallel_sim.py
```

实验	结果
1. TP SwiGLU（先列后行）	TP=2/4/8 与单卡误差 $\sim 1e-15$ ，每卡权重 $1/n$
2. 反例（先行切）	在部分和上做 SiLU，相对误差 52% ；要算对必须多一次通信
3. TP 注意力（按头切）	与单卡一致，每卡只存 $1/n$ 的 KV Cache
4. Ring All-Reduce	每卡发送 $2(n-1)/n$ 倍数据：2 卡 1.0、4 卡 1.5、8 卡 1.75
5. 70B Decode 延迟模型	NVLink TP=8 约 6.9x；PCIe TP=8 约 4.9x（通信占近 40%）；PP 不降延迟

27.3 动手练习

- 在实验 3 中实现 GQA（8 个 Q 头、2 个 KV 头）+ TP=4：KV 头少于卡数时，每个 KV 头要复制到哪几张卡上？KV Cache 总占用变成多少？
- 给实验 5 加上 batch 维度：batch=64 时每次 All-Reduce 的数据量变成多少？通信是否从”延迟主导”变成”带宽主导”？

3. 模拟专家并行：8 个专家分到 4 张卡，随机路由 1024 个 token (top-2)，统计每张卡收到的 token 数，计算最忙的卡比平均多多少（负载不均衡）。再让路由分布偏向少数专家，看看会怎样。
4. 用实验 4 的环形结构实现 Ring Attention：每张卡持有一段 KV，环形传递 KV 块，用第 9 篇的合并公式累加，验证与完整注意力一致。

27.4 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码，重点实现细节如下：

27.4.1 源码实现：tensor_parallel_sim.py

Listing 27.1: tensor_parallel_sim.py

```

1  """
2  Lab 13: 用 NumPy 模拟多卡并行推理 —— 张量并行、Ring All-Reduce 与多卡 Decode 延迟模型
3
4  对应教程：zero_to_hero_tutorial/13_多卡并行推理：张量并行、流水线并行与专家并行.md
5
6  实验清单：
7      1. 张量并行 SwiGLU: gate/up 按列切、down 按行切，一次 All-Reduce 后与单卡结果一致
8      2. 反例：如果 gate/up 按行切（切输入维度），在部分和上直接做 SiLU 会算错
9      3. 张量并行注意力：按头切分，每张"卡"只存自己那几个头的 KV，o_proj 按行切，一次 All-Reduce
10     4. Ring All-Reduce: Reduce-Scatter + All-Gather，每张卡发送  $2(n-1)/n$  倍数据
11     5. 多卡 Decode 延迟模型：Llama-3-70B, TP=1/2/4/8, NVLink vs PCIe；以及流水线并行为什么
        不降延迟
12
13  运行：
14      .venv/bin/python lab13_parallelism/tensor_parallel_sim.py
15  """
16
17  import math
18
19  import numpy as np
20
21  rng = np.random.default_rng(0)
22
23
24  def title(s):
25      print("\n" + "=" * 78)
26      print(f"[实验] {s}")
27      print("=" * 78)
28
29
30  def silu(x):

```

```

31     return x / (1 + np.exp(-x))
32
33
34 def all_reduce(parts):
35     """模拟 All-Reduce: 所有卡的张量求和, 每张卡都拿到结果"""
36     total = sum(parts)
37     return [total.copy() for _ in parts]
38
39
40 # =====
41 # 实验 1 & 2
42 # =====
43 def exp1_2_mlp():
44     T, d, h = 4, 64, 176
45     X = rng.standard_normal((T, d))
46     Wg, Wu = rng.standard_normal((d, h)) * 0.1, rng.standard_normal((d, h)) * 0.1
47     Wd = rng.standard_normal((h, d)) * 0.1
48     Y_ref = (silu(X @ Wg) * (X @ Wu)) @ Wd
49
50     title("实验 1: 张量并行 SwiGLU —— 先按列切, 再按行切, 只需一次 All-Reduce")
51     for n in [2, 4, 8]:
52         cols = np.array_split(np.arange(h), n)
53         partial = []
54         for c in cols:                                     # 每张卡只持有 1/n 的
权重
55             local = silu(X @ Wg[:, c]) * (X @ Wu[:, c])    # 本地完成非线性, 无需
通信
56             partial.append(local @ Wd[c, :])                # 得到一个部分和 [T,
d]
57         Y = all_reduce(partial)[0]
58         print(f"    TP={n}: 每卡权重 {Wg[:, cols[0]].size * 3} 个参数 (单卡 {Wg.size *
3}), 与单卡结果最大误差 {np.abs(Y - Y_ref).max():.1e}")
59         assert np.abs(Y - Y_ref).max() < 1e-12
60
61     title("实验 2: 反例 —— gate/up 按行切 (切输入维度), 在部分和上做 SiLU")
62     n = 2
63     rows = np.array_split(np.arange(d), n)
64     wrong = sum((silu(X[:, r] @ Wg[r]) * (X[:, r] @ Wu[r])) @ Wd for r in rows)
65     print(f"    错误做法  $\sum f(X_i A_i)$  与正确结果的最大误差: {np.abs(wrong - Y_ref).max
():.3f} (相对 {np.abs(wrong - Y_ref).max() / np.abs(Y_ref).max():.0%}) ")
66     assert np.abs(wrong - Y_ref).max() > 0.1, '错误切法应产生明显误差'
67     g = sum(X[:, r] @ Wg[r] for r in rows)                # 必须先 All-Reduce 出
完整的 XA
68     u = sum(X[:, r] @ Wu[r] for r in rows)
69     right = (silu(g) * u) @ Wd
70     print(f"    正确做法 (先 All-Reduce 再做 SiLU) 误差: {np.abs(right - Y_ref).max()
:.1e} —— 但多了一次通信")
71     print("    --> 非线性对加法不分配:  $f(a+b) \neq f(a)+f(b)$ 。'先列后行'让非线性在本地完成, 这

```

```

是 Megatron 切法的精髓。")
72
73
74 # =====
75 # 实验 3
76 # =====
77 def attention(Q, K, V):
78     s = Q @ K.T / math.sqrt(Q.shape[1])
79     s = s + np.triu(np.full(s.shape, -np.inf), 1)
80     p = np.exp(s - s.max(1, keepdims=True))
81     return (p / p.sum(1, keepdims=True)) @ V
82
83
84 def exp3_attention():
85     title("实验 3: 张量并行注意力 —— 按头切分")
86     T, d, n_h, hd = 6, 64, 8, 8
87     X = rng.standard_normal((T, d))
88     Wq, Wk, Wv = (rng.standard_normal((d, n_h * hd)) * 0.1 for _ in range(3))
89     Wo = rng.standard_normal((n_h * hd, d)) * 0.1
90
91     def heads_out(hs):
92         outs = []
93         for h in hs:
94             c = slice(h * hd, (h + 1) * hd)
95             outs.append(attention(X @ Wq[:, c], X @ Wk[:, c], X @ Wv[:, c]))
96         return np.concatenate(outs, axis=1)
97
98     Y_ref = heads_out(range(n_h)) @ Wo
99     for n in [2, 4]:
100         groups = np.array_split(np.arange(n_h), n)
101         partial = []
102         for g in groups:
103             cols = np.concatenate([np.arange(h * hd, (h + 1) * hd) for h in g])
104             partial.append(heads_out(g) @ Wo[cols, :]) # o_proj 按行切
105         Y = all_reduce(partial)[0]
106         print(f"    TP={n}: 每卡 {len(groups[0])} 个头、只存 1/{n} 的 KV Cache, 与单卡
107             最大误差 {np.abs(Y - Y_ref).max():.1e}")
108         assert np.abs(Y - Y_ref).max() < 1e-12
109         print("    --> 一层 Transformer = 注意力 1 次 All-Reduce + FFN 1 次 All-Reduce, 共
110             2 次。")
111
112 # =====
113 # 实验 4
114 # =====
115 def ring_all_reduce(tensors):
116     """
117     Ring All-Reduce: n 张卡排成环, 数据切成 n 块。

```

```

117 阶段 1 Reduce-Scatter (n-1 步) : 每步把一块发给右邻居并累加 → 结束时卡 i 拥有第 (i+1)%
    n 块的完整和
118 阶段 2 All-Gather (n-1 步) : 每步把完整的块传给右邻居 → 结束时人人拥有全部块
119 """
120 n = len(tensors)
121 chunks = [np.array_split(t.copy(), n) for t in tensors]
122 sent = [0] * n
123 for step in range(n - 1):                                     # Reduce-Scatter
124     msgs = [(i, (i - step) % n, chunks[i][(i - step) % n].copy()) for i in
    range(n)]
125     for i, c, data in msgs:
126         chunks[(i + 1) % n][c] += data
127         sent[i] += data.size
128     for step in range(n - 1):                                     # All-Gather
129         msgs = [(i, (i + 1 - step) % n, chunks[i][(i + 1 - step) % n].copy()) for
    i in range(n)]
130         for i, c, data in msgs:
131             chunks[(i + 1) % n][c] = data
132             sent[i] += data.size
133     return [np.concatenate(c) for c in chunks], sent
134
135
136 def exp4_ring():
137     title("实验 4: Ring All-Reduce 的正确性与通信量")
138     size = 1 << 16
139     for n in [2, 4, 8]:
140         ts = [rng.standard_normal(size) for _ in range(n)]
141         out, sent = ring_all_reduce(ts)
142         ref = sum(ts)
143         err = max(np.abs(o - ref).max() for o in out)
144         print(f"    {n} 张卡: 结果误差 {err:.1e} | 每卡发送 {sent[0] / size:.3f} × 数据
    量, 理论 2(n-1)/n = {2 * (n - 1) / n:.3f}")
145         assert err < 1e-12 and abs(sent[0] / size - 2 * (n - 1) / n) < 1e-9
146         print("    --> 卡数从 2 到 8, 每卡发送量只从 1.0 倍涨到 1.75 倍, 上限 2 倍 —— 带宽代价
    几乎与卡数无关。")
147
148
149 # =====
150 # 实验 5
151 # =====
152 def exp5_latency_model():
153     title("实验 5: 多卡 Decode 延迟模型 —— Llama-3-70B BF16, batch=1")
154     L, d, params = 80, 8192, 70.6e9
155     hbm_bw = 3.35e12                                             # H100 SXM 显存带宽
156     # 每次 All-Reduce 的固定延迟是粗略假设 (NVLink 约 5 μs、PCIe 约 20 μs), 真实值取决于
    拓扑与 NCCL 版本
157     links = {"NVLink (H100, ~450 GB/s)": (450e9, 5e-6), "PCIe 5.0 (~50 GB/s)": (50
    e9, 20e-6)}

```

```

158 msg = 1 * d * 2 # 每次 All-Reduce: 1
    个 token × d × 2 字节
159 print(f" 每 token: 读权重 {params * 2 / 1e9:.0f} GB; 每步 {2 * L} 次 All-
Reduce, 每次仅 {msg / 1024:.0f} KB (很小 → 由固定延迟主导)")
160 print(f"    {'互联':<24} | {'TP':>3} | {'读权重 ms':>9} | {'通信 ms':>8} | {'每
token ms':>11} | {'相对 TP=1':>9}")
161 speedups = {}
162 for name, (bw, lat) in links.items():
163     base = None
164     for n in [1, 2, 4, 8]:
165         t_mem = params * 2 / n / hbm_bw
166         t_comm = 0 if n == 1 else 2 * L * (lat + 2 * (n - 1) / n * msg / bw)
167         t = t_mem + t_comm
168         base = base or t
169         speedups[(name, n)] = base / t
170         print(f"    {name:<22} | {n:>3} | {t_mem * 1e3:9.2f} | {t_comm * 1e3
:8.2f} | {t * 1e3:11.2f} | {base / t:8.2f}x")
171 # 加速比必须是次线性的: 通信吃掉一部分收益, 而且卡越多占比越高
172 for name in links:
173     sp = [speedups[(name, n)] for n in (1, 2, 4, 8)]
174     assert sp[0] == 1.0, f"{name}: TP=1 的加速比应为 1, 实际 {sp[0]}"
175     assert all(sp[i] < sp[i + 1] for i in range(3)), f"{name}: 加速比应随 n 单调
递增, 实际 {sp}"
176     assert sp[3] < 8.0, f"{name}: TP=8 的加速比 {sp[3]:.2f} 不该达到理想的 8 倍
(通信不可能免费)"
177 # NVLink 的通信带宽远高于 PCIe, 所以它的扩展效率必须更好
178 nv = next(v for (name, n), v in speedups.items() if name.startswith("NVLink")
and n == 8)
179 pc = next(v for (name, n), v in speedups.items() if name.startswith("PCIe")
and n == 8)
180 assert nv > pc * 1.2, f"NVLink 上 TP=8 的加速比 {nv:.2f} 应显著优于 PCIe 的 {pc
:.2f}"
181 print("    (TP=1 需要 141 GB 显存, 单张 80GB 卡其实放不下, 这里只作为计算基准)")
182 print("    --> NVLink 上 TP=8 仍有约 7 倍加速; PCIe 上通信的固定延迟每 token 要累加
160 次, TP=8 时占总耗时近 40%, ")
183 print("    扩展效率明显下降。而且通信时间几乎不随 n 减少, 卡越多占比越高 —— 所以 TP
通常只在 NVLink 连接的机内使用。")
184
185 print("\n 流水线并行 PP=4 (每卡 20 层): ")
186 t_pp = params * 2 / hbm_bw + 3 * 20e-6 # 各段依次执行, 总读取
量不变, 外加 3 次段间传输
187 t_single = params * 2 / hbm_bw
188 # PP 的要点: 延迟几乎不变 (只多了几次段间传输), 换的是吞吐
189 assert t_pp > t_single and t_pp < t_single * 1.5, \
    f"PP 的延迟应接近单卡 ({t_single * 1e3:.2f} ms), 实际 {t_pp * 1e3:.2f} ms"
190 print(f"    单请求每 token ≈ {t_pp * 1e3:.2f} ms —— 与单卡的 {t_single * 1e3:.2f}
ms 基本相同, "
191 f"PP 不降低延迟; 但 4 段可以同时处理 4 批不同的请求, 吞吐 ×4。")
192

```

```
193
194
195 if __name__ == "__main__":
196     exp1_2_mlp()
197     exp3_attention()
198     exp4_ring()
199     exp5_latency_model()
```

第 28 章 生产级推理引擎（vLLM / SGLang / TRT-LLM）压测实践

28.1 毕业设计：工业级推理引擎实践（vLLM / llama.cpp）

核心数学定理推导

这是整条路线的**毕业设计**。前面的 Lab 03~13 里你已经亲手实现了推理引擎的每一个核心部件；现在去使用真正的引擎，并且**每一个观测到的数字，都要能用前面的公式解释**。
前置：第 8、10、11、12 篇；Lab 08（实测带宽）、Lab 07b（手写 Qwen）、Lab 12b（PagedAttention）。

28.2 原则：先预测，再测量，再解释

每做一个实验，先用 `lab08_memory_and_roofline/memory_calculator.py` 和第 8 篇的公式写下你的**预测**，再运行，最后解释差距。这比跑出一个漂亮的数字重要得多。

28.3 [测试] 任务 1：用 vLLM 部署，并测出吞吐-延迟曲线

安装：建议单独建一个虚拟环境（vLLM 会固定自己需要的 PyTorch 版本，可能和本仓库的 `.venv` 冲突）。RTX 50 系显卡需要支持 CUDA 12.8 的较新版本，按 [vLLM 官方安装文档](#) 操作。WSL2 下可以正常使用。

```
1 vllm serve Qwen/Qwen2.5-1.5B-Instruct --max-model-len 4096 --gpu-memory-utilization 0.85
```

先预测：

- Qwen2.5-1.5B: 28 层、2 个 KV 头、`head_dim 128` → 每 token KV = $2 \times 28 \times 2 \times 128 \times 2 = 28,672$ 字节 ≈ 28 KB;
- 权重 BF16 约 3.1 GB，按你实测的带宽（本机空闲热态约 380 GB/s，负载下会更低），单流 Decode 上限 ≈ 120 token/s;
- $16\text{GB} \times 0.85$ 扣掉权重和激活后，大约还剩多少 GB 给 KV？能放多少 token？

再测量：

1. 看 vLLM 启动日志里的 KV Cache 容量（不同版本措辞不同，通常会打印 KV Cache 的 token 数或 GPU block 数与最大并发）。和你的预测一致吗？ $\text{block 数} \times 16 = \text{token 数}$ ？

2. 压测：

```
1 python3 capstone_engines_practice/bench_openai_server.py --concurrency 1 4 16
   64 --max-tokens 256
```

1. 画出（或列出）”并发 → 总吞吐、TPOT P50”的关系。

避坑指南与核心警示

[注意] 压测本身有三个坑，不知道就会得出错误的结论（脚本已在文件头 docstring 里写明，这里再强调一次）：- **前缀缓存会污染跨档位的对比**：脚本各档位复用同一批问题，档位之间不清缓存，而 vLLM 默认开启前缀缓存 → 靠后的档位 TTFT 偏低。要得到干净的曲线，请在档位之间重启服务。- **输出长度必须一致**：默认加了 `ignore_eos: true` (vLLM/SGLang 扩展)，让每个请求都恰好输出 `--max-tokens` 个 token。否则”总吞吐 tok/s”测的是模型有多话唠，不是引擎有多快。标准 OpenAI 服务会忽略这个字段，那就只能靠 `max_tokens` 近似。- **必须预热**：脚本每档先发 `--warmup`（默认 2）个不计入统计的请求。GPU 从空闲频率爬到稳态要时间——lab08 里同一份代码在不同时钟状态下读到过 217~380 GB/s，差近一倍。第一档不预热会显著偏慢。

核心导读与背景

[重点] 另外，脚本会自检 `usage.completion_tokens` 和实际收到的内容分片数是否一致：不一致时说明服务端一个 SSE chunk 带了多个 token (llama.cpp 常见)，此时 TPOT 按分片间隔计算并打印告警——否则 TPOT 会被系统性高估。样本不足（例如 `--max-tokens 1`）时它会打印”样本不足”而不是崩溃。

要能解释：

- 并发 1 的单流速度为什么低于 120 token/s，但比 Lab 07b 的手写实现（约 20ms/token）快得多？（提示：CUDA Graph、融合 kernel，第 12 篇 §7）
- 并发从 1 到 16，总吞吐为什么几乎线性增长、TPOT 却几乎不变？到多少并发开始 TPOT 明显变差？（第 8 篇 §4：拐点约 130）

28.4 [测试] 任务 2：前缀缓存

```
1 python3 capstone_engines_practice/bench_openai_server.py --shared-prefix-tokens
   3000 --concurrency 4 --max-tokens 64
```

分别在开启和关闭前缀缓存时运行(新版 vLLM 默认开启;关闭参数通常是 `--no-enable-prefix-cache` 以你的版本 `vllm serve --help` 为准)。

预测: 3000 token 的 Prefill 按第 8 篇 $2N \cdot S / (\pi \cdot \text{MFU})$ 估算要多少毫秒? 命中缓存后 TTFT 应该降到多少?

28.5 [测试] 任务 3: 长 Prefill 对 Decode 的干扰 (Chunked Prefill)

同时跑两个压测: 一个并发 16、短问题、长输出(模拟正在聊天的用户); 另一个偶尔发送 3000+ token 的长提示词。观察第一个的 **TPOT P99** 是否出现尖刺。调整 `--max-num-batched-tokens` (每步 token 预算) 重复实验, 用 Lab 12b 实验 5 的模型解释你看到的现象。

28.6 [测试] 任务 4: llama.cpp + GGUF 量化

```
1 git clone https://github.com/ggml-org/llama.cpp && cd llama.cpp
2 cmake -B build -DGGML_CUDA=0N && cmake --build build --config Release -j
3 # 从 Hugging Face 上 Qwen 官方的 GGUF 仓库下载 Qwen2.5-7B-Instruct 的 Q4_K_M 版本
4 ./build/bin/llama-bench -m <你的 gguf 文件> -ngl 99
```

llama-bench 会报告 **pp512** (Prefill 512 token 的吞吐) 和 **tg128** (Decode 的吞吐)。

预测:

- Q4_K_M 平均约 4.8 bit/参数, 7.6B 参数约 4.6 GB $\rightarrow$ **tg** 上限 $\approx 380 / 4.6 \approx 80$ token/s;
- **pp** 是算力受限: 约 $\frac{48 \times 10^{12} \times \text{MFU}}{2 \times 7.6 \times 10^9}$ token/s。
- BF16 的 7B (15.2 GB) 在 16GB 卡上几乎放不下——这就是你这张卡上量化的第一价值 (第 11 篇)。

要能解释: 为什么量化让 **tg** 提升了约 3 倍, 而 **pp** 提升远没有那么多, 甚至可能变慢? (提示: W4A16 在计算前要反量化, Prefill 是算力受限的)

28.7 [测试] 任务 5: 投机解码

在 vLLM (配置草稿模型或 n-gram 投机) 或 llama.cpp (`llama-speculative`, 目标 Qwen2.5-7B + 草稿 Qwen2.5-0.5B) 上开启投机解码, 测单流速度与接受率。用第 10 篇的公式 $\frac{1-\alpha^{K+1}}{(1-\alpha)(Kc+1)}$ 代入实测接受率和两模型的耗时比, 预测加速比并与实测对比。再把并发调到 32, 投机解码还有收益吗? 为什么?

28.8 [阅读] 任务 6: 源码阅读路线

读源码的顺序很重要——从小到大:

1. **nano-vllm** (约 1200 行 Python, 实现了 vLLM 的核心: 调度、块管理、前缀缓存、CUDA Graph、张量并行)。对照表:

nano-vllm 里的部分	本仓库对应
调度器 (Prefill / Decode、抢占)	Lab 12a、第 12 篇 §2、§3.4
块管理器(引用计数、哈希前缀缓存)	Lab 12b 的 BlockAllocator 、实验 4
模型实现 (Qwen)	Lab 07b 的 QwenFromScratch
注意力 (调用 FlashAttention 的变长 / 分页接口)	Lab 09、第 9 篇
CUDA Graph 捕获	Lab 07b 性能分析、练习 5
张量并行的线性层	Lab 13、第 13 篇

1. **gpt-fast** (不到 1000 行, 用 `torch.compile` + 量化 + 投机解码把单流速度逼近带宽极限)。
2. **vLLM 本体**: 先读官方文档的架构设计部分, 再按”入口 → 引擎核心 → 调度器 → KV Cache 管理 → 模型执行器 → 注意力后端”的顺序读。
3. **llama.cpp**: 从 `ggml` 的计算图与量化格式 (Q4_K 等) 入手, 看它如何在 CPU / GPU 上做融合的反量化矩阵乘。

28.9 [OK] 毕业标准

完成下面这件事, 你就真正走完了这条路线:

核心数学定理推导

选定一个模型和你的 5060 Ti, 写一份一页纸的报告: **预测**它在 vLLM 上的单流速度、最大并发、TTFT (带/不带前缀缓存)、W4 量化后的速度, 给出每个预测的公式; 然后**实测**, 逐项解释预测与实测之间的差距来自哪里。

如果每一个差距你都能说清楚是哪一篇、哪一个公式、哪一个工程因素造成的, 你就不再是”库的使用者”了。

28.10 实验核心源码精选与解析

本实验配套有完整可运行的工业级验证代码, 重点实现细节如下:

28.10.1 源码实现：bench_openai_server.py

Listing 28.1: bench_openai_server.py

```
1  """
2  毕业设计：对任何 OpenAI 兼容的推理服务（vLLM / SGLang / llama.cpp 的 llama-server / TGI
   ）做延迟与吞吐压测（零外部依赖）
3
4  对应教程：zero_to_hero_tutorial/12_推理服务系统：PagedAttention、前缀缓存与调度.md（$1 指
   标体系）
5
6  测量内容（第 12 篇 $1）：
7      - TTFT：从发出请求到收到第一个 token
8      - TPOT：相邻两个输出 token 的平均间隔（每个请求算一个值）
9      - 端到端延迟、系统总输出吞吐（token/s）
10     - 在不同并发数下重复，观察"吞吐-延迟权衡"（第 8 篇 $4.4）
11
12 [注意] 测量前先读这三条，否则数字会骗你（每一条都会让结果系统性偏乐观）：
13     1. **前缀缓存**：所有并发档位复用同一批问题，档位之间不清缓存；vLLM 默认开启前缀缓存，
14       所以靠后的档位 TTFT 会偏低。要干净的数字，请在各档位之间重启服务，或换用互不相同的问题。
15     2. **输出长度**：默认带 `ignore_eos: true`，让每个请求都恰好产出 `--max-tokens` 个
16       token，
17       否则"总吞吐 tok/s"实际测的是"模型有多话唠"而不是引擎有多快。
18       注意这个字段是 vLLM/SGLang 的扩展，标准 OpenAI 服务可能忽略它——那就只能靠 max_tokens
19       一致来近似。
20     3. **预热**：脚本会先发一轮不计入统计的预热请求。GPU 从空闲频率爬到稳态需要时间，
21       不预热的第一档会明显偏慢（见 lab08 README 里 217~380 GB/s 的实测差异）。
22
23 用法示例：
24     # 1) 先启动服务，例如
25     #     vllm serve Qwen/Qwen2.5-1.5B-Instruct --max-model-len 4096
26     #     llama-server -m qwen2.5-7b-instruct-q4_k_m.gguf -ngl 99 -c 8192 --port 8000
27     # 2) 再压测
28     python3 capstone_engines_practice/bench_openai_server.py --base-url http://
29         localhost:8000 --concurrency 1 4 16 32
30     # 测前缀缓存：所有请求共享一段很长的系统提示词
31     python3 capstone_engines_practice/bench_openai_server.py --shared-prefix-tokens
32         2000 --concurrency 8
33
34  """
35
36 import argparse
37 import json
38 import statistics
39 import threading
40 import time
41 import urllib.error
42 import urllib.request
```

```

39 QUESTIONS = [
40     "用三句话解释什么是 KV Cache。",
41     "为什么大模型的 Decode 阶段是访存受限的?",
42     "简要介绍 PagedAttention 解决了什么问题。",
43     "投机解码为什么不会改变输出分布?",
44     "比较 GPTQ 和 AWQ 两种量化方法。",
45     "什么是 RoPE 旋转位置编码?",
46     "解释连续批处理与静态批处理的区别。",
47     "FlashAttention 为什么能减少显存占用?",
48 ]
49
50
51 def get_model(base_url):
52     try:
53         with urllib.request.urlopen(f"{base_url}/v1/models", timeout=10) as r:
54             return json.load(r)["data"][0]["id"]
55     except (urllib.error.URLError, OSError) as e:
56         raise SystemExit(
57             f"[X] 连不上 {base_url}/v1/models: {e}\n"
58             f"    先启动一个 OpenAI 兼容的推理服务, 例如: \n"
59             f"        vllm serve Qwen/Qwen2.5-1.5B-Instruct --max-model-len 4096\n"
60             f"        llama-server -m <model>.gguf -ngl 99 -c 8192 --port 8000\n"
61             f"    再重新运行本脚本 (默认端口 8000) 。") from None
62
63
64 def one_request(base_url, model, prompt, max_tokens, system, ignore_eos=True):
65     """发送一个流式请求, 返回 (ttft, 各 token 到达时间列表, 输出 token 数)"""
66     messages = ([{"role": "system", "content": system}] if system else []) + [{"role": "user", "content": prompt}]
67     payload = {
68         "model": model, "messages": messages, "max_tokens": max_tokens,
69         "temperature": 0, "stream": True, "stream_options": {"include_usage": True
70     },
71     }
72     if ignore_eos:
73         # vLLM / SGLang 的扩展: 忽略 EOS, 保证每个请求输出长度一致, 吞吐才可比
74         payload["ignore_eos"] = True
75     body = json.dumps(payload).encode()
76     req = urllib.request.Request(f"{base_url}/v1/chat/completions", data=body,
77                                 headers={"Content-Type": "application/json"})
78     t0 = time.perf_counter()
79     arrivals, usage_tokens = [], None
80     with urllib.request.urlopen(req, timeout=600) as resp:
81         for raw in resp:
82             line = raw.decode("utf-8").strip()
83             if not line.startswith("data:"):
84                 continue
85             payload = line[5:].strip()

```

```

85         if payload == "[DONE]":
86             break
87         chunk = json.loads(payload)
88         if chunk.get("usage"):
89             usage_tokens = chunk["usage"].get("completion_tokens")
90         for ch in chunk.get("choices", []):
91             if ch.get("delta", {}).get("content"):
92                 arrivals.append(time.perf_counter() - t0)
93     n_tokens = usage_tokens or len(arrivals)  # 有 usage 就用精确 token 数；否则按流
式分片数近似
94     return (arrivals[0] if arrivals else float("nan")), arrivals, n_tokens, time.
perf_counter() - t0
95
96
97 def pct(xs, p):
98     xs = sorted(xs)
99     return xs[min(len(xs) - 1, int(round(p / 100 * (len(xs) - 1))))]
100
101
102 def run_level(args, model, concurrency, system):
103     # 预热：不计入统计。GPU 从空闲频率爬到稳态要时间，不预热的第一档会明显偏慢。
104     for _ in range(args.warmup):
105         try:
106             one_request(args.base_url, model, QUESTIONS[0], min(args.max_tokens,
32), system,
107                         ignore_eos=args.ignore_eos)
108         except (urllib.error.URLError, OSError) as e:
109             print(f"  并发 {concurrency}: 预热失败 {e}")
110             return
111
112     results, lock = [], threading.Lock()
113     n_total = max(args.num_requests, concurrency)
114     counter = iter(range(n_total))
115
116     def worker():
117         while True:
118             with lock:
119                 i = next(counter, None)
120             if i is None:
121                 return
122             try:
123                 r = one_request(args.base_url, model, QUESTIONS[i % len(QUESTIONS)
], args.max_tokens, system,
124                                 ignore_eos=args.ignore_eos)
125             except (urllib.error.URLError, OSError) as e:
126                 r = e
127             with lock:
128                 results.append(r)

```

```

129
130 t0 = time.perf_counter()
131 threads = [threading.Thread(target=worker) for _ in range(concurrency)]
132 for t in threads:
133     t.start()
134 for t in threads:
135     t.join()
136 wall = time.perf_counter() - t0
137
138 ok = [r for r in results if not isinstance(r, Exception)]
139 if not ok:
140     print(f"    并发 {concurrency}: 全部失败, 例如 {results[0]}")
141     return
142 ttft = [r[0] for r in ok if r[0] == r[0]]          # 过滤 NaN (没收到任何
content 分片的请求)
143 # TPOT 的分子是「内容分片到达时间的跨度」, 跨过的是 len(r[1])-1 个间隔;
144 # 分母必须用同一个数。用 usage.completion_tokens 会和服务端是否一个 chunk 带多个
token 耦合,
145 # 那样算出来的 TPOT 口径不一致 (llama.cpp 等会一个 chunk 塞多个 token, TPOT 会被系统性
高估)。
146 tpot, n_mismatch = [], 0
147 for r in ok:
148     if len(r[1]) > 1:
149         tpot.append((r[1][-1] - r[1][0]) / (len(r[1]) - 1))
150         if r[2] != len(r[1]):
151             n_mismatch += 1
152 out_tokens = sum(r[2] for r in ok)
153
154 if not ttft or not tpot:
155     print(f"    {concurrency:>4} | {len(ok):>4} | 样本不足 (TTFT {len(ttft)} 个 /
TPOT {len(tpot)} 个), "
156           f"调大 --num-requests 或 --max-tokens")
157     return
158 if n_mismatch:
159     print(f"    [注意] 并发 {concurrency}: {n_mismatch}/{len(ok)} 个请求的 usage.
completion_tokens "
160           f"与收到的内容分片数不一致 (一个 chunk 带了多个 token?) —— TPOT 已按分片间
隔计算。")
161     print(f"    {concurrency:>4} | {len(ok):>4} | {statistics.median(ttft) * 1e3
:8.0f} | {pct(ttft, 99) * 1e3:8.0f} | "
162           f"{statistics.median(tpot) * 1e3:8.1f} | {pct(tpot, 99) * 1e3:8.1f} | {
out_tokens / wall:10.1f} | "
163           f"{1 / statistics.median(tpot):8.1f}")
164
165
166 def main():
167     ap = argparse.ArgumentParser()
168     ap.add_argument("--base-url", default="http://localhost:8000")

```

```

169 ap.add_argument("--model", default=None, help="默认取 /v1/models 返回的第一个")
170 ap.add_argument("--concurrency", type=int, nargs="+", default=[1, 4, 16])
171 ap.add_argument("--num-requests", type=int, default=32, help="每个并发档位总共发
    多少个请求")
172 ap.add_argument("--max-tokens", type=int, default=256)
173 ap.add_argument("--ignore-eos", action=argparse.BooleanOptionalAction, default
    =True,
174                 help="让每个请求都恰好输出 --max-tokens 个 token, 吞吐才可比 (vLLM/
    SGLang 支持; "
175                 "标准 OpenAI 服务会忽略该字段)。用 --no-ignore-eos 关闭。")
176 ap.add_argument("--warmup", type=int, default=2,
177                 help="每个并发档位开测前先发这么多个不计入统计的请求, 把 GPU 频率拉起来"
    )
178 ap.add_argument("--shared-prefix-tokens", type=int, default=0,
179                 help=">0 时所有请求共享一段约这么多 token 的系统提示词, 用来观察前缀缓
    存对 TTFT 的影响")
180 args = ap.parse_args()
181
182 model = args.model or get_model(args.base_url)
183 system = None
184 if args.shared_prefix_tokens > 0:
185     # 大约每个汉字 1 个 token; 内容固定不变, 才能命中前缀缓存 (第 12 篇 §5)
186     system = ("你是一个严谨的大模型推理系统专家。" * (args.shared_prefix_tokens //
    16 + 1))[: args.shared_prefix_tokens]
187     print(f"[目标] 服务 {args.base_url} | 模型 {model} | 每档 {args.num_requests} 个
    请求 | max_tokens={args.max_tokens}"
188           + (f" | 共享前缀约 {args.shared_prefix_tokens} token" if system else ""))
189     print(f"    {'并发':>4} | {'完成':>4} | {'TTFT P50':>8} | {'TTFT P99':>8} | {'
    TPOT P50':>8} | {'TPOT P99':>8} | "
190           f"{'总吞吐 tok/s':>10} | {'单流 tok/s':>8}")
191     print(f"    {'':>4} | {'':>4} | {'(ms)':>8} | {'(ms)':>8} | {'(ms)':>8} | {'(ms
    )':>8} | {'':>10} | {'':>8}")
192     for c in args.concurrency:
193         run_level(args, model, c, system)
194     print("\n[思考] 对照第 8 篇: 并发 1 时的单流速度应接近 (但低于) 带宽下界 = 实测带宽 / 权
    重字节数; ")
195     print("    并发增大时总吞吐上升、单流速度与 TTFT 变差 —— 这就是吞吐-延迟权衡。")
196
197
198 if __name__ == "__main__":
199     main()

```